

SYSTEM DESIGN FUNDAMENTALS

From First Principles to the Interview Whiteboard

SYSTEM DESIGN FUNDAMENTALS

A Complete Guide to High-Level Design, Basic Low-Level Design, and the System Design Interview

30 Chapters · 57 Diagrams · Trade-off Tables · Interview Q&A Banks · 8 Case Studies

AMRESH KUMAR

First Edition

System Design Fundamentals

First Edition — 2026

Copyright © 2026 Amresh Kumar. All rights reserved.

No part of this publication may be reproduced, distributed, or transmitted in any form or by any means — including photocopying, recording, or other electronic or mechanical methods — without the prior written permission of the author, except in the case of brief quotations embodied in reviews and certain other non-commercial uses permitted by copyright law.

Trademarks: Product, company and service names referenced in this book — including but not limited to Amazon, AWS, Google, Netflix, Uber, WhatsApp, Meta, Redis, PostgreSQL, MongoDB, Kafka and Cassandra — are the trademarks or registered trademarks of their respective owners. They are used here purely for identification and educational commentary, and their use does not imply any endorsement or affiliation.

Disclaimer: This book is provided for educational purposes. Distributed systems evolve quickly; the architectures, limits and vendor behaviours described here reflect widely documented industry practice at the time of writing and should be validated against current official documentation before being relied upon in production. The author accepts no liability for any loss arising from the use of the material in this book.

Feedback and corrections are welcome and will be incorporated into future editions.

Contents

Preface.....	5
Who This Book Is For.....	6
How to Use This Book.....	7
How Each Chapter Is Structured	8
What Is System Design & How Interviews Work.....	9
Client-Server Model & How the Internet Works.....	17
Networking Basics for System Design	24
APIs & Communication Styles	32
The Core Trade-off: CAP Theorem & PACELC.....	39
Scalability Fundamentals	46
Databases 101 — SQL vs NoSQL	54
Database Indexing	61
Database Scaling — Replication	68
Database Scaling — Sharding & Partitioning.....	75
Caching	82
Transactions, Locks & Concurrency Control.....	89
Load Balancing	97
Message Queues & Pub-Sub	104
API Gateway & Reverse Proxy.....	112
Content Delivery Networks (CDN) & Edge Computing.....	119
Consistent Hashing Deep Dive	126
Microservices vs Monolith & Service Communication.....	133
Availability, Fault Tolerance & Redundancy	141
Monitoring, Logging & Observability	149
Security Fundamentals for System Design.....	156
Data Storage at Scale — Blob/Object Storage & File Systems.....	163
HLD Framework & Capacity Estimation Practice.....	170
Case Study: Design a URL Shortener (TinyURL).....	176
Case Study: Design a Rate Limiter & a Distributed Cache.....	182
Case Study: Design a Chat / Messaging App (WhatsApp)	188
Case Study: Design a Social Media Feed.....	194
Case Study: Design a Ride-Sharing App (Uber) & a Video Platform (YouTube/Netflix).....	200
LLD / OOD Fundamentals.....	208
The Interview Playbook - Master Cheat Sheet.....	219

Preface

Most engineers do not struggle with system design because the subject is hard. They struggle because it is taught in fragments. A blog post explains caching. A video explains sharding. A cheat sheet lists the CAP theorem. Nothing explains how these pieces fit together, why an engineer would reach for one instead of another, or what an interviewer is actually listening for when they ask you to design a system used by two hundred million people.

This book was written to close that gap. It takes you from the first question — what is a client and what is a server? — all the way to designing a ride-sharing platform under interview conditions, in one ordered path, with no step assumed and no jargon left undefined.

Two principles shaped every chapter. The first is that intuition comes before terminology: every concept opens with a plain-language explanation and an everyday analogy before a single technical term appears. The second is that engineering is the study of trade-offs, not of correct answers. There is no universally right database, protocol or architecture — there is only the option whose costs you are willing to pay for the problem in front of you. Every chapter therefore ends by making those costs explicit.

The book is deliberately written for two audiences at once: the engineer preparing for an interview next month, and the engineer who will have to keep a real system alive at three in the morning. Those two needs overlap far more than people expect, because interviewers are simply testing for the reasoning that operating real systems demands.

Who This Book Is For

You will get the most from this book if you recognise yourself in one of the following:

- Software engineers with roughly one to eight years of experience preparing for system design rounds at product companies.
- Students and recent graduates who can write good code but have never been asked to design a system that outlives a single machine.
- Backend and full-stack developers moving from feature work towards architecture and technical leadership.
- Self-taught engineers who have read dozens of scattered articles and want one sequenced, coherent path through the material.
- Experienced engineers who want a structured reference and a revision layer they can sweep through the night before an interview.

What you need to know already

Very little. You should be comfortable writing code in at least one language and have some passing familiarity with databases and web requests. You do not need any background in distributed systems, networking theory or infrastructure. Where such a background is required, this book supplies it.

What you will be able to do by the end

- Break an open-ended problem such as "design WhatsApp" into requirements, estimates, components and trade-offs, using a repeatable framework.
- Size a system on the back of an envelope — storage, bandwidth, queries per second — without a calculator.
- Choose deliberately between SQL and NoSQL, replication and sharding, REST and gRPC, strong and eventual consistency, and defend the choice.
- Recognise the failure modes — hot partitions, replication lag, cache stampedes, single points of failure — before they appear in production.
- Structure and pace a forty-five minute design interview so that you finish with a complete answer instead of a half-drawn diagram.

How to Use This Book

The thirty chapters are ordered as a single progression: foundations first, then data, then the components that sit between your users and your data, then whole-system case studies. Reading straight through is the intended experience, and each chapter assumes the vocabulary of the ones before it. That said, three practical reading paths work well.

Path 1 — The interview sprint (about two weeks)

Read Chapters 1 to 13 in order, since almost every interview question is built from them. Then jump directly to Chapter 23 for the design framework and Chapters 24 to 28 for the case studies. Finish with Chapter 30, the master cheat sheet. Treat Chapters 14 to 22 as reference material and read only the ones a case study sends you to.

Path 2 — The thorough path (about eight weeks)

Four chapters a week, in order, cover to cover. Before reading a chapter's answers, try to answer its Interviewer Q&A Bank yourself out loud. This is slower and substantially more durable, and it is the right path if you want the knowledge to survive past the interview.

Path 3 — The reference path

Use the Table of Contents to jump to the concept you need. Read the chapter's opening explanation, the Core Concepts section and the Trade-offs Table, and skip the rest until you need it.

Getting the most out of it

- Redraw every diagram by hand. A diagram you have drawn once is a diagram you can reproduce on a whiteboard under pressure; one you have only looked at is not.
- Answer the Q&A banks aloud, not in your head. Interviews are spoken, and the gap between knowing something and being able to say it clearly is the gap most candidates fail in.
- Whenever you make a design choice, say the cost out loud as well as the benefit. This single habit separates strong candidates from average ones more reliably than any amount of memorised terminology.
- Use the one-page revision sheet at the end of each chapter for spaced repetition rather than re-reading the whole chapter.

How Each Chapter Is Structured

Every chapter follows the same ten-part structure, so that once you have read one chapter you know exactly where to find anything in all the others.

1. Simple Explanation First — the idea in plain language, with an everyday analogy, before any technical vocabulary is introduced.
2. Why This Topic Exists — the real problem that forced this concept into existence, so you learn it as a solution rather than as trivia.
3. Core Concepts in Depth — the substance of the chapter, broken into numbered sub-sections and supported by diagrams.
4. Trade-offs Table — an at-a-glance comparison of the options, with what each one costs you. This is the section to revisit before an interview.
5. How Real Companies Use This — how the concept appears in production at well-known engineering organisations.
6. Common Mistakes & Misconceptions — the errors that most often appear in interviews and in real systems, and the correct reasoning in each case.
7. Interviewer Q&A Bank — the questions you should expect, with model answers you can adapt in your own words.
8. HLD Mini-Scenario — the concept applied to a realistic design problem, showing the reasoning end to end.
9. Questions YOU Should Ask the Interviewer — the clarifying questions that demonstrate senior judgement and buy you thinking time.
10. Scenario-Based Practice Bank — open problems to work through yourself, followed by a one-page revision sheet summarising the chapter.

A note on the diagrams

The book contains fifty-seven diagrams, numbered by chapter (Figure 3.2 is the second figure of Chapter 3). Each is captioned with the single idea it is meant to convey. If a diagram and the surrounding text ever seem to disagree, trust the text and treat the diagram as the simplified view.

CHAPTER 1

What Is System Design & How Interviews Work

The map before the journey — understand the game before you play it

1. Simple Explanation First

System design is the skill of deciding how the different parts of a large software system talk to each other, store data, and stay reliable when millions of people use it at the same time. It is not about writing code — it is about drawing boxes (servers, databases, caches) and arrows (requests, data flow) that solve a real problem at scale.

Analogy: Think of building a system like planning a city. A single house (one app on one laptop) is easy — you don't need traffic lights or water pipelines. But a city of 10 million people needs roads (networks), water supply (databases), traffic police (load balancers), and backup power (redundancy). System design is city planning for software.

2. Why This Topic Exists

Companies don't hire engineers only to write correct code — they hire engineers who can build things that don't fall over when 1 million users show up instead of 100. Early in your career, you write code that works. As you grow senior, you're expected to design systems that keep working under failure, growth, and unpredictable traffic. System design interviews exist purely to test this judgment, because it's nearly impossible to test it through a coding question alone.

Why this matters: If you can't reason about trade-offs at a whiteboard level, no amount of coding skill convinces an interviewer you can own a large piece of infrastructure. This is why FAANG-level and senior interviews almost always include a dedicated system design round.

3. Core Concepts in Depth

3.1 HLD vs LLD — the two flavors of system design

High-Level Design (HLD) is about the big picture: which services exist, how they communicate, which databases are used, where caching happens. You draw boxes and arrows — you rarely write actual code. Low-Level Design (LLD), also called Object-Oriented Design (OOD), zooms into ONE box from the HLD and asks: what classes, interfaces, and methods would actually implement this? LLD is closer to real code — you draw class diagrams and define relationships (inheritance, composition).

HLD vs LLD — Zooming Into One Box

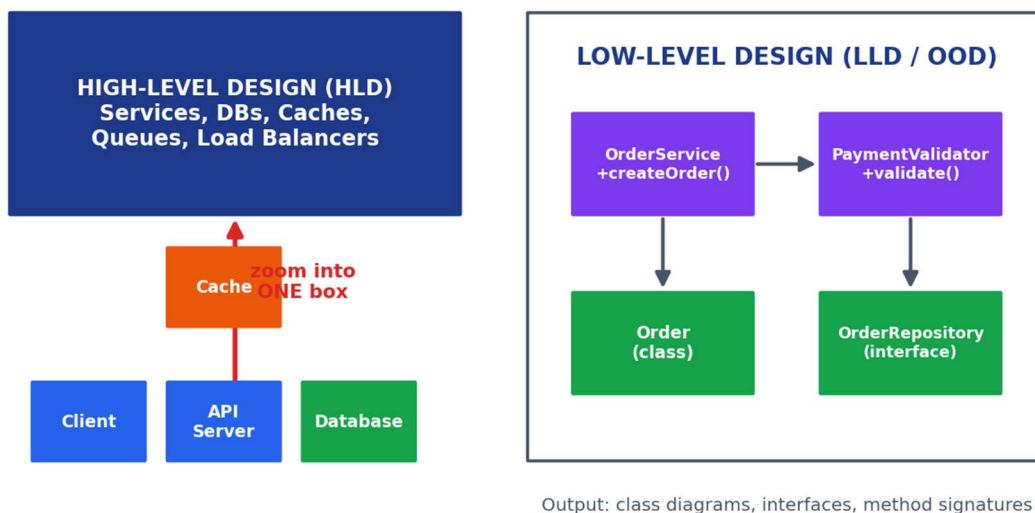


Figure 1.1 — HLD gives you the boxes and arrows; LLD zooms into one box and designs its internal classes.

HLD	LLD / OOD
Answers: what are the pieces and how do they connect?	Answers: how is one piece actually coded?
Output: architecture diagram	Output: class diagram, interfaces, method signatures
Example question: "Design Twitter"	Example question: "Design a Parking Lot system"
Tools of the trade: load balancers, databases, caches, queues	Tools of the trade: SOLID principles, design patterns

3.2 The 45-Minute Interview Structure

Most HLD interviews follow a loose but predictable timeline. Knowing this lets you self-pace instead of running out of time on one section.

The 45-Minute Interview Timeline



Figure 1.2 — A typical 45-minute HLD interview, minute by minute.

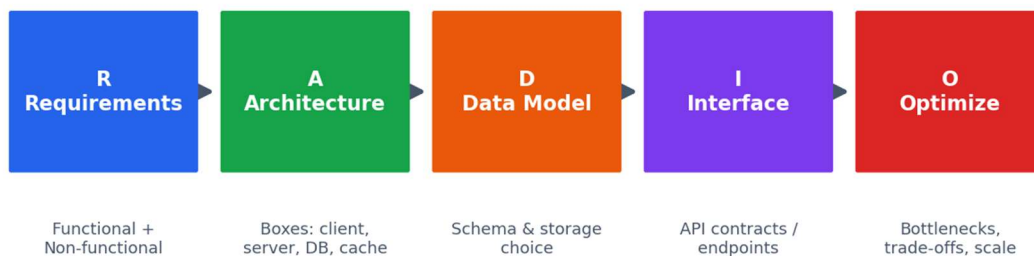
Time	What Happens
0 – 5 min	Interviewer gives a vague, intentionally underspecified prompt (e.g. "Design Instagram"). You ask clarifying questions.
5 – 10 min	You define scope, requirements, and do rough capacity estimation (back-of-envelope math).
10 – 15 min	You propose a high-level architecture — the major boxes and how data flows.
15 – 35 min	Deep dive into 1-2 components the interviewer is curious about (often database schema, or a specific bottleneck).
35 – 45 min	Discuss trade-offs, failure scenarios, scaling further, and answer interviewer's follow-up 'what if' questions.

Common mistake: Spending 20 minutes drawing a 'perfect' diagram before discussing any trade-offs. Interviewers want to see your thinking evolve, not a finished mural. Start rough, refine as you talk.

3.3 The RADIO Framework — your repeatable method

RADIO is a simple acronym to make sure you never freeze in front of a vague prompt. We will reuse this exact framework in every HLD case study chapter (Part 5), so memorize it now.

The RADIO Framework



Used at the start of every HLD interview — tune the RADIO before you build anything.

Figure 1.3 — The RADIO framework: run through these five steps in order for any 'design X' prompt.

Letter	Stands For & What You Do
R	Requirements — clarify functional requirements (what must it do) and non-functional requirements (scale, latency, availability).
A	Architecture — draw the high-level boxes: client, servers, database, cache, queue.
D	Data Model — define what data looks like and where it's stored (which DB, what schema).
I	Interface (API) — define the key API endpoints/contracts between components.
O	Optimize & scale — discuss bottlenecks, trade-offs, and how the design evolves as load grows.

Memory hook: RADIO: you're tuning into the right frequency before you build anything — Requirements, Architecture, Data, Interface, Optimize.

3.4 Functional vs Non-Functional Requirements

This distinction is one of the first things you must nail down out loud, because it silently shapes every later decision.

- Functional requirements: what the system actually does. Example for a chat app — "users can send messages, see read receipts, create groups."
- Non-functional requirements: how well it does it. Example — "must support 50 million daily active users, message delivery under 200ms, 99.99% uptime."

Why this matters: Non-functional requirements (scale, latency, consistency needs) are what actually determine your architecture. Two systems with identical functional requirements can have completely different designs if one needs to serve 100 users and the other needs to serve 100 million.

4. Trade-offs in How You Run the Interview

Approach	Pros	Cons
Ask many clarifying questions upfront	Avoids designing the wrong thing; shows maturity	Eats into your limited time if overdone
Jump straight into drawing a diagram	Looks fast and confident initially	Risk of solving a problem the interviewer didn't ask for
Go very deep on one component early	Shows technical depth	May run out of time to cover the full system breadth

5. How Real Companies Approach This

Big tech companies structure interviews differently, but the underlying skill tested is identical: can you reason about scale and trade-offs out loud?

- Google/Meta: typically one dedicated 45-minute HLD round in senior-level loops, graded on a fixed rubric (problem navigation, high-level design, deep dive, communication).
- Amazon: often blends system design with their Leadership Principles — expect questions like "tell me about a time you made a trade-off," mixed into the design discussion.
- Startups: usually less formal, but want to see if you can ship something reasonable fast — they value pragmatism over textbook-perfect designs.

6. Common Mistakes & Misconceptions

- Believing system design is about memorizing one 'correct' architecture for each famous app — there isn't one. Interviewers care about your reasoning, not a memorized diagram.
- Treating it like a trivia round (naming technologies without justifying them). Saying 'I'll use Kafka' without explaining why is a red flag.
- Staying silent while thinking. Interviewers can't grade a silent brain — narrate your thought process constantly.
- Never asking the interviewer anything — it can come across as not caring about the real constraints.

7. Interviewer Q&A Bank

Q1. What is the difference between HLD and LLD?

A: HLD defines the overall architecture — the services, databases, and how they communicate at a system level. LLD zooms into a single component and defines the actual classes, interfaces, and methods that implement it. HLD answers 'what pieces exist,' LLD answers 'how is one piece coded.'

Q2. How would you approach a vague prompt like 'Design Instagram'?

A: I'd start by narrowing scope through clarifying questions — which features matter most (photo upload, feed, stories, DMs?), expected scale (users, requests per second), and key non-functional needs

(latency, consistency vs availability). Only after scoping would I sketch the high-level architecture, following a structured framework like RADIO so I don't miss a step.

Q3. Why do companies test system design instead of just more coding rounds?

A: Coding rounds test whether you can solve a well-defined, bounded problem correctly. System design tests a different skill: reasoning under ambiguity, trade-off judgment, and the ability to design something that survives real-world scale and failure — skills that become more important as engineers grow senior and start owning larger systems.

Q4. What makes a system design answer 'senior level' versus 'junior level'?

A: A junior answer names technologies. A senior answer justifies each decision with a trade-off, anticipates failure modes, and adapts the design when the interviewer changes a constraint (e.g., '10x the traffic') without starting over from scratch.

Q5. How do you handle a question you genuinely don't know the answer to?

A: I'd be honest that I haven't worked with that exact technology, then reason from first principles about what trade-off it's likely solving, based on similar tools I do know. Interviewers value honest reasoning over confident guessing — pretending to know something and getting caught is far worse than admitting a gap and reasoning through it.

8. HLD Mini-Scenario: Practicing the RADIO Framework

Prompt: "Design a system that shortens long URLs into short ones." (We'll do this in full depth in Chapter 24 — here we just practice the RADIO skeleton.)

- Requirements: Functional — users submit a long URL and get a short one; visiting the short URL redirects to the original. Non-functional — low latency redirects, high read:write ratio, eventually consistent is fine.
- Architecture: A client talks to a web server, which talks to a database storing the short-to-long mapping; a cache sits in front of the database for fast redirects.
- Data Model: A simple table — short_code (primary key), long_url, created_at, expiry.
- Interface: POST /shorten {long_url} returns {short_url}; GET /{short_code} returns a 302 redirect.
- Optimize: Add caching for popular short links, use a CDN/edge layer for redirects, and plan sharding once the database grows beyond one machine's capacity.

Notice we didn't write a single line of code — that's the nature of HLD. Every later chapter builds your vocabulary so each of these five steps gets richer and more technically precise.

9. Questions YOU Should Ask the Interviewer (Process-Level)

These are general questions about the interview itself — useful at the very start of any system design round, regardless of topic:

- Should I focus more on breadth across the whole system, or depth on one or two components?
- Is there a specific scale (users, requests/sec) you'd like me to design for, or should I propose one?
- Would you like me to talk through trade-offs as I go, or present a design first and discuss after?
- Is this evaluating HLD, LLD, or both in this session?

10. Scenario-Based Practice Bank (New)

These scenarios go beyond definitions — they test whether you can apply Chapter 1's ideas when the situation shifts under you, the way it does in a real interview.

Scenario 1: The interviewer says 'don't worry about scale, just get the logic right' for what would normally be a scale-heavy prompt.

***How to approach it:** Compress the R and A steps of RADIO, and spend most of your time on D (data model) and I (API/interface) since the O (optimize/scale) step has been explicitly de-prioritized. Still mention non-functional requirements in one sentence so the interviewer sees you're aware of them even though you're not designing for them.*

Scenario 2: Halfway through your 45 minutes, the interviewer says: 'Now assume this needs to support 100x more users.'

***How to approach it:** Don't restart the design. Treat this as the natural 'Optimize' step of RADIO — identify the current bottleneck in your existing design (usually the single database or a single application server), and propose targeted changes: caching, read replicas, horizontal scaling, sharding. Reusing and evolving your design signals seniority far more than a redesign from scratch.*

Scenario 3: You're asked to design something you've genuinely never worked with in production, like a stock exchange matching engine.

***How to approach it:** State plainly that you haven't built this exact system, then reason from first principles using concepts you DO know (ordering guarantees, low-latency messaging, consistency needs), and explicitly flag your assumptions. This is far stronger than bluffing confidence you don't have.*

Scenario 4: The prompt is extremely vague: 'Design something like Uber,' with 45 minutes total.

***How to approach it:** Resist the urge to start drawing immediately. Spend the first few minutes scoping: which part of Uber (rider-driver matching, pricing, live tracking, payments)? What scale? Only then move into architecture — an unscoped 45 minutes on 'all of Uber' guarantees a shallow, unfocused answer.*

Scenario 5: You finish your high-level architecture in 12 minutes, well ahead of the expected 15-minute mark, and the interviewer hasn't jumped in with a deep-dive question yet.

***How to approach it:** Don't sit in silence waiting to be told what to do. Proactively suggest 1-2 components you'd like to explore deeper (e.g., 'I'd like to go deeper on how we shard the database, or how we handle the feed generation — which would be more useful to you?'). This shows initiative and helps you control pacing.*

Scenario 6: You're doing an asynchronous, written take-home system design assessment instead of a live 45-minute interview.

***How to approach it:** Since there's no interviewer to ask clarifying questions to in real time, write an explicit 'Assumptions' section up front covering scale, key features in scope, and non-functional targets. You can afford a more complete diagram and more thorough trade-off discussion since there's no live time pressure, but keep it as tight and structured as a live answer would be.*

Chapter 1 — One-Page Revision Sheet

- System design = deciding how system parts communicate, store data, and stay reliable at scale.
- HLD = big picture (boxes & arrows, services/DBs/caches). LLD/OOD = one box zoomed in (classes, methods, design patterns).
- 45-min interview shape: clarify (5-10m) -> high-level architecture (10-15m) -> deep dive (15-35m) -> trade-offs/scaling (35-45m).
- RADIO framework: Requirements -> Architecture -> Data Model -> Interface -> Optimize. Use this every time you face a 'design X' prompt.
- Functional requirements = what it does. Non-functional requirements = how well (scale, latency, availability) — these drive your architecture more than the features do.
- Interviewers grade reasoning and trade-offs, not memorized diagrams. Narrate your thinking out loud, constantly.
- Always ask scoping questions before designing; always ask process questions if unsure how the round is structured.

Memory hook: *Before drawing a single box, tune the RADIO.*

CHAPTER 2

Client-Server Model & How the Internet Works

What actually happens between you typing a URL and a page appearing

1. Simple Explanation First

The client-server model is the basic shape of almost every system you'll ever design: one side (the client) asks for something, the other side (the server) provides it. Your phone's Instagram app is a client. Instagram's data centers are the server. Every button tap sends a request across the internet to a server, which sends back a response.

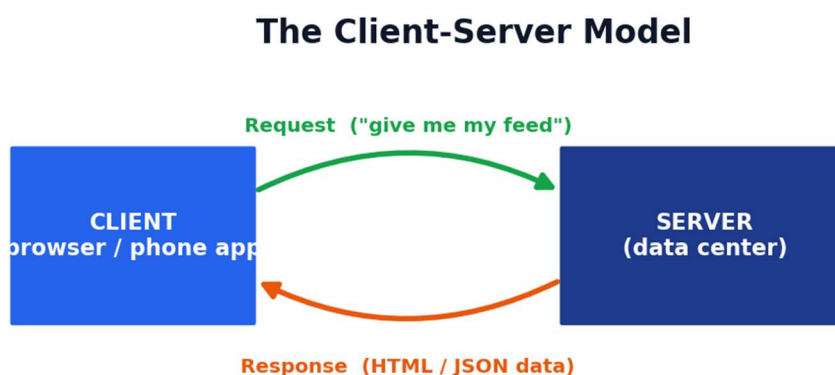


Figure 2.1 — The client always initiates; the server always responds.

Analogy: Think of a restaurant. You (the client) don't walk into the kitchen and cook your own food. You give your order to a waiter, who carries it to the kitchen (the server), which prepares the food and sends it back. You never need to know how the kitchen works internally — you just need the menu (the interface) and a way to place an order (the request).

2. Why This Topic Exists

Before the client-server model, software ran entirely on one machine — your data, your logic, all local. The internet forced a split: data and processing power live on remote, more powerful machines, while a lightweight client just displays results. This separation is what allows millions of cheap, low-powered devices (phones) to use the same shared, powerful backend.

Why this matters: Every system design problem starts by assuming this split exists. Understanding exactly what travels between client and server, and how, is the foundation everything else in this book builds on.

3. Core Concepts in Depth

3.1 What Happens When You Type a URL and Press Enter

This is one of the most classic warm-up questions in system design and networking interviews. Walking through it cleanly signals strong fundamentals.



Figure 2.2 — The full journey from keystroke to rendered page.

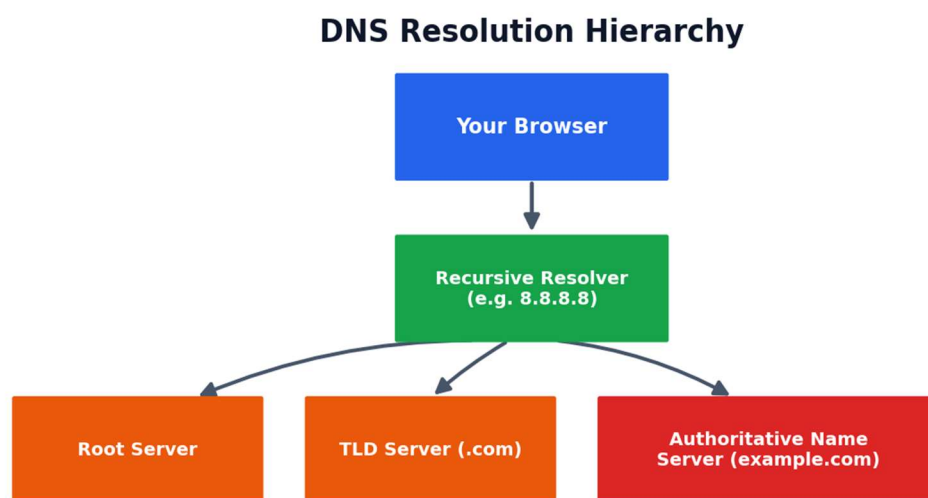
- 1. Browser checks its cache for a recent DNS lookup of the domain; if not found, it asks the OS, which may also have a cached result.
- 2. If nothing is cached, a DNS resolution happens: the request goes to a DNS resolver, which queries root servers, then TLD servers (e.g. .com), then the domain's authoritative name server, to translate the domain name into an IP address.
- 3. The browser opens a TCP connection to that IP address on port 443 (HTTPS) via a 3-way handshake (SYN, SYN-ACK, ACK).
- 4. A TLS handshake happens to set up encryption (certificates are exchanged and verified, a shared secret key is agreed upon).
- 5. The browser sends an HTTP GET request for the page.
- 6. The request may first hit a load balancer, which forwards it to one of many backend servers.

- 7. The server processes the request (possibly querying a database or cache) and sends back an HTTP response with HTML/JSON.
- 8. The browser parses the response, makes further requests for assets (CSS, JS, images — often served via a CDN), and renders the page.

Common mistake: Forgetting DNS resolution entirely and jumping straight to 'the browser sends a request to the server.' Interviewers specifically listen for DNS, TCP handshake, and TLS handshake — skipping them signals a shallow understanding of networking.

3.2 DNS — The Internet's Phonebook

DNS (Domain Name System) converts human-readable names (google.com) into machine-usable IP addresses (e.g., 142.250.183.14). Without DNS, you'd have to memorize numbers instead of names.



Each layer is cached with a TTL, so most lookups skip the full round-trip.

Figure 2.3 — The DNS lookup chain, from your browser to the authoritative source of truth.

DNS Server Type	Role
Recursive Resolver	The first stop — usually run by your ISP or a public service (e.g. Google's 8.8.8.8); does the legwork of querying other servers on your behalf.
Root Server	Knows where to find servers responsible for top-level domains (.com, .org, .net).
TLD Server	Knows where to find the authoritative server for a specific domain (e.g. all .com domains).
Authoritative Name Server	Holds the actual IP address record for the specific domain you requested.

DNS results are cached aggressively at every layer (browser, OS, ISP) using a TTL (time-to-live) value, because DNS lookups add latency and most domains rarely change IP addresses.

3.3 IP Addresses & Ports

An IP address identifies a specific machine on a network (like a building's street address). A port identifies a specific application or service running on that machine (like an apartment number within the building). A server can run multiple services simultaneously, each listening on a different port.

- IPv4: 32-bit addresses (e.g., 192.168.1.1) — roughly 4.3 billion possible addresses, now largely exhausted.
- IPv6: 128-bit addresses — astronomically more addresses, designed to solve IPv4 exhaustion.
- Well-known ports: 80 (HTTP), 443 (HTTPS), 22 (SSH), 53 (DNS), 3306 (MySQL default).

3.4 Stateless vs Stateful Communication (Preview)

HTTP, the protocol used for most web communication, is stateless by design — each request is independent and the server doesn't remember previous requests from the same client unless you explicitly add mechanisms like cookies or sessions. This single design decision deeply affects how you scale servers horizontally (covered fully in Chapter 6).

Why this matters: Statelessness is why you can add more servers behind a load balancer freely — any server can handle any request, since no server is 'remembering' a particular user. This is foundational to horizontal scaling, which we revisit constantly throughout this book.

4. Trade-offs Table

Decision	Pros	Cons
Low DNS TTL (cache expires quickly)	Can redirect traffic / change IPs quickly (useful during failover)	More frequent DNS lookups, slightly higher latency on average
High DNS TTL (cache expires slowly)	Fewer lookups, faster average response	Slower to propagate IP/server changes during incidents
Stateless servers	Easy horizontal scaling, any server can serve any request	Need external storage (DB/cache) for session data — extra complexity

5. How Real Companies Use This

- Cloudflare and Google Public DNS (8.8.8.8) run massive recursive resolver networks specifically to speed up DNS lookups globally.
- Netflix and large streaming companies use very low DNS TTLs intentionally, so they can reroute traffic away from a failing data center within seconds during an outage.
- Most companies put a CDN (Cloudflare, Akamai, Fastly) in front of their DNS so that the IP address returned can even vary by your geographic location — sending you to the nearest data center.

6. Common Mistakes & Misconceptions

- Thinking a domain name maps to a single fixed IP forever — in reality, DNS can return different IPs depending on location, load, or time (DNS-based load balancing).
- Confusing TCP handshake with TLS handshake — they are two separate steps; TCP establishes a raw connection, TLS adds encryption on top of it.

- Assuming HTTP is always stateless in practice — cookies/sessions can simulate state, but the protocol itself has no memory between requests.

7. Interviewer Q&A Bank

Q1. What happens when you type a URL into a browser and press Enter?

A: Briefly: the browser checks DNS caches, resolves the domain to an IP if not cached, opens a TCP connection via a 3-way handshake, performs a TLS handshake for HTTPS, sends an HTTP request (often through a load balancer), and the server responds with the page content, which the browser then renders, fetching any additional assets.

Q2. Why is DNS resolution cached at multiple layers?

A: Because DNS lookups add real latency to every single network request, and most domain-to-IP mappings change rarely. Caching at the browser, OS, and ISP level avoids repeating an expensive multi-hop lookup (root -> TLD -> authoritative server) for every request.

Q3. What is the difference between an IP address and a port?

A: An IP address identifies a specific machine on a network. A port identifies a specific application or service running on that machine. The combination of IP + port uniquely identifies where a particular service is reachable — this is why one server can run a web server on port 80/443 and a database on port 3306 simultaneously.

Q4. Why does HTTP being stateless matter for system design?

A: Statelessness means any server in a pool can handle any incoming request, since no server holds client-specific memory between requests. This is what makes horizontal scaling simple — you can freely add or remove servers behind a load balancer without worrying about which server 'remembers' a given user. To preserve user state across requests, we externalize it into cookies, tokens, or a shared session store.

Q5. What's the difference between a recursive resolver and an authoritative name server?

A: A recursive resolver is the middleman that does the work of tracking down an answer on your behalf, querying multiple servers if needed. An authoritative name server is the final source of truth — it actually holds the DNS record for a specific domain and gives the definitive answer.

8. HLD Mini-Scenario: Where Does DNS Fit in a Real Architecture?

Prompt: "A user in India requests a video from a US-based streaming service. How does the request get routed efficiently?"

- 1. The user's browser resolves the domain via DNS. The DNS provider uses geo-aware routing, returning the IP of a server or CDN edge node physically closer to India rather than the US origin server.
- 2. The request reaches a regional CDN edge node first — if the video is cached there, it's served immediately without ever reaching the US data center.
- 3. If not cached, the edge node fetches it from the origin server in the US, caches it for future requests, and returns it to the user.

This single mini-scenario foreshadows Chapter 16 (CDNs) — notice how DNS isn't just 'name to IP,' it can be an active load-balancing and geo-routing tool.

9. Questions YOU Should Ask the Interviewer

- Should I assume a single data center, or do we need to consider multi-region/global users for this design?
- Is low latency for first-time DNS lookups a real concern here, or is this more about steady-state traffic?
- Do we control the DNS layer (e.g., can we do geo-routing), or are we just consuming a fixed entry point?

10. Scenario-Based Practice Bank (New)

Scenario 1: Users in a country with poor mobile networks report your app takes noticeably longer to load than in the US.

***How to approach it:** Check DNS geo-routing first — are they even being sent to the nearest edge node? Then look at handshake overhead: TCP + TLS round trips are far more painful on high-latency links, so consider TLS session resumption and evaluate whether HTTP/3 (QUIC over UDP) would help, since it removes a round trip and tolerates packet loss better.*

Scenario 2: Your app serves users globally and you want to minimize DNS lookup latency without sacrificing the ability to react quickly to an incident.

***How to approach it:** Use geo-DNS/anycast so users are routed to their nearest region by default, but keep TTLs low specifically on the records you might need to redirect during a failover, while using longer TTLs for stable, rarely-changing records — this balances lookup speed against incident-response speed.*

Scenario 3: A legacy application stores session data in each server's local memory, and the company wants to move it behind a load balancer with multiple horizontally-scaled servers.

***How to approach it:** Before anything else, externalize session state into a shared store (Redis, a database, or signed client-side tokens/JWTs). Without this, the load balancer would need 'sticky sessions' tying each user to one server, which defeats much of the benefit of horizontal scaling.*

Scenario 4: The security team asks why your app still uses TLS on port 443 if handshake latency is a concern for repeat visitors.

***How to approach it:** Explain that TLS overhead is mostly a first-connection cost: TLS session resumption (and TLS 1.3's 0-RTT) lets returning clients skip much of the handshake, so keeping HTTPS doesn't have to mean paying full handshake latency on every single request.*

Scenario 5: Users in one specific region report intermittent failures reaching your site, while everywhere else is fine.

***How to approach it:** Suspect the DNS/routing layer for that region first — it may be resolving to an unhealthy IP or a regional load balancer/data center having issues. Use tools like dig and traceroute from that region (or a service that simulates it) to see exactly which IP is being returned and where the request is failing.*

Scenario 6: An interviewer says: 'Assume this internal admin tool doesn't use HTTPS, just plain HTTP.'

***How to approach it:** Flag this clearly as a security concern even in a hypothetical: without TLS, all data — including credentials — travels in plain text and can be intercepted or tampered with on the network. Note that most modern load balancers and CDNs also assume TLS termination, so skipping it removes options later, not just security.*

Chapter 2 — One-Page Revision Sheet

- Client-server model: client requests, server responds — separates lightweight devices from powerful shared backends.
- URL-to-page flow: DNS resolution -> TCP 3-way handshake -> TLS handshake -> HTTP request (often via load balancer) -> server response -> render + fetch assets.
- DNS hierarchy: Recursive Resolver -> Root Server -> TLD Server -> Authoritative Name Server. Cached at multiple layers using TTL.
- IP address = machine identity. Port = specific service on that machine. IP + port = exact reachable endpoint.
- HTTP is stateless by design — no memory between requests unless you add cookies/sessions/tokens. This is the foundation of horizontal scaling.
- DNS can do more than name resolution — geo-aware DNS and CDNs route users to the nearest server, reducing latency.
- Always distinguish TCP handshake (raw connection) from TLS handshake (encryption setup) — they are separate steps.

Memory hook: *DNS is the internet's phonebook; TCP is the handshake; TLS is the secret code; HTTP is the actual conversation.*

CHAPTER 3

Networking Basics for System Design

TCP, UDP, HTTP versions, WebSockets, and TLS — the protocols underneath every system

1. Simple Explanation First

Networking protocols are simply agreed-upon rules for how two machines talk to each other. TCP and UDP are two different 'languages' for sending data packets across a network. HTTP is built on top of TCP and defines the format of web requests/responses. WebSockets allow a constant two-way conversation instead of one-off question-answer exchanges. TLS is the encryption layer that keeps the conversation private.

TCP (Registered Letter) vs UDP (Shouting Across a Room)

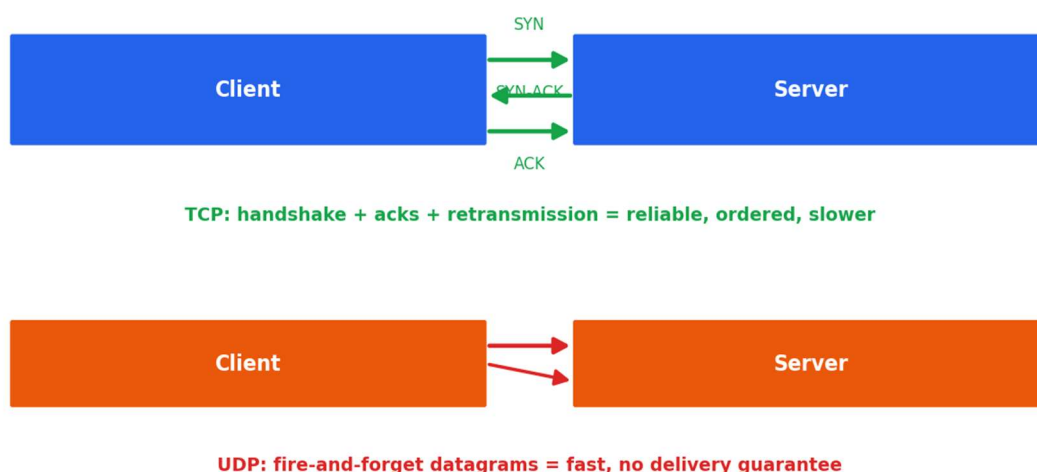


Figure 3.1 — TCP confirms every step of delivery; UDP just sends and moves on.

Analogy: TCP is like sending a registered letter — the postal service confirms delivery, and if a page is missing, they resend it. You're guaranteed completeness, but it's slower. UDP is like shouting across a room — fast, but if someone doesn't hear you, nobody resends it. You choose TCP when correctness matters (loading a webpage); you choose UDP when speed matters more than perfection (a live video call where a dropped frame is fine, but lag is not).

2. Why This Topic Exists

Different applications have wildly different needs. A bank transfer must arrive completely and in order — losing one byte is unacceptable. A live video call can tolerate losing a frame here and there, but cannot tolerate the delay of resending lost data. No single protocol satisfies both needs well, so multiple protocols exist, each optimized for a different trade-off between reliability and speed.

Why this matters: Choosing the wrong protocol for a system component is a classic 'gotcha' in interviews — e.g., suggesting TCP for real-time gaming, or UDP for a payments API. Interviewers test whether you understand why a protocol fits a specific use case, not just that the protocols exist.

3. Core Concepts in Depth

3.1 TCP vs UDP

TCP (Transmission Control Protocol) is connection-oriented: it establishes a connection first (3-way handshake), guarantees ordered and complete delivery by acknowledging packets and resending lost ones, and uses flow/congestion control to avoid overwhelming the network. UDP (User Datagram Protocol) is connectionless: it just fires packets ('datagrams') with no guarantee of delivery, order, or duplication-prevention — but it's much faster because it skips all the handshake and acknowledgment overhead.

TCP	UDP
Connection-oriented (handshake required)	Connectionless (no handshake)
Guarantees delivery & order	No guarantee of delivery or order
Slower due to acknowledgments & retransmission	Faster — minimal overhead
Used for: web browsing, file transfer, email, APIs	Used for: video streaming, VoIP, gaming, DNS lookups

Common mistake: Saying 'UDP is unreliable so we should never use it.' Unreliable doesn't mean useless — many real-time systems (video calls, live sports streaming, multiplayer games) deliberately choose UDP because a slightly imperfect, fast stream beats a perfect but laggy one.

3.2 HTTP/1.1 vs HTTP/2 vs HTTP/3

HTTP is the application-layer protocol that defines how clients and servers exchange requests and responses. It has evolved significantly to solve performance problems.

HTTP/1.1 vs HTTP/2 vs HTTP/3

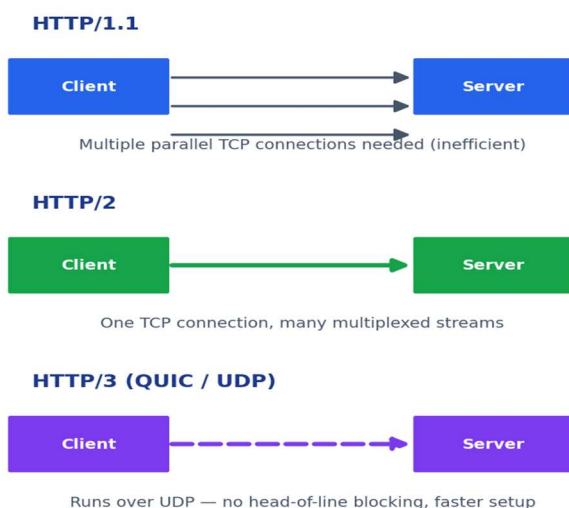


Figure 3.2 — HTTP/1.1 needs multiple connections; HTTP/2 multiplexes one; HTTP/3 moves to UDP-based QUIC.

Version	Key Characteristics
HTTP/1.1	One request per TCP connection at a time (without pipelining); browsers open multiple parallel connections to compensate, which is inefficient.
HTTP/2	Multiplexing — many requests/responses share a single TCP connection simultaneously; adds header compression; introduces server push.
HTTP/3	Built on QUIC (which runs over UDP instead of TCP) — avoids TCP's head-of-line blocking entirely, faster connection setup, better performance on unreliable networks (e.g., mobile).

Why this matters: *Head-of-line blocking — where one lost packet stalls all other data behind it on the same connection — is a recurring real-world performance problem. HTTP/2 reduced it; HTTP/3 (via QUIC over UDP) eliminates it almost entirely. This is exactly the kind of nuance that separates a strong answer from a generic one.*

3.3 WebSockets — Persistent Two-Way Communication

Regular HTTP is request-response: the client always initiates. But some applications (chat apps, live dashboards, multiplayer games) need the server to push data to the client at any time without the client asking first. WebSockets solve this by upgrading a normal HTTP connection into a persistent, full-duplex (two-way) TCP connection that stays open, allowing either side to send messages anytime.

- Handshake: starts as a normal HTTP request with an 'Upgrade: websocket' header; once both sides agree, the connection switches protocols.
- Used for: chat apps, live notifications, collaborative editing (e.g., Google Docs), stock tickers, multiplayer games.
- Trade-off: keeping many persistent connections open is more resource-intensive on the server than stateless HTTP requests — this affects how you scale WebSocket servers (often needs sticky sessions or a pub-sub backend to fan out messages across server instances).

3.4 Polling, Long-Polling, and Streaming — Alternatives to WebSockets

Before reaching for WebSockets, it's worth knowing the simpler alternatives, since interviewers want to see you choose the simplest solution that fits, not the most advanced one.

Polling vs Long-Polling vs WebSockets

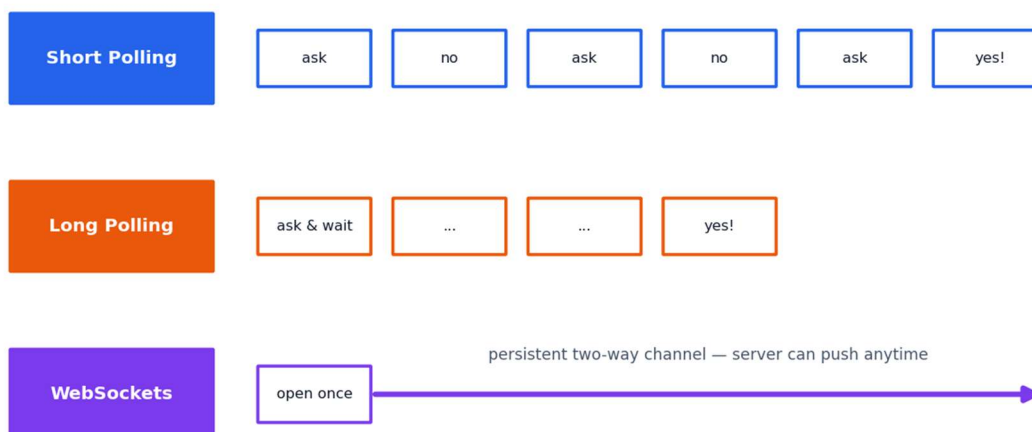


Figure 3.3 — Short polling asks repeatedly, long-polling waits, WebSockets stay open.

Technique	How It Works	Best For
Short Polling	Client repeatedly asks the server 'anything new?' every few seconds	Simple, low-frequency updates where some delay is fine
Long Polling	Client asks once; server holds the request open until new data exists, then responds	Near-real-time updates without full WebSocket complexity
WebSockets	Persistent two-way connection; either side pushes anytime	True real-time, frequent bidirectional updates (chat, gaming)

3.5 The TLS Handshake (Encryption Setup)

TLS (Transport Layer Security) sits on top of TCP and encrypts the data so eavesdroppers can't read it. The handshake establishes a shared secret key without ever sending that key in plain text over the network — using asymmetric cryptography to safely exchange it, then switching to faster symmetric encryption for the actual data.

- 1. Client sends a 'Client Hello' — supported TLS versions and encryption methods.
- 2. Server responds with a 'Server Hello,' its TLS certificate (proving its identity), and a chosen encryption method.
- 3. Client verifies the certificate against trusted Certificate Authorities, then both sides derive a shared symmetric session key.
- 4. All further communication is encrypted using this fast symmetric key.

4. Trade-offs Table

Protocol Choice	Pros	Cons
TCP for an API service	Reliable, ordered, guaranteed delivery — correct data every time	Higher latency overhead from handshakes & acknowledgments
UDP for live video/audio	Low latency, no resend delays	Possible dropped frames/packets; no delivery guarantee
WebSockets for chat	True real-time push from server, low per-message overhead after setup	More server resources per open connection; harder to scale horizontally

5. How Real Companies Use This

- WhatsApp and Slack use WebSocket-like persistent connections (often via XMPP or custom protocols) to push messages instantly without polling.
- YouTube and Netflix use a mix — initial video manifest requests over HTTP/TCP, but underlying media transport for live streaming often leans on UDP-based protocols for lower latency.
- Google was a primary driver behind QUIC (the protocol underlying HTTP/3), specifically to speed up search and ad-serving on unreliable mobile networks.

6. Common Mistakes & Misconceptions

- Assuming WebSockets are always 'better' than HTTP polling — for infrequent updates, long-polling is simpler to operate and sufficient.
- Forgetting that HTTP/3 runs over UDP, not TCP — many candidates assume all HTTP traffic is TCP-based.
- Mixing up TLS with TCP's handshake — they are layered, sequential, and separate steps.

7. Interviewer Q&A Bank

Q1. When would you choose UDP over TCP in a system design?

A: When low latency matters more than perfect delivery — real-time video/audio calls, live game state updates, or live sports streaming. A dropped UDP packet just causes a brief glitch; waiting for TCP to detect loss and retransmit would introduce noticeable lag, which is worse for these use cases.

Q2. How would you design real-time notifications for a chat application?

A: I'd use WebSockets for the persistent, low-latency, bidirectional channel needed for instant message delivery. For scaling across multiple WebSocket server instances, I'd add a pub-sub layer (like Redis Pub/Sub or Kafka) so a message sent to a user connected to Server A can be delivered even if it originated from Server B.

Q3. What problem does HTTP/2 multiplexing solve?

A: In HTTP/1.1, browsers had to open multiple parallel TCP connections to fetch resources concurrently, since one connection could only handle one request at a time effectively. HTTP/2 multiplexing allows many requests and responses to share a single TCP connection simultaneously, reducing connection overhead and improving page load performance.

Q4. Why might long-polling be preferred over WebSockets for some systems?

A: Long-polling is simpler to implement and operate using standard HTTP infrastructure (load balancers, proxies) without special handling, and it doesn't require maintaining many persistent open connections on the server. For features where updates aren't extremely frequent, it offers near-real-time behavior with far less operational complexity.

Q5. What does the TLS handshake actually accomplish?

A: It lets the client verify the server's identity using a certificate, and lets both sides agree on a shared secret encryption key without ever transmitting that key in plain text — using asymmetric cryptography briefly to bootstrap a faster symmetric encryption session for the rest of the communication.

8. HLD Mini-Scenario: Choosing the Right Protocol

Prompt: "Design the real-time communication layer for a multiplayer online game where players see each other's positions update live."

- 1. Requirement check: position updates need to be extremely low-latency; occasionally losing one position update is acceptable since the next update arrives in milliseconds anyway.
- 2. Protocol choice: UDP for position/movement data (speed over perfection), but TCP (or a reliable layer on top of UDP) for critical events like 'player purchased item' or 'match result,' where correctness matters more than speed.
- 3. This shows a key senior-level insight: a single system can use multiple protocols for different types of data, rather than forcing one protocol everywhere.

9. Questions YOU Should Ask the Interviewer

- How latency-sensitive is this feature — is a small delay acceptable, or does it need to feel instantaneous?
- Is occasional data loss acceptable for this use case, or must every message be guaranteed to arrive?
- Should I assume clients are on reliable WiFi/broadband, or do we need to account for unstable mobile networks?

10. Scenario-Based Practice Bank (New)

Scenario 1: Your multiplayer game currently sends player position updates over TCP, and players report characters 'warping' or stuttering under any packet loss.

How to approach it: This is the classic TCP-for-real-time mistake: a single lost packet forces TCP to stall everything behind it while it retransmits, causing the warp. Switch position/movement data to UDP, where an occasional dropped update just gets superseded by the next one milliseconds later.

Scenario 2: A payments team proposes using UDP for their transaction API to 'make it faster.'

How to approach it: Push back: payments need guaranteed, ordered delivery — a dropped or duplicated packet here means real money is wrong. The milliseconds UDP saves aren't worth the correctness risk; TCP's overhead is the right trade-off for this use case.

Scenario 3: Your chat app's WebSocket-based real-time messaging needs to scale to millions of concurrent connections across many servers, and a user on Server A isn't receiving messages sent by a user on Server B.

How to approach it: A single WebSocket server can't know about connections held by other servers. Add a pub-sub backend (Redis Pub/Sub, Kafka, or similar) so that when Server B receives a message for a user connected to Server A, it publishes the event and Server A's process picks it up and delivers it over that user's open socket.

Scenario 4: Mobile users report especially slow page loads right when their phone switches from WiFi to cellular data.

How to approach it: This is a good fit for HTTP/3 (QUIC): unlike TCP, QUIC connections aren't tied to a fixed IP/port pair, so they can survive a network change without a full reconnect — directly addressing the WiFi-to-cellular handoff problem.

Scenario 5: A dashboard needs to show 'somewhat fresh' data updated roughly every 30 seconds, and the team has limited time to build and operate new infrastructure.

How to approach it: Don't reach for WebSockets here. Short or long polling every 30 seconds is simpler to build on existing HTTP infrastructure, needs no persistent connection management, and easily meets a 30-second freshness bar — save WebSockets for genuinely sub-second, bidirectional needs.

Scenario 6: In a video conferencing app, users complain far more about audio lag than about occasional brief glitches.

How to approach it: This confirms UDP is the right choice for the media stream: users clearly prefer a small, quickly-forgotten glitch over the lag that TCP's retransmission-and-reorder guarantees would introduce. Reassure the team that occasional artifacts are an accepted, deliberate trade-off, not a bug to 'fix' by switching to TCP.

Chapter 3 — One-Page Revision Sheet

- TCP = reliable, ordered, connection-based (handshake + acks + retransmission). Use when correctness matters (APIs, file transfer, payments).
- UDP = fast, connectionless, no delivery guarantee. Use when speed matters more than perfection (video calls, live gaming, DNS).
- HTTP/1.1: one request per connection at a time. HTTP/2: multiplexing over one connection. HTTP/3: runs on QUIC/UDP, removes head-of-line blocking.
- WebSockets: persistent, full-duplex connection — true real-time push from server. Costs more server resources per connection than stateless HTTP.
- Polling < Long-polling < WebSockets in real-time-ness, but also in operational complexity. Choose the simplest one that meets the latency requirement.
- TLS handshake (encryption setup) is a separate, later step after the TCP handshake (connection setup) — don't conflate them.
- A single system can mix protocols: e.g., UDP for live position data, TCP for critical transactional events, in the same application.

Memory hook: TCP = registered letter (guaranteed, slower). UDP = shouting across a room (fast, no guarantee). Pick based on what you can afford to lose: time, or data.

CHAPTER 4

APIs & Communication Styles

REST, GraphQL, gRPC, and the difference between sync and async communication

1. Simple Explanation First

An API (Application Programming Interface) is a contract that defines how two pieces of software talk to each other — what you can ask for, and what shape the answer comes back in. REST, GraphQL, gRPC, and SOAP are different styles of writing that contract, each with different trade-offs. Separately, synchronous vs asynchronous describes whether the caller waits for an immediate answer or moves on and gets notified later.

REST vs GraphQL vs gRPC

REST — Fixed Menu



Multiple fixed endpoints -> multiple round trips

GraphQL — Build Your Own Bowl



One query, one endpoint -> exact shape, no over/under-fetch

gRPC — Drive-Through Code



Figure 4.1 — REST's fixed endpoints vs GraphQL's single flexible query vs gRPC's fast binary calls.

Analogy: Think of ordering at different types of restaurants. REST is like a fixed menu — you order combo #3 and get exactly what's listed, even if you only wanted half of it. GraphQL is like a build-your-own-bowl restaurant — you specify exactly which ingredients you want, nothing more, nothing less. gRPC is like a drive-through with a dedicated, pre-agreed ordering code between regular customers and staff — extremely fast, but you both need to know the code in advance.

2. Why This Topic Exists

As systems split into many independent services (Chapter 18 covers this fully), those services need standardized, predictable ways to exchange data. Different scenarios demand different trade-offs: a public API consumed by thousands of third-party developers needs simplicity and wide compatibility (REST); a mobile app fetching deeply nested data needs flexibility to avoid over-fetching (GraphQL); two internal microservices exchanging millions of messages per second need raw speed (gRPC).

Why this matters: Picking an API style is rarely 'right or wrong' — it's a trade-off interviewers expect you to articulate. Defaulting to REST without explaining why is a missed opportunity to show depth.

3. Core Concepts in Depth

3.1 REST (Representational State Transfer)

REST is an architectural style built on standard HTTP methods (GET, POST, PUT, DELETE) acting on 'resources' identified by URLs (e.g., /users/123). It's stateless, widely understood, cacheable by default via HTTP caching mechanisms, and works with virtually every client and tool.

HTTP Method	Typical Meaning
GET	Retrieve a resource (safe, no side effects, cacheable)
POST	Create a new resource
PUT	Replace/update an existing resource entirely
PATCH	Partially update an existing resource
DELETE	Remove a resource

Common mistake: Over-fetching and under-fetching with REST: a mobile client might need only a user's name and photo, but a REST endpoint returns the entire user object (over-fetching), or it might need data from three different endpoints to render one screen (under-fetching, requiring multiple round trips). This exact pain point is what GraphQL was built to solve.

3.2 GraphQL — Asking for Exactly What You Need

GraphQL exposes a single endpoint where the client sends a query describing exactly which fields it wants, possibly across multiple related resources, and the server returns exactly that shape — nothing more. This avoids both over-fetching and under-fetching, which is especially valuable for mobile apps on slow networks.

- Single endpoint (typically /graphql) instead of many REST URLs.
- Client defines the response shape via the query itself.
- Harder to cache at the HTTP layer (since it's usually a single POST endpoint), requiring more deliberate caching strategy on the server or client.
- More complex to implement on the server side — you need a schema and resolvers for every field.

3.3 gRPC — High-Performance Internal Communication

gRPC is a binary protocol built on HTTP/2, using Protocol Buffers (protobuf) to define strict, strongly-typed contracts between services. Because data is sent as compact binary instead of human-readable JSON/XML, it's significantly faster and uses less bandwidth — ideal for internal microservice-to-microservice communication where both sides are services you control.

REST/JSON	gRPC/Protobuf
Human-readable text format (JSON)	Compact binary format — smaller payloads
Loosely typed, flexible to consume from any language/tool	Strongly typed via a shared .proto contract
Easy to debug with a browser or curl	Needs special tooling to inspect traffic
Great for public/external APIs	Great for internal service-to-service calls needing speed

3.4 SOAP (Brief, Mostly Legacy Context)

SOAP (Simple Object Access Protocol) is an older, XML-based protocol with strict standards for security and transactions, still common in banking, healthcare, and government systems. It's verbose and rigid compared to REST, but its built-in standards for things like formal contracts (WSDL) and enterprise-grade security made it a fit for industries with heavy compliance requirements. You mainly need to recognize it exists and why legacy systems still use it — new systems rarely choose SOAP today.

3.5 Synchronous vs Asynchronous Communication

Synchronous communication means the caller sends a request and blocks/waits until it gets a response before continuing — like a phone call. Asynchronous communication means the caller fires off a request (often via a message queue) and continues with other work, getting notified or checking back later for the result — like sending a text message.

Synchronous vs Asynchronous Communication

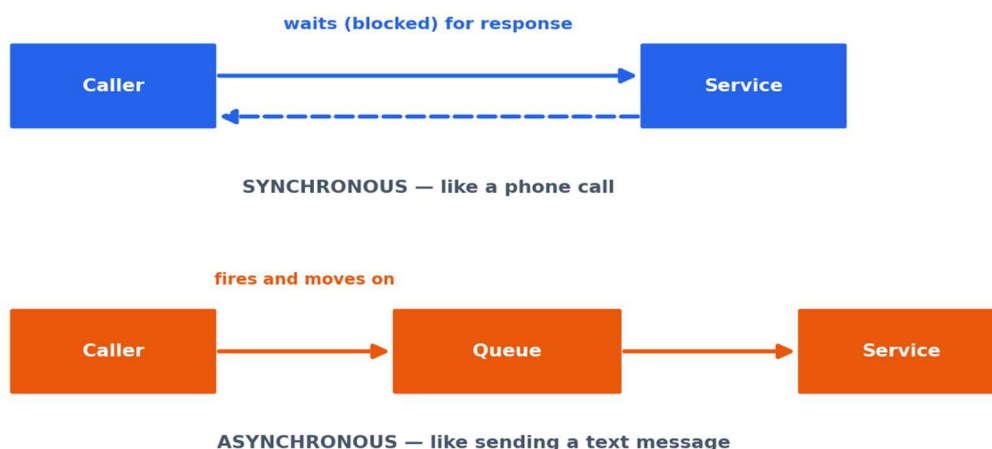


Figure 4.2 — Synchronous calls block and wait; asynchronous calls hand off and move on.

Synchronous	Asynchronous
Caller waits for an immediate response	Caller continues without waiting
Simpler to reason about, but couples services together in time	Decouples services — one being slow/down doesn't block the other
Good for: user-facing requests needing an immediate answer (e.g., login)	Good for: background jobs, notifications, processing pipelines (e.g., sending an email after signup)
Failure cascades quickly if the downstream service is slow	Failures can be retried independently without blocking the caller

Why this matters: This sync/async distinction is the conceptual seed of Chapter 14 (Message Queues). Recognizing when a step in your design should NOT block the user is one of the highest-leverage instincts you can develop for system design interviews.

3.6 Webhooks

A webhook flips the normal client-server direction: instead of you repeatedly asking a service 'did anything happen yet?' (polling), you register a URL with that service, and it calls YOUR server automatically when an event occurs. Common in payment systems (Stripe notifying your server when a payment succeeds) and GitHub (notifying your CI system when code is pushed).

4. Trade-offs Table

API Style	Best Fit	Watch Out For
REST	Public APIs, simple CRUD apps, broad compatibility	Over/under-fetching for complex nested data needs
GraphQL	Mobile apps, complex/nested data, many client types with different needs	Harder caching, more server-side complexity, risk of very expensive queries
gRPC	Internal microservice-to-microservice, performance-critical paths	Not browser-friendly by default, steeper learning curve, less human-readable

5. How Real Companies Use This

- Facebook/Meta invented GraphQL specifically to solve over-fetching problems on the mobile News Feed, where different screens needed wildly different subsets of data.
- Google created gRPC and uses it extensively for internal service-to-service communication across its massive microservices fleet.
- Stripe relies heavily on webhooks so merchant servers get notified instantly when a payment's status changes, instead of polling Stripe's API repeatedly.

6. Common Mistakes & Misconceptions

- Believing GraphQL replaces REST entirely — many companies use both: GraphQL for client-facing flexibility, REST or gRPC internally.

- Assuming async always means 'better' — async adds complexity (you need to track job status, handle retries, possibly notify the user later). Don't reach for it if a request truly needs an immediate synchronous answer (e.g., checking login credentials).
- Forgetting that gRPC isn't naturally browser-compatible without a proxy layer (like gRPC-Web) — it's primarily for server-to-server communication.

7. Interviewer Q&A Bank

Q1. When would you choose GraphQL over REST?

A: When clients have very different data needs (e.g., a mobile app vs a web dashboard) and a fixed REST response shape would cause significant over-fetching or require many round trips to assemble one screen's worth of data. GraphQL lets each client request exactly the fields it needs in a single call.

Q2. Why would two internal microservices use gRPC instead of REST?

A: Because gRPC's binary protobuf format is more compact and faster to serialize/deserialize than JSON, and HTTP/2 multiplexing reduces connection overhead — both matter a lot when services call each other millions of times per second internally. The strongly-typed .proto contract also reduces integration bugs between teams.

Q3. How would you decide whether a step in your system should be synchronous or asynchronous?

A: I'd ask whether the user needs an immediate result to proceed. If yes (e.g., 'is my login correct?'), it should be synchronous. If the work can happen in the background without blocking the user's experience (e.g., 'send a welcome email,' 'generate a video thumbnail'), I'd make it asynchronous via a queue, which also improves resilience since a slow downstream service won't block the main request path.

Q4. What's a downside of GraphQL that REST doesn't have?

A: GraphQL's flexibility means a client can construct a deeply nested, expensive query that's hard to predict and cache, potentially overloading the server — REST's fixed endpoints are easier to rate-limit and cache predictably. GraphQL servers often need query complexity limits or depth limiting to guard against this.

Q5. What is a webhook, and when would you use one?

A: A webhook is a way for a service to push event notifications to your server by calling a URL you registered, instead of you repeatedly polling for changes. I'd use it when I need near-real-time notification of an event from a third-party system (like a payment confirmation) without the cost and latency of constant polling.

8. HLD Mini-Scenario: Choosing an API Style

Prompt: "You're designing the backend for a food delivery app. The mobile app needs restaurant details, menu items, and reviews on one screen. Internally, the order service needs to talk to the payment service and the delivery-tracking service. How would you choose API styles?"

- 1. Client-facing API (mobile app to backend): GraphQL is a strong fit — the mobile screen needs nested, related data (restaurant + menu + reviews) in one call, avoiding multiple round trips on a potentially slow mobile network.

- 2. Order service to payment service (internal, synchronous need): gRPC is a good fit for speed and type-safety, since 'charge the customer' needs a fast, reliable, immediate response before confirming the order.
- 3. Order service notifying the delivery-tracking service: this can be asynchronous via a message queue, since tracking setup doesn't need to block the customer's order confirmation screen.

This shows the senior-level instinct: don't pick one API style for the whole system — pick per-edge based on who's talking to whom and what they need.

9. Questions YOU Should Ask the Interviewer

- Are we designing the public/client-facing API, the internal service-to-service APIs, or both?
- Do different clients (web, mobile, third-party) have meaningfully different data needs?
- Are there steps in this flow that don't need to block the user's response — can any of this be made asynchronous?

10. Scenario-Based Practice Bank (New)

Scenario 1: The mobile team complains that rendering one profile screen currently requires 5 separate REST calls (user info, posts, followers, settings, notifications).

How to approach it: This is a textbook GraphQL fit: expose a single query that asks for exactly those five related pieces of data in one round trip, cutting five requests (and five round-trip latencies on a mobile network) down to one.

Scenario 2: Two internal microservices call each other millions of times per second, and profiling shows JSON serialization/deserialization is a measurable CPU cost.

How to approach it: Migrate that internal edge to gRPC: the binary protobuf format is cheaper to (de)serialize than JSON, and HTTP/2 multiplexing cuts connection overhead — both directly address a JSON-serialization bottleneck at this call volume.

Scenario 3: You're building a public API platform for thousands of third-party developers who need to debug requests easily with tools like curl or Postman.

How to approach it: REST is the right default here: it's human-readable, cacheable via standard HTTP mechanisms, and universally supported by every language and tool, which matters far more for a broad, external developer audience than gRPC's raw speed.

Scenario 4: Your server currently polls a payment processor's API every 5 seconds to check if a charge succeeded, adding both latency and unnecessary load.

How to approach it: Switch to a webhook: register a URL with the payment processor so it calls your server the instant the charge status changes, eliminating both the polling delay and the wasted repeated requests.

Scenario 5: A 'send a welcome email after signup' step is currently implemented as a synchronous call inside the signup request, and signups feel slow whenever the email provider is slightly delayed.

How to approach it: Move the email send to an asynchronous background job via a message queue. The user doesn't need the email to arrive before their signup confirms, so this step shouldn't be allowed to block or slow down the main synchronous request path.

Scenario 6: A team wants to expose their internal GraphQL schema directly to the public internet without any additional safeguards.

How to approach it: *Flag the risk: GraphQL's flexibility lets a client construct a deeply nested, expensive query that could overload the server. Recommend adding query depth/complexity limiting and rate limiting before exposing it publicly — the same flexibility that makes GraphQL powerful also makes it easy to abuse without guardrails.*

Chapter 4 — One-Page Revision Sheet

- REST: resource-based, standard HTTP verbs, simple & cacheable, but prone to over/under-fetching.
- GraphQL: client specifies exact data shape needed, solves over/under-fetching, but harder to cache and can allow expensive queries.
- gRPC: binary protobuf over HTTP/2, fast & strongly typed, best for internal service-to-service calls, not browser-native.
- SOAP: legacy, XML-based, strict standards — mostly seen in banking/government/healthcare systems today.
- Synchronous = caller waits for the answer (use when an immediate result is required). Asynchronous = caller moves on, gets notified later (use for background work, decoupling, resilience).
- Webhooks flip the direction: the other service calls YOU when an event happens, avoiding wasteful polling.
- Real systems mix API styles: GraphQL/REST for client-facing flexibility, gRPC for fast internal calls, queues for async background work.

Memory hook: *REST = fixed menu. GraphQL = build-your-own-bowl. gRPC = drive-through code only regulars know. Sync = phone call. Async = text message.*

CHAPTER 5

The Core Trade-off: CAP Theorem & PACELC

The single idea that quietly drives 80% of every system design decision you'll make

1. Simple Explanation First

CAP theorem says that in a distributed system (data spread across multiple machines), when a network failure happens between those machines (a 'partition'), you are forced to choose between two things: keep answering requests even if the answer might be slightly outdated (Availability), or refuse to answer until you're sure the answer is correct and up to date (Consistency). You cannot have perfect versions of both at the same time during that failure.

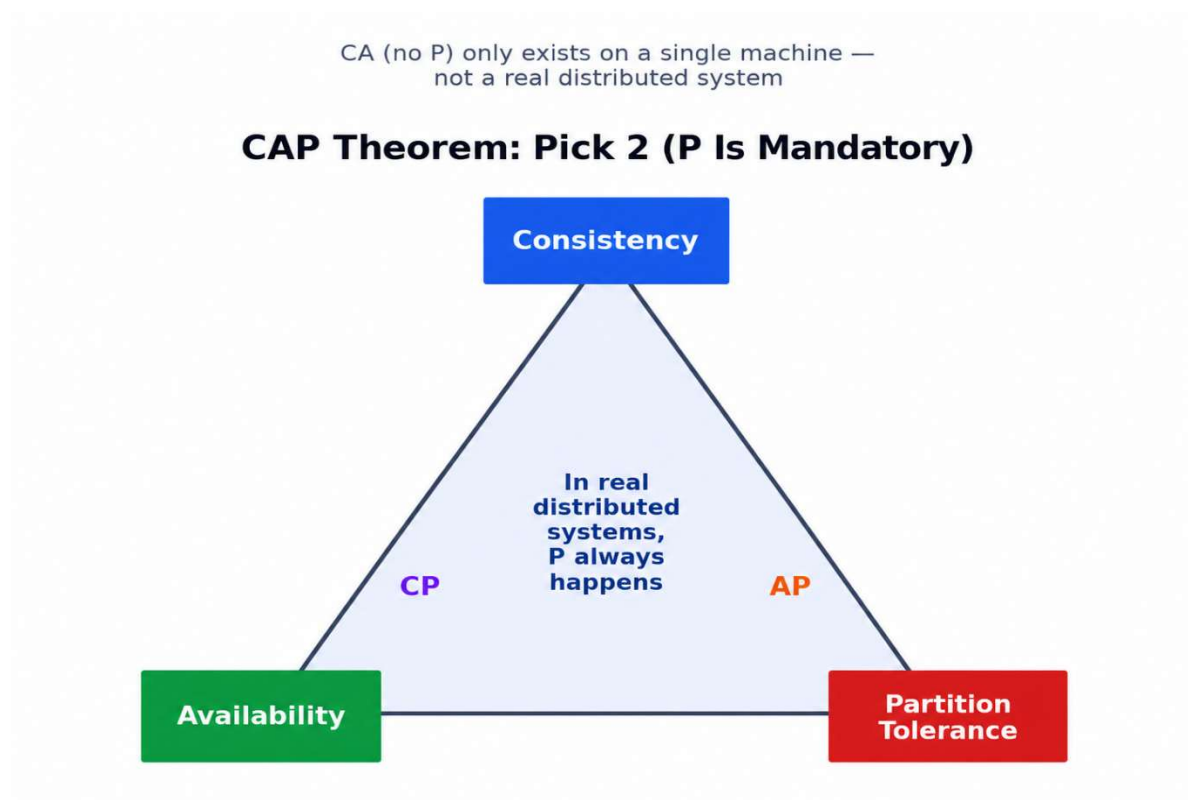


Figure 5.1 — Partition tolerance is mandatory for any real distributed system, so the real choice is CP vs AP.

Analogy: Imagine two bank branches in different cities, connected by a phone line that suddenly goes dead. A customer walks into Branch A and asks for their balance. Branch A has its last known balance, but can't confirm with Branch B whether a withdrawal just happened there. Branch A has two choices: refuse to answer until the phone line is fixed (Consistency — but the customer is stuck waiting), or answer immediately using its last known data (Availability — but the answer might be stale if Branch B already processed a withdrawal). CAP theorem is just this dilemma, formalized.

2. Why This Topic Exists

Single-machine databases never face this dilemma — there's only one copy of the data, no network between copies, no partition possible. But the moment you scale beyond one machine (which every large system eventually must), you introduce a network between your data copies, and networks fail. CAP theorem isn't a design choice you can avoid — it's a mathematical reality you must consciously navigate once your system is distributed.

Why this matters: CAP theorem is one of the most frequently invoked concepts in system design interviews, because almost every 'should we use Database X or Y' or 'should this be eventually consistent' discussion ultimately traces back to it. Interviewers use it to test whether you understand WHY certain databases behave the way they do, not just which ones to use.

3. Core Concepts in Depth

3.1 Breaking Down the Three Letters

Property	Meaning
Consistency (C)	Every read receives the most recent write, or an error. All nodes show the exact same data at the exact same time.
Availability (A)	Every request receives a (non-error) response, without guarantee that it contains the most recent write. The system always 'answers,' even if the answer is stale.
Partition Tolerance (P)	The system continues to operate despite network failures (dropped/delayed messages) between nodes.

Common mistake: Saying 'I'll just pick CA and skip P.' In any real distributed system spanning more than one machine, network partitions WILL eventually happen — it's not optional. So in practice, you must always choose P, and the real decision is C vs A when a partition occurs. CA-only systems essentially only exist as single-node systems, which defeats the purpose of distributing in the first place.

3.2 The Real Choice: CP vs AP

Since Partition Tolerance is non-negotiable for any real distributed system, CAP theorem in practice becomes a choice between CP (Consistency + Partition Tolerance, sacrificing Availability during a partition) and AP (Availability + Partition Tolerance, sacrificing strict Consistency during a partition).

CP Systems (favor Consistency)	AP Systems (favor Availability)
During a partition, some nodes may refuse requests rather than risk returning stale data	During a partition, all nodes keep answering, possibly with outdated data
Example databases: traditional relational databases in strict mode, MongoDB (configurable), HBase, Zookeeper	Example databases: Cassandra, DynamoDB, CouchDB (configurable per use case)
Good for: banking balances, inventory counts, anything where wrong data is worse than no data	Good for: social media likes/views, product recommendations, anything where slightly stale data is acceptable but downtime is not

3.3 Eventual Consistency — The Most Common AP Pattern

'Eventually consistent' means that if no new writes occur, all nodes WILL eventually converge to the same value — but there's a window of time where different nodes might disagree. This isn't a flaw; it's an explicit, accepted trade-off for higher availability and lower latency.

- Example: when you post on Instagram, your followers in different regions might see the post at slightly different times because it hasn't fully replicated everywhere yet — but it WILL show up for everyone shortly.
- This trade-off is acceptable here because a few seconds of delay seeing a post is harmless, but the app being completely down (the alternative under strict consistency) would be far worse for user experience.

3.4 PACELC — The More Complete Picture

CAP theorem only describes behavior DURING a network partition. But what about normal operation, when there's no partition at all? PACELC extends CAP to cover this: 'if Partitioned, choose Availability or Consistency; Else (normal operation), choose Latency or Consistency.'

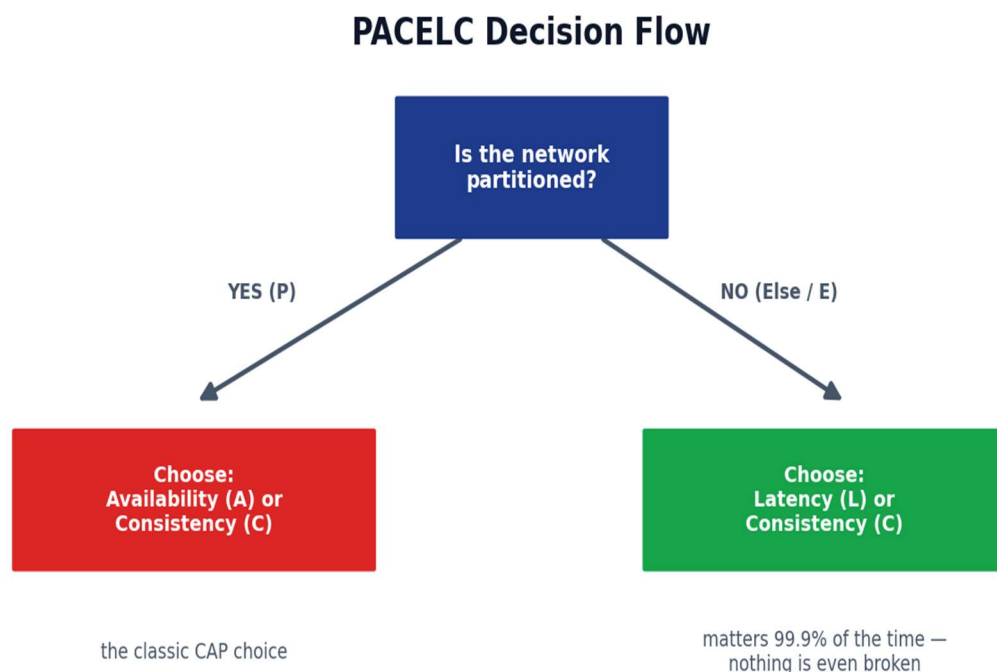


Figure 5.2 — PACELC covers both the failure case (P) and everyday operation (Else).

Scenario	Trade-off
P — during a Partition	Choose Availability (A) or Consistency (C) — this is the original CAP choice
E — Else, normal operation (no partition)	Choose Latency (L) or Consistency (C) — even with a healthy network, waiting to confirm data is consistent across replicas adds latency

In other words, even when everything is working fine, a system that always waits for all replicas to confirm a write (strong consistency) will be slower than one that responds as soon as one replica confirms and syncs the rest in the background (lower latency, weaker consistency). This is why PACELC is considered a more complete and practically useful framework than CAP alone — it explains design trade-offs even in the 99.9% of the time when nothing is broken.

Why this matters: Mentioning PACELC after CAP theorem in an interview is a strong signal of depth — it shows you understand that consistency trade-offs aren't just about disaster scenarios, they're a constant, everyday cost-benefit decision.

3.5 Strong vs Eventual vs Causal Consistency (Quick Map)

Consistency isn't binary — it's a spectrum. Knowing a few named points on this spectrum helps you speak precisely.

Consistency Model	What It Guarantees	Example Use
Strong Consistency	Every read sees the latest write, immediately, everywhere	Bank account balance, inventory stock count
Eventual Consistency	All replicas converge eventually, but reads may be temporarily stale	Social media post counts, product reviews
Causal Consistency	Operations that are causally related are seen in the same order by everyone; unrelated ones may differ	Comment threads — replies always appear after the comment they reply to

4. Trade-offs Table

Choice	Pros	Cons
CP (favor Consistency)	Data is always correct and up to date when read	System may become unavailable/reject requests during a network partition
AP (favor Availability)	System always responds, even during failures — better uptime	Risk of reading stale/conflicting data temporarily
Strong consistency always (ignore PACELC's Else case)	Simplest mental model, no stale reads ever	Higher latency even during normal, healthy operation

5. How Real Companies Use This

- Amazon's DynamoDB was explicitly designed AP-first, born from Amazon's realization that a shopping cart being briefly 'wrong' was far less costly than the cart being completely unavailable during Black Friday traffic.

- Banking systems and Google Spanner prioritize strong consistency for account balances and transactions, accepting higher latency and more complex infrastructure (like atomic clocks in Spanner's case) to guarantee correctness.
- WhatsApp and most chat apps use eventual consistency for message delivery status (delivered/read receipts) since a few hundred milliseconds of staleness is invisible to users, but favor stronger guarantees for message ordering within a single conversation.

6. Common Mistakes & Misconceptions

- Saying 'NoSQL = AP, SQL = CP' as a blanket rule — many modern databases (MongoDB, Cassandra, even some SQL setups) are configurable and can be tuned toward either side depending on settings.
- Treating CAP as a permanent, system-wide label — in real architectures, DIFFERENT parts of the same system can make different choices (e.g., payment data is CP, view counts are AP).
- Forgetting that CAP theorem only applies DURING a partition — many candidates misapply it to describe normal-operation trade-offs, which is actually what PACELC is for.

7. Interviewer Q&A Bank

Q1. Explain CAP theorem in your own words.

A: In a distributed system, when a network partition happens between nodes, you must choose between Consistency (only returning confirmed, up-to-date data, even if that means refusing some requests) and Availability (always responding, even if the data might be slightly stale). Partition tolerance itself isn't really optional for a real distributed system, so practically, CAP becomes a CP vs AP decision.

Q2. Would you choose a CP or AP database for an e-commerce shopping cart?

A: I'd lean AP. Showing a shopping cart that's occasionally a few seconds stale is a minor inconvenience, but the cart being completely unavailable during high-traffic events would directly cost sales. I'd reserve strong consistency for the actual payment and inventory-deduction step, where correctness genuinely matters more than uptime.

Q3. What is PACELC and how does it extend CAP theorem?

A: CAP only describes trade-offs during a network partition. PACELC extends this: if there IS a partition, choose Availability or Consistency (the CAP choice); Else, even during normal healthy operation, choose Latency or Consistency, since waiting for strong consistency guarantees adds latency even when nothing is broken. PACELC gives a more complete picture because it acknowledges this trade-off exists all the time, not just during failures.

Q4. Can a single system use both CP and AP behavior for different data?

A: Yes, and a strong design usually does. For example, in a ride-sharing app, payment processing might use a CP, strongly consistent approach since billing errors are costly, while live driver location updates can be AP/eventually consistent, since a one-second-stale location on a map is harmless.

Q5. What does 'eventual consistency' actually guarantee?

A: It guarantees that if no new writes occur to a piece of data, all replicas will eventually converge to the same value. It does NOT guarantee how long that convergence takes, and reads during that window may return different values from different replicas. It's a deliberate trade-off for lower latency and higher availability.

8. HLD Mini-Scenario: Applying CAP/PACELC to a Design

Prompt: "Design the backend data layer for a ticket-booking platform (e.g., movie tickets) where two people might try to book the last seat at the same time."

- 1. Identify the critical consistency point: seat availability/booking confirmation must be strongly consistent — if two users both see 'seat available' due to stale data, both could pay for the same seat, a real business and trust problem.
- 2. Choose CP for the seat-booking transaction itself: use a database/locking mechanism that guarantees only one booking succeeds for a given seat, even if it means brief unavailability under a partition.
- 3. Choose AP for less critical reads, like browsing showtimes or viewing seat maps before committing to book — staleness here is harmless and we'd rather the browsing experience stay fast and available.

This demonstrates the senior-level instinct from question 4 above: apply CAP/PACELC selectively, component by component, not as one blanket decision for the whole system.

9. Questions YOU Should Ask the Interviewer

- For this specific piece of data, is it worse to show stale information, or to show an error/unavailable state?
- Are there parts of this system where strict correctness matters much more than others (e.g., payments vs view counts)?
- How tolerant are users of a few seconds of delay before seeing updated data in this feature?

10. Scenario-Based Practice Bank (New)

Scenario 1: An e-commerce flash sale causes a network partition between database replicas right as thousands of shoppers try to buy the last few units of a hot item.

***How to approach it:** Treat inventory decrement/stock-count-to-zero as the CP-critical path — use a strongly consistent mechanism (locking or a single source of truth) so two shoppers can't both successfully buy the last unit. Meanwhile, keep product browsing and search AP so the rest of the storefront stays available even while that narrow path is more cautious.*

Scenario 2: A social media 'like' counter shows slightly different numbers to different users for a few seconds after a partition.

***How to approach it:** This is fine — likes/view counts are a textbook AP case. A briefly inconsistent count is invisible to the user experience and far less costly than the feature being unavailable; let it converge via eventual consistency.*

Scenario 3: A bank's transfer system experiences a network partition between its two data centers mid-transaction.

***How to approach it:** This must be CP: refuse or hold the transfer rather than risk applying it inconsistently (e.g., debiting one account without confirming the matching credit). Showing a temporary 'try again' error is far preferable to a silently wrong balance.*

Scenario 4: There's no partition at all — the network is completely healthy — but writes still feel slow because every write waits for all regional replicas to confirm.

How to approach it: This is a PACELC 'Else' trade-off, not a CAP one. Consider whether every write genuinely needs to wait for all replicas (strong consistency, higher latency) or whether some paths can acknowledge after one replica confirms and replicate the rest asynchronously (lower latency, weaker consistency) — a decision that applies even when nothing is broken.

Scenario 5: An interviewer asks you to justify choosing MongoDB versus Cassandra for a given feature, expecting more than 'MongoDB is SQL-like, Cassandra is AP.'

How to approach it: Point out that both are configurable rather than fixed to one side of CAP — the real answer depends on the specific consistency and availability requirements of that feature, not a blanket label on the database technology itself.

Scenario 6: A comments feature needs replies to always appear after the comment they're responding to, even though the app is generally fine with some staleness elsewhere.

How to approach it: This calls for causal consistency rather than full strong consistency or plain eventual consistency: causally related writes (a comment and its reply) must be seen in the correct order, while unrelated comments elsewhere in the app can still tolerate being briefly out of sync.

Chapter 5 — One-Page Revision Sheet

- CAP theorem: during a network partition, you must choose Consistency (correct but maybe unavailable) or Availability (always responds, maybe stale) — Partition tolerance is not optional in real distributed systems.
- In practice, CAP becomes a CP vs AP decision, since P must always be assumed for any multi-node system.
- PACELC extends CAP: if Partitioned -> choose Availability or Consistency; Else (normal operation) -> choose Latency or Consistency. This covers the 99.9% of time when nothing is broken.
- Consistency spectrum: Strong (always latest data) > Causal (related events ordered correctly) > Eventual (converges eventually, may be briefly stale).
- CP fits: banking balances, inventory/seat booking, anything where wrong data is worse than no data.
- AP fits: social counters, browsing/view data, anything where staleness is harmless but downtime is costly.
- Different components of the SAME system can make different CAP choices — this is normal and expected in real architectures, not a contradiction.

Memory hook: Two bank branches, a dead phone line: answer now and risk being wrong (Availability), or stay silent until you're sure (Consistency). You can't do both when the line is down.

CHAPTER 6

Scalability Fundamentals

Vertical vs horizontal scaling, statelessness, and the back-of-envelope math interviewers love

1. Simple Explanation First

Scalability is a system's ability to handle growing amounts of work by adding resources. There are two ways to add resources: make one machine bigger and more powerful (vertical scaling), or add more machines working together (horizontal scaling). Most large-scale systems eventually favor horizontal scaling, but understanding why requires understanding the limits of vertical scaling first.

Analogy: Vertical scaling is like replacing a small restaurant kitchen with a bigger, more powerful one — more stoves, faster ovens, a bigger fridge, all in the same building. Horizontal scaling is like opening multiple restaurant branches across the city instead. A bigger kitchen eventually hits a physical ceiling (only so much equipment fits in one building), but you can keep opening new branches almost indefinitely — at the cost of needing a system to coordinate between them (which branch should this order go to?).

2. Why This Topic Exists

Every system starts small and simple — one server, one database, handling modest traffic. But user growth, viral moments, or business expansion can multiply traffic by 10x, 100x, or 1000x. A system designed without scalability in mind hits a wall: the single server runs out of CPU, memory, or disk I/O, and the entire application slows down or crashes. Scalability planning exists to make growth a smooth, predictable, and survivable event rather than a five-alarm emergency.

Why this matters: Almost every system design interview eventually asks some version of 'how would this handle 10x the traffic?' Your answer reveals whether you actually understand scaling, or were just describing a fixed architecture. This chapter gives you the vocabulary to answer that question convincingly, every time.

3. Core Concepts in Depth

3.1 Vertical Scaling (Scaling Up)

Vertical scaling means adding more power to an existing machine — more CPU cores, more RAM, faster disks (SSD vs HDD). It's the simplest scaling strategy because your application code usually doesn't need to change at all — the same single server just becomes more capable.

- Pros: simplicity (no architecture changes needed), no distributed-systems complexity (no need to worry about data consistency across multiple machines).
- Cons: there's a hard physical ceiling — even the most powerful single machine available has a maximum CPU/RAM/disk capacity. It also creates a single point of failure — if that one (very expensive) machine goes down, everything goes down.

3.2 Horizontal Scaling (Scaling Out)

Horizontal scaling means adding more machines and distributing the workload across them, rather than making one machine bigger. This requires a load balancer (Chapter 13) to distribute incoming requests, and it requires your application to be designed so any machine can handle any request.

- Pros: virtually unlimited growth potential (keep adding machines), better fault tolerance (if one machine dies, others keep serving traffic), and you can use cheaper, commodity hardware instead of one extremely expensive machine.
- Cons: added complexity — you now need to handle data consistency across machines, coordinate state, and manage a load balancer. This is where CAP theorem (Chapter 5) and database scaling techniques (Chapters 9-10) become unavoidable.

Vertical Scaling vs Horizontal Scaling

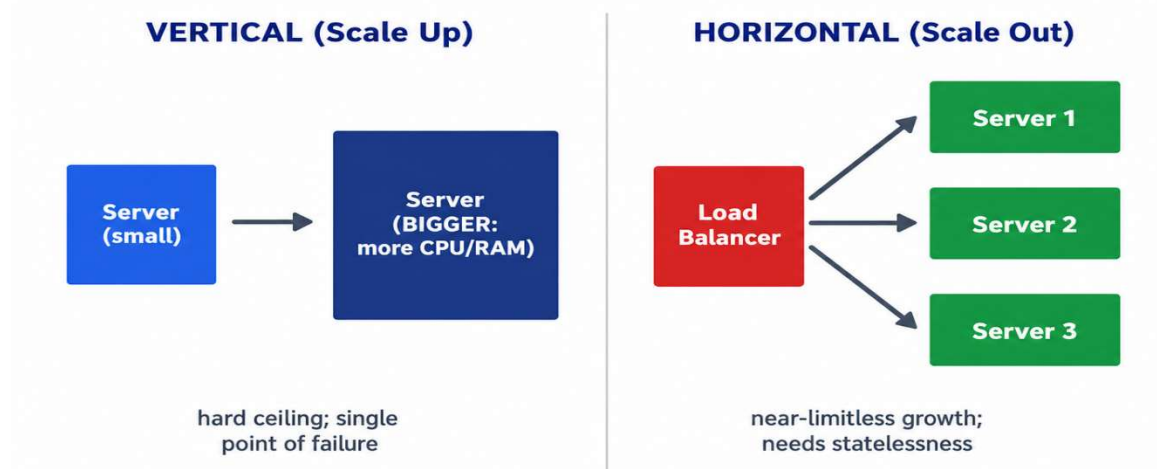


Figure 6.1 — One bigger machine hits a ceiling; many coordinated machines scale further.

Vertical Scaling	Horizontal Scaling
Add power to one machine	Add more machines
Simple — no architecture change needed	Requires load balancing & distributed coordination
Has a hard ceiling (max hardware specs)	Near-limitless growth by adding more nodes
Single point of failure	Naturally more fault-tolerant

Common mistake: Assuming horizontal scaling is always the 'correct' answer. For small-to-medium systems, or early-stage startups, vertical scaling is often the pragmatic first move — it's simpler, cheaper to operate, and avoids premature distributed-systems complexity. Jumping straight to a complex horizontally-scaled architecture for a system with modest traffic can be a red flag of over-engineering in an interview.

3.3 Stateless vs Stateful Services

This concept (first introduced in Chapter 2) is the precondition for horizontal scaling to work smoothly. A stateless service doesn't store any client-specific data between requests — every request contains all the information needed to process it (or fetches it fresh from a shared database/cache). A stateful service remembers something about a specific client locally (e.g., a user's session stored only in that one server's memory).

If a service is stateful and tied to one specific server, you can't freely route a user's next request to a DIFFERENT server without losing that state — this is exactly why stateless design is preferred for anything you intend to scale horizontally. When state is genuinely needed, it should live in a shared, external store (a database, a distributed cache like Redis) rather than in one server's local memory.

Why this matters: This is one of the most important 'invisible' design decisions in system design. Choosing to keep your application servers stateless is often what makes horizontal scaling, load balancing, and fault tolerance possible in the first place — without it, the rest of this book's techniques don't work cleanly.

3.4 Back-of-the-Envelope Estimation — The Napkin Math

Interviewers frequently ask you to estimate scale — requests per second, storage needed, bandwidth required — to ground your architecture decisions in real numbers rather than vague intuition. You're not expected to be perfectly precise; you're expected to reason with reasonable assumptions and simple math, out loud.

3.4.1 Useful Numbers to Memorize

Quantity	Approximate Value
Seconds in a day	~86,400 (often rounded to 100,000 for quick math)
1 KB	1,000 bytes (approx, for estimation)
1 MB	1,000 KB / 1,000,000 bytes
1 GB	1,000 MB / 1 billion bytes
Average read:write ratio for many social apps	Often 100:1 or higher (far more reads than writes)

3.4.2 A Worked Example

Prompt: estimate storage needed per year for a service where 1 million users each post one 1KB message per day.

- 1. Messages per day = 1,000,000 users x 1 message = 1,000,000 messages/day.
- 2. Storage per day = 1,000,000 messages x 1 KB = 1,000,000 KB = ~1 GB/day.
- 3. Storage per year = 1 GB x 365 = ~365 GB/year.
- 4. Quick sanity check: this is a very manageable amount of storage for a single modern database, suggesting we don't need aggressive sharding purely for storage volume reasons at this scale — though we might still need it for read/write throughput reasons.

Why this matters: Notice the last step: the goal of back-of-envelope math isn't just to produce a number, it's to let that number *INFORM* a design decision (here: we probably don't need to shard yet, just for storage). Always close the loop — connect the math back to your architecture.

3.4.3 QPS (Queries Per Second) Estimation

QPS tells you how much traffic your servers need to handle concurrently — critical for deciding how many servers you need and whether you need caching/load balancing.

Back-of-Envelope: Estimating Peak QPS



Figure 6.2 — Turning daily active users into a peak QPS number you can design capacity for.

- 1. Example: 10 million daily active users, each making 5 requests/day on average.
- 2. Total requests/day = 10,000,000 x 5 = 50,000,000 requests/day.
- 3. Average QPS = 50,000,000 / 86,400 seconds ~ ~580 QPS.
- 4. Peak QPS is typically estimated as 2-3x the average (traffic isn't evenly spread across 24 hours) ~ ~1,200-1,800 QPS at peak — this peak number is what you actually design capacity for, not the average.

4. Trade-offs Table

Decision	Pros	Cons
Vertical scaling first (small/medium scale)	Simple, fast to implement, no distributed complexity	Hard ceiling; eventually must migrate to horizontal anyway
Horizontal scaling from day one	Scales further, more fault-tolerant	Higher upfront complexity; may be over-engineering for small systems
Keeping services stateless	Enables horizontal scaling & easy failover	Requires externalizing state (extra DB/cache calls), slightly more design effort

5. How Real Companies Use This

- Most startups begin with vertical scaling (a single larger database/server) and only invest in horizontal scaling and sharding once growth data justifies the added complexity — a deliberate, pragmatic sequencing.
- Netflix and Amazon design nearly all of their services to be stateless by default, storing session and user state in external, distributed data stores, specifically so any request can be served by any server in any data center.
- Companies running flash-sale or viral-traffic events (e.g., ticket sales, product drops) explicitly provision for peak QPS, not average QPS, often using auto-scaling horizontal infrastructure that adds servers automatically as load increases.

6. Common Mistakes & Misconceptions

- Treating back-of-envelope estimation as a precision math test — interviewers care about your reasoning process and reasonable assumptions, not getting an exact 'correct' number.
- Forgetting to account for peak traffic vs average traffic — designing only for the daily average can leave a system crippled during real-world traffic spikes.
- Assuming horizontal scaling 'just works' without addressing statelessness first — adding more servers behind a load balancer doesn't help if those servers can't interchangeably handle any request.

7. Interviewer Q&A Bank

Q1. What is the difference between vertical and horizontal scaling?

A: Vertical scaling means increasing the resources (CPU, RAM, disk) of a single existing machine. Horizontal scaling means adding more machines and distributing load across them. Vertical scaling is simpler but has a hard physical ceiling and creates a single point of failure; horizontal scaling has near-limitless growth potential and better fault tolerance, but requires load balancing and careful handling of distributed state.

Q2. Why does statelessness matter for scaling a service horizontally?

A: If a server stores client-specific state locally (e.g., session data in memory), then that client's future requests must always go back to that exact server, which breaks the flexibility a load balancer needs to distribute traffic freely. Making services stateless — storing any needed state in a shared external database or cache — means any server can handle any request, which is what makes horizontal scaling and failover actually work in practice.

Q3. How would you estimate the number of servers needed for a new service?

A: I'd start by estimating expected daily active users and requests per user per day to get total daily requests, then divide by seconds in a day to get average QPS. I'd then multiply by a peak factor (commonly 2-3x average) to estimate peak QPS, since systems need to be provisioned for peak load, not average load. From there, I'd estimate how many requests a single server can handle per second (based on expected response time and concurrency) to calculate the number of servers needed, while leaving headroom for failures and traffic spikes.

Q4. When would you choose vertical scaling over horizontal scaling?

A: For early-stage systems with modest, predictable traffic, vertical scaling is often the pragmatic choice — it avoids the operational complexity of distributed coordination, load balancing, and data consistency challenges that horizontal scaling introduces. I'd reach for horizontal scaling once traffic, fault-tolerance requirements, or growth projections genuinely demand it, rather than defaulting to it for its own sake.

Q5. What's the risk of designing a system only for average traffic instead of peak traffic?

A: Real-world traffic is rarely evenly distributed across the day — there are predictable peaks (e.g., evening hours) and unpredictable spikes (e.g., a viral post, a flash sale). If a system is only provisioned for the average load, it can become overwhelmed and degrade or crash exactly when it matters most: during high-visibility peak moments.

8. HLD Mini-Scenario: Scaling Decision in Action

Prompt: "Your photo-sharing app currently runs on a single server and database, handling 10,000 daily active users smoothly. The product team announces a marketing campaign expected to bring this to 5 million daily active users within 3 months. What do you change?"

- 1. Quick estimation: going from 10,000 to 5,000,000 DAUs is a 500x increase — this is well beyond what a single vertically-scaled server can absorb, no matter how powerful, especially combined with peak-traffic multipliers.
- 2. Make application servers stateless if they aren't already, so we can place multiple of them behind a load balancer (Chapter 13) and scale horizontally.
- 3. Plan for database scaling: introduce read replicas (Chapter 9) for the heavy read traffic typical of a photo-sharing feed, and evaluate whether sharding (Chapter 10) will be needed based on storage and write-throughput estimates.
- 4. Add a caching layer (Chapter 11) for frequently accessed data (popular photos, profile data) to reduce direct database load.

This mini-scenario is intentionally a preview — by the end of Part 2 and Part 3 of this book, every one of these steps will have its own full chapter with deep technical detail.

9. Questions YOU Should Ask the Interviewer

- What's the expected scale — current and projected — in terms of users and requests per second?
- Is traffic expected to be steady, or are there predictable peaks/spikes we should specifically design for?
- Are there budget or infrastructure constraints (e.g., on-prem vs cloud, cost sensitivity) that should influence whether we favor simplicity or aggressive horizontal scaling?

10. Scenario-Based Practice Bank (New)

Scenario 1: A pre-seed startup with a small engineering team has moderate, predictable traffic and wants to ship features quickly.

How to approach it: Recommend vertical scaling first: bump up a single server/database's specs as needed. It avoids the operational overhead of load balancers, distributed state, and multi-node consistency issues — complexity this team doesn't yet need to pay for.

Scenario 2: Marketing announces a campaign expected to bring 10x normal traffic in two weeks, and the current app stores user sessions in each server's local memory.

How to approach it: Before adding more servers, externalize session state into a shared store like Redis — otherwise a load balancer can't freely distribute requests, since a user's session would only exist on the one server they first hit. Only after statelessness is fixed does adding horizontal capacity actually help.

Scenario 3: You're migrating a legacy stateful application to a horizontally-scaled, load-balanced architecture.

How to approach it: Sequence the migration: first externalize all per-user state (sessions, in-progress uploads, etc.) to a shared database or cache, then introduce the load balancer and additional server instances. Doing it in the reverse order causes subtle bugs where a user's state 'disappears' whenever they're routed to a different server.

Scenario 4: Leadership asks how many servers to provision for a new feature launch, given 20 million daily active users each making 3 requests/day on average.

How to approach it: Walk through the QPS estimation: $20,000,000 \times 3 = 60,000,000$ requests/day; divide by $\sim 86,400$ seconds for an average of ~ 695 QPS; multiply by a 2-3x peak factor to get $\sim 1,400$ - $2,100$ QPS at peak, and provision server capacity for that peak number, not the average.

Scenario 5: A cost-sensitive client wants minimal ongoing infrastructure spend but occasionally sees short, unpredictable traffic spikes (e.g., a product mentioned on social media).

How to approach it: Suggest a hybrid: run a right-sized vertically-scaled baseline for normal traffic, paired with auto-scaling horizontal capacity (or serverless compute) that only spins up extra instances during a spike and scales back down afterward — avoiding the cost of permanently over-provisioning for rare peaks.

Scenario 6: A system provisioned for its average daily traffic keeps crashing every evening between 7-9pm when real usage peaks.

How to approach it: This is the classic average-vs-peak mistake: re-run the capacity estimate using peak QPS (2-3x average) instead of average QPS, and add auto-scaling so capacity grows automatically as evening traffic ramps up rather than staying fixed at an average-case size.

Chapter 6 — One-Page Revision Sheet

- Vertical scaling = bigger single machine. Simple, but has a hard ceiling and is a single point of failure.
- Horizontal scaling = more machines working together. Near-limitless growth, better fault tolerance, but needs load balancing & distributed coordination.
- Statelessness is the precondition for clean horizontal scaling — any server should be able to handle any request; externalize state into a shared DB/cache.
- Back-of-envelope math: estimate daily requests -> divide by 86,400 for average QPS -> multiply by 2-3x for peak QPS -> design capacity for the peak, not the average.
- Useful estimation rule: 1 KB ~ 1,000 bytes, 1 day ~ 86,400 seconds (or ~100,000 for quick rounding).
- Always connect your math back to a design decision (e.g., 'this storage volume means we don't need sharding yet') — the number alone isn't the point.
- Don't default to horizontal scaling for small systems — match the scaling strategy to actual, justified scale requirements.

Memory hook: *One bigger kitchen (vertical) hits a ceiling. Many branches (horizontal) scale forever — but only if any branch can serve any customer (statelessness).*

CHAPTER 7

Databases 101 — SQL vs NoSQL

ACID, BASE, and choosing the right kind of database for the job

1. Simple Explanation First

A database is just an organized place to store and retrieve data reliably. SQL (relational) databases store data in strict tables with rows and columns, where relationships between tables are explicitly defined. NoSQL databases store data in more flexible shapes — documents, key-value pairs, wide columns, or graphs — trading some structure and strictness for flexibility and scale.

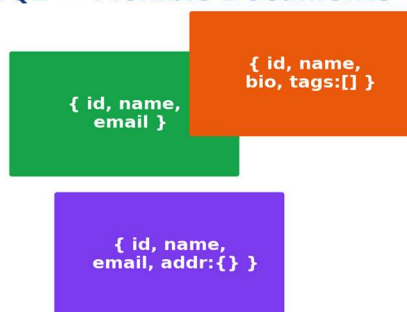
SQL Tables vs NoSQL Documents

SQL — Filing Cabinet

...	...	...
...	...	...
...	...	...
id	name	email

Every row: same columns, same shape

NoSQL — Flexible Documents



Each document can have a different shape

Figure 7.1 — SQL enforces one fixed shape per table; NoSQL lets each record take its own shape.

Analogy: A SQL database is like a perfectly organized filing cabinet, where every drawer (table) has the same labeled folders in the same order — predictable, but rigid if you suddenly need a different folder shape. A NoSQL database is like a pile of labeled boxes — each box (record) can hold whatever shape of stuff makes sense for it, which is flexible, but you lose the guarantee that every box looks the same.

2. Why This Topic Exists

For decades, relational (SQL) databases were the default choice for almost everything, because they offered strong guarantees about correctness through ACID transactions. But as internet companies grew to planet-scale (Amazon, Google, Facebook), the rigid structure and the difficulty of scaling SQL databases across many machines became a real bottleneck. NoSQL databases emerged specifically to trade some of SQL's strict guarantees for the ability to scale horizontally and handle less-structured data.

Why this matters: Almost every system design interview eventually asks 'would you use SQL or NoSQL here, and why?' This is rarely a trick question — it's testing whether you understand the actual trade-offs instead of reaching for whichever database is trendy.

3. Core Concepts in Depth

3.1 ACID Properties (SQL's Core Promise)

ACID is a set of guarantees that relational databases make about transactions (a transaction is a group of operations that should succeed or fail together, e.g., transferring money between two accounts).

Property	Meaning
Atomicity	A transaction either fully completes, or fully fails and rolls back — no partial updates. (E.g., money is deducted from one account AND added to another, or neither happens.)
Consistency	A transaction takes the database from one valid state to another valid state, never violating defined rules/constraints (e.g., a foreign key must reference an existing row).
Isolation	Concurrent transactions don't interfere with each other — it should look as if transactions ran one at a time, even if they actually overlapped.
Durability	Once a transaction is committed, it survives even a crash immediately afterward — the data is permanently saved.

Common mistake: Confusing 'Consistency' in ACID with 'Consistency' in CAP theorem (Chapter 5) — they are related in spirit but are formally different concepts. ACID consistency is about a single database obeying its own defined rules/constraints; CAP consistency is about multiple replicas/nodes agreeing on the same value. Conflating the two in an interview is a common, noticeable slip.

3.2 BASE Properties (NoSQL's Core Promise)

Many NoSQL databases relax ACID guarantees in favor of BASE, which prioritizes availability and scalability over strict immediate correctness.

Letter	Meaning
Basically Available	The system guarantees availability, in the CAP theorem sense — it will respond, even during failures.
Soft state	The state of the system may change over time, even without new input, as data propagates/syncs between replicas.
Eventual consistency	Given enough time without new writes, all replicas will converge to the same value (the same eventual consistency concept from Chapter 5).

Why this matters: ACID vs BASE is essentially the database-level expression of the CP vs AP decision from Chapter 5. SQL databases traditionally lean CP (favor correctness); many NoSQL databases lean AP (favor availability). Connecting this back to CAP theorem in an interview shows you see the bigger pattern, not just isolated facts.

3.3 Types of NoSQL Databases

'NoSQL' isn't one thing — it's an umbrella term for several very different data models, each suited to different access patterns.

Four Flavors of NoSQL



Pick the shape that matches your data's natural structure and access pattern.

Figure 7.2 — The four common NoSQL data models and where each is typically used.

Type	How Data Is Stored	Example & Best Use Case
Key-Value Store	Simple key maps to a value (often a blob); no structure enforced on the value	Redis, DynamoDB — caching, session storage, simple lookups by ID
Document Store	Each record is a flexible document (often JSON-like), can have nested fields	MongoDB, Couchbase — content management, user profiles, catalogs with varying fields
Column-Family (Wide-Column) Store	Data organized by column families, optimized for writing/reading huge volumes of sparse data	Cassandra, HBase — time-series data, IoT sensor data, large-scale logging
Graph Database	Data stored as nodes and edges, optimized for traversing relationships	Neo4j — social networks, recommendation engines, fraud detection

3.4 When to Choose SQL vs NoSQL

This decision should be driven by your data's shape, your consistency needs, and your query patterns — not by trend or familiarity.

Choose SQL When...	Choose NoSQL When...
Data is highly structured with clear relationships (e.g., orders belong to customers, which have addresses)	Data is unstructured or varies in shape between records (e.g., user-generated content, varying product attributes)
You need strong transactional guarantees (e.g., financial transactions, inventory counts)	You need to scale horizontally across many machines easily, or have huge write throughput
Complex queries with joins across multiple tables are common	Access patterns are simple (lookup by key) and known in advance
Examples: PostgreSQL, MySQL, Oracle, SQL Server	Examples: MongoDB, Cassandra, DynamoDB, Redis, Neo4j

4. Trade-offs Table

Database Type	Pros	Cons
SQL (Relational)	Strong consistency (ACID), powerful queries with joins, mature tooling	Harder to scale horizontally; rigid schema requires migrations for changes
NoSQL (Document/Key-Value)	Flexible schema, scales horizontally well, often faster for simple access patterns	Weaker consistency guarantees by default; complex relational queries are harder or impossible
NoSQL (Graph)	Extremely fast relationship traversal (friends-of-friends, recommendations)	Niche use case; not a general-purpose substitute for SQL or document stores

5. How Real Companies Use This

- Banks and payment processors (e.g., Visa, traditional banking cores) rely heavily on relational databases for transactional integrity — a wrong balance is unacceptable, so ACID guarantees matter enormously.
- Amazon's DynamoDB (NoSQL key-value/document store) powers the shopping cart and many high-traffic services, explicitly chosen for horizontal scalability and availability during massive traffic events like Black Friday.
- LinkedIn and Facebook use graph databases (or graph-like data structures) to power 'people you may know' and friend-of-friend recommendation features, since these are naturally relationship-traversal problems.
- Most large companies use BOTH — SQL for core transactional data (payments, orders), and various NoSQL stores for caching, logging, search, and high-velocity unstructured data, in the same overall system.

6. Common Mistakes & Misconceptions

- Believing 'NoSQL is always more scalable than SQL' as an absolute rule — modern SQL databases (with sharding, read replicas) can scale very far too; the real difference is in how naturally each model accommodates that scaling, not a hard scalability ceiling.

- Treating the choice as permanent and singular for an entire system — most real architectures mix SQL and NoSQL for different components based on each component's needs.
- Assuming NoSQL means 'no schema at all' — most NoSQL databases still benefit from a planned, consistent shape for your data, even if it's not strictly enforced by the database engine.

7. Interviewer Q&A Bank

Q1. What is the core difference between SQL and NoSQL databases?

A: SQL databases store data in structured tables with defined relationships and enforce strong consistency through ACID transactions. NoSQL databases store data in more flexible formats (documents, key-value pairs, wide columns, or graphs) and often relax strict consistency (following BASE principles instead) in exchange for easier horizontal scaling and flexible schemas.

Q2. Would you use SQL or NoSQL for an e-commerce order and payment system?

A: I'd use SQL for the core order and payment data, since this requires strong transactional guarantees — an order being placed and a payment being processed must be atomic and consistent, and the structured, relational nature of orders, customers, and line items maps naturally to relational tables. I might still use NoSQL elsewhere in the same system, like for product catalog search or session caching.

Q3. Explain ACID properties and why they matter.

A: ACID stands for Atomicity, Consistency, Isolation, and Durability. Atomicity ensures a transaction either fully completes or fully fails. Consistency ensures the database moves between valid states honoring its constraints. Isolation ensures concurrent transactions don't corrupt each other's results. Durability ensures committed data survives a crash. Together, they guarantee that a database stays correct even under concurrent access and failures — critical for systems like banking where partial or incorrect updates are unacceptable.

Q4. How does BASE relate to CAP theorem?

A: BASE — Basically Available, Soft state, Eventual consistency — is essentially how many NoSQL databases implement the AP side of the CP vs AP trade-off from CAP theorem. Instead of guaranteeing immediate strong consistency like ACID does, BASE-following systems prioritize staying available and let data converge to consistency over time.

Q5. When would you choose a graph database over a document or relational database?

A: When the core access pattern is about traversing relationships — like 'friends of friends' in a social network, or detecting fraud rings by following chains of connected accounts. Graph databases are purpose-built to make these traversal queries fast, whereas the same query in a relational database would require expensive multi-table joins that get slower as the relationship depth grows.

8. HLD Mini-Scenario: Choosing Databases for Different Components

Prompt: "Design the data layer for a ride-sharing app: it needs to store user profiles, ride transactions/payments, live driver locations, and a 'find nearby drivers' search feature."

- 1. User profiles & ride/payment transactions: SQL (e.g., PostgreSQL) — structured, relational data with strong consistency needs for billing correctness.
- 2. Live driver locations: a fast key-value store (e.g., Redis), since this data changes constantly, doesn't need long-term history, and benefits from extremely fast reads/writes.

- 3. 'Find nearby drivers' search: could use a specialized geospatial index (built into Redis or a dedicated search engine), since this is a specific access pattern that a general relational query wouldn't handle efficiently.

This demonstrates the core takeaway of this chapter: a single real system almost always uses multiple database types, each chosen for what that specific component actually needs.

9. Questions YOU Should Ask the Interviewer

- How structured is the data — do we know the shape in advance, or will it vary significantly between records?
- How critical is strong consistency for this specific data — would a brief staleness cause real problems?
- What are the dominant query/access patterns — simple lookups by key, complex joins, or relationship traversal?

10. Scenario-Based Practice Bank (New)

Scenario 1: A product team wants to store user-generated survey responses where every survey can have a completely different set of questions/fields.

***How to approach it:** This is a strong document-store fit (e.g., MongoDB) since each response's shape genuinely varies. Forcing this into rigid SQL tables would mean either a huge sparse table full of unused columns, or a painful schema migration every time a new survey type appears.*

Scenario 2: An interviewer asks you to design storage for a banking ledger, then adds: 'and we're expecting 50x growth in transaction volume next year.'

***How to approach it:** Don't abandon SQL just because of the growth number — banking ledgers need ACID guarantees that NoSQL typically can't match. Instead, discuss scaling the SQL database itself: read replicas (Chapter 9), sharding by account or region (Chapter 10), and aggressive indexing, while keeping the strong consistency the domain requires.*

Scenario 3: A social network wants to launch a 'people you may know' feature based on mutual friends and shared interests.

***How to approach it:** Recommend a graph database (or graph-like structure) here — this is precisely the relationship-traversal problem graph databases are optimized for. Doing the equivalent multi-hop friend-of-friend query in a relational database would require expensive, slow joins that get worse as the traversal depth grows.*

Scenario 4: A startup's engineer says 'let's use MongoDB for everything since it's schema-flexible and easy to start with,' including the core payments and order data.

***How to approach it:** Push back specifically on payments/orders: even though MongoDB can enforce structure at the application level, the core need here is strong transactional guarantees (atomicity across an order + payment), which is exactly what SQL's ACID model is built for. Recommend SQL for that slice, while MongoDB may still be fine for less consistency-sensitive parts of the app.*

Scenario 5: A logging/monitoring platform needs to ingest millions of write events per second from IoT devices, with queries mostly being 'give me device X's readings between time A and B.'

How to approach it: Recommend a wide-column/column-family store (e.g., Cassandra) — this write-heavy, time-series access pattern is exactly what wide-column stores are optimized for, unlike a traditional relational database that would struggle under that write volume with standard B-Tree indexing (Chapter 8).

Scenario 6: An interviewer asks: 'Isn't NoSQL just objectively more scalable, so why would anyone still choose SQL today?'

How to approach it: Correct the premise gently: modern SQL databases scale very far too, using replication and sharding — the real distinction isn't a hard scalability ceiling, it's which model naturally fits the data's structure and consistency needs. Reiterate that most large systems use both, chosen per component.

Chapter 7 — One-Page Revision Sheet

- SQL = structured tables, defined relationships, ACID guarantees (Atomicity, Consistency, Isolation, Durability).
- NoSQL = flexible data shapes, often follows BASE (Basically Available, Soft state, Eventual consistency) instead of strict ACID.
- ACID vs CAP 'Consistency' are different concepts — don't conflate them. ACID = single DB obeying its own rules. CAP = multiple nodes agreeing on a value.
- NoSQL types: Key-Value (Redis, DynamoDB), Document (MongoDB), Wide-Column (Cassandra), Graph (Neo4j) — each fits a different access pattern.
- Choose SQL for: structured relational data, strong transactional needs, complex joins. Choose NoSQL for: flexible/unstructured data, horizontal scale, simple known access patterns.
- Real systems almost always mix SQL and NoSQL across different components — this is normal, expected architecture, not an inconsistency.
- BASE is essentially the AP side of CAP theorem, applied at the database level.

Memory hook: SQL = a filing cabinet with identical labeled drawers. NoSQL = a pile of boxes, each shaped for what's inside. Pick the cabinet when structure matters; pick the boxes when flexibility and scale matter more.

CHAPTER 8

Database Indexing

Why some queries are instant and others crawl — B-Trees, composite indexes, and trade-offs

1. Simple Explanation First

An index is a separate, pre-organized data structure that lets a database find rows quickly without scanning every single row in a table. Without an index, finding one row in a million-row table means checking every row one by one (a 'full table scan'). With an index, the database can jump almost directly to the right row, similar to how a book's index lets you jump to a page instead of reading the whole book to find a topic.

Index Lookup (B-Tree) vs Full Table Scan

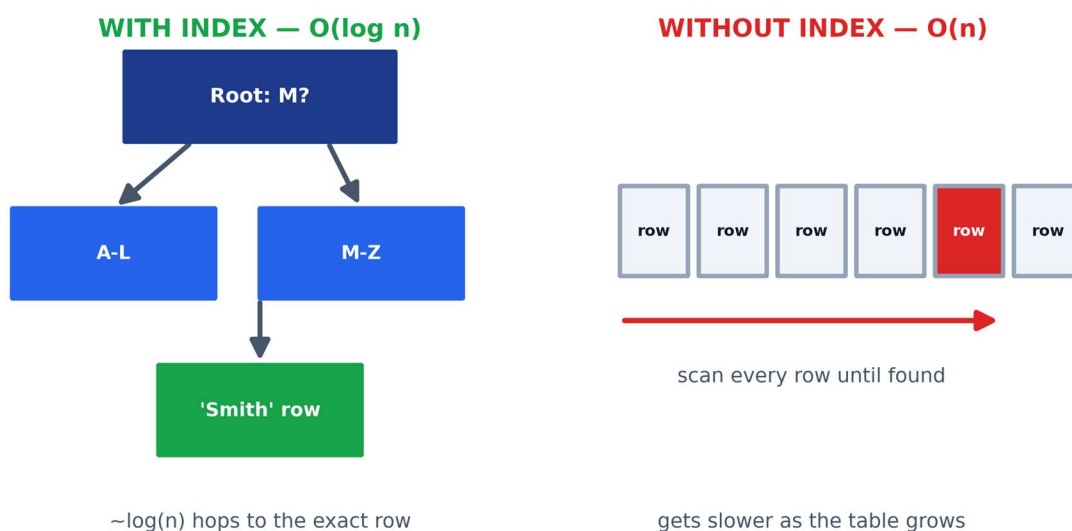


Figure 8.1 — An index narrows the search in $O(\log n)$ hops; without one, the database checks every row.

Analogy: Imagine a phone book with a million names, sorted alphabetically. If you want to find 'Smith, John,' you don't read every name from the start — you flip to the S section directly. A database index is exactly this: a sorted, organized shortcut that lets the database 'flip directly' to the relevant data instead of reading everything.

2. Why This Topic Exists

As tables grow from thousands to millions or billions of rows, a full table scan for every query becomes catastrophically slow — a query that took 1 millisecond on a 1,000-row table might take many seconds on a 100-million-row table without an index. Indexing exists purely to keep query performance fast as data grows, by trading some extra storage and slower writes for dramatically faster reads.

Why this matters: Indexing is one of the most practical, high-leverage topics in real-world database work — and a very common 'why is this slow?' debugging question interviewers ask to test if you understand WHY a query is slow, not just that adding an index 'fixes' it.

3. Core Concepts in Depth

3.1 How Indexes Work Internally — B-Trees

Most relational databases implement indexes using a data structure called a B-Tree (or its variant, a B+ Tree). A B-Tree keeps data sorted and organized in a balanced tree shape, where each level narrows down the search range, similar to a multi-level table of contents. This allows the database to find any row in roughly $O(\log n)$ steps instead of $O(n)$ steps — meaning even with a billion rows, finding one row might only take ~30 comparisons instead of up to a billion.

- B+ Tree (the most common variant used in databases like MySQL's InnoDB): all actual data values are stored only in the leaf nodes, and leaf nodes are linked together, making range queries (e.g., 'find all orders between Jan 1 and Jan 31') very efficient.
- The tree stays balanced automatically as data is inserted/deleted, which is why lookups remain fast ($O(\log n)$) even as the table grows.

Why this matters: Knowing the name 'B-Tree' and roughly why it gives $O(\log n)$ lookups is usually enough depth for an interview — you're rarely expected to implement one from scratch, but explaining WHY an index speeds things up (not just THAT it does) is what shows real understanding.

3.2 Primary Index vs Secondary Index

A primary index is automatically built on a table's primary key (the column that uniquely identifies each row) and often determines the physical storage order of the data itself (called a 'clustered index' in some databases). A secondary index is any additional index you create on other columns to speed up queries that filter or sort by those columns.

Primary / Clustered Index	Secondary Index
Built on the primary key automatically	Built manually on other frequently-queried columns
Often determines physical row storage order (in clustered designs)	A separate structure pointing back to the actual row location
Usually one per table	A table can have many secondary indexes
Example: index on user_id in a Users table	Example: index on email, to speed up login lookups by email

3.3 Composite (Multi-Column) Indexes

A composite index covers multiple columns together, in a specific order. The order matters enormously: a composite index on (last_name, first_name) can efficiently serve queries filtering by last_name alone, or by last_name AND first_name together — but it generally cannot efficiently serve a query filtering by first_name alone, because the index is sorted by last_name first.

Composite Index Order Matters: (last_name, first_name)

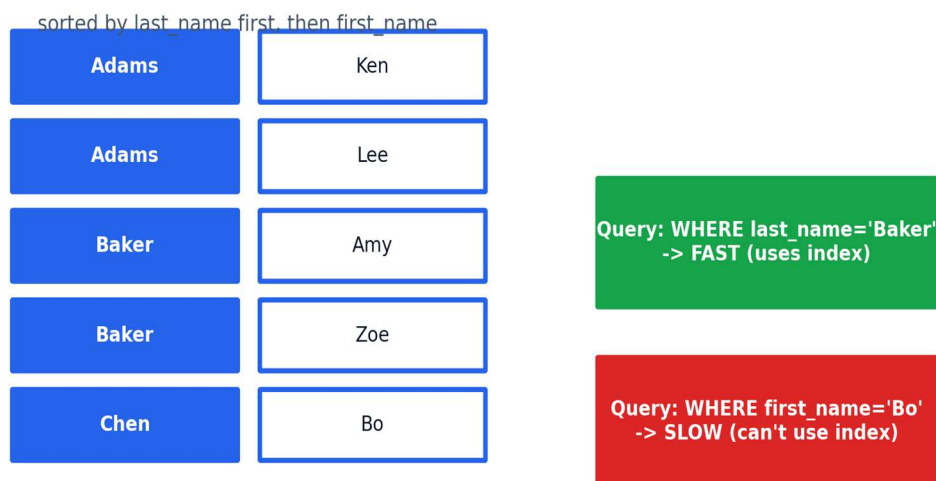


Figure 8.2 — A composite index is sorted by its first column first; queries that skip it can't use the index.

Common mistake: Creating a composite index in the wrong column order for your actual query patterns. A common interview trap: candidates create indexes without considering which columns are filtered most often or in what combination, leading to indexes that exist but never actually get used by the query planner.

3.4 The Trade-off: Indexes Speed Up Reads, Slow Down Writes

Indexes aren't free. Every time you insert, update, or delete a row, the database must also update every index that covers the affected columns — keeping the B-Tree balanced and sorted. More indexes mean more work on every write, and more disk space consumed to store the index structures themselves.

More Indexes	Fewer Indexes
Faster reads/queries on indexed columns	Slower reads/queries that rely on those columns (full table scans)
Slower writes (inserts/updates/deletes) — every index must be updated	Faster writes — less overhead per write operation
More disk space used to store index structures	Less disk space used

Why this matters: This read/write trade-off is exactly why you shouldn't index every column 'just in case.' A write-heavy system (e.g., logging, IoT sensor ingestion) often deliberately keeps indexes minimal, while a read-heavy system (e.g., a product catalog) can afford more indexes since reads vastly outnumber writes.

3.5 When NOT to Use an Index

- Very small tables: a full scan of a few hundred rows is often faster than the overhead of using an index at all.
- Columns with very low cardinality (few unique values, e.g., a boolean 'is_active' column) — an index here often doesn't help much, since it can't narrow the search space significantly.
- Columns that are rarely used in WHERE clauses, JOIN conditions, or ORDER BY — indexing them adds write overhead for little to no read benefit.

4. Trade-offs Table

Decision	Pros	Cons
Add an index on a frequently-queried column	Dramatically faster reads ($O(\log n)$ vs $O(n)$)	Slower writes, more disk space used
Composite index covering common multi-column queries	Can serve several related query patterns efficiently from one index	Order-sensitive; wrong order makes the index far less useful
No index, rely on full table scans	Zero write overhead, no extra storage	Reads become extremely slow as the table grows large

5. How Real Companies Use This

- Most production incidents labeled 'the database is slow' trace back to a missing or poorly-designed index on a query that used to be fast when the table was small but degraded as data grew — index review is a routine part of database performance tuning at every major tech company.
- E-commerce platforms heavily index columns used in product search and filtering (category, price range, brand) since these are read constantly, while deliberately avoiding excessive indexing on high-write tables like order logs.
- Companies running write-heavy time-series workloads (IoT, monitoring/metrics platforms) often use specialized storage engines or column-family databases (Chapter 7) partly because traditional B-Tree indexing overhead becomes a bottleneck at extreme write volumes.

6. Common Mistakes & Misconceptions

- Believing 'adding more indexes is always good' — every index has a write-performance and storage cost, so indexes should be added deliberately based on actual query patterns, not preemptively on every column.
- Forgetting that composite index column order matters — an index on (A, B) is not the same as an index on (B, A) for query optimization purposes.
- Assuming an index always gets used — the database's query planner may decide a full table scan is actually faster in some cases (e.g., when a query would return most of the table anyway), so simply having an index doesn't guarantee it's used.

7. Interviewer Q&A Bank

Q1. How does a database index speed up queries?

A: An index is a separate, sorted data structure — typically a B+ Tree — that lets the database locate rows in roughly $O(\log n)$ steps instead of scanning every row ($O(n)$). It works similarly to a phone book's alphabetical sorting: instead of reading every entry, the database can navigate directly to the relevant range.

Q2. What's the trade-off of adding more indexes to a table?

A: Indexes speed up reads on the indexed columns, but every insert, update, or delete must also update each relevant index, slowing down writes and consuming additional disk space. The right number of indexes depends on the table's actual read/write ratio and query patterns — write-heavy tables should be indexed more sparingly than read-heavy ones.

Q3. Why does the column order matter in a composite index?

A: A composite index is sorted by its first column, then its second column within each value of the first, and so on. This means it can efficiently serve queries that filter on the first column alone, or the first and second together, but generally cannot efficiently serve a query that filters only on the second column without the first — the order must match how you actually query the data.

Q4. When would you decide NOT to add an index to a column?

A: If the table is small enough that a full scan is already fast, if the column has very low cardinality (few unique values, so the index doesn't meaningfully narrow the search), or if the column is rarely used in filtering, joining, or sorting — in these cases the write-overhead cost of maintaining the index isn't justified by the read benefit.

Q5. A query that used to be fast is now slow as the table has grown. What would you investigate?

A: I'd first check whether the columns used in the query's WHERE clause, JOIN conditions, or ORDER BY are indexed, and whether the existing indexes match the actual query pattern (right columns, right order for composite indexes). I'd also check if the query planner is actually using the available index (via an EXPLAIN query plan) — sometimes an index exists but isn't being used due to how the query is written.

8. HLD Mini-Scenario: Indexing Decisions for a Product Search Feature

Prompt: "Design the indexing strategy for an e-commerce product table that users search by category and filter by price range, and that gets updated frequently by sellers."

- 1. Identify the dominant query pattern: 'find products in category X within price range Y' — this strongly suggests a composite index on (category, price), in that order, since category is typically the higher-level filter applied first.
- 2. Acknowledge the write trade-off: since sellers update product data frequently, we shouldn't over-index every column — only category and price (the proven hot query path) get dedicated indexes, not every minor product attribute.
- 3. Consider separating concerns: for more complex, flexible search (e.g., free-text search across product names/descriptions), a dedicated search engine (like Elasticsearch) might be a better fit than relying purely on database indexes, since full-text search has very different indexing needs than range/equality filtering.

This shows the senior-level instinct again: indexing decisions should be driven by actual, named query patterns — not applied generically.

9. Questions YOU Should Ask the Interviewer

- What are the most frequent queries against this data — what columns are typically filtered, joined, or sorted on?
- Is this table more read-heavy or write-heavy, and roughly how large is it expected to grow?
- Are there specific latency requirements for these queries that should guide how aggressively we index?

10. Scenario-Based Practice Bank (New)

Scenario 1: A query filtering on `WHERE country = 'US' AND email = 'x@y.com'` is slow, even though there's an index on `(country, email)`.

***How to approach it:** Check whether the query planner is actually using the index (via `EXPLAIN`) — but also consider `country`'s cardinality: if the vast majority of rows are 'US', the index on `country` barely narrows the search, so the database may be right to fall back to a fuller scan. A composite index led by the higher-cardinality column (`email` first) might serve this query far better.*

Scenario 2: An engineer proposes adding an index to every column on a heavily-written orders table 'just to be safe' before a big launch.

***How to approach it:** Push back: every additional index adds write overhead (each insert/update must update every index) and disk space. Recommend identifying the actual hot query patterns first, and indexing only those columns — over-indexing a write-heavy table can hurt exactly the launch-day write throughput the team is trying to protect.*

Scenario 3: A reporting dashboard needs to filter orders by `first_name`, but the only existing index is a composite one on `(last_name, first_name)`.

***How to approach it:** Explain that this existing index can't efficiently serve a `first_name`-only filter, since it's sorted by `last_name` first. Either add a separate index on `first_name` (if this query pattern is common enough to justify the write cost), or reconsider whether this report can be restructured to also filter by `last_name`.*

Scenario 4: A boolean `is_verified` column on a Users table currently has its own dedicated index, and a teammate wants to know if that's helpful.

***How to approach it:** Point out that a boolean column has very low cardinality (only two possible values), so an index here rarely narrows the search meaningfully — it likely isn't earning its write-overhead cost unless paired with other columns in a composite index that does narrow things down.*

Scenario 5: A free-text product search feature ('wireless bluetooth headphones noise cancelling') is being built directly on top of standard database indexes.

***How to approach it:** Recommend a dedicated search engine (like Elasticsearch) instead of stretching relational B-Tree indexes to do full-text search — B-Tree indexes are built for equality/range filtering, not for ranking free-text relevance across multiple words, which is a fundamentally different indexing problem.*

Scenario 6: A query was fast in staging (10,000 rows) but is now timing out in production (80 million rows), and the table has no indexes at all on the filtered column.

How to approach it: This is the classic full-table-scan-at-scale problem: the query looked fine on a small table because $O(n)$ is trivial at 10,000 rows but becomes catastrophic at 80 million. Add an index on the filtered column and verify with EXPLAIN that the query planner is now using it.

Chapter 8 — One-Page Revision Sheet

- An index is a sorted, separate data structure (typically a B+ Tree) that lets the database find rows in $O(\log n)$ instead of scanning every row ($O(n)$).
- Primary/clustered index: built on the primary key, often determines physical row order. Secondary index: built manually on other queried columns.
- Composite (multi-column) indexes are order-sensitive — index (A, B) efficiently serves queries on A alone or A+B, not B alone.
- Trade-off: more indexes = faster reads, but slower writes (every index must update) and more disk space used.
- Don't index everything — skip indexing small tables, low-cardinality columns, and rarely-queried columns.
- A query that worked fast on a small table can become slow at scale if it lacks a matching index — this is one of the most common real-world performance issues.
- Use EXPLAIN/query plans to verify an index is actually being used by the database's query planner, not just present.

Memory hook: An index is a phone book's alphabetical order — you flip straight to 'S' instead of reading every name. But maintaining that order costs effort every time a new name is added (write overhead).

CHAPTER 9

Database Scaling — Replication

Copying data across machines for speed and survival

1. Simple Explanation First

Replication means keeping multiple copies of the same data on different machines (replicas). It serves two purposes: surviving failure (if one machine dies, others still have the data) and handling more traffic (multiple machines can serve read requests in parallel instead of overloading one).

Master-Slave (Leader-Follower) Replication

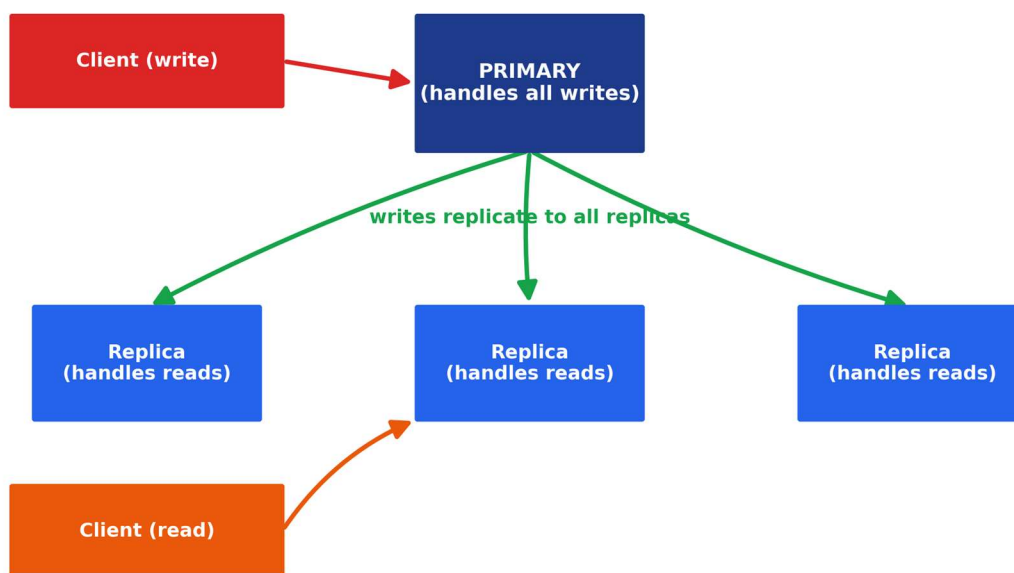


Figure 9.1 — One primary absorbs all writes; replicas absorb the read traffic.

Analogy: Imagine a popular library with only one copy of a bestselling book — everyone has to wait in line for that single copy. Replication is like the library buying 10 copies of that book and placing them on different shelves (or even different branches). Now 10 people can read it simultaneously. But if the original publisher releases a corrected edition, all 10 copies need to be updated — and there's a brief window where some shelves have the old edition and some have the new one.

2. Why This Topic Exists

A single database server has two hard limits: it can crash (and take your only copy of data down with it), and it can only handle so many simultaneous read/write requests before performance degrades. Replication solves both problems by maintaining synchronized copies of the data across multiple machines — but it introduces a new, fundamental challenge: keeping those copies in sync, which directly connects back to CAP theorem (Chapter 5).

Why this matters: Replication is usually the *FIRST* scaling technique applied to a database, before reaching for more complex solutions like sharding (Chapter 10). Understanding it deeply is foundational — almost every 'how would you scale this database' answer starts with replication.

3. Core Concepts in Depth

3.1 Master-Slave (Primary-Replica) Replication

In this most common setup, one node (the master/primary) handles all writes. Changes are then propagated to one or more replica (slave) nodes, which typically handle read traffic. This is sometimes called 'leader-follower' replication in modern terminology, avoiding the older master-slave naming.

- All writes go to the single primary, which keeps write logic simple and avoids conflicting simultaneous writes to the same data.
- Reads can be distributed across many replicas, dramatically increasing how much read traffic the system can handle (this is the most common way to scale read-heavy workloads).
- If the primary fails, one replica must be promoted to become the new primary — this failover process needs careful handling to avoid data loss or split-brain scenarios (two nodes both thinking they're the primary).

3.2 Master-Master (Multi-Leader) Replication

In this setup, multiple nodes can each accept writes, and they sync changes with each other. This avoids having a single point of failure for writes, but introduces a much harder problem: what happens when two masters receive conflicting writes to the same piece of data at nearly the same time?

Master-Slave (Single Leader)	Master-Master (Multi-Leader)
Simple — one source of truth for writes, no write conflicts	Complex — must handle conflicting concurrent writes
Write availability depends on the single primary being up	Higher write availability — multiple nodes can accept writes
Failover requires promoting a replica (some delay/risk)	No single point of failure for writes, but conflict resolution adds complexity
Common in most traditional setups (e.g., standard PostgreSQL/MySQL replication)	Used when write availability across regions matters more than simplicity

Common mistake: Suggesting master-master replication as a default 'better' choice without acknowledging the conflict-resolution complexity it introduces. Interviewers often probe with 'what happens if two masters get conflicting writes at the same time?' — be ready to discuss conflict resolution strategies (e.g., last-write-wins, vector clocks, or application-level merge logic) rather than assuming it 'just works.'

3.3 Synchronous vs Asynchronous Replication

This decision determines how 'in sync' your replicas really are, and it's a direct, practical application of the PACELC framework from Chapter 5 — even with no network partition, you're trading latency for consistency.

Synchronous vs Asynchronous Replication

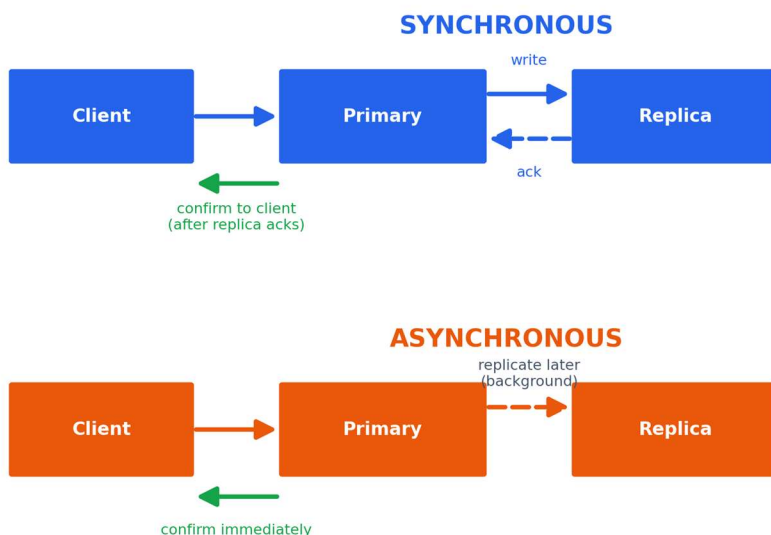


Figure 9.2 — Synchronous replication waits for the replica to confirm; asynchronous confirms immediately and replicates in the background.

Synchronous Replication	Asynchronous Replication
Primary waits for replica(s) to confirm the write before acknowledging success to the client	Primary acknowledges the write immediately, then propagates to replicas in the background
Stronger consistency — replicas are always (nearly) up to date	Weaker consistency — replicas may lag behind momentarily
Higher write latency (waiting for confirmation across the network)	Lower write latency (no waiting)
Risk: if a replica is slow/down, it can block or slow writes entirely	Risk: if the primary fails before replicating, the most recent writes could be lost

3.4 Replication Lag and Its Real Consequences

Replication lag is the delay between a write happening on the primary and that write becoming visible on a replica. In asynchronous replication, this lag is unavoidable and can range from milliseconds to (in unhealthy systems) minutes. This has very real, sometimes surprising consequences for application behavior.

- Read-your-own-writes problem: a user updates their profile picture, the write goes to the primary, but their next page load reads from a replica that hasn't caught up yet — they see their OLD picture momentarily, which feels like a bug even though the system is working as designed.
- Common mitigation: route a user's own immediate post-write reads to the primary (or a replica known to be caught up), while routing general browsing traffic to any replica.

Why this matters: The 'read-your-own-writes' problem is a fantastic example to bring up in interviews — it shows you understand replication lag isn't just a theoretical concept, it causes concrete, named user-facing bugs that real engineering teams have to design around.

4. Trade-offs Table

Decision	Pros	Cons
Master-slave with async replication	Fast writes, simple conflict-free model, good read scaling	Replication lag; possible small data loss if primary fails before replicating
Master-master replication	No single point of failure for writes; good for multi-region write availability	Conflict resolution complexity; harder to reason about correctness
Master-slave with sync replication	Strong consistency across replicas, no data loss on primary failure	Higher write latency; write availability tied to replica health

5. How Real Companies Use This

- Most read-heavy applications (news sites, social feeds, e-commerce catalogs) use master-slave replication with several read replicas to absorb the vastly higher read traffic compared to write traffic.
- Globally distributed systems (e.g., multi-region deployments at companies like Google, Amazon) sometimes use master-master or more advanced consensus-based replication specifically so users in different regions can write locally with low latency, accepting the added complexity of conflict resolution.
- Financial systems typically favor synchronous replication (or strong consensus protocols) for critical transactional data, explicitly accepting higher latency in exchange for guaranteeing no data loss.

6. Common Mistakes & Misconceptions

- Assuming replication alone solves write scalability — master-slave replication scales READS well, but writes are still bottlenecked through the single primary (sharding, covered in Chapter 10, addresses write scaling).
- Forgetting that asynchronous replication can lose recent writes if the primary crashes before replicating — this is an important trade-off to state explicitly, not gloss over.
- Treating replication and backups as the same thing — replication keeps live, near-real-time copies for availability/scaling; backups are point-in-time snapshots for disaster recovery. You need both, for different reasons.

7. Interviewer Q&A Bank

Q1. What is the difference between master-slave and master-master replication?

A: In master-slave (leader-follower) replication, only one node accepts writes, and changes propagate to read replicas — simple, with no write conflicts, but the primary is a single point of failure for writes. In master-master replication, multiple nodes can accept writes, removing that single point of failure but

requiring conflict resolution logic for when two nodes receive conflicting writes to the same data nearly simultaneously.

Q2. What's the trade-off between synchronous and asynchronous replication?

A: Synchronous replication waits for replicas to confirm a write before acknowledging it to the client, guaranteeing replicas stay in sync but adding latency to every write and creating risk if a replica is slow. Asynchronous replication acknowledges writes immediately and replicates in the background, giving lower latency but risking replication lag and even data loss if the primary fails before the write propagates.

Q3. What is replication lag, and how can it cause a noticeable bug?

A: Replication lag is the delay between a write happening on the primary and that change becoming visible on a replica. A classic resulting bug is the 'read-your-own-writes' problem: a user updates something, but their next read is served from a replica that hasn't caught up yet, so they briefly see stale data — even though the underlying system is functioning exactly as designed.

Q4. Does replication help scale write throughput?

A: Not directly in a master-slave setup — all writes still go through the single primary, so replication mainly scales read throughput by distributing reads across replicas. To scale write throughput, you typically need sharding/partitioning (splitting data across multiple independently-writable databases), which is a separate technique covered alongside replication.

Q5. How would you handle the 'read-your-own-writes' problem in a replicated system?

A: A common approach is to route a user's reads immediately following their own write to the primary (or a replica confirmed to be caught up), while routing general, non-personalized read traffic to any available replica. Some systems also use 'read-your-writes' session tokens that track the last write a user made, ensuring subsequent reads in that session come from a sufficiently up-to-date replica.

8. HLD Mini-Scenario: Designing Replication for a Blogging Platform

Prompt: "A blogging platform has far more readers than writers — most traffic is people reading posts, while only a small number of users publish or edit posts. How would you use replication here?"

- 1. Set up a master-slave replication topology: one primary database handles all writes (new posts, edits), and multiple read replicas handle the much higher volume of read traffic (people viewing posts).
- 2. Choose asynchronous replication for most replicas, since reading a blog post that's a few seconds stale is a non-issue for most users — this keeps writes fast and avoids unnecessary latency.
- 3. Handle the author's own experience specially: immediately after an author publishes/edits a post, route THEIR next read (e.g., viewing their own published post) to the primary or a guaranteed-fresh replica, avoiding the awkward 'I just published this, why does it look unedited?' experience.

This mini-scenario shows how replication strategy decisions should be grounded in actual read/write ratios and real user experience considerations, not applied uniformly without thought.

9. Questions YOU Should Ask the Interviewer

- What's the approximate read-to-write ratio for this system's data?
- Is brief data staleness on reads acceptable, or do certain flows need guaranteed up-to-date data immediately after a write?
- Are we deploying across multiple geographic regions, or is a single-region setup sufficient for now?

10. Scenario-Based Practice Bank (New)

Scenario 1: A user updates their profile bio, refreshes the page immediately, and sees their old bio — then it 'fixes itself' a second later.

How to approach it: This is the read-your-own-writes problem caused by asynchronous replication lag. Fix it by routing that specific user's next read (right after their own write) to the primary or a replica confirmed caught up, while general browsing traffic continues to use any replica.

Scenario 2: A financial services company asks whether they should use asynchronous replication for their core ledger to reduce write latency.

How to approach it: Recommend against it for this specific data: async replication risks losing the most recent writes if the primary crashes before replicating, which is unacceptable for a ledger. Synchronous replication (or a strong consensus protocol) is worth the added write latency here, even though async might be fine elsewhere in the same company's less critical systems.

Scenario 3: Two regional offices of a multi-region app both need to accept writes locally for low latency, and conflicting edits to the same record occasionally happen.

How to approach it: This calls for master-master replication, but the team must explicitly design conflict resolution (last-write-wins, vector clocks, or application-level merge logic) rather than assuming the database 'just handles it' — flag this complexity proactively rather than waiting for it to surface as a bug.

Scenario 4: A read replica has fallen far behind the primary (minutes of lag) after a burst of write traffic, and dashboards relying on it show clearly outdated numbers.

How to approach it: Investigate why replication lag spiked (network issues, replica under-provisioned, write burst exceeding replica catch-up capacity), and consider routing time-sensitive dashboards to the primary temporarily, or adding replica capacity/monitoring so lag spikes are caught before they visibly affect users.

Scenario 5: A team assumes that adding 5 more read replicas will fix their database's write-throughput bottleneck during a big sale event.

How to approach it: Correct this misconception: replicas scale read throughput, not write throughput, since all writes still funnel through the single primary. If writes are the actual bottleneck, the team needs sharding (Chapter 10) or a more powerful/optimized primary, not more read replicas.

Scenario 6: After a primary database crash, the team promotes a replica to be the new primary, but discovers a few of the last transactions before the crash are missing.

How to approach it: This is the expected risk of asynchronous replication: writes acknowledged by the primary just before the crash may not have propagated yet. If this data loss is unacceptable for this

system, the team should move to synchronous replication (or a quorum-based write acknowledgment) for that specific data, accepting the latency trade-off.

Chapter 9 — One-Page Revision Sheet

- Replication = keeping multiple copies of data across machines, for fault tolerance (survive crashes) and read scaling (distribute read load).
- Master-slave (leader-follower): one primary handles writes, replicas handle reads — simple, no write conflicts, but primary is a single point of failure for writes.
- Master-master (multi-leader): multiple nodes accept writes — no single write point of failure, but requires conflict resolution for simultaneous conflicting writes.
- Synchronous replication: waits for replica confirmation -> strong consistency, higher latency. Asynchronous: acknowledges immediately -> lower latency, risk of lag/data loss.
- Replication lag causes real bugs like 'read-your-own-writes' — mitigate by routing a user's post-write reads to the primary or a guaranteed fresh replica.
- Replication scales READS well via master-slave, but does NOT by itself scale WRITES — that requires sharding (Chapter 10).
- Replication is not the same as backups — replication is for live availability/scaling, backups are for point-in-time disaster recovery; you need both.

Memory hook: *A library with 10 copies of a bestseller (replication) — 10 people can read at once, but if a corrected edition is released, there's a window where some shelves are out of date.*

CHAPTER 10

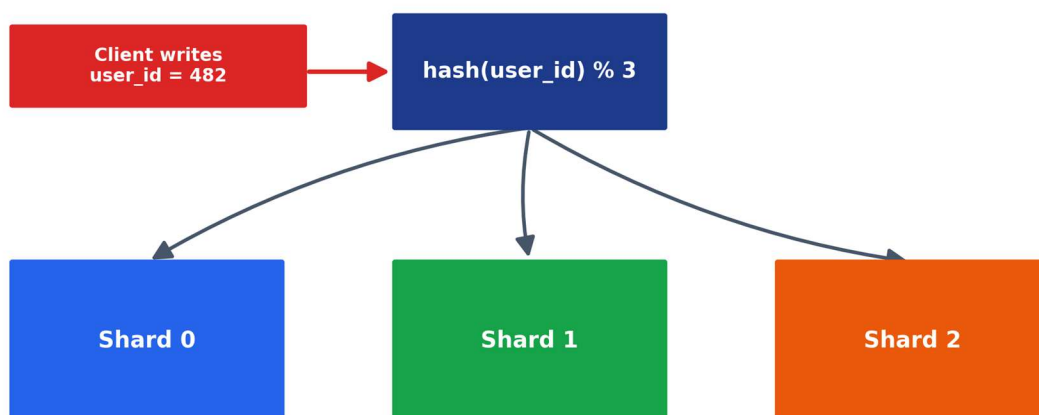
Database Scaling — Sharding & Partitioning

Splitting data across machines so no single machine carries the whole burden

1. Simple Explanation First

Sharding (also called horizontal partitioning) means splitting one large dataset into smaller pieces (shards), each stored on a different machine. Instead of one giant database holding all 100 million users, you might have 10 smaller databases, each holding 10 million users. Each shard is independently capable of handling both reads AND writes for its slice of data, which is what makes sharding solve the write-scaling problem that replication (Chapter 9) alone cannot.

Hash-Based Sharding



Each shard independently handles its own reads AND writes for its slice of users.

Figure 10.1 — A hash function decides which shard a given key belongs to.

Analogy: Imagine a single massive supermarket trying to serve an entire country — the checkout lines would be endless. Sharding is like opening many smaller supermarkets across different neighborhoods, each one independently serving and processing transactions for the customers nearest to it. Each branch doesn't need to know about every transaction happening at every other branch — it just needs to handle its own neighborhood's full shopping needs, including both browsing (reads) and buying (writes).

2. Why This Topic Exists

Replication (Chapter 9) scales read traffic beautifully, but every write still funnels through a single primary database — eventually, even one powerful primary machine hits a ceiling on write throughput or storage capacity. Sharding exists specifically to break through that ceiling by distributing both the data AND the write workload across multiple independent machines, each handling only a fraction of the total.

Why this matters: Sharding is usually the *SECOND* major scaling step after replication — interviewers expect you to reach for it specifically when write throughput or total data volume genuinely exceeds what a single (even replicated) database can handle, not as a default starting point.

3. Core Concepts in Depth

3.1 Horizontal Partitioning Strategies — How Do You Decide What Goes Where?

The hardest part of sharding isn't splitting the data — it's deciding WHICH data goes to WHICH shard, in a way that distributes load evenly and lets you find any given piece of data quickly later.

Strategy	How It Works	Trade-off
Range-based sharding	Data split by ranges of a key (e.g., users A-F on shard 1, G-M on shard 2)	Simple and supports range queries well, but can create uneven load if data isn't evenly distributed across ranges (hot spots)
Hash-based sharding	A hash function is applied to a key (e.g., user_id), and the result determines the shard	Distributes data much more evenly, but makes range queries across shards harder, and resharding is painful (see consistent hashing)
Directory-based sharding	A separate lookup service/table tracks exactly which shard holds which piece of data	Very flexible (can rebalance freely), but the lookup directory itself becomes a critical, must-scale component

Common mistake: Choosing range-based sharding by a naturally 'hot' key without considering data distribution — e.g., sharding by signup date range means the newest shard gets ALL new signups, creating a massive imbalance where one shard is overwhelmed while older shards sit idle.

3.2 The Hot Partition / Hot Key Problem

A hot partition (or hot key/hot shard) happens when one shard receives disproportionately more traffic than others — defeating the entire purpose of sharding, since that one overloaded shard becomes the new bottleneck. This commonly happens when the sharding key correlates with popularity or recency.

The Hot Partition Problem

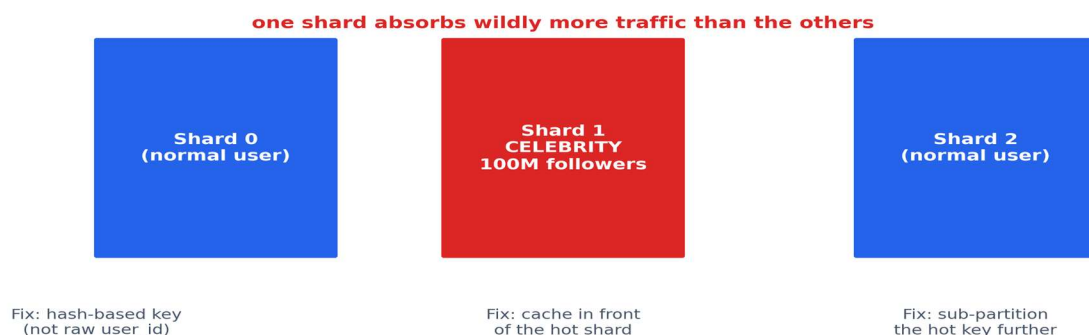


Figure 10.2 — One shard absorbing a celebrity account gets far more traffic than its neighbours.

- Example: sharding a social media platform by `user_id` seems reasonable, until a celebrity with 100 million followers happens to land on one shard — that shard now handles wildly more read/write traffic than every other shard.
- Example: sharding e-commerce orders by date means the 'today' shard is always the hottest, since all new orders land there, while older date-shards go nearly idle.
- Mitigations: choose a sharding key that distributes load more evenly (e.g., a hash of `user_id` instead of raw `user_id`), add a secondary layer of sub-partitioning for known hot keys, or use caching in front of the hot shard to absorb read pressure.

Why this matters: *Bringing up the hot partition problem proactively — even when the interviewer hasn't asked — is one of the clearest signals of real sharding experience. It shows you know sharding isn't 'solved' just by splitting data; the split has to actually be balanced in practice.*

3.3 Resharding — The Painful Part

Resharding means changing how data is distributed across shards — usually because you're adding more shards as the system grows, or rebalancing because of a hot partition. With naive hash-based sharding ($\text{hash}(\text{key}) \% \text{number_of_shards}$), adding even ONE new shard changes the modulo result for almost every key, meaning almost ALL data would need to move to a different shard — an enormous, disruptive operation on a live system.

This exact problem is why Chapter 17 is dedicated entirely to consistent hashing — a technique specifically designed to minimize how much data needs to move when shards are added or removed. For now, just remember: naive modulo-based hashing is simple but creates a massive resharding problem; consistent hashing exists to fix this.

3.4 What Sharding Costs You — The Real Trade-offs

Sharding solves scale problems but introduces real complexity, which is why it should be a deliberate, justified decision rather than a default.

- Cross-shard queries become hard: a query that needs data from multiple shards (e.g., 'find the top 10 products across all regions') can no longer be a single simple database query — it requires querying multiple shards and merging results in application code.
- Cross-shard transactions become hard: an operation that needs to atomically update data on two different shards (e.g., transferring a balance between two users on different shards) loses the simple single-database ACID guarantee, often requiring distributed transaction patterns (Chapter 12, and the Saga pattern in Chapter 18).
- Operational complexity increases: more machines to monitor, back up, and maintain; schema changes now need to be applied consistently across every shard.

4. Trade-offs Table

Approach	Pros	Cons
Range-based sharding	Simple, supports efficient range queries	Risk of uneven load (hot spots) if data isn't evenly distributed by range
Hash-based sharding	Even data distribution, avoids obvious hot spots from sequential keys	Range queries across shards become hard; naive rehashing on resize is very disruptive
Sharding overall (vs staying single-database)	Breaks through write-throughput and storage ceilings of a single machine	Cross-shard queries/transactions become significantly harder; more operational overhead

5. How Real Companies Use This

- Instagram famously shards its PostgreSQL database by `user_id` using a custom ID-generation scheme that embeds shard information directly into each generated ID, simplifying lookups.
- Discord shards its message storage to handle the enormous, ever-growing volume of chat messages across millions of servers, since no single database could hold or serve that volume efficiently.
- Uber shards ride and trip data geographically in some systems, since most queries naturally relate to a specific city/region, making geographic sharding a natural fit for their access patterns.

6. Common Mistakes & Misconceptions

- Reaching for sharding before exhausting simpler options (vertical scaling, replication, caching) — sharding adds substantial complexity and should be justified by genuine, demonstrated need, not used as a first resort.
- Picking a sharding key without checking for hot-partition risk — always ask 'could this key be very unevenly distributed in the real world?' before committing to it.
- Forgetting that cross-shard queries and transactions are NOT free after sharding — failing to mention this cost in an interview suggests you haven't actually grappled with sharding's real-world complexity.

7. Interviewer Q&A Bank

Q1. What problem does sharding solve that replication alone cannot?

A: Replication scales read throughput by distributing read traffic across multiple replica copies, but all writes still go through a single primary database, which eventually hits a throughput or storage ceiling. Sharding splits the data itself across multiple independent databases, each capable of handling its own reads AND writes, which is what allows write throughput and total storage to scale beyond what one machine can handle.

Q2. What is a 'hot partition' and how would you avoid one?

A: A hot partition is a shard that receives disproportionately more traffic than others, often because the sharding key correlates with popularity (e.g., a viral user) or recency (e.g., sharding by today's date). I'd avoid this by choosing a sharding key that distributes load more evenly — often a hash of an ID rather

than the raw ID or a time-based value — and by monitoring for emerging hot spots so they can be mitigated with caching or further sub-partitioning.

Q3. Why is resharding difficult with naive hash-based sharding?

A: With a naive scheme like $\text{hash}(\text{key}) \% \text{number_of_shards}$, adding or removing even one shard changes the modulo result for nearly every key, meaning almost all existing data would need to be moved to different shards — a massive, disruptive operation on a live system. This is exactly the problem consistent hashing is designed to solve, by minimizing how much data needs to move when the number of shards changes.

Q4. What becomes harder after you shard a database?

A: Cross-shard queries (needing data from multiple shards, like a global leaderboard) can no longer be a single simple query — they require querying multiple shards and merging results in application logic. Cross-shard transactions (atomically updating data across two different shards) lose the simple single-database ACID guarantee and typically need distributed transaction patterns like the Saga pattern.

Q5. How would you choose a sharding key for a multi-tenant SaaS application?

A: I'd likely shard by tenant/organization ID, since most queries in a multi-tenant system naturally scope to a single tenant — this keeps the vast majority of queries within a single shard, avoiding expensive cross-shard operations. I'd also check whether tenant sizes vary wildly (a few huge enterprise customers vs many small ones), since that could create hot partitions requiring special handling for the largest tenants.

8. HLD Mini-Scenario: Sharding Decision for a Messaging Platform

Prompt: "Design the message storage layer for a chat application with billions of messages across millions of conversations. A single database can no longer keep up with write volume."

- 1. Choose a sharding key: shard by conversation_id (or a hash of it), since most queries naturally scope to a single conversation (fetching message history for one chat) — this keeps the dominant query pattern within a single shard.
- 2. Check for hot-partition risk: a small number of extremely large group conversations (e.g., a 'company-wide announcements' channel with thousands of active members) could become hot shards — mitigate with caching the most recent messages of high-traffic conversations, or further sub-partitioning very large conversations.
- 3. Acknowledge the cross-shard cost: a feature like 'search across all of a user's conversations' now requires querying multiple shards and merging results, since a user's various conversations may live on different shards — this would likely be better served by a separate search index (foreshadowing search/indexing patterns) rather than querying shards directly.

This shows the full sharding thought process: pick a key matching the dominant access pattern, proactively check for hot spots, and honestly name what becomes harder as a result.

9. Questions YOU Should Ask the Interviewer

- What's the dominant query pattern — do most queries naturally scope to a single entity (a user, a conversation, a tenant)?
- Is there a risk of certain keys being far more popular/active than others (celebrities, large tenants, viral content)?

- Do we need cross-shard queries or transactions for any critical features, and how important is that consistency?

10. Scenario-Based Practice Bank (New)

Scenario 1: A team shards their e-commerce orders table by order date (e.g., one shard per month), and the current month's shard is consistently overloaded while last year's shards sit nearly idle.

***How to approach it:** This is the classic recency-correlated hot partition problem. Recommend re-sharding by a hash of a more evenly-distributed key (e.g., hash of customer_id or order_id) instead of raw date, so new orders spread across all shards rather than concentrating on 'today's' shard.*

Scenario 2: After sharding a social platform by raw user_id, one particular shard is handling 50x the traffic of any other because it happens to contain a celebrity account.

***How to approach it:** Confirm this is a hot-key problem tied to the sharding key correlating with popularity. Mitigate with a hash-based key (spreading any single popular user's associated data more evenly won't fully fix a single mega-user, so also add caching in front of that specific hot shard and consider sub-partitioning that one account's data specifically).*

Scenario 3: A finance feature needs to atomically transfer a balance between two user accounts that happen to live on two different shards after sharding by user_id.

***How to approach it:** Flag that this cross-shard transaction can no longer rely on simple single-database ACID guarantees. Recommend a distributed transaction pattern — either a controlled two-phase commit (Chapter 12) for strict atomicity, or more likely a Saga pattern (Chapter 18) that uses compensating actions, trading strict atomicity for better availability.*

Scenario 4: The team wants to add 3 new shards to handle growth, and someone points out the current sharding scheme is $\text{hash}(\text{user_id}) \% \text{current_shard_count}$.

***How to approach it:** Warn that this naive modulo scheme means changing the shard count remaps almost every key to a different shard — a massive, disruptive data migration. Recommend adopting consistent hashing (Chapter 17) before scaling shard count further, since it minimizes how much data needs to move when shards are added or removed.*

Scenario 5: A 'global leaderboard showing the top 10 players across all regions' feature is requested, but player data is sharded by region.

***How to approach it:** Point out this is now a cross-shard query: no single shard has the full picture, so the application must query each regional shard for its local top players, then merge and re-rank results in application code — a real cost of sharding that should be named upfront rather than discovered during implementation.*

Scenario 6: A startup with modest traffic and a single small database is proactively asking whether they should shard now 'to be ready for scale.'

***How to approach it:** Recommend against premature sharding: it adds substantial cross-shard query/transaction and operational complexity that isn't justified without genuine demonstrated need. Suggest exhausting simpler options first — vertical scaling, replication, caching — and revisiting sharding only when write throughput or storage genuinely approaches a single database's ceiling.*

Chapter 10 — One-Page Revision Sheet

- Sharding = splitting data across multiple independent databases, each handling its own reads AND writes — solves write/storage scaling that replication alone cannot.
- Range-based sharding: simple, supports range queries, but risks uneven hot spots. Hash-based sharding: even distribution, but range queries are hard and naive resizing is disruptive.
- Hot partition problem: one shard gets disproportionate traffic (e.g., a celebrity user, a 'today' date shard) — mitigate via better key choice, caching, or sub-partitioning.
- Naive $\text{hash}(\text{key}) \% N$ sharding makes resharding extremely disruptive — almost all keys remap when N changes. This is why consistent hashing exists (Chapter 17).
- Sharding costs: cross-shard queries need application-level merging; cross-shard transactions lose simple ACID guarantees and need distributed transaction patterns (Saga, Chapter 18).
- Reach for sharding only after simpler options (vertical scaling, replication, caching) are genuinely insufficient — it's a deliberate complexity trade-off, not a default.
- Choose a sharding key matching your dominant query pattern (e.g., shard chat messages by `conversation_id`) to minimize expensive cross-shard operations.

Memory hook: *One giant supermarket = endless checkout lines. Many neighborhood branches (shards) each handle their own customers fully — but a shopper needing items from two different branches (cross-shard query) now has extra work to do.*

CHAPTER 11

Caching

Storing the answer ahead of time so you almost never have to compute it again

1. Simple Explanation First

A cache is a small, fast storage layer that keeps a copy of frequently-accessed data so future requests for that data can be served instantly, without repeating expensive work (like a slow database query or complex computation). Caches are typically much faster but much smaller than your main database — they trade completeness for speed.

Cache-Aside (Lazy Loading) Flow

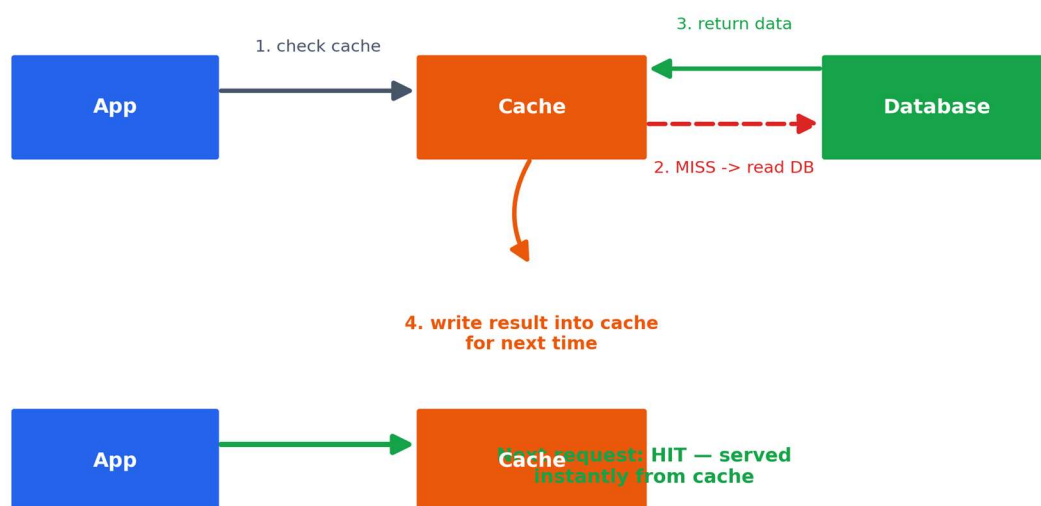


Figure 11.1 — On a miss the app reads the database and populates the cache; the next request hits the cache directly.

Analogy: Think of a cache like keeping your most-used kitchen spices on the counter instead of in the back of the pantry. You don't put everything on the counter (it would get cluttered and slow you down finding things) — just the items you reach for constantly. Going to the pantry (the database) is still possible for rare items, but for the common case, the counter (the cache) saves you a trip every single time.

2. Why This Topic Exists

Databases are durable and accurate, but relatively slow compared to in-memory storage — a database query might take 10-100 milliseconds, while reading from memory (RAM) can take microseconds, often 100-1000x faster. For data that's read far more often than it changes (the vast majority of real-world data),

repeatedly hitting the database for the same answer wastes time and database capacity. Caching exists to absorb that repetitive read load and serve it almost instantly.

Why this matters: Caching is frequently the single highest-leverage performance improvement you can add to a system — it's cheap to add, dramatically reduces database load, and directly improves user-perceived speed. It's also one of the most commonly discussed components in HLD interviews across almost every case study.

3. Core Concepts in Depth

3.1 Caching Patterns — How Reads Flow Through a Cache

Pattern	How It Works	Trade-off
Cache-Aside (Lazy Loading)	Application checks cache first; on a miss, reads from the database, then writes the result into the cache for next time	Simple and widely used; first request after a cache miss is always slow; cache can become stale if data changes without updating the cache
Write-Through	Every write goes to the cache AND the database at the same time (synchronously)	Cache is always fresh; but write latency increases since you're writing to two places before confirming success
Write-Back (Write-Behind)	Write goes to the cache immediately and is acknowledged; the database is updated asynchronously later	Very fast writes; risk of data loss if the cache fails before the database write completes
Write-Around	Writes go directly to the database, bypassing the cache; cache is only populated on subsequent reads	Avoids filling the cache with data that might never be read again; first read after a write is always a cache miss

Common mistake: Defaulting to write-through caching for everything without considering write latency cost. Write-through is great for data that's read immediately after being written, but adds unnecessary latency for write-heavy data that's rarely read back soon after — cache-aside or write-around often fit better there.

3.2 Cache Eviction Policies — What Gets Removed When the Cache Is Full

A cache is intentionally much smaller than the full dataset, so when it's full and a new item needs to be cached, something has to be removed (evicted). The eviction policy decides what to remove.

LRU Eviction — Cache Is Full, New Item Arrives

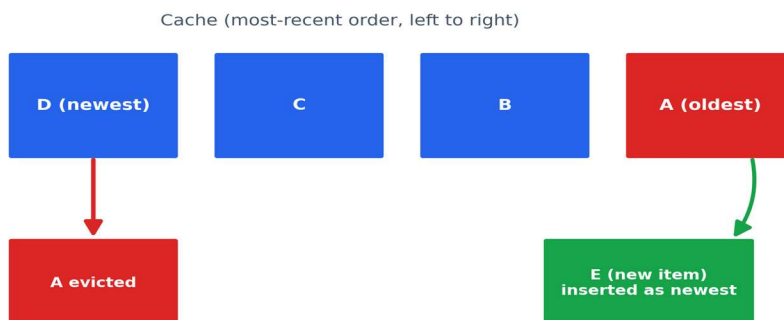


Figure 11.2 — LRU evicts whatever hasn't been touched in the longest time to make room for the newest item.

Policy	How It Decides What to Evict
LRU (Least Recently Used)	Removes the item that hasn't been accessed for the longest time — assumes recently-used data is likely to be used again soon
LFU (Least Frequently Used)	Removes the item with the fewest total accesses — assumes popularity over time matters more than recency
FIFO (First In, First Out)	Removes the oldest-added item regardless of how often it's been accessed — simple but often less effective than LRU/LFU
TTL-based (Time To Live)	Items automatically expire after a set duration, regardless of usage — useful when data has a natural 'freshness window'

LRU is by far the most commonly used default in real-world systems (and the most commonly asked-about in interviews) because it's a reasonable, simple-to-reason-about approximation of 'what's likely to be needed again soon' for most access patterns.

3.3 Cache Invalidation — 'One of the Two Hard Things in Computer Science'

There's a famous joke in computing: 'there are only two hard things in computer science: cache invalidation, and naming things.' Cache invalidation means knowing WHEN cached data has become stale (because the underlying data changed) and needs to be refreshed or removed — get this wrong, and your system confidently serves outdated, incorrect data.

- TTL-based expiration: simplest approach — just let cached data expire automatically after a fixed time, accepting some staleness in exchange for simplicity. Works well when some staleness is tolerable.
- Explicit invalidation on write: when data changes, the application explicitly deletes or updates the corresponding cache entry at the same time — more accurate, but requires remembering to do this everywhere the data can change, which is error-prone in complex systems.
- Event-driven invalidation: a change event (e.g., from a message queue/Chapter 14) triggers cache invalidation across all relevant cache nodes — more robust for distributed caches, but adds architectural complexity.

Why this matters: Bringing up cache invalidation challenges proactively — not just caching's benefits — is a strong signal of real-world experience. Interviewers specifically probe for 'how do you keep the cache from serving stale data?' because it's where caching designs most often go wrong in practice.

3.4 Redis vs Memcached

These are the two most common caching technologies referenced in interviews — knowing their core difference is high-value, low-effort knowledge.

Redis	Memcached
Supports rich data structures: strings, lists, sets, sorted sets, hashes	Simple key-value store only (strings/blobs)
Supports persistence (can save data to disk, survive a restart)	Purely in-memory — data is lost on restart
Supports built-in replication and clustering	Simpler, multi-threaded, often slightly faster for pure simple key-value workloads
Can be used as a cache AND a lightweight database/message broker (Pub/Sub)	Designed to do exactly one thing — simple caching — very well

3.5 CDN as a Caching Layer (Preview of Chapter 16)

A CDN (Content Delivery Network) is essentially a geographically-distributed cache for static content (images, videos, JS/CSS files) and sometimes dynamic content, placed physically close to users to reduce latency. This is conceptually the exact same caching idea covered in this chapter, just applied at a global, network-edge scale rather than within a single data center. Chapter 16 covers this in full depth.

4. Trade-offs Table

Decision	Pros	Cons
Cache-aside pattern	Simple, only caches what's actually requested (efficient use of cache space)	First request after a miss is slow; risk of staleness if invalidation is handled poorly
Write-through pattern	Cache always fresh and consistent with the database	Higher write latency; cache may fill with data that's never actually read
Aggressive caching (long TTLs, cache almost everything)	Maximizes cache hit rate, minimizes database load	Higher risk of serving stale data; more memory required

5. How Real Companies Use This

- Twitter/X caches timelines and tweet data heavily in Redis-like systems, since the same popular tweets are read by millions of users far more often than they're written once.
- Amazon caches product page data and pricing aggressively (with careful invalidation for fast-changing data like stock levels and flash-sale pricing) to keep page loads fast under enormous read traffic.

- Facebook built and open-sourced Memcached specifically to handle their massive read-heavy social graph workload, later supplementing it with more advanced caching layers as their needs grew more complex.

6. Common Mistakes & Misconceptions

- Treating caching as a free performance win with no downsides — every cache introduces a staleness risk and an invalidation responsibility that must be deliberately designed, not assumed away.
- Caching data that changes extremely frequently relative to how often it's read — this can result in a cache that's almost always stale or constantly being invalidated, providing little real benefit.
- Forgetting to discuss eviction policy when a cache is mentioned — interviewers often follow up with 'what happens when the cache is full?' expecting you to know LRU or a similar policy.

7. Interviewer Q&A Bank

Q1. Explain the cache-aside pattern and when you'd use it.

A: In cache-aside, the application checks the cache first; on a miss, it reads from the database and then populates the cache for next time. It's the most common and simplest caching pattern, well-suited for read-heavy data where you want the cache to only hold data that's actually been requested, rather than pre-populating everything.

Q2. Why is cache invalidation considered one of the hardest problems in computer science?

A: Because you need to reliably know exactly when cached data has become outdated due to an underlying change, and ensure that stale data either gets refreshed or removed before being served to a user — missing an invalidation path means confidently serving wrong data, which can be a worse failure mode than no caching at all. As systems grow more distributed and data can change from multiple places, keeping invalidation logic correct and complete becomes genuinely difficult.

Q3. What eviction policy would you use for a cache, and why?

A: LRU (Least Recently Used) is my default choice for most general-purpose caching scenarios, since it's a simple, effective approximation that recently-accessed data is likely to be accessed again soon. I'd consider LFU if I had reason to believe access frequency over a longer window matters more than recency for a specific dataset, or TTL-based expiration if the data has a natural freshness window regardless of access patterns.

Q4. What's the difference between write-through and write-back caching?

A: Write-through writes to both the cache and the database synchronously before acknowledging success, keeping them always consistent but adding write latency. Write-back writes to the cache first and acknowledges immediately, updating the database asynchronously afterward — faster writes, but a risk of losing that data if the cache fails before the database write completes.

Q5. When would you choose Redis over Memcached?

A: When I need more than simple key-value storage — for example, using Redis's sorted sets to build a leaderboard, its lists for a queue, or its Pub/Sub feature for real-time notifications — or when I need persistence so cached data can survive a restart. For a purely simple, very high-throughput key-value cache with no need for these extra features, Memcached's simplicity can be a perfectly good and sometimes slightly faster fit.

8. HLD Mini-Scenario: Caching Strategy for a News Website Homepage

Prompt: "Design the caching strategy for a news website's homepage, which shows the top 20 trending articles and is viewed by millions of users per minute, but the trending list only changes every few minutes."

- 1. Recognize the access pattern: an extremely high read-to-write ratio (millions of reads per minute, but the underlying trending list changes only every few minutes) — a textbook case for aggressive caching.
- 2. Use cache-aside with a TTL: cache the rendered homepage/trending list with a TTL matching the update frequency (e.g., 2-5 minutes), so the cache naturally refreshes on a predictable schedule without needing complex explicit invalidation logic.
- 3. Choose Redis (or a similar in-memory store) for this cache, given the simple key-value nature of 'homepage content,' and pair it with a CDN (Chapter 16) at the edge for further latency reduction, since this content is identical for nearly all users globally.

This demonstrates matching the caching strategy (pattern, eviction/expiration approach, and technology) to the actual read/write characteristics of the specific data — not applying a generic caching answer.

9. Questions YOU Should Ask the Interviewer

- How often does this data actually change, relative to how often it's read?
- How tolerant is this feature of briefly serving slightly stale data?
- Is this data identical for all users, or personalized per user — does that change how we'd structure the cache keys?

10. Scenario-Based Practice Bank (New)

Scenario 1: A product page's cached price still shows the old value for a few minutes after a flash-sale discount was applied in the database.

***How to approach it:** This is a cache invalidation gap: the write path isn't explicitly clearing/updating the cached price. Add explicit invalidation on write for price changes specifically (since this is exactly the kind of fast-changing, high-impact data that TTL-only expiration handles poorly), while leaving less volatile fields on a longer TTL.*

Scenario 2: A cache is filling up with rarely-reused, one-off computation results from a bulk data-export job, crowding out genuinely popular data.

***How to approach it:** This points to using write-around instead of write-through/cache-aside for the export job's writes — bypass the cache for data unlikely to be read again soon, so the cache's limited space stays dedicated to data with real reuse value.*

Scenario 3: An interviewer asks what happens when your Redis cache is full and a new, popular item needs to be cached.

***How to approach it:** Explain the eviction policy: with LRU (the sensible default here), the least-recently-accessed item gets evicted to make room. Mention that LRU is a reasonable approximation for most workloads, but note you'd consider LFU if long-term popularity matters more than recency for this specific dataset.*

Scenario 4: A write-through cache was added to a write-heavy logging table, and now write latency has visibly increased.

***How to approach it:** This is the classic 'wrong pattern for this access profile' mistake: write-through pays synchronous write cost to the cache on every write, which makes no sense for data that's rarely read back soon. Switch to write-around or drop caching entirely for this specific write-heavy, rarely-read table.*

Scenario 5: A cache node crashes and restarts, and now some values that had only been written via write-back caching are missing from the database.

***How to approach it:** This confirms the known risk of write-back (write-behind) caching: an acknowledged write can be lost if the cache fails before the asynchronous database write completes. For any data where this loss is unacceptable, use write-through instead, accepting its higher write latency.*

Scenario 6: A team wants to cache personalized 'recommended for you' data per user, but keys are colliding because everyone's recommendations are stored under a single shared cache key.

***How to approach it:** Point out that personalized data needs per-user cache keys (e.g., 'recommendations:user_id') rather than one shared key — otherwise every user's request overwrites the previous user's cached result, making the cache effectively useless for personalization.*

Chapter 11 — One-Page Revision Sheet

- Caching = a fast, small storage layer holding frequently-accessed data to avoid repeating expensive database reads/computation.
- Patterns: Cache-aside (lazy, on-miss load — most common), Write-through (sync write to both, always fresh, slower writes), Write-back (fast writes, risk of loss), Write-around (bypass cache on write).
- Eviction policies: LRU (most common default), LFU (frequency over time), FIFO (simple, less effective), TTL (natural expiration window).
- Cache invalidation is one of the hardest problems in CS — always explain HOW stale data gets cleared (TTL, explicit invalidation on write, or event-driven) when discussing a cache.
- Redis: rich data structures, persistence, clustering, can double as a message broker. Memcached: simple, fast, pure key-value, no persistence.
- A CDN is the same caching concept applied globally at the network edge for static (and sometimes dynamic) content — full depth in Chapter 16.
- Always pair a caching answer with: what's the eviction policy, and how does invalidation work — these are the two follow-up questions interviewers almost always ask.

***Memory hook:** Keep your most-used spices on the counter (cache), not buried in the pantry (database) — but if you swap out a spice, remember to update the counter too, or you'll cook with the wrong one (stale cache).*

CHAPTER 12

Transactions, Locks & Concurrency Control

Keeping data correct when many things try to change it at the same time

1. Simple Explanation First

When multiple users or processes try to read and write the same data at the same time, things can go wrong in subtle ways — two people might both think a concert ticket is available and both buy it, or a balance update might get partially overwritten. Concurrency control is the set of techniques databases use to make sure simultaneous operations don't corrupt data, even though they're happening at nearly the same instant.

Analogy: *Imagine two people editing the same shared spreadsheet cell at the exact same moment, both typing different numbers. Without any coordination, whichever save happens last just silently overwrites the other — the first person's work vanishes with no warning. Locks and concurrency control are like a 'someone is editing this cell, please wait' indicator — they make sure only one change wins in a controlled, predictable way, instead of a silent, random overwrite.*

2. Why This Topic Exists

Real systems aren't used by one person at a time — thousands or millions of users might be reading and writing the same underlying data simultaneously. Without concurrency control, simultaneous operations can interleave in unpredictable ways, leading to lost updates, inconsistent reads, or outright incorrect data (e.g., double-booking a hotel room, or a bank account balance becoming wrong after two simultaneous withdrawals). This isn't a rare edge case — it's a constant, everyday reality for any system with real concurrent traffic.

Why this matters: *Concurrency bugs are notoriously hard to catch in testing because they often only appear under real production load with genuine simultaneous access — they're exactly the kind of subtle correctness issue interviewers probe for to see if you think about more than just the 'happy path' single-user case.*

3. Core Concepts in Depth

3.1 ACID Recap in the Distributed Context

Chapter 7 introduced ACID for a single database. In distributed systems (multiple databases, multiple shards, multiple services), maintaining these same guarantees becomes significantly harder, because a 'transaction' might now need to span multiple machines that could each independently fail or be slow.

3.2 Optimistic vs Pessimistic Locking

These are two fundamentally different philosophies for handling concurrent access to the same data.

Pessimistic vs Optimistic Locking

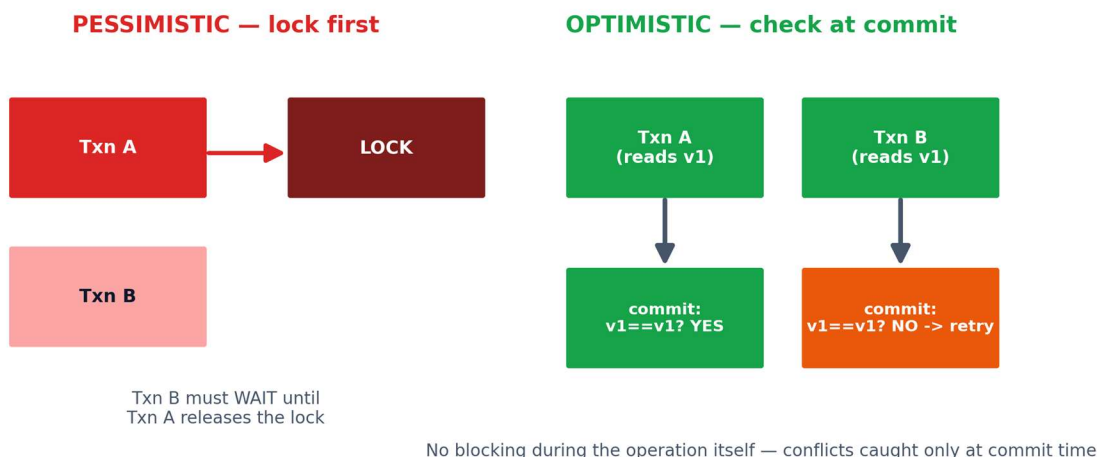


Figure 12.1 — Pessimistic locking blocks and waits; optimistic locking proceeds freely and checks only at commit time.

Pessimistic Locking	Optimistic Locking
Assumes conflicts are likely — locks the data BEFORE making changes, blocking other operations until the lock is released	Assumes conflicts are rare — allows operations to proceed without locking, then checks for conflicts before committing
Other operations must wait for the lock to be released, which can hurt performance under high contention	No waiting/blocking during the operation, but a conflict at commit time means the operation must be retried
Good for: high-contention scenarios where conflicts are frequent and expensive to redo (e.g., booking the last seat on a flight)	Good for: low-contention scenarios where conflicts are rare and retrying is cheap (e.g., most user profile edits)
Implementation: explicit database locks (e.g., SELECT ... FOR UPDATE)	Implementation: version numbers or timestamps checked at commit time (e.g., 'update only if version still matches what I read')

Common mistake: Defaulting to pessimistic locking everywhere 'to be safe.' Pessimistic locks reduce concurrency and can hurt performance significantly at scale, especially when contention is actually low. Optimistic locking is often the better default for most everyday operations, reserving pessimistic locks for genuinely high-contention, high-stakes scenarios.

3.3 Distributed Transactions & Two-Phase Commit (2PC)

When a single logical transaction needs to update data across multiple databases or services (e.g., deduct money from Account A on Database 1, and add it to Account B on Database 2), you need a way to guarantee BOTH updates happen, or NEITHER does — even if one of the databases crashes mid-operation. Two-Phase Commit is a classic protocol for this.

Two-Phase Commit (2PC)

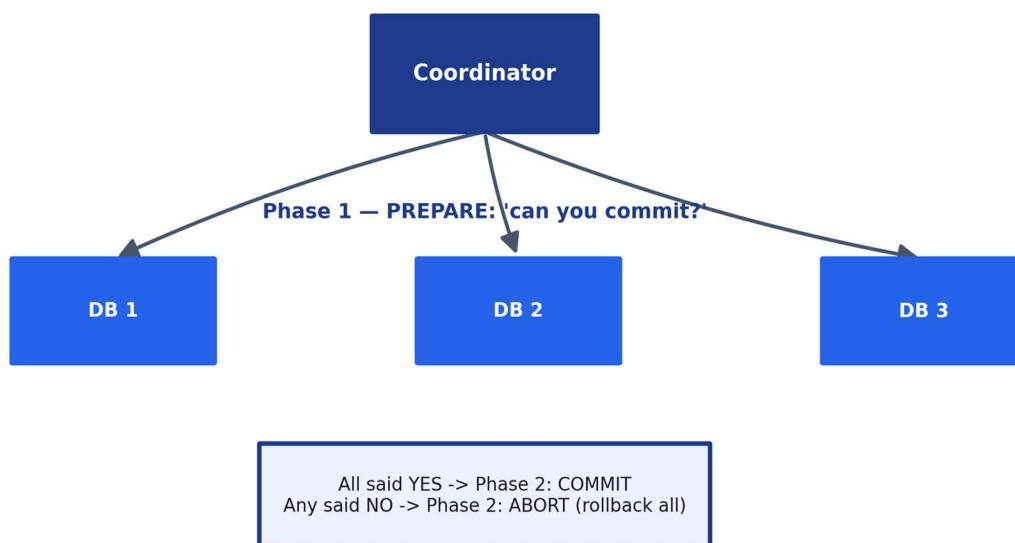


Figure 12.2 — The coordinator asks everyone to prepare before telling anyone to actually commit.

- 1. Phase 1 (Prepare): a coordinator asks all participating databases 'are you ready to commit this change?' Each database checks if it CAN commit (no conflicts, enough resources) and locks the relevant data, responding yes or no, without actually committing yet.
- 2. Phase 2 (Commit/Abort): if ALL participants said yes, the coordinator tells everyone to actually commit. If ANY participant said no (or didn't respond), the coordinator tells everyone to abort/rollback instead.

2PC guarantees atomicity across multiple databases, but at a real cost: it's slow (multiple network round trips), and if the coordinator itself crashes between phases, participants can be left holding locks indefinitely, blocking other operations — this is why many large-scale systems avoid 2PC in favor of patterns like the Saga pattern (covered fully in Chapter 18), which sacrifices strict atomicity for better availability and performance.

Why this matters: Knowing that 2PC exists, roughly how it works, and WHY many modern large-scale systems avoid it in favor of eventual-consistency-based patterns (like Sagas) is a strong signal of architectural maturity — it directly echoes the CAP/PACELC trade-offs from Chapter 5 applied to multi-database transactions.

3.4 Isolation Levels — How Much Can Concurrent Transactions See of Each Other?

Isolation levels define how much one transaction is protected from seeing the in-progress, uncommitted effects of another concurrent transaction. Higher isolation means stronger correctness guarantees, but typically lower concurrency/performance.

Isolation Level	What It Prevents
Read Uncommitted	Almost nothing — transactions can see other transactions' uncommitted changes ('dirty reads'). Rarely used in practice.
Read Committed	Prevents dirty reads — you only ever see data that's been committed, but the same query run twice in one transaction might see different results if another transaction commits in between.
Repeatable Read	Prevents dirty reads AND ensures that if you read the same row twice in one transaction, you get the same result — but new rows matching your query might still appear ('phantom reads').
Serializable	The strongest level — transactions behave as if they ran one at a time, completely isolated. Prevents all anomalies, but at the cost of the most reduced concurrency/performance.

You don't need to memorize every anomaly by name for most interviews — what matters is understanding that isolation is a SPECTRUM, and that choosing a stricter level trades away performance/concurrency for stronger correctness guarantees, mirroring the consistency-vs-performance theme that runs throughout this entire book.

4. Trade-offs Table

Decision	Pros	Cons
Pessimistic locking	Prevents conflicts proactively, simpler to reason about correctness	Reduces concurrency; risk of deadlocks; can hurt performance under load
Optimistic locking	Higher concurrency/throughput, no blocking during normal operation	Conflicts discovered late (at commit), requiring retries under high contention
Two-Phase Commit for distributed transactions	Guarantees atomicity across multiple databases	Slow (multiple network round trips); coordinator failure can leave system blocked

5. How Real Companies Use This

- Ticketmaster and airline booking systems use pessimistic locking (or equivalent mechanisms) for the final seat/ticket reservation step specifically, since double-booking the literal last seat is a costly, high-visibility failure worth the performance trade-off.
- Most e-commerce platforms use optimistic locking for general product/cart updates (low contention, cheap to retry), reserving stricter locking only for the actual checkout/payment/inventory-deduction step.
- Modern large-scale companies (Uber, Netflix, Amazon) largely avoid traditional 2PC across services in favor of the Saga pattern and eventual consistency, explicitly accepting more complex application-level compensation logic in exchange for not blocking on slow, fragile distributed locks.

6. Common Mistakes & Misconceptions

- Assuming 'transactions' only matter within a single database — in distributed systems with sharding (Chapter 10) and microservices (Chapter 18), maintaining transactional correctness across boundaries is a much harder, ongoing design challenge.
- Treating Serializable isolation as the obvious 'safest, so always best' choice without acknowledging its real performance cost — the right isolation level depends on the specific correctness needs of the data involved.
- Forgetting that pessimistic locks can cause deadlocks (two transactions each waiting on a lock the other holds) — a real operational concern that needs timeout/detection mechanisms.

7. Interviewer Q&A Bank

Q1. What's the difference between optimistic and pessimistic locking?

A: Pessimistic locking assumes conflicts are likely and locks data upfront before making changes, blocking other operations until the lock releases — safer under high contention but reduces concurrency. Optimistic locking assumes conflicts are rare, lets operations proceed without locking, and checks for conflicts only at commit time (often via a version number), retrying if a conflict is found — better throughput when contention is genuinely low.

Q2. How would you guarantee two people don't both successfully book the last seat on a flight?

A: I'd use pessimistic locking on that specific seat record during the booking transaction — the first request to reach the seat locks it, performs the booking, and releases the lock; the second concurrent request either waits and then sees the seat is taken, or is rejected immediately. Given this is low-frequency but high-stakes (a double-booking is a real business and trust problem), the performance cost of locking is well justified here, even though I'd use optimistic concurrency for lower-stakes operations elsewhere in the same system.

Q3. Explain Two-Phase Commit and why some systems avoid it.

A: 2PC coordinates a transaction across multiple databases in two steps: first, a 'prepare' phase where all participants confirm they CAN commit and lock their data; second, a 'commit' phase where the coordinator tells everyone to actually commit, only if all participants agreed. It guarantees atomicity across databases, but it's slow due to multiple network round trips, and if the coordinator crashes between phases, participants can be left holding locks indefinitely. Many modern large-scale systems avoid it in favor of patterns like Sagas, trading strict atomicity for better availability and performance.

Q4. What are isolation levels, and why wouldn't you always use the strictest one?

A: Isolation levels define how much one transaction can be affected by another concurrent transaction's in-progress changes, ranging from Read Uncommitted (weakest) to Serializable (strongest, fully isolated). The strictest level, Serializable, prevents all concurrency anomalies but does so by significantly reducing how many transactions can run truly in parallel, hurting throughput. The right level depends on how much correctness risk a specific piece of data can tolerate versus how much concurrency/performance the system needs.

Q5. What is a deadlock, and how can it happen with pessimistic locking?

A: A deadlock happens when two (or more) transactions each hold a lock the other one needs, and each is waiting for the other to release it — neither can proceed, and without intervention, they'd wait forever. Databases typically detect this and resolve it by aborting one of the transactions (forcing a

retry). It's a real operational risk of pessimistic locking that needs to be accounted for, often by being consistent about the order in which locks are acquired.

8. HLD Mini-Scenario: Concurrency Control for a Flash Sale

Prompt: "Design the inventory-checkout flow for a flash sale where 100,000 users might try to buy the same limited-stock item (only 500 units) within the same few seconds."

- 1. Recognize this as a genuinely high-contention scenario — many concurrent attempts to decrement the same limited counter, where overselling (more sales than stock) is a costly, visible failure.
- 2. Use a controlled, atomic decrement operation (e.g., a database-level atomic 'decrement if greater than zero' operation, or a distributed lock/semaphore on the inventory counter) rather than naive read-then-write logic, which would be vulnerable to race conditions under this volume.
- 3. Consider a queue-based approach (foreshadowing Chapter 14) in front of the actual inventory decrement — letting users 'get in line' and processing purchase attempts sequentially against the inventory counter, rather than allowing 100,000 simultaneous direct database hits, which also protects the database from being overwhelmed.

This shows how concurrency control decisions connect directly to capacity and architecture decisions — the right answer here isn't just 'use a lock,' it's recognizing the contention pattern and designing the surrounding system (queueing) to make that lock's job manageable.

9. Questions YOU Should Ask the Interviewer

- How frequently do we expect real conflicts/contention on this specific piece of data?
- What's the cost of a conflict being resolved incorrectly here — is overselling/double-booking acceptable in any way, or strictly forbidden?
- Does this transaction need to span multiple services/databases, or can it be contained within one?

10. Scenario-Based Practice Bank (New)

Scenario 1: A user profile edit feature uses pessimistic locking, and users occasionally see 'please wait, this record is locked' errors even though almost no one edits the same profile simultaneously.

How to approach it: This is over-engineering for the actual contention level: profile edits are low-contention. Switch to optimistic locking (a version number checked at commit) so normal edits proceed without blocking, and conflicts — genuinely rare here — are handled with a retry instead of upfront locking.

Scenario 2: A distributed transaction spanning three microservices uses 2PC, and once a month the coordinator crashes mid-transaction, leaving all three services holding locks for hours.

How to approach it: This is the well-known 2PC coordinator-failure risk. Recommend migrating this flow to a Saga pattern (Chapter 18): break the transaction into a sequence of local transactions with compensating actions for rollback, trading strict atomicity for much better availability and no indefinite lock-holding.

Scenario 3: Two transactions each need a lock the other is currently holding, and both are stuck waiting indefinitely.

How to approach it: This is a deadlock. Most databases detect this automatically and abort one transaction for retry — make sure the application has retry logic for this case, and where possible, standardize the order in which locks are acquired across the codebase to reduce how often deadlocks occur in the first place.

Scenario 4: An interviewer asks why you wouldn't just always use Serializable isolation to be safe from every possible concurrency bug.

How to approach it: Explain the trade-off directly: Serializable prevents all anomalies but does so by drastically reducing how many transactions can truly run in parallel, hurting throughput. The right level depends on the data's actual correctness needs — many parts of a system are fine with Read Committed, reserving Serializable for genuinely correctness-critical operations.

Scenario 5: A ticket-booking flow currently reads the remaining seat count, checks if it's greater than zero in application code, then writes a decremented value back — and double-bookings started appearing under load.

How to approach it: This is a classic read-then-write race condition: two requests can both read the same 'seats available' number before either writes back. Replace it with an atomic, database-level conditional decrement (or a proper lock on that row) so the check-and-decrement happens as one atomic operation instead of two separate steps.

Scenario 6: A checkout flow needs to charge a customer's card (via an external payment service) and decrement inventory in the same database — and the team is debating whether to wrap both in a single ACID transaction.

How to approach it: Point out that an external payment service call can't participate in a local database ACID transaction — it's a separate system. Recommend a pattern like: reserve inventory locally (a local transaction), call the payment service, then confirm or release the reservation based on the payment result, with compensating logic if the payment fails after inventory was reserved — the Saga-style approach rather than trying to force ACID across a boundary it can't cover.

Chapter 12 — One-Page Revision Sheet

- Concurrency control prevents simultaneous reads/writes from corrupting data — essential anytime more than one user/process can touch the same data at once.
- Pessimistic locking: lock upfront, block others, safer under high contention, reduces concurrency. Optimistic locking: no upfront lock, check for conflict at commit, retry if conflicted — better for low contention.
- Two-Phase Commit (2PC): Prepare phase (can everyone commit?) then Commit/Abort phase — guarantees atomicity across multiple databases but is slow and risky if the coordinator fails. Many modern systems prefer Sagas instead (Chapter 18).
- Isolation levels (weakest to strongest): Read Uncommitted -> Read Committed -> Repeatable Read -> Serializable. Stronger isolation = stronger correctness, but less concurrency/performance.
- Deadlocks happen when transactions wait on each other's locks indefinitely — databases detect and resolve these by aborting one transaction for retry.
- Match the locking strategy to actual contention: pessimistic for rare-but-critical high-contention moments (last seat, flash sale), optimistic for everyday low-contention operations.
- For very high-contention scenarios (flash sales), pair locking/atomic operations with a queueing layer to protect the database from being overwhelmed by simultaneous direct hits.

Memory hook: *Two people editing the same spreadsheet cell at once — without a 'someone's editing this' lock, the last save silently wins and the other person's work just vanishes.*

CHAPTER 13

Load Balancing

Spreading traffic across many servers so no single one gets overwhelmed

1. Simple Explanation First

A load balancer is a component that sits between clients and your servers, deciding which server should handle each incoming request. Instead of every user hitting the same single server, the load balancer spreads requests across a pool of servers, so no single machine becomes a bottleneck.

Analogy: Think of a load balancer like the host at the entrance of a busy restaurant with multiple identical dining sections. Instead of every customer walking straight to the same section (overcrowding it while others sit empty), the host looks at which sections have free tables and directs each new customer accordingly. The customer doesn't need to know or care which section they end up in — they just want a table, fast.

2. Why This Topic Exists

Once you've decided to scale horizontally (Chapter 6) — running multiple identical servers instead of one giant one — you need something to actually decide which server handles each incoming request. Without a load balancer, you'd need to give clients a list of server addresses and have them somehow pick one themselves, which is fragile, doesn't adapt to server failures, and doesn't balance load fairly. The load balancer is the missing piece that makes horizontal scaling actually usable.

Why this matters: A load balancer is one of the first components mentioned in almost every HLD answer, because it's the literal entry point that makes everything else (multiple servers, fault tolerance, scaling) possible. Interviewers expect it to appear early in your architecture diagram, right after the client.

3. Core Concepts in Depth

3.1 Layer 4 vs Layer 7 Load Balancing

Load balancers operate at different layers of the network stack, and the layer determines how much information they can use to make routing decisions.

Layer 4 vs Layer 7 Load Balancing

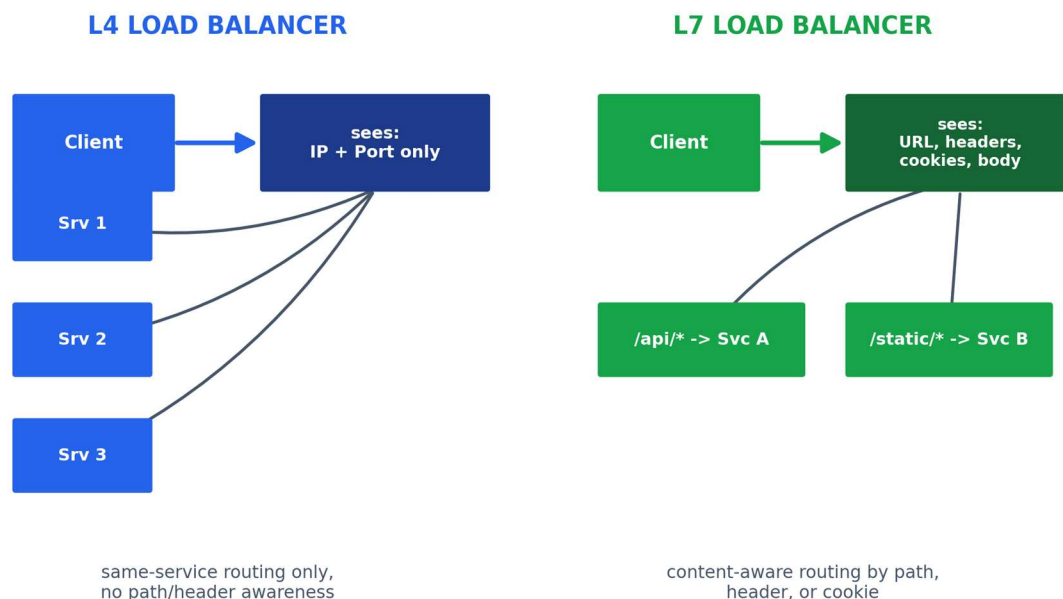


Figure 13.1 — L4 load balancers see only IP/port; L7 load balancers read the full HTTP request.

Layer 4 (Transport Layer)	Layer 7 (Application Layer)
Routes based on IP address and port only — doesn't look inside the actual request content	Routes based on the actual HTTP request — URL path, headers, cookies, even request body
Faster, since it does minimal inspection of each packet	Slower (relatively), since it must parse the full HTTP request
Can't make 'smart' routing decisions like 'send /api/video requests to video servers'	Can make smart, content-aware routing decisions
Good for: simple, very high-throughput routing needs	Good for: routing different parts of an application to different specialized services

Common mistake: Assuming load balancers are interchangeable — an L4 load balancer literally cannot read a URL path or a cookie to make a routing decision, since it operates below the HTTP layer entirely. If a design needs path-based or header-based routing (e.g., routing /api/* differently from /static/*), it must use an L7 load balancer.

3.2 Load Balancing Algorithms

Once the load balancer knows it needs to pick a server, it needs a rule for HOW to pick. Different algorithms suit different traffic patterns.

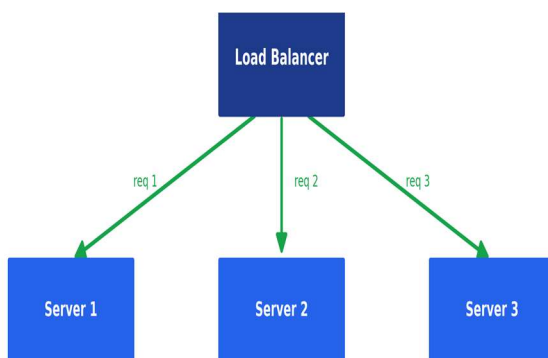


Figure 13.2 — Round robin cycles through servers in a fixed, repeating order: request 1 → S1, request 2 → S2, request 3 → S3, request 4 → S1, and so on.

Algorithm	How It Works	Best Fit
Round Robin	Cycles through servers in order: S1, S2, S3, S1, S2...	Simple, works well when all servers and requests are roughly equal in capacity/cost
Least Connections	Sends the next request to whichever server currently has the fewest active connections	Better when requests vary significantly in how long they take to process
Consistent Hashing	Routes based on a hash of some request attribute (e.g., user ID), so the same client tends to reach the same server	Useful when you want session affinity or to maximize cache hits on a specific server
Weighted Round Robin	Like round robin, but more powerful servers receive proportionally more requests	Useful when your server pool has machines of different capacities

3.3 Health Checks

A load balancer is only useful if it sends traffic to servers that are actually healthy and able to respond. Health checks are periodic probes (e.g., an HTTP request to a /health endpoint every few seconds) that the load balancer uses to determine whether each server in its pool is alive and ready to handle traffic. If a server fails its health check, the load balancer stops routing new requests to it until it recovers — this is a core part of how systems achieve fault tolerance (covered fully in Chapter 19).

- Active health checks: the load balancer proactively pings each server on a schedule.
- Passive health checks: the load balancer observes real traffic and marks a server unhealthy if it starts returning errors or timing out.

Why this matters: Mentioning health checks unprompted, right after introducing a load balancer, is a strong signal — it shows you understand a load balancer isn't just a traffic splitter, it's also the first line of defense against routing users to a broken server.

3.4 Sticky Sessions

Sometimes an application needs a specific user's requests to consistently land on the same server — for example, if session data is stored in that server's local memory rather than externalized to a shared store (recall the statelessness discussion in Chapter 6). Sticky sessions (also called session affinity) configure the

load balancer to route a given client to the same backend server for the duration of their session, often using a cookie to track this.

Common mistake: Reaching for sticky sessions as a default solution instead of making services stateless. Sticky sessions reintroduce the exact coupling problem statelessness was designed to avoid — if that one server goes down, that user's session data goes down with it, and the load balancer's freedom to balance load evenly is reduced. Sticky sessions should be a deliberate exception, not a habit.

3.5 Single Point of Failure — and How Load Balancers Avoid Being One

It might seem like the load balancer itself is now a single point of failure — if it goes down, no traffic gets through at all, no matter how many healthy servers sit behind it. Real production systems solve this by running multiple load balancer instances themselves, often using DNS round robin or a floating IP / virtual IP technique to fail over between them, sometimes coordinated by a separate health-checking mechanism. The load balancer layer is designed to be redundant for the exact same reasons your application servers are.

4. Trade-offs Table

Decision	Pros	Cons
L7 load balancing	Smart, content-aware routing (path, headers, cookies)	More CPU overhead, slightly higher latency per request
Least connections algorithm	Adapts well to uneven request processing times	Slightly more bookkeeping overhead than simple round robin
Sticky sessions	Simple way to support stateful servers without externalizing session storage	Reduces load-balancing flexibility; user loses session if their server fails

5. How Real Companies Use This

- Amazon's Elastic Load Balancer (ELB) and Google Cloud Load Balancing offer both L4 and L7 options, letting engineering teams pick based on whether they need simple high-throughput routing or content-aware routing.
- Netflix runs load balancing at multiple layers — DNS-based geographic routing to the nearest region, then L7 load balancers within a region routing to specific microservices based on the request path.
- Most companies running stateful real-time features (like WebSocket-based chat, from Chapter 3) use sticky sessions or a pub-sub backend specifically to handle the tension between statelessness and persistent connections.

6. Common Mistakes & Misconceptions

- Forgetting to mention health checks when describing a load balancer — interviewers often probe with 'what happens if one of the servers crashes?' expecting this answer.
- Treating the load balancer as a single, unscalable component — in reality, load balancers themselves need redundancy, just like application servers.
- Confusing load balancing with API gateways (Chapter 15) — a load balancer's core job is distributing traffic across replicas of the SAME service, while an API gateway routes to DIFFERENT

services and handles cross-cutting concerns like auth. In practice the two are often layered together, but they solve different problems.

7. Interviewer Q&A Bank

Q1. What's the difference between Layer 4 and Layer 7 load balancing?

A: Layer 4 load balancing routes based only on IP address and port, without inspecting the actual request content — it's fast but can't make content-aware decisions. Layer 7 load balancing operates at the application layer, reading the full HTTP request including the URL path, headers, and cookies, allowing smarter routing decisions like sending different paths to different backend services, at the cost of slightly more processing overhead per request.

Q2. How does a load balancer know if a server is healthy?

A: Through health checks — either active checks, where the load balancer periodically pings a health endpoint on each server, or passive checks, where it monitors real traffic and flags a server as unhealthy if it starts erroring or timing out. Once a server is marked unhealthy, the load balancer stops routing new traffic to it until it recovers, which is a key part of the system's overall fault tolerance.

Q3. When would you use the least-connections algorithm instead of round robin?

A: When requests vary significantly in how expensive or long-running they are to process. Round robin assumes each request takes roughly the same effort, which can lead to one server being overloaded with several slow requests while another sits idle. Least connections adapts to this by always routing to whichever server currently has the fewest active connections, balancing actual load rather than just request count.

Q4. Isn't the load balancer itself a single point of failure?

A: It can be if deployed naively as a single instance, but production systems run multiple load balancer instances with their own failover mechanism — often DNS-based routing or a floating/virtual IP that can shift to a healthy load balancer instance if one fails. The load balancer layer is designed to be redundant using the same fault-tolerance principles applied to application servers.

Q5. When would you need sticky sessions, and what's the trade-off?

A: Sticky sessions are needed when a server holds session state locally in memory rather than in an external shared store, and a user's subsequent requests must reach that same server to access their session data. The trade-off is that it reduces the load balancer's flexibility to balance traffic evenly, and if that specific server fails, the user loses their session — which is why making services stateless (storing session data externally) is generally the preferred long-term solution over relying on sticky sessions.

8. HLD Mini-Scenario: Load Balancer Choice for a Video Platform

Prompt: "Design the load balancing layer for a platform that serves both lightweight API requests (e.g., 'get video metadata') and heavy video-streaming requests, from two different specialized backend services."

- 1. Choose L7 load balancing, since we explicitly need content-aware routing: requests to `/api/*` should go to the lightweight metadata service, while requests to `/stream/*` should go to the video-streaming service — this is impossible with L4 alone.

- 2. Apply least-connections (or a similar load-aware algorithm) for the video-streaming service specifically, since streaming requests vary wildly in duration and resource usage, unlike short-lived metadata API calls which could reasonably use simple round robin.
- 3. Ensure both services have independent health checks, since a failure in the streaming service shouldn't affect routing decisions for the unrelated metadata service.

This shows how the choice of load balancer layer and algorithm should be driven by the actual shape of the traffic, not applied as a one-size-fits-all default.

9. Questions YOU Should Ask the Interviewer

- Do different request types need to be routed to different backend services, or is this a single homogeneous service pool?
- Do we need session affinity for any part of this system, or can everything be designed stateless?
- Are there latency-sensitive paths where L4's lower overhead would matter more than L7's smarter routing?

10. Scenario-Based Practice Bank (New)

Scenario 1: A design needs to route `/api/*` requests to a metadata service and `/video/*` requests to a streaming service, but the team currently uses an L4 load balancer.

How to approach it: Flag that this is impossible with L4 alone, since it can only see IP/port, not the URL path. Upgrade to an L7 load balancer, which can inspect the actual HTTP request and route based on path — accept the modest added latency in exchange for the content-aware routing the design requires.

Scenario 2: One backend server keeps getting far more slow, resource-heavy requests than the others under round robin, and its response times are visibly degrading.

How to approach it: Round robin assumes every request costs about the same, which isn't true here. Switch to a least-connections algorithm so the load balancer actively favors servers with fewer in-flight requests, adapting to the uneven request cost rather than blindly rotating.

Scenario 3: A server silently stopped responding to new connections an hour ago, but the load balancer is still sending it a share of traffic, causing user-facing errors.

How to approach it: This points to missing or misconfigured health checks. Add active health checks (periodic pings to a `/health` endpoint) so the load balancer can detect and stop routing to unresponsive servers automatically, rather than relying only on user complaints to notice.

Scenario 4: A team wants to add sticky sessions to fix a bug where users occasionally get logged out mid-session.

How to approach it: Investigate why session state isn't surviving — the actual bug is likely that session data lives only in one server's memory. Recommend externalizing session state to a shared store (Redis, a database) instead of defaulting to sticky sessions, which would only mask the coupling problem and still lose sessions entirely if that one server fails.

Scenario 5: An interviewer points out that the load balancer in your diagram is drawn as a single box and asks what happens if it goes down.

How to approach it: Acknowledge this directly: production systems run multiple load balancer instances with a failover mechanism (DNS round robin or a floating/virtual IP), so the load balancer layer itself is redundant — the same fault-tolerance principle applied to application servers, just one layer earlier in the request path.

Scenario 6: A server pool has one much more powerful machine (16 cores) alongside several smaller ones (4 cores), and simple round robin is sending them all equal traffic.

How to approach it: Recommend weighted round robin instead, assigning the more powerful server a proportionally higher share of requests so capacity is used efficiently, rather than treating all servers as equally capable under plain round robin.

Chapter 13 — One-Page Revision Sheet

- A load balancer distributes incoming requests across a pool of servers, making horizontal scaling (Chapter 6) actually usable.
- L4 (Transport): routes by IP/port only, fast, no content awareness. L7 (Application): reads full HTTP request (path/headers/cookies), smarter but slower.
- Algorithms: Round Robin (simple rotation), Least Connections (adapts to uneven load), Weighted Round Robin (accounts for different server capacities), Consistent Hashing (session affinity).
- Health checks (active or passive) let the load balancer stop routing to unhealthy servers — always mention this when describing a load balancer.
- Sticky sessions route a user consistently to the same server — useful for stateful servers, but reduces balancing flexibility; statelessness is the better long-term fix.
- The load balancer itself needs redundancy (multiple instances + failover) — it is not exempt from the single-point-of-failure problem.
- Load balancer != API gateway: a load balancer spreads traffic across replicas of the SAME service; a gateway routes to DIFFERENT services and handles cross-cutting concerns (Chapter 15).

Memory hook: A restaurant host directing customers to open tables across identical sections — they don't care which section, they just want fast service, and the host adapts if one section gets backed up.

CHAPTER 14

Message Queues & Pub-Sub

Letting services talk without waiting on each other

1. Simple Explanation First

A message queue is a buffer that sits between two services, letting one service (the producer) drop off work without waiting for the other service (the consumer) to process it immediately. Pub-Sub (publish-subscribe) extends this idea further: one event can be delivered to MANY independent subscribers, not just one.

Analogy: A message queue is like dropping a letter in a mailbox instead of waiting on someone's doorstep until they personally answer. You drop it off and walk away immediately — the recipient picks it up and deals with it on their own schedule. Pub-Sub is like a notice board in a building lobby: you post one notice once, and every resident who cares to check the board sees it independently, without you needing to hand a copy to each person individually.

2. Why This Topic Exists

Chapter 4 introduced the idea that not every step in a system needs to be synchronous — some work can happen in the background without blocking the user. Message queues are the concrete infrastructure that makes this possible. They exist to decouple services in time (the producer and consumer don't need to be available at the exact same moment) and in failure (if the consumer is temporarily down, messages just wait in the queue instead of being lost or blocking the producer).

Why this matters: Message queues are one of the most common 'how would you make this more resilient/scalable' answers in HLD interviews. Any time you spot a step that doesn't need an immediate response — sending an email, processing a video, updating analytics — introducing a queue is usually a strong, expected improvement to suggest.

3. Core Concepts in Depth

3.1 Why Asynchronous Messaging? (Visualizing the Decoupling)

Synchronous Call vs Message Queue

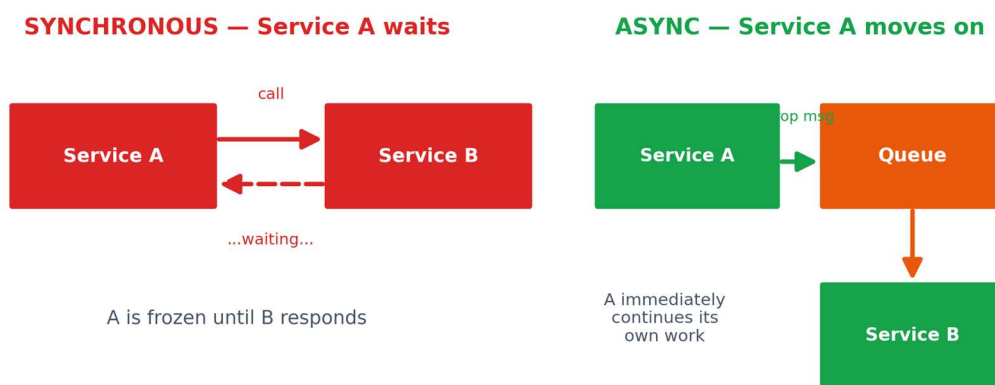


Figure 14.1 — Synchronous calls block the caller; a queue lets the caller move on immediately.

The core value of a queue is visible in this diagram: in the synchronous case, Service A is frozen, doing nothing useful, while it waits for Service B. In the asynchronous case, Service A drops its message in the queue and immediately moves on to other work, fully trusting that Service B will eventually process it. This single shift — from waiting to trusting — is what allows systems to remain responsive even when downstream services are slow or temporarily unavailable.

3.2 Kafka vs RabbitMQ vs SQS — The Big Three

These three are the most commonly referenced messaging technologies in interviews. You don't need implementation-level expertise, but knowing their core philosophical differences is high-value.

Technology	Core Model	Best Fit
Kafka	Distributed log — messages are appended to a durable, ordered log and can be replayed by multiple independent consumer groups	High-throughput event streaming, analytics pipelines, when you need to replay/reprocess historical events
RabbitMQ	Traditional message broker — messages are routed via exchanges to queues, typically consumed once and removed	Complex routing logic, task queues, when you need fine-grained delivery guarantees per message
Amazon SQS	Fully managed, simple cloud queue service — minimal setup, but fewer advanced routing/streaming features	Simple, reliable task queues in AWS-based systems without wanting to operate the messaging infrastructure yourself

A useful mental shortcut: Kafka behaves more like a durable, replayable log (good for streaming and analytics); RabbitMQ and SQS behave more like a traditional task queue (good for 'do this job once'). This distinction matters more than memorizing exact feature lists.

3.3 Delivery Guarantees

When a message is sent through a queue, what's the actual guarantee that the consumer will process it — and process it only once? This is a frequently misunderstood area, and a precise answer here is a strong interview signal.

Guarantee	What It Means
At-most-once	A message is delivered zero or one times — it might be lost, but it will NEVER be processed twice. Fastest, but risks silent message loss.
At-least-once	A message is guaranteed to be delivered one or more times — it will never be silently lost, but the consumer might process the same message twice (e.g., if an acknowledgment is lost after processing but before confirming).
Exactly-once	A message is delivered and processed exactly one time, no more, no less — the ideal, but genuinely hard to guarantee in a distributed system and often requires additional coordination/deduplication logic.

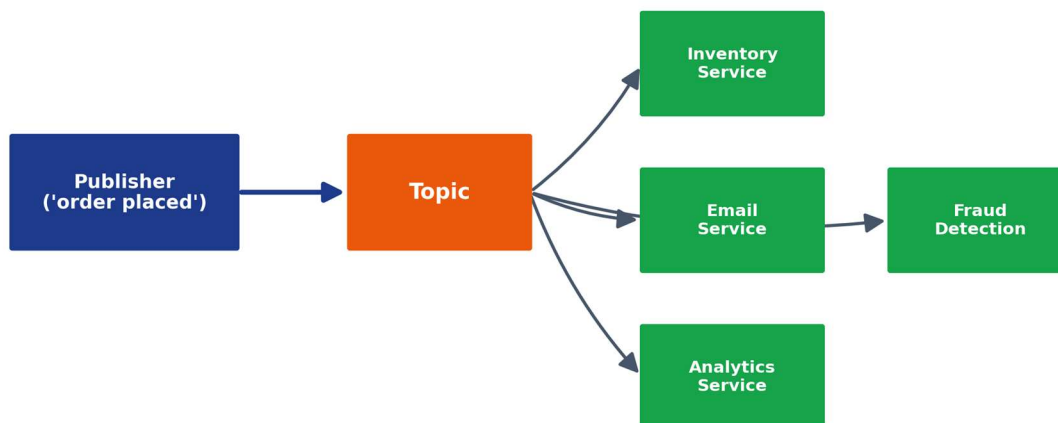
Common mistake: Claiming a system uses 'exactly-once' delivery without acknowledging the real complexity behind that guarantee. In practice, most real systems use at-least-once delivery combined with idempotent consumers (processing the same message twice has no extra effect, e.g., 'set status to PAID' rather than 'add \$10 to balance') to achieve effectively-exactly-once behavior at the application level — saying this explicitly shows real depth.

3.4 Dead-Letter Queues

Sometimes a message simply cannot be processed successfully — maybe it's malformed, or it repeatedly fails due to a bug. Without special handling, a consumer might retry that same poisoned message forever, blocking progress on every message behind it. A dead-letter queue (DLQ) is a separate queue where messages get moved after failing processing some maximum number of times, so they can be inspected and handled separately (manually reviewed, logged, or retried later) without blocking the main queue.

3.5 The Fan-Out Pattern (Pub-Sub in Action)

Pub-Sub Fan-Out



Each subscriber processes the SAME event independently — add a new one without touching the publisher.

Figure 14.2 — One published event reaches every independent subscriber.

Fan-out is what separates Pub-Sub from a simple point-to-point queue. In a simple queue, one message is typically consumed by exactly one consumer (useful for distributing work across a pool of workers). In Pub-Sub, one published event can be delivered to MANY independent subscribers, each maintaining its own position/copy — this is the pattern that lets you add a brand-new service (e.g., a new fraud-detection system) that reacts to 'order placed' events without ever touching the original order-placing code.

Why this matters: The fan-out pattern is exactly why event-driven architectures are praised for extensibility — new functionality can be added by simply subscribing to existing events, with zero changes to the producer. This is a recurring theme you'll see again in Chapter 18 (microservices) as a way to decouple services from each other.

4. Trade-offs Table

Decision	Pros	Cons
Using a message queue for background work	Decouples services, improves resilience, keeps user-facing requests fast	Added infrastructure complexity; eventual consistency for the deferred work
At-least-once delivery + idempotent consumers	Never silently loses messages, practical to implement	Requires extra application-level logic to handle duplicate processing safely
Pub-Sub fan-out architecture	Highly extensible — new subscribers can be added without touching producers	Harder to trace a single request's full effect across many independent subscribers

5. How Real Companies Use This

- LinkedIn created Kafka originally to handle its massive internal stream of activity events (views, clicks, connections) feeding into many downstream analytics and recommendation systems.
- Uber uses message queues extensively to decouple ride-matching, payments, and notifications — a driver's location update doesn't need to wait for the payment system to be available.
- E-commerce platforms commonly use the fan-out pattern for 'order placed' events: inventory, email confirmation, analytics, and fraud detection all subscribe independently, so adding a new feature (like loyalty points) just means adding a new subscriber, not modifying the checkout code.

6. Common Mistakes & Misconceptions

- Using a queue for something that genuinely needs an immediate response — e.g., queuing a login authentication check would add unacceptable latency and complexity to a flow that needs a synchronous answer.
- Assuming messages are always processed in the exact order they were sent — many distributed queue systems only guarantee ordering within a single partition/shard, not globally, which can surprise candidates who assume strict global ordering.
- Forgetting that consumers need to be idempotent under at-least-once delivery — a common interview follow-up is 'what if this message gets delivered twice?'

7. Interviewer Q&A Bank

Q1. When would you introduce a message queue into a system design?

A: Whenever a step in the workflow doesn't need to block the user's immediate response — like sending a confirmation email, generating a thumbnail, or updating analytics after an action. I'd put that step behind a queue so the user-facing request returns quickly, while the actual work happens asynchronously, and so a temporary slowdown or outage in that background service doesn't impact the main user flow.

Q2. What's the difference between a simple queue and Pub-Sub?

A: In a simple point-to-point queue, a message is typically consumed by exactly one consumer, which is useful for distributing units of work across a pool of workers. In Pub-Sub, one published event can be delivered to many independent subscribers, each processing it for their own purpose — this fan-out pattern is what enables loosely-coupled, easily extensible event-driven architectures.

Q3. Explain at-least-once vs exactly-once delivery, and which is more realistic in practice.

A: At-least-once guarantees a message won't be silently lost but might be delivered and processed more than once. Exactly-once guarantees precisely one delivery and processing, which sounds ideal but is genuinely hard to achieve in a distributed system due to network failures and partial acknowledgments. In practice, most real systems use at-least-once delivery combined with idempotent consumer logic — designing the processing so handling the same message twice causes no extra effect — to achieve effectively-exactly-once behavior without the full complexity of true exactly-once guarantees.

Q4. What is a dead-letter queue and why is it useful?

A: A dead-letter queue is a separate queue where messages get moved after repeatedly failing processing, instead of being retried forever and blocking the messages behind them. It lets engineers

inspect and handle problematic messages separately — logging them, alerting on them, or manually reprocessing them — without stalling the rest of the queue's throughput.

Q5. Would you choose Kafka or a simpler queue like SQS for a notification system that sends a push notification after a user action?

A: For a straightforward 'do this job once' notification task, I'd lean toward a simpler managed queue like SQS, since it's easier to operate and the notification doesn't need replayability or multiple independent consumer groups reading the same stream. I'd reach for Kafka instead if I needed to support multiple different systems consuming the same event stream independently (e.g., notifications AND analytics AND a fraud system all reading the same 'user action' events) or needed to replay historical events.

8. HLD Mini-Scenario: Async Workflow for Order Processing

Prompt: "Design the order-processing flow for an e-commerce checkout: payment must be confirmed synchronously, but inventory updates, confirmation emails, and analytics tracking should not slow down the checkout response."

- 1. Keep payment processing synchronous: the user needs an immediate yes/no on whether their payment succeeded before the checkout flow can be considered complete — this cannot be deferred to a queue.
- 2. Once payment succeeds, publish an 'order placed' event to a Pub-Sub topic, and return success to the user immediately — the checkout response no longer waits on anything downstream.
- 3. Inventory service, email service, and analytics service all independently subscribe to the 'order placed' event and process it on their own schedule, each with their own retry logic and dead-letter queue for failures.

This demonstrates the core skill of this chapter: identifying exactly which steps genuinely need a synchronous response, and which can be pushed into an async, fan-out flow — and explaining why the line falls where it does.

9. Questions YOU Should Ask the Interviewer

- Which steps in this flow absolutely require an immediate response, versus which can happen in the background?
- Do we need strict message ordering, or is eventual, possibly out-of-order processing acceptable?
- How should we handle a message that repeatedly fails to process — what's the expected behavior?

10. Scenario-Based Practice Bank (New)

Scenario 1: A login endpoint was recently changed to publish an authentication event to a queue and wait for a consumer to validate credentials before responding to the user.

How to approach it: *This is a misuse of async messaging: login needs an immediate synchronous answer, and queuing it adds unacceptable latency and unnecessary complexity. Revert this specific step to a direct synchronous call, reserving queues for steps that genuinely don't need to block the user.*

Scenario 2: A payment confirmation email is sometimes sent twice to the same customer.

How to approach it: This is the expected behavior of at-least-once delivery without an idempotent consumer. Fix it at the consumer level: make the email-sending logic idempotent (e.g., check if a confirmation was already sent for this order ID before sending again) rather than assuming the queue will never redeliver a message.

Scenario 3: A new fraud-detection team wants to react to 'order placed' events without modifying the existing checkout code or slowing it down.

How to approach it: This is exactly what the Pub-Sub fan-out pattern is for: have the fraud-detection service subscribe independently to the existing 'order placed' topic. The checkout/producer code needs zero changes, and the new subscriber processes events on its own schedule without affecting checkout latency.

Scenario 4: A consumer keeps crashing on one specific malformed message, and every message behind it in the queue is now stuck waiting.

How to approach it: This is a textbook dead-letter queue situation: after a small number of failed processing attempts, move that poisoned message to a DLQ for separate inspection, so the rest of the queue keeps flowing instead of being blocked indefinitely by one bad message.

Scenario 5: A team assumes their Kafka-based pipeline delivers messages in the exact global order they were produced, and a subtle bug appears when messages from different partitions arrive 'out of order.'

How to approach it: Correct the assumption: most distributed queue systems (including Kafka) only guarantee ordering within a single partition, not globally across all partitions. If strict ordering matters for related events, ensure they're produced to the same partition (e.g., partitioned by a relevant key like user ID or order ID).

Scenario 6: A team building a simple 'send one push notification after signup' feature is debating whether to invest in a Kafka cluster.

How to approach it: Recommend against it: this is a straightforward 'do this job once' task with no need for replayability or multiple independent consumer groups reading the same stream. A simpler managed queue like SQS fits better and avoids the operational overhead of running Kafka for a task that doesn't need its specific strengths.

Chapter 14 — One-Page Revision Sheet

- A message queue decouples producer and consumer in time and failure — the producer drops a message and moves on; the consumer processes it independently.
- Kafka: durable, replayable log, great for streaming/analytics with multiple independent consumer groups. RabbitMQ/SQS: traditional task queues, simpler 'do this job once' model.
- Delivery guarantees: At-most-once (may lose, never duplicates), At-least-once (never loses, may duplicate — most common in practice), Exactly-once (ideal but hard; usually approximated via idempotent consumers).
- Dead-letter queues catch repeatedly-failing messages so they don't block the rest of the queue — always mention this for resilience.
- Pub-Sub fan-out: one event reaches many independent subscribers — enables adding new features (new subscribers) without touching the original producer code.
- Use a queue when a step doesn't need an immediate response (emails, thumbnails, analytics); keep genuinely blocking steps (payment confirmation, login) synchronous.
- Idempotent consumer design ('set status to X' not 'add to balance') is what makes at-least-once delivery safe to use in practice.

Memory hook: A queue is a mailbox, not a doorstep conversation — drop the letter and walk away, trusting it gets read. Pub-Sub is a notice board: post once, and every interested resident checks it on their own.

CHAPTER 15

API Gateway & Reverse Proxy

One front door for many services — auth, rate limiting, and routing in one place

1. Simple Explanation First

An API Gateway is a single entry point that sits in front of multiple backend services, handling shared concerns (authentication, rate limiting, logging) once, centrally, instead of every individual service reimplementing them. A reverse proxy is the more general underlying concept — a server that sits in front of one or more backend servers and forwards client requests to them, often used for load balancing, SSL termination, or caching, with an API gateway being a specialized, feature-rich form of reverse proxy purpose-built for APIs.

Analogy: *Imagine a large office building with dozens of departments (HR, Finance, Engineering, Sales), each on a different floor. Instead of visitors needing to know exactly which floor and department to find, badge in separately at each one, and figure out building rules themselves, there's a single reception desk at the entrance. Reception checks your ID once (authentication), tells you if you're visiting too often today (rate limiting), and directs you to the right floor (routing) — every department gets to skip building this front-desk logic themselves.*

2. Why This Topic Exists

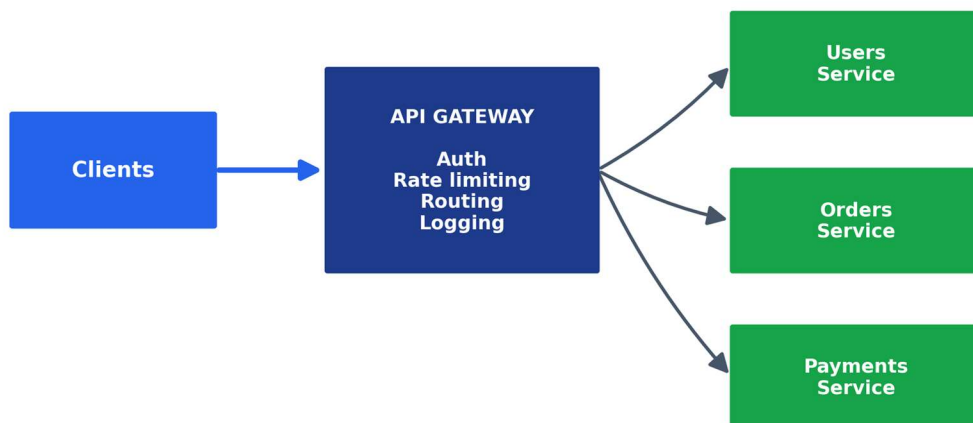
As systems grow into many independent services (previewed in Chapter 4, fully covered in Chapter 18), each service technically COULD handle its own authentication, rate limiting, and logging — but that means duplicating the same cross-cutting logic dozens of times, with inconsistent implementations and more places for security bugs to hide. An API Gateway exists to centralize these shared concerns in one place, so individual services can focus purely on their own business logic.

Why this matters: *The API Gateway pattern is a direct, practical answer to the question 'how do clients actually talk to a system made of many microservices?' — it's almost always part of the answer once your HLD design grows beyond a single service, making it a near-mandatory mention in larger system design interviews.*

3. Core Concepts in Depth

3.1 The Role of an API Gateway

API Gateway — One Front Door for Many Services



Clients never talk to services directly — every request passes through the gateway first.

Figure 15.1 — Clients only ever talk to the gateway; it handles shared concerns and routes internally.

The diagram shows the core shape of this pattern: clients never talk to individual backend services directly. Instead, every request passes through the gateway first, which can perform several jobs before forwarding the request onward (or rejecting it outright):

- Authentication & authorization: verifying who's making the request and what they're allowed to do, before it ever reaches a backend service.
- Rate limiting & throttling: protecting backend services from being overwhelmed by any single client (covered in depth in section 3.3).
- Routing: directing each request to the correct backend service based on the URL path or other request attributes (this overlaps with L7 load balancing from Chapter 13, and in practice the two are often combined in one component).
- Request/response transformation: adapting request or response formats (e.g., combining multiple backend calls into one client-facing response).
- Logging & monitoring: capturing a consistent record of all traffic in one place, rather than scattered across every individual service.

3.2 API Gateway vs Load Balancer vs Reverse Proxy — Untangling the Overlap

These three terms are often confused because they genuinely do overlap in practice — modern API gateways often include load-balancing functionality, and a load balancer IS technically a type of reverse proxy. The distinction is more about PRIMARY PURPOSE than a hard technical boundary.

Component	Primary Purpose	Typical Scope
Reverse Proxy	General term: forwards client requests to one or more backend servers	Can be used for caching, SSL termination, simple routing — the broadest, most general concept
Load Balancer	Distributes traffic across multiple replicas of the SAME service	Focused purely on traffic distribution and health-based routing within one service's server pool
API Gateway	Single entry point for many DIFFERENT services, with cross-cutting concerns built in	Auth, rate limiting, routing across services, request/response shaping — the most feature-rich of the three

3.3 Rate Limiting Algorithms

Rate limiting protects backend services from being overwhelmed — whether by a misbehaving client, a bug causing a retry storm, or a deliberate abuse attempt. Several algorithms exist, each with different trade-offs around burstiness and precision.

Token Bucket Rate Limiting

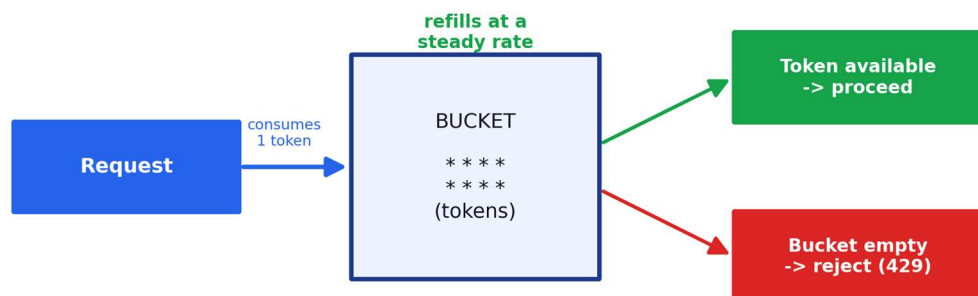


Figure 15.2 — Token bucket: each request consumes a token; the bucket refills at a steady rate.

Algorithm	How It Works	Trade-off
Token Bucket	A bucket holds tokens, refilled at a fixed rate; each request consumes one token; requests are rejected if the bucket is empty	Allows controlled bursts (using saved-up tokens) while enforcing a steady average rate — a very common, well-balanced default
Leaky Bucket	Requests are queued and processed at a strictly fixed output rate, like water leaking from a hole at a constant pace	Smooths out bursts completely into a steady stream, but can add latency for bursty legitimate traffic
Sliding Window	Counts requests within a moving time window (e.g., the last 60 seconds, continuously updated)	More accurate and fair than fixed time windows, but more computationally/memory expensive to track precisely

Common mistake: Confusing throttling with rate limiting. Rate limiting typically **REJECTS** requests once a limit is exceeded (e.g., HTTP 429 Too Many Requests). Throttling typically **SLOWS DOWN** or **queues** requests instead of outright rejecting them. They solve a similar underlying problem (protect the system from too much traffic) but behave differently from the client's perspective, and interviewers sometimes probe this exact distinction.

3.4 Authentication at the Edge

Centralizing authentication at the gateway means a request's identity (e.g., a validated JWT token, see Chapter 21) is established once, at the very edge of the system, before the request reaches any internal service. Internal services can then trust a lightweight, pre-validated identity passed along by the gateway (e.g., in a header) instead of each independently re-verifying credentials — reducing duplicated work and shrinking the attack surface, since only the gateway needs direct exposure to sensitive auth infrastructure.

4. Trade-offs Table

Decision	Pros	Cons
Centralizing concerns in an API Gateway	Consistent auth/rate-limiting/logging across all services; less duplicated logic	The gateway becomes a critical, must-scale component; adds an extra network hop
Token bucket rate limiting	Allows reasonable bursts while still enforcing an average rate — good real-world balance	Slightly more complex to reason about than a simple fixed-window counter
Rejecting (rate limiting) vs queuing (throttling) excess requests	Rejecting protects the system immediately and predictably	Throttling/queuing can provide a smoother client experience but risks queue buildup under sustained overload

5. How Real Companies Use This

- Netflix's Zuul (and later, Spring Cloud Gateway-based successors) was built specifically as an API Gateway to handle routing, auth, and resiliency for their large microservices fleet.

- Stripe and most large public APIs implement token-bucket-style rate limiting per API key, returning clear rate-limit headers so client developers can see their remaining quota.
- Kong, Amazon API Gateway, and Apigee are widely-used, purpose-built API Gateway products that companies adopt rather than building this layer from scratch.

6. Common Mistakes & Misconceptions

- Treating the API Gateway as just a fancier load balancer — its defining feature is handling cross-cutting concerns (auth, rate limiting) across MULTIPLE different services, not just distributing traffic to replicas of one service.
- Forgetting that the gateway itself needs to be highly available and horizontally scalable — since EVERY request passes through it, it can become a severe bottleneck or single point of failure if under-provisioned.
- Applying a single global rate limit instead of per-client/per-API-key limits — a single misbehaving client shouldn't be able to exhaust the rate budget for every other legitimate user.

7. Interviewer Q&A Bank

Q1. What problem does an API Gateway solve that a simple load balancer doesn't?

A: A load balancer distributes traffic across replicas of a single service, but doesn't inherently understand authentication, rate limiting, or routing across many DIFFERENT services. An API Gateway centralizes these cross-cutting concerns in one place for an entire system made of multiple services, so each individual service doesn't need to reimplement auth and rate limiting itself.

Q2. Explain the token bucket rate-limiting algorithm.

A: A bucket holds a fixed maximum number of tokens, which refill at a steady rate over time. Each incoming request consumes one token; if a token is available, the request proceeds, and if the bucket is empty, the request is rejected. This allows clients to make a reasonable burst of requests at once (using up saved tokens) while still enforcing a steady average rate over time, which tends to model real-world traffic patterns better than a strict fixed-rate approach.

Q3. What's the difference between rate limiting and throttling?

A: Rate limiting typically rejects requests outright once a defined limit is exceeded, usually returning an error like HTTP 429. Throttling instead slows down or queues excess requests rather than rejecting them, smoothing out traffic over time. Both protect the system from overload, but they produce different experiences for the client — an immediate rejection versus added latency.

Q4. Why would authentication be centralized at the API Gateway instead of in each service?

A: Centralizing authentication means identity is verified once, at the system's edge, reducing duplicated logic across services and shrinking the attack surface, since only the gateway needs direct access to sensitive credential-verification infrastructure. Internal services can then trust a lightweight, pre-validated identity (e.g., passed in a header) rather than each independently re-implementing and re-verifying authentication.

Q5. Doesn't putting everything through one API Gateway create a bottleneck?

A: It can, if the gateway isn't designed to scale — since literally every request passes through it, it needs to be highly available and horizontally scalable itself, often deployed as a cluster behind its own load balancing, similar to how the load balancer layer itself needs redundancy. This is a real trade-off of the

pattern, and it's worth proactively acknowledging rather than treating the gateway as a free, infinitely-capable layer.

8. HLD Mini-Scenario: Gateway Design for a Public Developer API

Prompt: "Design the API Gateway layer for a company offering a public API to third-party developers, where different customers pay for different tiers (free, pro, enterprise) with different rate limits."

- 1. Authenticate each request at the gateway using an API key, identifying which customer and which pricing tier is making the request — this lookup happens once, centrally, rather than in every backend service.
- 2. Apply per-API-key token bucket rate limiting, with bucket size/refill rate varying by tier (e.g., free tier gets a small, slowly-refilling bucket; enterprise gets a much larger one) — this directly ties a technical mechanism to a real business requirement.
- 3. Route validated, rate-limit-passed requests to the appropriate internal backend service based on the URL path, and log every request centrally at the gateway for billing and usage analytics, rather than each backend service needing its own logging/billing logic.

This shows how gateway features (auth, rate limiting, routing, logging) combine to solve a genuinely business-relevant problem, not just a technical checklist.

9. Questions YOU Should Ask the Interviewer

- Do different clients/customers need different rate limits or access levels?
- Should rejected requests fail immediately, or is some queuing/throttling behavior acceptable?
- Is authentication uniform across all routes, or do some endpoints have different auth requirements?

10. Scenario-Based Practice Bank (New)

Scenario 1: A single misbehaving free-tier client is exhausting the rate limit budget meant for all customers, causing paying enterprise customers to get rate-limited too.

How to approach it: *This is a sign the rate limit is applied globally instead of per-client. Switch to per-API-key token buckets, so one client's excessive usage only depletes their own bucket and can't affect anyone else's quota.*

Scenario 2: A team is deciding whether excess requests during a traffic spike should be rejected outright or queued and processed with delay.

How to approach it: *Frame this as the rate-limiting-vs-throttling trade-off: rejecting (rate limiting, e.g., HTTP 429) protects the system immediately and predictably but gives a harsher client experience; throttling smooths traffic by queuing but risks queue buildup under sustained overload. Choose based on whether the client can tolerate delay or needs an immediate clear signal.*

Scenario 3: Each of 12 microservices currently implements its own authentication logic, and a security review found 3 slightly different (and inconsistent) implementations.

How to approach it: *This is exactly the problem an API Gateway centralizes: move authentication to the gateway so identity is verified once, consistently, at the edge, and let internal services trust a pre-validated identity passed along (e.g., in a header) instead of each reimplementing auth independently.*

Scenario 4: An interviewer asks whether the API Gateway itself could become a bottleneck once traffic grows.

How to approach it: Acknowledge this directly: since every request passes through the gateway, it must be highly available and horizontally scalable itself — typically deployed as a cluster behind its own load balancing — rather than treated as an infinitely-capable, free layer.

Scenario 5: A public API returns bursty legitimate traffic from batch-processing clients, and a strict fixed-rate limiter is rejecting valid short bursts.

How to approach it: Recommend token bucket rate limiting instead of a strict fixed-rate approach: it allows a reasonable burst using saved-up tokens while still enforcing a steady average rate over time, better matching real-world bursty-but-legitimate traffic patterns.

Scenario 6: A team is debating whether their 'API Gateway' is really just their existing load balancer with a new name.

How to approach it: Clarify the distinction: a load balancer's primary purpose is distributing traffic across replicas of the SAME service. If this component also handles auth, per-client rate limiting, and routing across multiple DIFFERENT services, it has grown into a true API Gateway — the primary-purpose distinction matters more than the specific product name.

Chapter 15 — One-Page Revision Sheet

- API Gateway = single entry point for many services, centralizing auth, rate limiting, routing, and logging — avoids duplicating this logic in every service.
- Reverse proxy is the general concept (forwards requests to backend servers); load balancer focuses on distributing traffic within ONE service; API Gateway adds cross-cutting concerns across MANY services.
- Rate limiting algorithms: Token Bucket (allows bursts, steady average rate — common default), Leaky Bucket (smooths to a strict constant rate), Sliding Window (accurate but more expensive to track).
- Rate limiting rejects excess requests (e.g., 429 error); throttling slows/queues them instead — know the distinction.
- Centralizing auth at the gateway means identity is verified once at the edge; internal services trust a pre-validated identity instead of re-checking credentials themselves.
- The gateway itself must be highly available and horizontally scalable — it sees ALL traffic and can become a bottleneck or single point of failure if under-provisioned.
- Apply rate limits per-client/per-API-key, not globally, so one misbehaving client can't exhaust the budget for everyone else.

Memory hook: A building's reception desk: checks your ID once, tells you if you're visiting too often today, and points you to the right floor — every department skips building this front-desk logic themselves.

CHAPTER 16

Content Delivery Networks (CDN) & Edge Computing

Bringing the data physically closer to the user instead of making it travel further

1. Simple Explanation First

A CDN (Content Delivery Network) is a globally distributed network of servers (called edge nodes or PoPs — Points of Presence) that cache and serve content from a location physically close to the end user, instead of every request traveling all the way back to one central origin server. This is the exact same caching concept from Chapter 11, applied at a global, geographic scale.

Analogy: *Imagine a popular bakery in New York that ships its bread worldwide. Without a CDN, every single order — whether from Tokyo, London, or Mumbai — has to be baked fresh in New York and flown out individually, which is slow and expensive. With a CDN, the bakery sets up small local kitchens (edge nodes) in major cities worldwide, each pre-stocked with the most popular bread recipes. A customer in Tokyo gets their bread from the Tokyo kitchen almost instantly; only unusual, rarely-requested items need a slow shipment from the original New York bakery (the origin server).*

2. Why This Topic Exists

Physical distance directly causes latency — light (and data) can only travel so fast, and real-world networks add further delay at each hop. A user in India requesting an image hosted on a server in the US might wait 200+ milliseconds just for the data to physically travel back and forth, before any actual processing happens. CDNs exist to eliminate most of that physical distance for the vast majority of requests, by keeping copies of content much closer to where users actually are.

Why this matters: *Any system design question involving a global user base — and most real systems do have one — should mention a CDN for static assets almost as a reflex. It's one of the highest-impact, lowest-controversy additions you can propose, and skipping it is a noticeable omission for any user-facing system serving images, videos, JS/CSS, or similar content.*

3. Core Concepts in Depth

3.1 How a CDN Serves Content

CDN: Edge Nodes vs Origin Server

Cache HIT at the edge = fast, no trip to origin. Only a MISS reaches the origin (dashed).

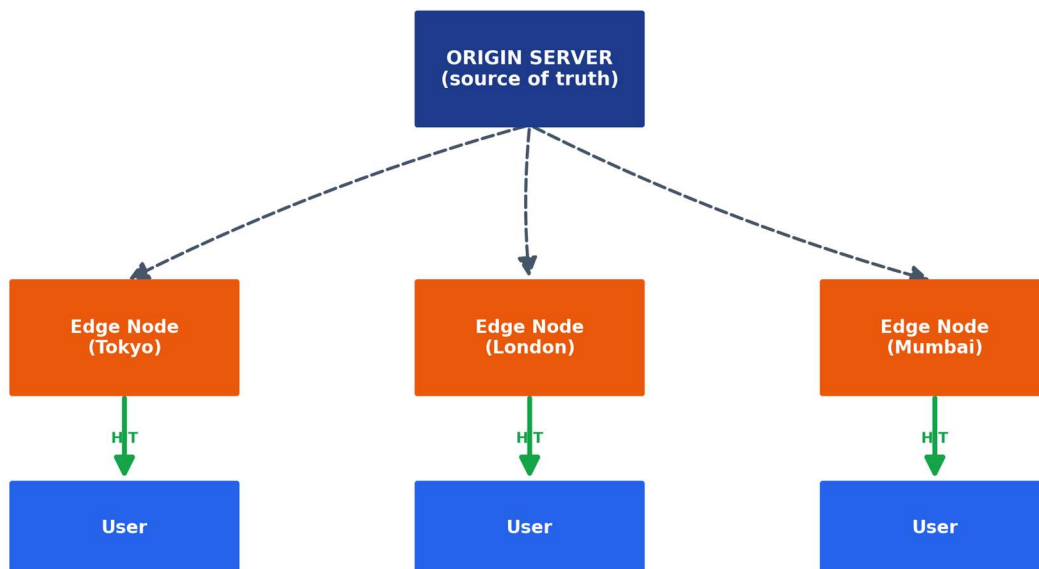


Figure 16.1 — Most requests are served entirely from a nearby edge node; the origin is hit only on a cache miss.

The diagram shows the typical flow: a user's request first reaches the nearest edge node (determined via geo-aware DNS, echoing the DNS routing discussion from Chapter 2). If that edge node already has the requested content cached, it serves it directly — fast, and without ever bothering the distant origin server. If not (a cache miss), the edge node fetches it from the origin, caches a copy for next time, and returns it to the user. Over time, popular content ends up cached at nearly every edge location, while only the origin server holds the complete, authoritative copy of everything.

3.2 Static vs Dynamic Content Caching

CDNs were originally designed for static content — files that don't change per-request or per-user, like images, videos, CSS, and JavaScript bundles. These are easy to cache because the same response is correct for every user. Modern CDNs increasingly also cache dynamic content (personalized or frequently-changing data), but this requires much more careful cache key design and invalidation strategy.

Static Content	Dynamic Content
Same response for every user (images, videos, JS/CSS bundles, static HTML)	Response varies per user or changes frequently (personalized feeds, search results, account pages)
Easy to cache with long TTLs	Requires careful cache key design (e.g., per-user) or very short TTLs
CDN caching is close to a 'free win' here	CDN caching here requires deliberate, careful design to avoid serving the wrong user's data

Common mistake: Assuming a CDN can blindly cache ANY content type with no extra thought. Naively caching a personalized or authenticated response at a shared edge node — without including the user's identity in the cache key — can leak one user's private data to another user who happens to hit the same edge node. This is a real, serious security mistake, not just a performance footgun.

3.3 Cache Invalidation for CDNs

Just like the application-level caching covered in Chapter 11, CDN-cached content can become stale, and the same fundamental challenge (knowing WHEN to refresh) applies, now multiplied across potentially hundreds of geographically distributed edge nodes.

- TTL-based expiration: the most common approach — content is cached with an expiration time, after which the edge node re-fetches from origin on the next request.
- Cache purging/invalidation API: most CDN providers offer an API to explicitly invalidate specific cached content across all edge nodes immediately (useful for urgent corrections, like removing an incorrect price).
- Versioned/fingerprinted URLs: a very common and elegant pattern — instead of invalidating /style.css, you deploy a new version as /style.a1b2c3.css with a new filename whenever the content changes, so the 'old' cached version simply becomes irrelevant and the new URL is naturally a cache miss that gets cached fresh. This sidesteps invalidation complexity entirely for many static assets.

Why this matters: Mentioning fingerprinted/versioned URLs is a great, concrete example of senior-level thinking — it reframes a hard problem (active invalidation across many distributed nodes) into an easy one (just change the URL), which is exactly the kind of elegant trade-off interviewers love to see.

3.4 Edge Computing — Beyond Just Caching

Modern CDN providers (Cloudflare, AWS CloudFront with Lambda@Edge, Fastly) increasingly let you run actual application code at edge locations, not just cache static files. This is called edge computing — it allows simple logic (authentication checks, A/B testing, request redirects, even some personalization) to run physically close to the user, without needing a full round trip to the origin server for every decision.

4. Trade-offs Table

Decision	Pros	Cons
Using a CDN for static assets	Dramatically lower latency for users worldwide; reduces load on origin servers	Added cost; cache invalidation complexity; another layer to monitor
Caching dynamic/personalized content at the edge	Can extend CDN benefits beyond static files	Requires careful, per-user cache key design — real risk of data leakage if done carelessly
Versioned/fingerprinted asset URLs	Sidesteps invalidation complexity entirely — simple and reliable	Requires a build/deploy process that generates and references these unique filenames

5. How Real Companies Use This

- Netflix operates its own purpose-built CDN (Open Connect), placing servers physically inside ISP networks worldwide specifically to deliver video content with minimal latency and ISP transit cost.
- Most major websites use third-party CDNs like Cloudflare, Akamai, or Fastly for static assets, SSL termination, and increasingly for edge-computing logic like bot detection and A/B testing.
- E-commerce sites commonly use fingerprinted asset URLs (e.g., `bundle.3f9a8c.js`) generated automatically by their build tools, so new deployments are immediately reflected without needing to manually purge CDN caches.

6. Common Mistakes & Misconceptions

- Believing a CDN is only useful for huge, global companies — even moderately-sized applications with users spread across a country or continent benefit meaningfully from reduced latency.
- Forgetting to mention HOW stale CDN content gets refreshed — just like Chapter 11's caching, simply saying 'use a CDN' without addressing invalidation is an incomplete answer.
- Caching personalized/authenticated responses without including user identity in the cache key — a serious, real-world security mistake, not just a minor inefficiency.

7. Interviewer Q&A Bank

Q1. What is a CDN and why would you use one?

A: A CDN is a globally distributed network of edge servers that cache content close to end users, so requests can be served from a nearby location instead of traveling all the way to a single origin server. I'd use one for any user-facing system with a geographically spread-out user base, primarily for static assets like images, video, and JS/CSS, since it dramatically reduces latency and offloads traffic from the origin.

Q2. How would you handle cache invalidation for content served through a CDN?

A: I'd primarily rely on TTL-based expiration for most content, since it's simple and requires no active intervention. For urgent corrections, I'd use the CDN provider's purge/invalidation API to immediately clear specific cached content. For frequently-changing build assets like JS/CSS bundles, I'd prefer

versioned or fingerprinted filenames — generating a new unique URL whenever the content changes — which avoids needing active invalidation altogether, since the old URL simply stops being referenced.

Q3. Can a CDN cache personalized content safely?

A: Yes, but it requires care — the cache key must include enough information (like a user or session identifier) to ensure one user's personalized response is never served to a different user from the shared edge cache. Done carelessly, this can cause a serious data-leakage bug, so personalized/dynamic caching needs deliberate design rather than being treated the same as simple static asset caching.

Q4. What's the difference between a CDN and edge computing?

A: A traditional CDN primarily caches and serves static content from edge locations. Edge computing extends this further by allowing actual application code to run at those edge locations — for tasks like authentication checks, redirects, or A/B testing — so some logic can execute physically close to the user without a full round trip back to the origin server for every decision.

Q5. If your origin server goes down, does a CDN help at all?

A: For content that's already cached at the edge with a still-valid TTL, yes — users can continue to be served that cached content even while the origin is down, which is a meaningful resilience benefit. However, any content not yet cached, or any cache miss, will fail until the origin recovers, so a CDN reduces but doesn't eliminate the impact of an origin outage; it's a mitigation, not a full substitute for origin redundancy (Chapter 19).

8. HLD Mini-Scenario: CDN Strategy for a Global News Site

Prompt: "Design the content delivery strategy for a news website with readers worldwide, serving article images, a JS/CSS frontend bundle, and personalized 'recommended for you' article lists."

- 1. Serve article images and the JS/CSS bundle through a CDN with long TTLs, since this content is identical for all users and changes only on deployment — use fingerprinted filenames for the JS/CSS bundle so new deployments are automatically reflected without manual invalidation.
- 2. Cache article text itself at the edge with a short TTL (e.g., a few minutes), since articles rarely change after publishing but breaking corrections do occasionally happen — short TTL balances freshness with performance.
- 3. Do NOT cache the personalized 'recommended for you' list at the shared edge layer; instead, serve it from the origin (or a regional application cache, Chapter 11) where it's safe to key by user identity, avoiding any risk of one user's recommendations leaking to another.

This demonstrates the key judgment call of this chapter: matching the caching strategy (CDN-cacheable vs not) to whether content is genuinely shared across users or specific to one.

9. Questions YOU Should Ask the Interviewer

- Is our user base concentrated in one region, or genuinely spread across multiple continents?
- Which parts of the content are identical for all users, versus personalized per user?
- How quickly do we need content updates/corrections to propagate to all users globally?

10. Scenario-Based Practice Bank (New)

Scenario 1: A user in Mumbai reports images loading slowly on a site whose origin server is in Virginia, USA, while US-based users report no issues.

How to approach it: This is the classic physical-distance latency problem CDNs solve. Recommend putting a CDN in front of the image assets so Mumbai users are served from a nearby edge node instead of round-tripping to Virginia on every request.

Scenario 2: A CDN was recently caught serving one logged-in user's account dashboard to a different logged-in user who happened to hit the same edge node.

How to approach it: This is a serious cache-key design flaw: the personalized response was cached without including user identity in the cache key. Fix by keying cached responses per-user (e.g., including a user or session identifier in the cache key), or simply exclude this personalized content from shared edge caching entirely.

Scenario 3: A company needs to fix an incorrect price displayed on a product page immediately, but the page is cached at the CDN with a 24-hour TTL.

How to approach it: Recommend using the CDN provider's purge/invalidation API to immediately clear that specific cached page across all edge nodes, rather than waiting for the 24-hour TTL to naturally expire — this is exactly the urgent-correction use case that invalidation APIs exist for.

Scenario 4: Every time the frontend team deploys a new JS bundle, users intermittently get a broken mix of old and new cached files until the CDN cache fully clears.

How to approach it: Recommend switching to versioned/fingerprinted filenames (e.g., bundle.a1b2c3.js) for each deployment, so a new deploy is naturally served from a brand-new URL that's a guaranteed cache miss — sidestepping invalidation timing issues entirely instead of relying on purges or TTL expiry.

Scenario 5: An interviewer asks what happens to a CDN-fronted site if the origin server goes completely down for an hour.

How to approach it: Explain the nuance: already-cached content at the edge with a still-valid TTL continues to be served normally during that hour, which is a real resilience benefit. But anything not yet cached, or any cache miss, will fail until the origin recovers — a CDN reduces but doesn't eliminate the impact of an origin outage.

Scenario 6: A small startup with users mostly in one country is debating whether a CDN is worth the added cost and complexity.

How to approach it: Even within one country, a CDN can meaningfully reduce latency by serving from a point of presence closer to users than a single origin data center, and it offloads traffic from the origin. It's a reasonable, low-controversy addition even at moderate scale, not just for huge global companies.

Chapter 16 — One-Page Revision Sheet

- A CDN is geographically-distributed caching (the Chapter 11 concept, applied globally) — edge nodes serve content close to users instead of every request reaching one distant origin.
- Static content (images, video, JS/CSS) is the classic, easy CDN use case — same response for everyone, safe to cache with long TTLs.
- Dynamic/personalized content CAN be cached at the edge, but only with careful per-user cache keys — getting this wrong risks leaking one user's data to another.
- Invalidation options: TTL expiration (simplest), explicit purge API (urgent fixes), versioned/fingerprinted URLs (sidesteps invalidation entirely for build assets).
- Edge computing extends CDNs beyond caching — running actual logic (auth checks, redirects, A/B tests) physically close to users.
- A CDN reduces but doesn't eliminate the impact of an origin outage — already-cached content with a valid TTL still works, but cache misses will fail.
- Mention a CDN almost by reflex for any user-facing system with a geographically spread-out user base — it's a high-impact, low-controversy addition.

Memory hook: *A bakery with small local kitchens in every major city (edge nodes), each pre-stocked with popular bread — only unusual orders need the slow flight from the original New York bakery (origin).*

CHAPTER 17

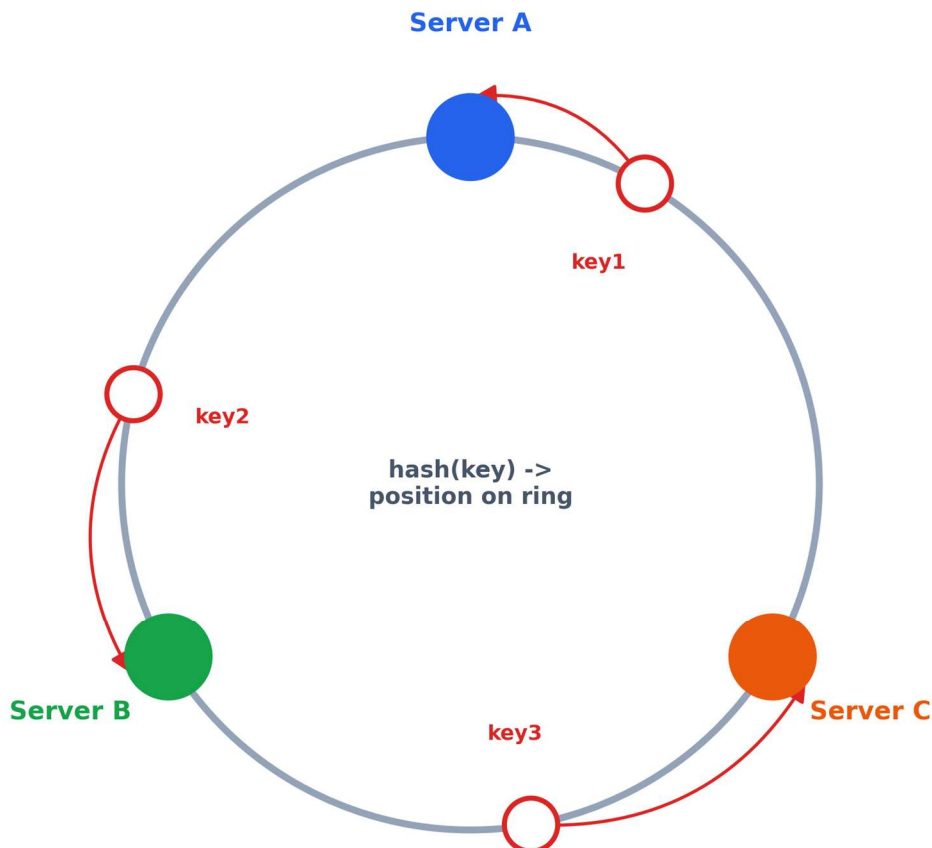
Consistent Hashing Deep Dive

The elegant trick that makes adding or removing a server painless

1. Simple Explanation First

Consistent hashing is a technique for distributing data (or requests) across a set of servers in a way that minimizes how much data needs to move when a server is added or removed. Instead of using a simple formula like $\text{hash}(\text{key}) \% \text{number_of_servers}$ (which reshuffles almost everything when the server count changes), consistent hashing places both servers AND data keys on a conceptual circle (a 'hash ring'), and each key simply belongs to the nearest server going clockwise around that ring.

Consistent Hashing — The Hash Ring



Each key belongs to the nearest server going clockwise around the ring.

Figure 17.1 — Servers and keys both map onto the same ring; each key belongs to the nearest server clockwise.

Analogy: Imagine assigning students to classrooms by writing each student's name and each classroom's name on a giant circular clock face, based on a fingerprint-like hash of their name. Each student simply walks clockwise from their position until they hit the first classroom — that's their assigned room. If a new classroom opens up, only students between the new classroom and the previous one need to switch rooms; everyone else's assignment is completely undisturbed. Compare this to the old system of 'classroom = student_number % number_of_classrooms,' where adding even one classroom would shuffle nearly every student into a totally different room.

2. Why This Topic Exists

Chapter 10 introduced the resharding problem: naive hash-based sharding ($\text{hash}(\text{key}) \% N$) requires moving almost all data whenever N (the number of shards/servers) changes, because the modulo result for nearly every key shifts. This makes scaling up or down a live, naively-sharded system extremely disruptive — exactly the kind of operation you want to avoid doing under production load. Consistent hashing exists specifically to solve this problem, by ensuring that adding or removing one server only affects the small slice of keys immediately next to it on the ring, not the entire dataset.

Why this matters: Consistent hashing is one of the most frequently asked 'explain this concept' questions in system design interviews, precisely because it elegantly solves a real, painful problem (Chapter 10's resharding issue) with a clever, visualizable idea. Being able to draw the ring and explain it clearly is a high-value, memorable skill.

3. Core Concepts in Depth

3.1 The Hash Ring

Both servers and data keys are run through the same hash function, producing a number that's treated as a position on a fixed-size circular ring (e.g., a ring numbered 0 to $2^{32} - 1$). To find which server owns a given key, you start at the key's position and move clockwise until you hit the first server — that server owns the key. As shown in the diagram, key1 belongs to Server A (the nearest server clockwise from key1's position), key2 belongs to Server B, and so on.

The critical property: if Server B is removed from the ring, only the keys that were assigned to Server B need to move (to the next server clockwise, which is now their new nearest neighbor) — every other key, assigned to Server A or Server C, is completely unaffected. This is a dramatic improvement over naive modulo hashing, where removing one server out of N reshuffles roughly $(N-1)/N$ of all keys.

3.2 Virtual Nodes — Solving Uneven Distribution

A basic hash ring with just a few real servers has a real weakness: depending on where each server happens to land on the ring (essentially random, based on the hash function), the amount of 'territory' each server owns can be wildly uneven — one server might end up owning 60% of the ring's keys while another owns just 5%. Virtual nodes solve this by having each PHYSICAL server appear at MANY different positions on the ring (e.g., 100-200 virtual positions per real server) instead of just one.

- Each virtual node still maps back to the same underlying physical server — it's just a trick for distributing that server's 'territory' more evenly around the ring.
- With enough virtual nodes per server, the law of averages takes over, and each physical server ends up owning a roughly equal, fair share of the total keyspace, regardless of where any single hash happened to land.

- Virtual nodes also make adding/removing a server smoother — instead of one large chunk of keys moving to/from a single neighbor, the affected keys are spread across many different neighboring virtual positions, distributing the rebalancing load more evenly too.

Why this matters: *Bringing up virtual nodes UNPROMPTED after explaining the basic ring is one of the clearest ways to demonstrate genuine depth on this topic — it's the detail that separates 'I memorized the concept' from 'I understand why the naive version isn't quite enough in practice.'*

3.3 Walking Through Adding and Removing a Server

It's worth being able to narrate this step by step in an interview, since it's often asked as a direct follow-up to 'explain consistent hashing.'

- 1. Adding a server: the new server (and its virtual nodes) are placed onto the ring at positions determined by hashing its identifier. Only the keys that fall between the new server's position(s) and the previous server(s) clockwise need to move to the new server — everything else stays exactly where it was.
- 2. Removing a server: all keys that were owned by the removed server (and its virtual nodes) simply move to whatever is now the next server clockwise from each of those positions. Again, every other key elsewhere on the ring is completely unaffected.
- 3. In both cases, the amount of data that needs to move is roughly proportional to $1/N$ of the total data (where N is the number of servers), not the near-total reshuffling that naive modulo hashing would cause.

3.4 Where Consistent Hashing Shows Up Beyond Database Sharding

While Chapter 10 introduced consistent hashing in the context of database sharding, the same technique appears in many other places throughout distributed systems — it's a genuinely general-purpose tool, not a single-use trick.

- Distributed caches: Memcached and similar systems use consistent hashing to decide which cache node a given key lives on, so adding/removing a cache node doesn't invalidate the entire cache.
- Load balancers (Chapter 13): consistent hashing can be used as a load-balancing algorithm to provide session affinity, consistently routing a given client to the same backend server without a centralized session table.
- Distributed hash tables (DHTs): peer-to-peer systems (like parts of BitTorrent's architecture) use consistent hashing to let nodes find which peer is responsible for a given piece of data, without any central coordinator.

4. Trade-offs Table

Decision	Pros	Cons
Consistent hashing over naive modulo hashing	Minimal data movement when servers are added/removed — much friendlier to live resharding	More complex to implement and reason about than simple modulo arithmetic
Using virtual nodes	Much more even data distribution across physical servers	Slightly more memory/bookkeeping overhead to track many virtual positions per server
Few virtual nodes per server	Simpler, less bookkeeping	Risk of uneven, unfair load distribution across servers

5. How Real Companies Use This

- Amazon's DynamoDB paper (a hugely influential systems design document) popularized consistent hashing as a core technique for distributing data across nodes in a way that handles node failure and growth gracefully.
- Discord and other large-scale chat platforms use consistent hashing to distribute their sharded message storage, so adding capacity doesn't require a massive, disruptive data migration.
- Content delivery and caching systems (including some CDN internals from Chapter 16) use consistent hashing to decide which edge node or cache server should handle a given piece of content.

6. Common Mistakes & Misconceptions

- Explaining the basic ring concept but forgetting virtual nodes — a complete answer should proactively mention this, since the basic ring alone has a real, well-known weakness.
- Assuming consistent hashing means ZERO data movement when scaling — it dramatically REDUCES data movement compared to naive hashing, but some movement (proportional to $1/N$) is still expected and normal.
- Treating consistent hashing as exclusively a database sharding technique — it's a general distributed-systems tool that also applies to caching, load balancing, and peer-to-peer systems.

7. Interviewer Q&A Bank

Q1. Explain consistent hashing and the problem it solves.

A: Consistent hashing places both servers and data keys on a conceptual circular ring using a hash function, where each key belongs to the nearest server clockwise. It solves the resharding problem of naive $\text{hash}(\text{key}) \% N$ sharding, where changing the number of servers reshuffles almost all keys — with consistent hashing, adding or removing a server only affects the small portion of keys near that server's position on the ring, leaving everything else undisturbed.

Q2. What problem do virtual nodes solve in consistent hashing?

A: With only one position per physical server on the ring, the amount of keyspace each server owns can be very uneven, since server positions are essentially randomly determined by the hash function. Virtual nodes have each physical server appear at many different positions on the ring, which evens out the

distribution of keys across servers through the law of averages, and also spreads the rebalancing load more evenly when a server is added or removed.

Q3. How much data needs to move when you add a new server with consistent hashing?

A: Roughly $1/N$ of the total data, where N is the resulting number of servers — only the keys that fall between the new server's ring position and its previous neighbor need to move to it. This is a dramatic improvement over naive modulo hashing, where adding or removing even one server can reshuffle nearly all the data, but it's worth being precise that consistent hashing reduces data movement significantly rather than eliminating it entirely.

Q4. Besides database sharding, where else is consistent hashing used?

A: It's used in distributed caching systems (to decide which cache node owns a given key without invalidating the whole cache when nodes change), as a load-balancing algorithm for session affinity, and in peer-to-peer distributed hash tables where nodes need to find who's responsible for a given piece of data without a central coordinator. It's a general-purpose distributed-systems technique, not limited to databases.

Q5. What happens to the keys owned by a server that crashes unexpectedly, under consistent hashing?

A: Those keys' ownership automatically shifts to whatever is now the next server clockwise on the ring from each of the failed server's positions. If the underlying data was also replicated (Chapter 9) to that neighboring server beforehand, the system can continue serving those keys correctly with minimal disruption — though if the failed server held the only copy of that data, that specific data would be lost, which is why consistent hashing is usually paired with replication in real systems.

8. HLD Mini-Scenario: Scaling a Distributed Cache With Consistent Hashing

Prompt: "Your distributed cache cluster of 4 nodes is reaching capacity, and you need to add 2 more nodes without causing a massive cache miss storm that overwhelms your database."

- 1. Use consistent hashing (rather than naive modulo hashing) to assign cache keys to nodes, specifically because we're anticipating future scaling events like this one, and want to minimize disruption when they happen.
- 2. When adding the 2 new nodes, only the keys that land between the new nodes' ring positions and their nearest previous neighbors will be considered 'missing' from the new nodes initially — the vast majority of existing cached keys remain correctly assigned to their original nodes, avoiding a full cache wipeout.
- 3. Use virtual nodes for each of the 6 total cache nodes to ensure the keyspace (and thus memory usage) stays evenly distributed, rather than risking one node becoming a hot spot due to an unlucky ring position.

This connects directly back to Chapter 11's caching concepts — consistent hashing is precisely what prevents a scaling event from causing a sudden, system-wide wave of cache misses hitting the database all at once.

9. Questions YOU Should Ask the Interviewer

- How frequently do we expect to add or remove servers/nodes in this system?
- Is the data evenly accessed, or do we need to worry about some keys being much more popular (hot keys) than others?

- Is this data replicated elsewhere, or would losing one node's data be a real concern requiring extra safeguards?

10. Scenario-Based Practice Bank (New)

Scenario 1: A distributed cache cluster uses $\text{hash}(\text{key}) \% \text{node_count}$, and every time a node is added, the team observes almost the entire cache invalidating at once, hammering the database.

How to approach it: This is exactly the naive-modulo resharding problem. Migrate to consistent hashing so that adding a node only affects the small slice of keys near its ring position — the vast majority of cached keys remain correctly assigned, avoiding the cache-miss storm.

Scenario 2: After implementing a basic consistent hashing ring with only 4 real server positions, one server ends up owning 55% of all keys while another owns only 8%.

How to approach it: This is the classic uneven-distribution weakness of a basic ring with too few positions per server. Add virtual nodes — give each physical server 100-200 positions on the ring — so the law of averages evens out each server's share of the keyspace.

Scenario 3: An interviewer asks how much data moves when a 5th node is added to a 4-node consistent-hashing cluster.

How to approach it: Answer precisely: roughly $1/5$ of the total data (proportional to $1/N$), not zero and not a full reshuffle — only the keys between the new node's ring position(s) and its previous neighbor(s) move, everything else stays put.

Scenario 4: A node in a consistent-hashing cluster crashes unexpectedly, and the team wants to know if that node's data is simply 'reassigned' with no consequence.

How to approach it: Clarify that ownership of those keys does shift to the next server clockwise, but if that data wasn't also replicated to another node beforehand, it's genuinely lost, not just reassigned. This is why consistent hashing is typically paired with replication (Chapter 9) in real systems.

Scenario 5: A load balancer needs to consistently route the same user to the same backend server (for cache-hit efficiency) without maintaining a centralized session table.

How to approach it: Recommend consistent hashing as the load-balancing algorithm itself (Chapter 13), hashing on something like user ID — this gives natural session affinity without needing a separate lookup table, and handles server pool changes gracefully just like it does for database sharding.

Scenario 6: A candidate explains the hash ring correctly but stops there, and the interviewer asks 'is there anything else you'd want to add to make this production-ready?'

How to approach it: This is the cue to proactively bring up virtual nodes if you haven't already — explaining that a basic ring with few positions per server can be badly uneven, and that adding many virtual positions per physical server smooths out both the initial distribution and the rebalancing load during scaling events.

Chapter 17 — One-Page Revision Sheet

- Consistent hashing places servers AND keys on the same circular ring (via a shared hash function); each key belongs to the nearest server clockwise.
- Solves the resharding problem from Chapter 10: adding/removing a server only moves the small slice of keys near it on the ring, not nearly everything.
- Virtual nodes: each physical server appears at MANY positions on the ring, evening out keyspace distribution and smoothing rebalancing — always mention this proactively.
- Data movement when scaling is roughly proportional to $1/N$ (where N = number of servers), NOT zero — be precise about this in interviews.
- Used far beyond database sharding: distributed caches, load-balancer session affinity, peer-to-peer DHTs.
- Often paired with replication (Chapter 9) — if a node fails and its data wasn't replicated elsewhere, that data is genuinely lost, not just 'reassigned.'
- This is one of the most commonly asked 'explain a concept' questions — practice drawing the ring and narrating add/remove scenarios out loud.

Memory hook: *Students assigned to classrooms by walking clockwise around a giant clock face from their hashed position — open one new classroom, and only the nearby students need to switch rooms; everyone else stays put.*

CHAPTER 18

Microservices vs Monolith & Service Communication

One big codebase, or many small ones talking to each other?

1. Simple Explanation First

A monolith is a single, unified application where all functionality (users, orders, payments, etc.) lives in one codebase and is deployed as one unit. Microservices break that same functionality into multiple small, independent services, each with its own codebase, deployment, and often its own database, communicating with each other over the network.

Analogy: A monolith is like one large department store where every section (groceries, electronics, clothing) is run by the same staff, uses the same checkout system, and the whole store opens and closes together. Microservices are like a shopping mall where each store is independently owned and operated — the electronics store can renovate or restock without affecting the clothing store at all, but now a customer wanting items from three different stores has to physically walk between them, and the mall needs shared infrastructure (parking, directories) to tie everything together.

2. Why This Topic Exists

Early-stage systems are almost always monoliths, because a single small team building a single small product benefits enormously from the simplicity of one codebase, one deployment, and no network calls between internal components. But as a company and engineering team grow, a monolith starts to strain: different teams stepping on each other's code, one team's bug crashing the entire application, and the whole system needing to be redeployed for even a tiny change in one area. Microservices emerged as a response to these specific organizational and technical scaling pains — they exist to let teams and services scale and deploy independently.

Why this matters: This is one of the most consequential, foundational architecture decisions in any system design discussion, and interviewers expect you to recognize it's a genuine trade-off, not an obvious 'microservices are always better' choice — many real, successful companies run on well-structured monoliths far longer than tech culture stereotypes suggest.

3. Core Concepts in Depth

3.1 Monolith vs Microservices — Visualizing the Shift

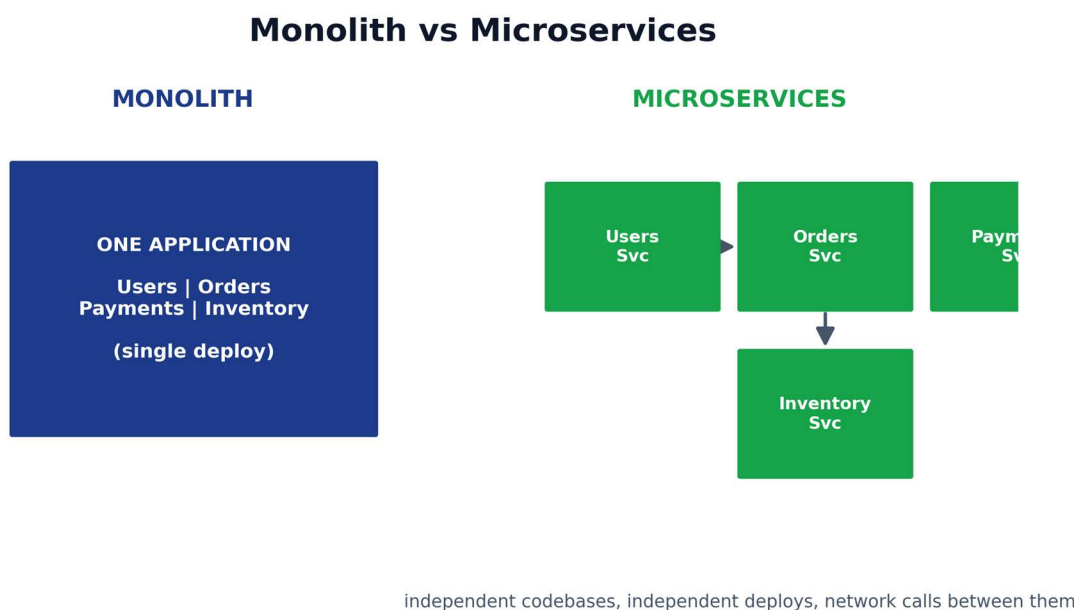


Figure 18.1 — A single deployable app vs independent services communicating over a network.

Monolith	Microservices
Single codebase, single deployment	Multiple independent codebases, independent deployments
Simple to develop and test locally (no network calls between components)	Components communicate over the network — adds latency and failure modes
One bug or memory leak can crash the entire application	A failure in one service can be isolated and doesn't necessarily crash others
Scaling means scaling the WHOLE application, even if only one part is the bottleneck	Each service can be scaled independently based on its own specific load
One team's code changes can affect everyone; deployments are often coordinated	Teams can develop, deploy, and release their service independently

Common mistake: Treating 'microservices' as an automatic upgrade from 'monolith.' Microservices introduce substantial new complexity — network communication, distributed data consistency (Chapter 12), more complex monitoring and debugging — that a small team or simple product may not be equipped to handle well. Many interviewers specifically probe whether you'll default to microservices regardless of context, which is itself a red flag.

3.2 Service Discovery

In a microservices architecture, services need to find and call each other — but with many independent services potentially scaling up and down, moving between machines, and being deployed independently, hardcoding IP addresses simply doesn't work. Service discovery solves this: a registry (like Consul, etcd, or built-in cloud platform tooling) keeps track of which service instances are currently running and where, letting services look each other up dynamically by name instead of a fixed address.

3.3 Inter-Service Communication Patterns

This directly builds on Chapter 4's API styles and Chapter 14's messaging concepts, now applied specifically to how microservices talk to each other internally.

- Synchronous (REST/gRPC): used when one service genuinely needs an immediate response from another before it can proceed (e.g., checking inventory availability before confirming an order).
- Asynchronous (message queues/events): used when a service just needs to notify others that something happened, without waiting for a response (e.g., 'order placed' triggering several independent downstream reactions, as seen in Chapter 14's fan-out pattern).
- The general principle from Chapter 4 applies directly here: minimize synchronous coupling between services where possible, since every synchronous call is a potential point of cascading failure — if Service B is slow or down, and Service A calls it synchronously, Service A's response time and reliability are now directly tied to Service B's.

3.4 Circuit Breakers

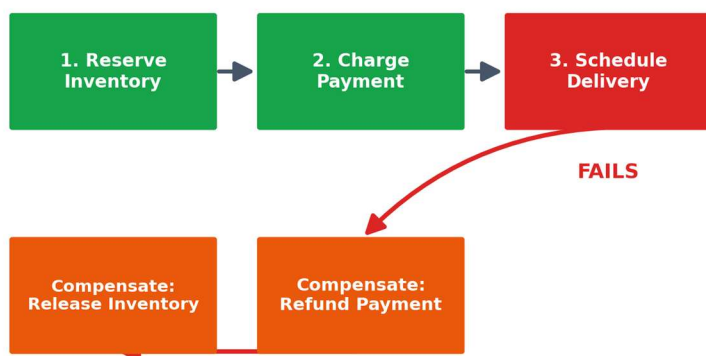
When Service A makes synchronous calls to Service B, what happens if Service B becomes slow or completely unresponsive? Without protection, Service A might keep waiting on every call, exhausting its own resources (threads, connections) and potentially becoming unresponsive itself — a failure cascading outward from one struggling service to its callers. A circuit breaker is a pattern that monitors calls to a downstream service, and after detecting repeated failures, 'trips' (opens the circuit) and starts failing fast — immediately returning an error instead of waiting on a service that's likely to fail anyway, giving the struggling service room to recover and protecting the caller from being dragged down with it.

- Closed state: calls flow through normally; failures are being counted.
- Open state: after too many failures, calls immediately fail without even attempting to reach the downstream service.
- Half-open state: after a cooldown period, a few test calls are allowed through to check if the downstream service has recovered, before fully closing the circuit again.

Why this matters: Circuit breakers are a direct, practical answer to 'how do you prevent one failing service from taking down the whole system?' — a very common follow-up question once a microservices architecture is on the table.

3.5 The Saga Pattern — Transactions Across Services

Saga Pattern — Compensating Actions



Undo earlier steps in reverse order when a later step fails.

Figure 18.2 — If a later step fails, compensating actions undo earlier steps in reverse order.

Chapter 12 introduced Two-Phase Commit as a way to guarantee atomicity across multiple databases, and noted that many modern systems avoid it due to its performance and availability costs. The Saga pattern is the alternative most commonly used in microservices architectures: instead of one big atomic transaction, a business process is broken into a sequence of smaller local transactions, each in a different service. If a later step fails, the Saga executes compensating actions to UNDO the effects of the earlier steps, in reverse order — as shown in the diagram, a failed delivery triggers a refund and an inventory release, rather than the system trying to roll back a single giant cross-service transaction.

- Choreography-based Sagas: each service listens for events and decides on its own what to do next (and what to compensate), with no central coordinator — simpler initially, but can become hard to track as the number of steps grows.
- Orchestration-based Sagas: a central coordinator explicitly tells each service what to do and in what order, including triggering compensations — easier to reason about and monitor, but introduces a more central, coordinating component.

4. Trade-offs Table

Decision	Pros	Cons
Monolith architecture	Simple development, testing, and deployment; no network overhead between components	Scaling and deploying are all-or-nothing; one bug can affect the whole app
Microservices architecture	Independent scaling and deployment per service; failure isolation	Network communication overhead; distributed data consistency challenges; more operational complexity
Saga pattern for cross-service transactions	Avoids the performance/availability cost of distributed locking (2PC)	Requires carefully designing compensating actions for every step; more complex application logic

5. How Real Companies Use This

- Amazon famously transitioned from a large monolith to a microservices architecture specifically to let thousands of engineering teams deploy independently without coordinating with each other.
- Shopify and many successful companies have deliberately stayed on a well-modularized monolith far longer than industry trends suggest, citing the operational simplicity benefits for their team size and stage.
- Netflix popularized the circuit breaker pattern through their open-source Hystrix library, born directly from real production incidents where one slow service cascaded into a much larger outage.

6. Common Mistakes & Misconceptions

- Defaulting to microservices for a small system or early-stage product 'because that's what big tech companies do' — the organizational and technical complexity often isn't justified at small scale, and many large companies started as monoliths for good reason.
- Designing microservices with too much synchronous coupling between them, effectively recreating a 'distributed monolith' that has all the network overhead of microservices with none of the failure-isolation benefits.
- Forgetting to mention how cross-service data consistency is handled (Saga pattern, eventual consistency) when describing a microservices design — this is a near-certain follow-up question.

7. Interviewer Q&A Bank

Q1. When would you choose a monolith over microservices?

A: For an early-stage product with a small team, a monolith is often the more pragmatic choice — it avoids the network communication overhead, distributed data consistency challenges, and operational complexity (deployment, monitoring, service discovery) that microservices introduce, when the team is too small to benefit from independent service ownership. I'd lean toward microservices once specific, real pains emerge — different components needing very different scaling profiles, or multiple teams needing to deploy independently without blocking each other.

Q2. What is a circuit breaker, and why is it important in a microservices architecture?

A: A circuit breaker monitors calls to a downstream service and, after detecting repeated failures, 'opens' and starts failing fast on further calls instead of letting them wait on a likely-failing service. This prevents a slow or failing downstream service from cascading its problems to every service that calls it, which is a critical protection in a microservices architecture where many services depend on each other over the network.

Q3. How would you handle a transaction that needs to span multiple microservices, like an order that involves inventory, payment, and shipping?

A: I'd use the Saga pattern rather than a distributed transaction like Two-Phase Commit, since 2PC's locking and coordination overhead doesn't fit well with microservices' independence and availability goals. The Saga breaks the process into a sequence of local transactions in each service, and if a later step fails, it triggers compensating actions to undo the effects of earlier steps — for example, refunding a payment and releasing reserved inventory if the final shipping step fails.

Q4. What is service discovery and why is it needed in microservices?

A: Service discovery is a mechanism (often a registry like Consul or etcd) that tracks which instances of each service are currently running and where, letting services find and call each other dynamically by name rather than hardcoded IP addresses. It's needed because microservice instances scale up and down and move between machines far more dynamically than in a monolith, making static configuration impractical.

Q5. What's the risk of designing microservices with too much synchronous communication between them?

A: It can recreate many of the downsides of a monolith — tight coupling and cascading failures — while still paying the network latency and complexity cost of a distributed system, sometimes called a 'distributed monolith.' I'd look for opportunities to replace synchronous calls with asynchronous, event-driven communication (Chapter 14) wherever a service doesn't genuinely need an immediate response, to get the real benefits of loose coupling that microservices are meant to provide.

8. HLD Mini-Scenario: Deciding Architecture for a Growing Startup

Prompt: "A startup's e-commerce monolith is struggling because the product-recommendation feature is computationally expensive and needs very different scaling than the rest of the app, and a separate data science team wants to deploy improvements to it independently without going through the main app's release process."

- 1. Recognize this as a strong, specific signal for extracting JUST the recommendation feature into its own microservice — not a blanket call to fully decompose the entire monolith into many tiny services.
- 2. Have the main application call the new recommendation service asynchronously where possible (e.g., pre-computing recommendations and caching them) rather than synchronously blocking page loads on it, protected by a circuit breaker so a slow recommendation service degrades gracefully (e.g., falling back to generic recommendations) instead of breaking the whole page.
- 3. Keep the rest of the application as a monolith for now, since there's no stated pain point justifying further decomposition — this is a deliberately incremental, justified architecture change rather than a wholesale rewrite.

This demonstrates a mature, pragmatic instinct: extract a microservice when there's a specific, real organizational or scaling reason to do so, rather than treating 'microservices' as an all-or-nothing architectural religion.

9. Questions YOU Should Ask the Interviewer

- What's the team size and structure — are multiple independent teams expected to own different parts of this system?
- Are there specific components with very different scaling or reliability needs than the rest of the system?
- How important is independent deployability for different parts of this system versus overall simplicity?

10. Scenario-Based Practice Bank (New)

Scenario 1: A 5-person startup with one simple product is debating microservices 'because that's what Netflix and Amazon do.'

***How to approach it:** Push back on this reasoning: at this team size, a monolith avoids network overhead, distributed consistency challenges, and operational complexity (service discovery, multi-service monitoring) that this team isn't yet positioned to manage well. Recommend a well-modularized monolith until a specific, real pain point (divergent scaling needs, multiple independent teams) actually emerges.*

Scenario 2: Service A calls Service B synchronously on every request, and when Service B recently became slow, Service A's threads backed up and it became unresponsive too.

***How to approach it:** This is precisely the cascading-failure problem circuit breakers solve. Add a circuit breaker around calls to Service B so that after repeated failures/timeouts, Service A fails fast instead of waiting, giving Service B room to recover and protecting Service A from being dragged down with it.*

Scenario 3: An order-processing flow spans inventory, payment, and shipping services, and the team is debating wrapping all three in a single 2PC-style distributed transaction.

***How to approach it:** Recommend the Saga pattern instead: break the process into local transactions per service, with compensating actions (e.g., refund payment, release inventory) if a later step like shipping fails — avoiding 2PC's locking/coordinator-failure risks while still handling the multi-service transaction correctly.*

Scenario 4: A microservices architecture has service instances that scale up and down constantly, and hardcoded IP addresses in configuration files keep going stale.

***How to approach it:** This is a service discovery gap. Introduce a registry (Consul, etcd, or built-in cloud tooling) so services can look each other up dynamically by name, reflecting whichever instances are currently running, instead of relying on static, quickly-outdated IP configuration.*

Scenario 5: A team split their monolith into a dozen 'microservices,' but nearly every request still makes several synchronous calls across services, and one slow service now regularly stalls the whole request chain.

***How to approach it:** This is the 'distributed monolith' trap: they've paid for network overhead and operational complexity without gaining real failure isolation. Recommend replacing unnecessary*

synchronous coupling with asynchronous, event-driven communication (Chapter 14) wherever an immediate response genuinely isn't required.

Scenario 6: A data science team wants to deploy improvements to a computationally expensive recommendation feature independently of the main app's release cycle, without needing to coordinate with the core engineering team every time.

How to approach it: This is a specific, legitimate signal to extract just the recommendation feature into its own microservice, called asynchronously where possible and protected by a circuit breaker — while leaving the rest of the application as a monolith, since no other stated pain justifies further decomposition.

Chapter 18 — One-Page Revision Sheet

- Monolith: one codebase, one deployment, simple but tightly coupled — scaling/deploying is all-or-nothing. Microservices: independent services over a network — independent scaling/deployment, but added complexity.
- Service discovery lets services dynamically find each other by name, since microservice instances scale and move far more than monolith components.
- Inter-service communication: synchronous (REST/gRPC) when an immediate response is genuinely needed; asynchronous (queues/events, Chapter 14) to minimize coupling and cascading failure risk.
- Circuit breakers stop a failing downstream service from cascading failures upstream — closed (normal) -> open (fail fast) -> half-open (test recovery).
- Saga pattern handles transactions across services: a sequence of local transactions, with compensating actions to undo earlier steps if a later one fails. Choreography (decentralized) vs Orchestration (central coordinator).
- Don't default to microservices without a specific, justified reason (team independence, divergent scaling needs) — many successful systems run well-structured monoliths far longer than expected.
- Avoid the 'distributed monolith' trap: too much synchronous coupling between microservices recreates monolith-style fragility while still paying network/complexity costs.

Memory hook: A department store (monolith, one unified operation) vs a shopping mall (microservices, independent stores) — the mall gives each store independence, but now needs shared infrastructure to tie it all together, and a customer's one trip might require visiting several stores.

CHAPTER 19

Availability, Fault Tolerance & Redundancy

Designing systems that keep working even when pieces of them break

1. Simple Explanation First

Availability is the percentage of time a system is actually up and able to serve requests. Fault tolerance is a system's ability to keep functioning correctly even when some of its components fail. Redundancy — having extra, duplicate components ready to take over — is the primary technique used to achieve both.

Analogy: Think of a commercial airplane with four engines, even though it could technically fly on two. The extra engines aren't wasteful indulgence — they're redundancy, ensuring the plane can lose an engine mid-flight and still land safely. System design redundancy works the same way: you deliberately over-provision (extra servers, extra database replicas, extra data centers) specifically so a failure of any single piece doesn't bring down the whole flight.

2. Why This Topic Exists

Hardware fails. Networks drop. Software has bugs. Data centers lose power. At a small scale, these events are rare enough to mostly ignore — but at the scale of thousands of servers running continuously, SOME component is statistically failing at almost any given moment. Availability and fault tolerance exist as a discipline because 'just don't have failures' isn't an option at scale — the only real choice is whether your system is DESIGNED to survive failures gracefully, or whether each failure becomes an outage.

Why this matters: This is one of the 'invisible' qualities — like security — that senior engineers are expected to weave into every design without being explicitly asked. An interviewer who asks 'what happens if this server crashes?' partway through your design is testing exactly this instinct, and a strong candidate should ideally have already addressed it before being asked.

3. Core Concepts in Depth

3.1 The Nines of Availability

The Nines of Availability

99%	~3.65 days/year downtime
99.9%	~8.76 hours/year downtime
99.99%	~52.6 minutes/year downtime
99.999%	~5.3 minutes/year downtime

Each extra nine cuts allowable downtime by roughly 10x — and cost/complexity rises just as fast.

Figure 19.1 — Each additional 'nine' of availability cuts allowable downtime by roughly 10x.

Availability is usually expressed as a percentage of uptime over a year, informally referred to by how many '9s' it contains. The jump from 99% to 99.9% might look small as percentages, but as the diagram shows, it represents a roughly 10x reduction in actual allowed downtime — and each additional nine becomes progressively harder and more expensive to achieve, often requiring fundamentally different architecture (more redundancy, multi-region deployment, more sophisticated failover automation).

Why this matters: Being able to state roughly how much downtime 99.9% vs 99.99% represents (a memorized version of this table) is a small but noticeable detail that signals real familiarity with production systems, not just textbook knowledge.

3.2 Single Point of Failure (SPOF)

A single point of failure is any component whose failure alone is enough to take down the entire system — a single database with no replica, a single load balancer instance, a single server with no backup. Eliminating SPOFs is the central goal of fault-tolerant design: for every critical component, ask 'what happens to the whole system if this one thing breaks?' and if the answer is 'everything stops working,' that's a SPOF that needs redundancy.

Common mistake: Designing a system with redundant application servers but forgetting that the load balancer, the database, or even the DNS provider in front of everything is still a single point of failure. A

thorough SPOF analysis must walk through EVERY layer of the architecture, not just the most obvious one (usually app servers, which people tend to think about first since Chapter 6).

3.3 Active-Passive vs Active-Active Failover

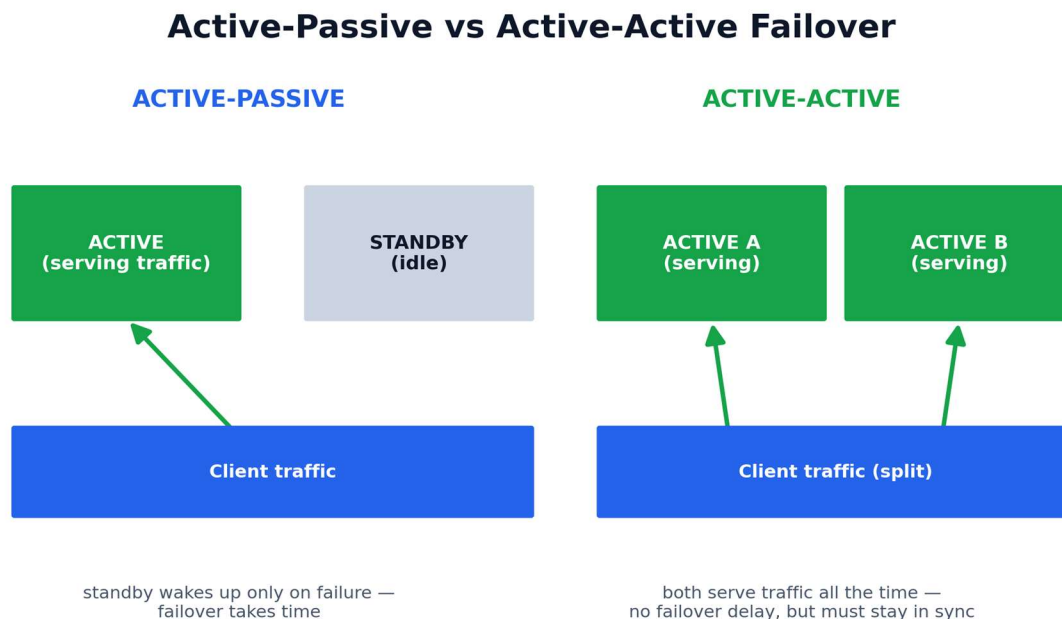


Figure 19.2 — A passive standby sits idle until needed; active-active has both nodes serving real traffic simultaneously.

Active-Passive	Active-Active
One server/region actively serves traffic; a standby sits idle, ready to take over on failure	Multiple servers/regions actively serve traffic simultaneously, all the time
Simpler to reason about — only one source of truth at any given moment	More complex — requires keeping multiple active nodes in sync (echoes CAP/replication themes from Chapters 5 and 9)
Wastes capacity — the standby does no useful work most of the time	Efficient — all provisioned capacity is actively used
Failover takes some time (detecting failure + promoting standby)	No failover delay needed for already-active nodes — but losing one still requires the others to absorb its load

3.4 Redundancy at Every Layer

Fault tolerance isn't a single technique — it's a mindset applied consistently across every layer of a system, and most of the individual techniques have already been covered earlier in this book under different names. This chapter's real contribution is connecting them as a unified theme.

- Application servers: multiple instances behind a load balancer (Chapter 13) — if one crashes, health checks route around it.

- Databases: replication (Chapter 9) provides redundant copies of data, with automated failover promoting a replica if the primary fails.
- Load balancers: redundant load balancer instances themselves, as discussed in Chapter 13's 'single point of failure' section.
- Data centers/regions: multi-region deployment, so an entire data center outage (power failure, natural disaster) doesn't take down the whole system.
- Message queues: replicated queue brokers (Chapter 14) so a single broker failure doesn't lose in-flight messages.

Why this matters: When asked 'how would you make this system more fault-tolerant,' the strongest answers systematically walk through each architectural layer like this list does, rather than only addressing the one component the interviewer happened to mention.

3.5 Disaster Recovery

Disaster recovery (DR) planning addresses the worst-case scenario: what happens if an entire region or data center becomes unavailable. Two key metrics define DR strategy:

Metric	What It Measures
RTO (Recovery Time Objective)	How long the system can be down before it must be restored — 'how fast do we need to recover?'
RPO (Recovery Point Objective)	How much data loss is acceptable, measured in time — 'how much recent data can we afford to lose?' (directly tied to how frequently backups/replication occur)

A low RTO/RPO (near-zero downtime, near-zero data loss) requires expensive, sophisticated infrastructure (multi-region active-active, continuous replication). A more relaxed RTO/RPO (a few hours of downtime and data loss is acceptable) can be achieved with much simpler, cheaper periodic backups — the right answer depends entirely on the business cost of downtime versus the cost of building redundancy.

4. Trade-offs Table

Decision	Pros	Cons
Active-passive failover	Simpler to implement and reason about	Wastes standby capacity; failover has some delay
Active-active failover	No idle capacity; better resource utilization	Significantly more complex to keep multiple active nodes consistent
Pursuing higher availability (more nines)	Less downtime, better user trust and experience	Exponentially more expensive and architecturally complex per additional nine

5. How Real Companies Use This

- Netflix deliberately runs chaos engineering tools (like Chaos Monkey) that randomly kill production instances, specifically to continuously verify their redundancy and failover mechanisms actually work, rather than assuming they do.

- Banks and financial institutions typically maintain strict RTO/RPO requirements with active-active multi-region setups for critical transaction systems, given the severe cost of downtime or data loss.
- Most cloud providers (AWS, GCP, Azure) structure their infrastructure into multiple availability zones within a region specifically so customers can build redundancy without needing fully separate geographic regions for moderate fault tolerance.

6. Common Mistakes & Misconceptions

- Adding redundancy to the obvious component (app servers) while overlooking less obvious single points of failure (DNS, a single load balancer, a single message queue broker).
- Assuming more nines of availability is always worth pursuing — for many systems, 99.9% is perfectly adequate, and chasing 99.999% would be a poor use of engineering effort and cost relative to the actual business need.
- Confusing redundancy with backups — redundancy provides immediate failover for continued availability; backups provide a recovery point after data loss or corruption. Both matter, but they solve different problems.

7. Interviewer Q&A Bank

Q1. What is a single point of failure, and how would you find them in a design?

A: A single point of failure is any component whose failure alone would take down the entire system. I'd find them by systematically walking through every layer of the architecture — load balancers, application servers, databases, message queues, even DNS — and asking 'what happens to the whole system if this specific piece fails?' Any component where the answer is 'everything stops' needs redundancy added.

Q2. What's the difference between active-passive and active-active failover?

A: In active-passive, one node actively serves traffic while a standby sits idle, ready to take over if the active node fails — simpler, but wastes the standby's capacity and has some failover delay. In active-active, multiple nodes serve traffic simultaneously all the time, making full use of all provisioned capacity, but requiring more complex coordination to keep multiple active nodes consistent with each other.

Q3. What do RTO and RPO mean, and how do they shape a disaster recovery strategy?

A: RTO (Recovery Time Objective) is how quickly the system must be restored after a disaster. RPO (Recovery Point Objective) is how much recent data loss is acceptable, which is directly tied to how frequently data is backed up or replicated. A near-zero RTO and RPO requires expensive, sophisticated infrastructure like continuous multi-region replication, while a more relaxed RTO/RPO can be met with simpler, cheaper periodic backups — the right choice depends on the actual business cost of downtime and data loss.

Q4. Is pursuing 99.999% availability always the right goal?

A: No — each additional nine of availability becomes exponentially more expensive and architecturally complex to achieve, and for many systems the business impact of occasional brief downtime simply doesn't justify that cost. I'd want to understand the actual cost of downtime for this specific system before committing to a particular availability target, rather than assuming higher is always better.

Q5. How would you test that your fault-tolerance mechanisms actually work?

A: Beyond just designing for redundancy, I'd advocate for actively testing failure scenarios in a controlled way — similar to chaos engineering practices like Netflix's Chaos Monkey, which deliberately kills production instances to verify failover actually happens as expected. Redundancy that's never actually been tested under real failure conditions is a common source of surprises during real incidents.

8. HLD Mini-Scenario: Fault-Tolerant Design for a Payments System

Prompt: "Design the fault tolerance strategy for a payment-processing system where even a few minutes of downtime is costly, and any data loss (a 'lost' transaction) is unacceptable."

- 1. Given the stated near-zero tolerance for downtime AND data loss, target a low RTO and RPO — this points toward active-active multi-region deployment with synchronous or near-synchronous replication (Chapter 9) for transaction data, accepting the added latency cost discussed in Chapter 5's PACELC trade-off, because correctness and uptime both matter enormously here.
- 2. Systematically eliminate single points of failure at every layer: redundant load balancers, multiple application server instances across availability zones, a replicated database with automated failover, and a replicated message queue for any asynchronous post-payment steps (Chapter 14).
- 3. Establish clear RTO/RPO targets with the business stakeholders first, since 'even a few minutes of downtime is costly' needs to be translated into a specific number before architecture decisions (and their real cost) can be properly justified.

This demonstrates connecting this chapter's concepts back to earlier ones (replication, CAP/PACELC, queues) — fault tolerance isn't a separate add-on, it's woven through every architectural decision already covered.

9. Questions YOU Should Ask the Interviewer

- What's the real business cost of downtime for this system — is this closer to a 99.9% or a 99.999% requirement?
- Is some data loss during a disaster acceptable, or must every transaction/write be preserved?
- Should we design for single-region failures (server/AZ level) or also full-region outages?

10. Scenario-Based Practice Bank (New)

Scenario 1: A team has redundant app servers behind a load balancer, but during a postmortem discovers the entire outage was caused by their single DNS provider going down.

How to approach it: This is a classic overlooked-SPOF case: redundancy at the app-server layer doesn't help if a layer above it (DNS) is a single point of failure. Recommend a systematic SPOF walk-through of every layer — DNS, load balancer, app servers, database, queues — not just the one everyone thinks of first.

Scenario 2: An executive asks why the team isn't targeting 99.999% availability 'since more is always better.'

How to approach it: Push back with the cost curve: each additional nine is exponentially more expensive and architecturally complex, often requiring multi-region active-active infrastructure. Ask what the

actual business cost of downtime is for this specific system — many systems are perfectly served by 99.9%, and chasing 99.999% would be a poor use of engineering effort relative to real need.

Scenario 3: A payments system needs near-zero data loss and near-zero downtime, and the team is debating active-passive vs active-active.

How to approach it: *Given the stated near-zero RTO/RPO requirements, recommend active-active with synchronous or near-synchronous replication, accepting the added latency (per PACELC) and coordination complexity — active-passive's failover delay and simpler model don't meet this system's strict uptime/data-loss bar.*

Scenario 4: A team has robust replication and redundancy on paper, but has never actually tested what happens when a primary database fails in production.

How to approach it: *Recommend proactive failure testing — chaos-engineering-style deliberate failure injection (like Netflix's Chaos Monkey) — since redundancy that's never been tested under real failure conditions is a common source of nasty surprises during actual incidents.*

Scenario 5: An interviewer asks you to distinguish 'redundancy' from 'backups' after you mention both in your design.

How to approach it: *Clarify precisely: redundancy provides immediate failover to keep the system available right now if a component fails; backups provide a recovery point to restore from after data loss or corruption, often used well after the fact. Both are necessary, but they solve genuinely different problems and shouldn't be conflated.*

Scenario 6: A company sets an RPO of 24 hours for a reporting system but an RPO of near-zero for its core transaction ledger, using the same nightly backup process for both.

How to approach it: *Flag the mismatch: a 24-hour-old backup is fine for the reporting system's stated RPO, but doesn't come close to meeting a near-zero RPO for the transaction ledger. The ledger needs continuous or near-continuous replication, not nightly backups, to actually meet its stricter data-loss tolerance.*

Chapter 19 — One-Page Revision Sheet

- Availability = % uptime. Each additional 'nine' (99% -> 99.9% -> 99.99%) cuts allowed downtime ~10x and gets exponentially harder/costlier to achieve.
- Single Point of Failure (SPOF): any component whose failure alone takes down the whole system — check EVERY layer (LB, app servers, DB, queues, DNS), not just the obvious ones.
- Active-Passive: standby idle until needed, simple but wastes capacity. Active-Active: all nodes serve traffic, efficient but harder to keep consistent.
- RTO (Recovery Time Objective) = how fast must we recover. RPO (Recovery Point Objective) = how much data loss is acceptable. Both drive how expensive/complex the DR strategy needs to be.
- Redundancy != backups: redundancy enables immediate failover for continued availability; backups enable recovery after data loss/corruption — you need both.
- Fault tolerance is woven through every layer already covered in this book: replication (Ch 9), load balancer redundancy (Ch 13), replicated queues (Ch 14) — it's a unifying theme, not a new add-on.
- Test failure scenarios proactively (chaos engineering) rather than assuming redundancy works until a real incident proves otherwise.

Memory hook: *A four-engine airplane doesn't need all four to fly — the extra engines are redundancy, not waste, so losing one mid-flight is a non-event instead of a crash.*

CHAPTER 20

Monitoring, Logging & Observability

How would you even know this system is failing?

1. Simple Explanation First

Observability is the ability to understand what's happening inside a system from the outside, by examining the data it produces — without needing to add new code every time you have a new question. It's built from three complementary types of signals: metrics (numbers over time), logs (records of discrete events), and traces (the full journey of one specific request across multiple services).

Analogy: Think of observability like the instruments in a car. The dashboard's speedometer and fuel gauge are metrics — constant numbers telling you the car's general state at a glance. The car's onboard event log (recorded fault codes, like 'door opened at 3:42 PM') is like application logs — discrete, timestamped events. And if a mechanic plugs in a diagnostic tool to trace exactly which sensor triggered a warning light through the car's full electrical system, that's a trace — following one specific signal's complete path through many interconnected components.

2. Why This Topic Exists

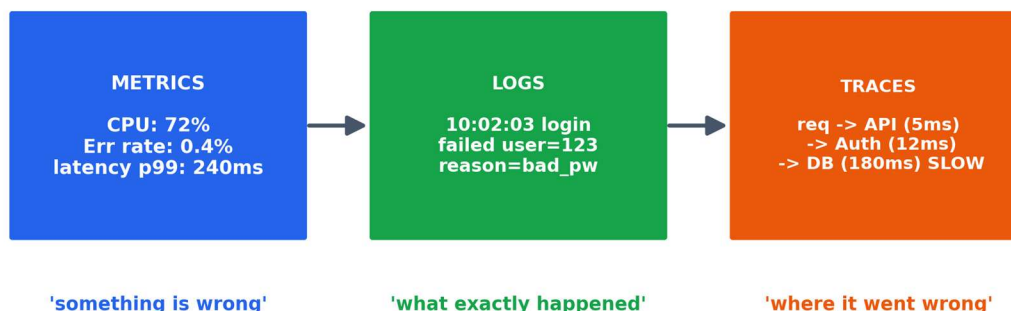
A distributed system made of dozens of services, databases, and caches is fundamentally a black box once it's running in production — you can't just attach a debugger and step through code on a live system serving millions of users. Observability tooling exists to make that black box transparent: to answer 'is the system healthy right now?', 'what just broke?', and 'where, across all these services, did this specific failure happen?' without guessing.

Why this matters: This chapter answers a question interviewers explicitly love to ask near the end of an HLD discussion: 'how would you know if this system is failing?' Many candidates who nail the architecture forget this question entirely, treating monitoring as an afterthought rather than a first-class design concern.

3. Core Concepts in Depth

3.1 Metrics vs Logs vs Traces

Metrics vs Logs vs Traces



Typical flow: metrics flag an issue -> logs explain what happened -> traces pinpoint where.

Figure 20.1 — Metrics tell you something is wrong; logs tell you what happened; traces tell you where it went wrong.

Signal	What It Is	Best For
Metrics	Numeric measurements aggregated over time (CPU usage, request count, error rate, latency percentiles)	Dashboards, alerting, spotting trends — answering 'is something wrong right now?'
Logs	Timestamped, discrete records of individual events, often with context (e.g., 'User 123 login failed at 10:02:03, reason: bad password')	Debugging a specific incident — answering 'what exactly happened?'
Traces	A record of one request's complete journey across every service it touched, with timing for each step	Diagnosing where in a multi-service request a slowdown or error occurred — answering 'where in this chain did it go wrong?'

These three signals work together in a typical investigation flow: a metrics dashboard shows error rate spiking (something is wrong) -> you check logs filtered to that time window to see specific error messages (what happened) -> for a multi-service issue, you pull a trace of an affected request to see exactly which downstream service introduced the delay or error (where it went wrong).

3.2 Latency Percentiles — Why Averages Lie

A very common mistake is reporting only the AVERAGE response time of a system, which can be deeply misleading. If 95% of requests take 50ms but 5% take 5 seconds (perhaps hitting an uncached, slow path),

the average might look like a perfectly fine 290ms — completely hiding the fact that a meaningful fraction of real users are having a terrible experience.

Why Averages Lie — Latency Percentiles



Average looks fine (~55ms) — but the slow p99 tail is a real, hidden user pain point.

Figure 20.2 — A handful of very slow requests barely move the average, but dominate the p99.

Metric	What It Tells You
p50 (median)	Half of all requests are faster than this — represents the 'typical' experience
p95	95% of requests are faster than this — captures most users, including some slower ones
p99	99% of requests are faster than this — captures the 'tail' experience, often where real problems hide

Common mistake: Reporting or optimizing only for average latency. Engineers who care about real user experience track p95/p99 latency specifically because slow outliers are often concentrated on certain code paths, certain users, or certain times — and averages mathematically smear that signal away. Mentioning 'p99 latency' instead of just 'average latency' in an interview is a small but reliable signal of production experience.

3.3 Alerting

Collecting metrics, logs, and traces is only useful if someone is notified when something goes wrong — that's the role of alerting. A good alerting strategy walks a careful line: alert too rarely, and real incidents go unnoticed; alert too often (or on noisy, non-actionable conditions), and engineers start ignoring alerts entirely — a phenomenon known as alert fatigue, which is genuinely dangerous because it means real alerts get dismissed along with the noise.

- Alert on symptoms that matter to users (e.g., error rate, p99 latency, availability), not just internal implementation details (e.g., 'CPU is at 70%' might be completely fine and expected).

- Set thresholds based on what's actually abnormal for THIS system, not arbitrary round numbers — this usually means establishing a baseline first.
- Route alerts to the right people with the right urgency — a minor degraded-but-functional issue shouldn't page someone awake at 3 AM the same way a full outage should.

3.4 Dashboards

A dashboard is a curated, visual summary of a system's key metrics, designed to answer 'is everything okay?' at a glance, without needing to query raw data every time. Well-designed dashboards typically follow a layered structure: a small set of top-level, business-critical metrics (overall error rate, request volume, p99 latency) visible immediately, with the ability to drill down into more granular, component-specific metrics when something looks off.

Why this matters: When asked 'how would you know this system is failing,' describing a dashboard built around the same key metrics you'd alert on (error rate, latency percentiles, availability) — rather than a vague 'we'd have monitoring' — demonstrates that you've actually thought through what 'healthy' looks like for this specific system, not just that monitoring exists in the abstract.

4. Trade-offs Table

Decision	Pros	Cons
Tracking p99 latency alongside averages	Reveals real tail-end user experience, hidden by averages alone	Slightly more complex to collect and reason about than a single average number
Aggressive, low-threshold alerting	Catches issues earlier	Risk of alert fatigue if thresholds are too sensitive or not user-impact-focused
Detailed distributed tracing on every request	Excellent debugging visibility across services	Added overhead (storage, slight latency) — often sampled rather than capturing 100% of requests at scale

5. How Real Companies Use This

- Google popularized the 'four golden signals' (latency, traffic, errors, saturation) as the minimal set of metrics worth monitoring for any service, a framework widely adopted across the industry.
- Companies like Datadog, New Relic, and Honeycomb exist specifically to provide unified metrics/logs/traces observability platforms, since building this tooling in-house is a significant undertaking most companies prefer to outsource.
- Distributed tracing standards like OpenTelemetry have become an industry-wide standard specifically to let traces flow consistently across services written in different languages and owned by different teams within the same company.

6. Common Mistakes & Misconceptions

- Treating monitoring as an afterthought added after a system is built, rather than a first-class design concern discussed alongside the architecture itself.

- Relying only on average latency/response time metrics, which can hide serious tail-latency problems affecting a meaningful fraction of real users.
- Setting up alerting without considering alert fatigue — too many low-value alerts trains engineers to ignore notifications, which defeats the entire purpose of alerting.

7. Interviewer Q&A Bank

Q1. How would you know if this system you just designed is failing in production?

A: I'd track key metrics like error rate, request throughput, and latency percentiles (especially p95/p99, not just average) on a dashboard, with alerts configured on user-impacting thresholds for those same metrics. For deeper investigation when something looks wrong, I'd rely on logs to see specific error details, and distributed tracing to pinpoint exactly which service in a multi-service request introduced a slowdown or failure.

Q2. Why is average latency potentially misleading?

A: Averages can hide the fact that a meaningful fraction of requests are much slower than the typical experience — if most requests are fast but a tail of requests are very slow, the average can still look acceptable while real users are having a bad experience. Tracking p95 and p99 latency specifically surfaces this tail behavior, which is often where genuine problems concentrate.

Q3. What's the difference between metrics, logs, and traces, and when would you use each?

A: Metrics are aggregated numbers over time, good for dashboards and alerting — answering 'is something wrong right now?' Logs are detailed records of individual events, good for understanding exactly what happened during a specific incident. Traces follow one request's full journey across multiple services, good for pinpointing exactly where in a distributed system a problem occurred. A typical investigation moves from metrics (spotting an issue) to logs (understanding details) to traces (locating the exact failure point across services).

Q4. What is alert fatigue, and how would you avoid it?

A: Alert fatigue happens when engineers receive so many low-value or non-actionable alerts that they start ignoring notifications altogether, including genuinely important ones. I'd avoid it by alerting on metrics that directly reflect user impact (error rate, latency, availability) rather than every internal implementation detail, and by setting thresholds based on what's genuinely abnormal for the specific system rather than arbitrary numbers.

Q5. Would you trace every single request in a high-traffic system?

A: Not necessarily at full volume — full tracing on every request adds storage and processing overhead that can become significant at very high scale, so many systems use sampling, tracing a representative percentage of requests (or specifically tracing requests that errored or were unusually slow) rather than 100% of all traffic, balancing debugging visibility against overhead cost.

8. HLD Mini-Scenario: Observability Strategy for a Checkout Service

Prompt: "Design the monitoring and observability strategy for an e-commerce checkout service, which calls inventory, payment, and shipping services as part of completing an order."

- 1. Set up dashboards tracking checkout-specific metrics: overall checkout success rate, p99 latency of the full checkout flow, and error rate broken down by which downstream service (inventory, payment, shipping) caused the failure.

- 2. Configure alerts on checkout success rate dropping below a baseline threshold and on p99 latency exceeding an acceptable bound — these are the metrics that directly reflect whether real customers can actually complete purchases.
- 3. Implement distributed tracing across the checkout flow specifically, since it touches multiple services — when checkout success rate drops, traces let engineers immediately see whether the bottleneck or failure is in inventory, payment, or shipping, rather than guessing.

This shows observability designed around the SPECIFIC business-critical flow (checkout) rather than generic, undifferentiated monitoring — exactly the kind of targeted thinking interviewers want to see.

9. Questions YOU Should Ask the Interviewer

- Are there existing observability tools/platforms in place, or should I assume we're choosing/building this from scratch?
- What's the business-critical user journey we most need visibility into?
- Is there an existing incident response process this monitoring needs to integrate with?

10. Scenario-Based Practice Bank (New)

Scenario 1: A dashboard shows average response time holding steady at 300ms, but customer support tickets about 'slow checkout' have been increasing.

***How to approach it:** This is the averages-lie problem: check p95/p99 latency specifically, since a growing tail of very slow requests can hide behind a stable-looking average while a real, growing subset of users suffers a bad experience.*

Scenario 2: An on-call engineer has started silencing entire alert channels because dozens of low-value pages arrive every night.

***How to approach it:** This is alert fatigue in action, and it's dangerous because real incidents now get silenced along with the noise. Audit existing alerts and keep only ones tied to genuine user-impacting symptoms (error rate, p99 latency, availability), removing or down-prioritizing internal-detail alerts like routine CPU fluctuations.*

Scenario 3: A checkout flow spans inventory, payment, and shipping services, and error rate just spiked, but nobody knows which service is responsible.

***How to approach it:** This is exactly what distributed tracing is for: pull traces of affected checkout requests to see the timing and status of each downstream call (inventory, payment, shipping) and immediately identify which specific service introduced the failure or slowdown, rather than guessing.*

Scenario 4: A team wants to trace 100% of requests in a very high-traffic API for maximum debugging visibility.

***How to approach it:** Flag the overhead cost: full tracing on every request adds real storage and processing burden at high volume. Recommend sampling — tracing a representative percentage of all requests, plus specifically capturing traces for requests that errored or were unusually slow — balancing visibility against cost.*

Scenario 5: An interviewer asks near the end of your HLD answer: 'how would you know if this system you just designed was failing right now?'

How to approach it: Don't treat this as an afterthought — describe concrete dashboards built on error rate, request throughput, and p95/p99 latency, with alerts on the same user-impacting thresholds, plus logs and tracing for deeper investigation. A vague 'we'd have monitoring' answer misses the point of this question.

Scenario 6: A new logging system is capturing every event with full detail, but during a recent incident, engineers still couldn't quickly tell which specific service in a 6-service request chain caused a slowdown.

How to approach it: Logs alone don't show cross-service timing relationships. Recommend adding distributed tracing (e.g., via an OpenTelemetry-based setup) so a single request's full journey — with per-service timing — can be inspected directly, rather than manually correlating timestamps across six separate services' logs.

Chapter 20 — One-Page Revision Sheet

- Observability = understanding system health from the outside via metrics (numbers over time), logs (discrete events), and traces (one request's full journey across services).
- Typical investigation flow: metrics show something is wrong -> logs show what happened -> traces show where, across services, it happened.
- Track p95/p99 latency, not just averages — averages hide tail-latency problems affecting a real subset of users.
- Alert on user-impacting metrics (error rate, latency, availability), not internal details — avoid alert fatigue from too many low-value alerts.
- Dashboards should surface top-level, business-critical metrics first, with drill-down into component-level detail when something looks off.
- Full tracing on every request has real overhead at scale — many systems sample, tracing a representative subset or specifically errored/slow requests.
- Always proactively address 'how would you know this is failing' in an HLD interview — don't wait to be asked.

Memory hook: A car's dashboard (metrics), its fault-code event log (logs), and a mechanic's full diagnostic trace through the electrical system (traces) — three different views of the same vehicle, each answering a different question.

CHAPTER 21

Security Fundamentals for System Design

Who are you, what are you allowed to do, and is anyone listening in?

1. Simple Explanation First

Security in system design centers on a few core questions: how do we verify WHO is making a request (authentication), what are they ALLOWED to do (authorization), and how do we keep data private as it travels and rests (encryption)? On top of these basics sit practical concerns like defending against attackers trying to overwhelm the system (DDoS) or steal data.

Analogy: Think of a secure office building. Authentication is showing your ID badge at the front door to prove who you are. Authorization is your badge only opening certain doors — the intern's badge doesn't open the executive floor, even though the building already confirmed who they are. Encryption is like sending documents in a locked briefcase that only the intended recipient has the key to, so even if someone intercepts it in transit, they can't read the contents.

2. Why This Topic Exists

Any system that handles user data, money, or anything valuable is a target. Security isn't an optional feature bolted on later — a system designed without security in mind from the start tends to have security woven so shallowly into its architecture that retrofitting real protection later is painful and error-prone. This topic exists because the cost of a security failure (data breach, financial loss, loss of user trust) is often catastrophic and disproportionate compared to the cost of designing for it upfront.

Why this matters: Interviewers — especially at companies handling sensitive data (fintech, healthcare) — routinely slip security questions into HLD interviews, even when the prompt doesn't explicitly mention security. Proactively addressing authentication and authorization when sketching an API layer, without being asked, is a mark of a mature engineer.

3. Core Concepts in Depth

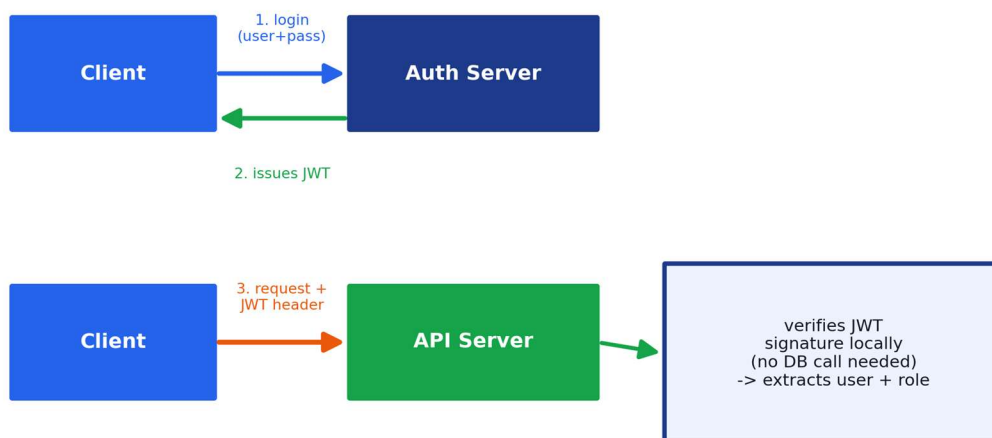
3.1 Authentication vs Authorization (AuthN vs AuthZ)

These two terms are frequently confused but answer fundamentally different questions, and conflating them is a common interview stumble.

Authentication (AuthN)	Authorization (AuthZ)
Answers: 'Who are you?'	Answers: 'What are you allowed to do?'
Verifies identity — username/password, biometrics, a valid token	Checks permissions — does this verified identity have access to this specific resource/action?
Happens first, establishing identity	Happens after authentication, using that established identity to make access decisions
Example: logging into an account	Example: a logged-in regular user being blocked from an admin-only page

3.2 The Authentication Flow: Login, Tokens, and Trusted Requests

Login Once, Then Trust the Token



The JWT itself carries identity — verified by signature, not a database lookup.

Figure 21.1 — After login, a token lets the API server verify identity without re-checking credentials on every request.

Re-checking a username and password on every single request would be slow and would mean sending sensitive credentials over the network constantly. Instead, the common pattern is: the client logs in once, receives a token (often a JWT — JSON Web Token), and includes that token on every subsequent request. The API server can verify the token's signature locally (without calling back to the auth server) to confirm it's genuine and hasn't been tampered with, extracting the user's identity and permissions directly from the token's contents.

- JWT (JSON Web Token): a self-contained token that includes encoded claims (like user ID, role, expiration time) and a cryptographic signature. Any server with the right public key can verify it's authentic without a database lookup.
- OAuth2: a broader authorization framework, commonly used for 'login with Google/Facebook' style flows, where a third-party service grants limited access to a user's data without ever sharing that user's actual password with the requesting application.

Common mistake: Treating JWTs as automatically secure just because they're signed. A signed JWT proves the token wasn't tampered with and was genuinely issued by the auth server — but if a JWT is stolen (e.g., via a compromised client), anyone holding it can use it until it expires. This is why JWTs should have a short expiration time and often a separate, revocable refresh token mechanism, rather than being trusted indefinitely.

3.3 Encryption: In Transit vs At Rest

Data needs protection in two different states, each requiring a different mechanism.

Encryption: In Transit vs At Rest

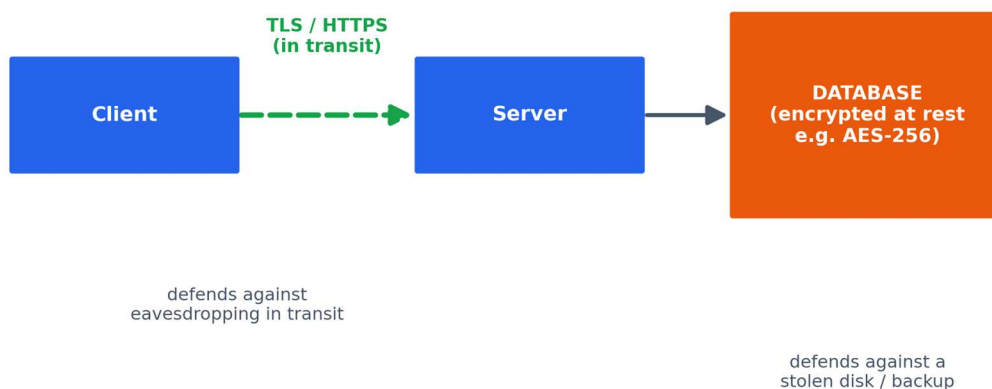


Figure 21.2 — TLS protects data on the wire; disk/database-level encryption protects data once it's stored.

Encryption in Transit	Encryption at Rest
Protects data while it's traveling across a network	Protects data while it's sitting in storage (database, disk, backups)
Implemented via TLS/HTTPS (recall Chapter 3's TLS handshake)	Implemented via disk-level or database-level encryption (e.g., AES-256)
Defends against: eavesdropping on network traffic, man-in-the-middle attacks	Defends against: a stolen hard drive, an unauthorized party gaining direct access to storage/backups

Both are necessary and address different threats — encrypting data in transit doesn't help if someone steals the physical hard drive (or unauthorized database snapshot) where data sits unencrypted, and encrypting data at rest doesn't help if it's transmitted in plain text and intercepted along the way.

3.4 DDoS Protection Basics

A DDoS (Distributed Denial of Service) attack attempts to overwhelm a system with a flood of traffic from many sources simultaneously, making it unavailable to legitimate users. This connects directly back to concepts already covered: rate limiting (Chapter 15) at the API Gateway helps absorb abusive traffic per client, while CDNs (Chapter 16) help absorb large-scale traffic floods at the edge, far from the origin servers, since edge nodes are specifically built to handle massive concurrent traffic.

- Rate limiting and throttling: limits how much any single client can request, blunting attacks from a limited number of sources.
- CDN/edge absorption: distributes and absorbs traffic across many geographically distributed edge nodes before it ever reaches origin infrastructure.
- Specialized DDoS protection services (e.g., Cloudflare, AWS Shield): purpose-built to detect and filter malicious traffic patterns at massive scale, something most individual companies can't replicate themselves.

3.5 Common Security Practices Worth Mentioning

- Principle of least privilege: every component, service, and user should have the minimum level of access needed to do its job, and nothing more — limiting the damage if any one piece is compromised.
- Input validation/sanitization: never trust data coming from a client; validate and sanitize it on the server to prevent injection attacks (SQL injection, XSS).
- Secrets management: API keys, database credentials, and encryption keys should be stored in dedicated secrets management tools (not hardcoded in source code), with controlled, audited access.

4. Trade-offs Table

Decision	Pros	Cons
Short-lived JWTs with refresh tokens	Limits damage window if a token is stolen	Slightly more complex client-side logic to handle token refresh
Centralizing auth at the API Gateway (Chapter 15)	Consistent enforcement, less duplicated security logic	Gateway becomes a high-value target requiring strong protection itself
Encrypting data at rest	Protects against physical theft / unauthorized storage access	Some performance overhead; key management adds operational complexity

5. How Real Companies Use This

- Most modern web applications use short-lived JWT access tokens (minutes to a couple hours) combined with longer-lived, revocable refresh tokens, balancing security with not forcing users to log in constantly.
- Companies handling payment data must comply with PCI-DSS standards, which mandate specific encryption, access control, and auditing practices for any system touching card data.
- Cloudflare and AWS Shield protect a significant percentage of the internet's traffic from DDoS attacks, since building this level of distributed traffic-absorption capability in-house is impractical for most companies.

6. Common Mistakes & Misconceptions

- Confusing authentication with authorization in interview answers — always be precise about which one a given check is performing.
- Assuming HTTPS alone (encryption in transit) is sufficient security — it says nothing about how data is protected once it's stored, or about authorization rules within the application.
- Treating security as a final checklist item rather than a consideration woven through the architecture (auth at the gateway, least privilege between services, encryption at every layer where it matters).

7. Interviewer Q&A Bank

Q1. What's the difference between authentication and authorization?

A: Authentication verifies who someone is — confirming their identity, typically via credentials or a token. Authorization determines what an already-authenticated identity is allowed to do — which resources or actions they have permission to access. Authentication always happens first, establishing identity that authorization then uses to make access decisions.

Q2. How does a JWT let an API server verify a user without hitting the database on every request?

A: A JWT is a self-contained token with encoded claims (like user ID and role) and a cryptographic signature from the auth server. The API server can verify that signature locally using a shared secret or public key, confirming the token is genuine and unmodified, without needing to call back to the auth server or database — this makes authentication checks fast and avoids that becoming a bottleneck on every request.

Q3. What's the risk if a JWT is stolen, and how do systems mitigate it?

A: Anyone holding a valid JWT can use it to make authenticated requests until the token expires, since the server only verifies the signature, not who's presenting it. Systems mitigate this by keeping JWT expiration times short and pairing them with a separate, longer-lived refresh token that can be revoked server-side if compromise is suspected, limiting the window of exposure from a stolen access token.

Q4. What's the difference between encryption in transit and encryption at rest, and why do you need both?

A: Encryption in transit (via TLS/HTTPS) protects data while it's traveling across a network, defending against eavesdropping. Encryption at rest protects data while it's stored, defending against threats like a stolen hard drive or unauthorized access to backups. They protect against different threat scenarios, so a complete security posture needs both — protecting data only in transit leaves stored data exposed, and vice versa.

Q5. How would you protect a public API from a DDoS attack?

A: I'd use rate limiting at the API Gateway (Chapter 15) to cap how much any single client can request, and route traffic through a CDN or dedicated DDoS protection service (Chapter 16) so a large-scale traffic flood is absorbed at distributed edge locations before it ever reaches origin infrastructure. For especially large-scale attacks, I'd rely on a specialized provider like Cloudflare or AWS Shield, since matching that scale of attack absorption in-house usually isn't practical.

8. HLD Mini-Scenario: Securing a Multi-Tenant SaaS Application

Prompt: "Design the authentication and authorization model for a multi-tenant SaaS product, where each customer organization has multiple users with different roles (admin, member, viewer), and one customer's data must never be accessible to another."

- 1. Use JWTs for authentication, with each token's claims including the user's identity, their organization (tenant) ID, and their role — issued after login and verified locally by backend services on each request.
- 2. Implement authorization checks at two levels: tenant isolation (every data query is automatically scoped to the requesting user's organization ID, so cross-tenant access is structurally prevented, not just policy-prevented) and role-based permissions within that tenant (e.g., only 'admin' role can modify billing settings).

- 3. Apply the principle of least privilege to internal service-to-service communication too — the billing service shouldn't have broad access to the entire user database, only the specific data it actually needs.

This shows security designed as a structural property of the data model (tenant ID scoping) rather than just an application-logic check that could be accidentally bypassed — a key distinction for multi-tenant systems specifically.

9. Questions YOU Should Ask the Interviewer

- What's the sensitivity of the data involved — are there specific compliance requirements (PCI-DSS, HIPAA, GDPR) we need to design around?
- Is this a single-tenant or multi-tenant system, and how strict does data isolation need to be?
- Should I assume a third-party identity provider (like Google/Okta login), or are we building authentication from scratch?

10. Scenario-Based Practice Bank (New)

Scenario 1: A mobile app's access token was intercepted, and the attacker was able to make authenticated requests for the next 6 hours before the token expired.

How to approach it: This is expected JWT-theft behavior at a 6-hour expiration — the fix is to shorten access token lifetime significantly (minutes, not hours) and pair it with a separate, revocable refresh token, so a stolen access token's window of usefulness shrinks dramatically.

Scenario 2: A multi-tenant SaaS product had a bug where an application-logic check was accidentally skipped on one endpoint, briefly exposing one customer's data to another.

How to approach it: Recommend structural tenant isolation instead of relying purely on application-logic checks: scope every database query automatically by the requesting user's organization ID at the data-access layer, so cross-tenant leakage becomes structurally prevented rather than dependent on every single endpoint remembering to check correctly.

Scenario 3: A company encrypts all traffic with HTTPS and considers their security posture complete.

How to approach it: Point out the gap: HTTPS only covers encryption in transit. Ask about encryption at rest (protecting stored data/backups from theft or unauthorized access) and authorization rules within the application — HTTPS alone says nothing about either.

Scenario 4: A public API starts receiving a massive flood of requests from thousands of different IP addresses simultaneously, degrading service for legitimate users.

How to approach it: This is a DDoS pattern too distributed for simple per-client rate limiting alone to fully solve. Recommend routing through a CDN or dedicated DDoS protection service (Cloudflare, AWS Shield) to absorb the flood at the edge, combined with API Gateway rate limiting for more moderate abuse.

Scenario 5: A junior engineer gives the billing microservice broad read access to the entire user database 'just in case it needs something later.'

How to approach it: Flag this as a violation of least privilege: scope the billing service's access to only the specific user fields it actually needs (e.g., billing address, payment method), not the entire user database — limiting the damage if that service is ever compromised.

Scenario 6: An interviewer asks you to explain the difference between authentication and authorization after you used the terms interchangeably in your answer.

How to approach it: Correct precisely: authentication answers 'who are you' (verifying identity via credentials or a token), while authorization answers 'what are you allowed to do' (checking permissions for an already-established identity). Authentication always happens first; authorization is the subsequent access-control decision.

Chapter 21 — One-Page Revision Sheet

- Authentication (AuthN) = who are you. Authorization (AuthZ) = what are you allowed to do. AuthN always happens first.
- JWTs let servers verify identity locally (via signature check) without a database call on every request — but a stolen JWT is valid until it expires, so keep them short-lived and pair with a revocable refresh token.
- OAuth2 enables 'login with X' flows, granting limited access without sharing the actual password with the requesting app.
- Encryption in transit (TLS/HTTPS) protects data on the network. Encryption at rest protects stored data. You need both — they defend against different threats.
- DDoS protection combines rate limiting (Chapter 15) at the gateway and CDN/edge absorption (Chapter 16) — plus specialized services (Cloudflare, AWS Shield) at extreme scale.
- Principle of least privilege: every service/user gets the minimum access needed, nothing more — limits blast radius if something is compromised.
- For multi-tenant systems, bake tenant isolation into the data model itself (structural prevention), not just application-logic checks that could be bypassed.

Memory hook: A secure office building: your badge proves who you are (authentication), but it only opens certain doors (authorization) — and sensitive documents travel in a locked briefcase (encryption in transit) and live in a locked safe once delivered (encryption at rest).

CHAPTER 22

Data Storage at Scale — Blob/Object Storage & File Systems

Where do you put a billion photos, videos, and PDFs?

1. Simple Explanation First

Object storage (also called blob storage — Binary Large Object) is a storage system purpose-built for storing large, unstructured files — images, videos, PDFs, backups — as whole objects, each with a unique key, rather than as rows in a database table. It's fundamentally different from the relational and NoSQL databases covered in Part 2, which are optimized for structured, queryable data, not massive binary files.

Analogy: Think of a database like a filing cabinet with neatly labeled folders containing structured forms — great for quickly finding 'all customers in California.' Object storage is more like a massive warehouse of sealed boxes, each with a barcode (the key). You can't search inside a box without opening it, but you can retrieve any specific box instantly if you know its barcode, and the warehouse can hold an effectively unlimited number of boxes of any size, from a tiny note to an entire shipping container.

2. Why This Topic Exists

Storing a 50MB video file as a row in a relational database would be wildly inefficient — databases are optimized for fast, structured queries over relatively small rows, not for serving massive binary blobs. As applications increasingly need to store huge volumes of images, videos, and documents (which now make up the vast majority of data volume on the internet), a specialized storage system was needed: one optimized for simple key-based retrieval of large files at virtually unlimited scale, rather than complex queries.

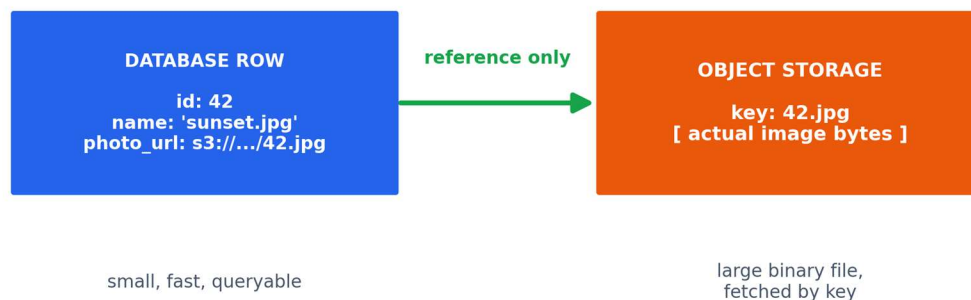
Why this matters: Almost every modern application that lets users upload photos, videos, or documents needs object storage as a core component, making it one of the most practically relevant topics in this book — yet it's often under-discussed in interviews because candidates default to 'we'll store it in the database' without thinking about file size and access patterns.

3. Core Concepts in Depth

3.1 Object Storage vs Traditional Databases

It's important to be precise about why object storage is a SEPARATE component from the databases covered in Part 2, not a competing alternative to them.

The Standard Pattern: DB Reference + Object Storage File



Never store the file bytes in the database row — store a reference/URL instead.

Figure 22.1 (new) — The standard pattern: a small database row references a large file living in object storage.

Object Storage (S3-like)	Relational/NoSQL Database
Optimized for storing and retrieving large binary files by key	Optimized for structured, queryable data with relationships
Simple operations: PUT (store), GET (retrieve), DELETE — no complex queries	Rich query capabilities: filters, joins, aggregations, indexes (Chapter 8)
Virtually unlimited storage capacity per object and per bucket	Practical limits on row size; not designed for huge binary blobs
Example: Amazon S3, Google Cloud Storage, Azure Blob Storage	Example: PostgreSQL, MongoDB, Cassandra (Chapter 7)

Common mistake: Storing large binary files directly in a relational database (e.g., as a BLOB column). This bloats the database, slows down backups and replication (Chapter 9), and wastes a resource optimized for structured queries on something it's not designed for. The correct pattern is to store the actual file in object storage, and store just a REFERENCE (a URL or object key) to that file in the database row — e.g., a 'profile_photo_url' column pointing to an S3 object, not the photo bytes themselves.

3.2 When to Use Blob Storage vs a Database

The decision is really about the SHAPE and SIZE of the data, and how it needs to be accessed.

- Use object storage for: images, videos, audio files, PDFs, backups, log archives, any large file that's retrieved as a whole unit by a known key, rather than queried by its internal content.
- Use a database for: structured data that needs to be queried, filtered, joined, or updated in small pieces — user profiles, orders, relationships between entities.

- Most real systems use BOTH together: the database stores structured metadata and a reference/URL, while object storage holds the actual large file — this is the dominant real-world pattern, not an either/or choice.

3.3 Distributed File Systems (Brief Context)

Before cloud object storage became dominant, large-scale systems often used distributed file systems like HDFS (Hadoop Distributed File System) to store and process huge volumes of data across many machines, particularly for big-data processing workloads (e.g., running analytics jobs across petabytes of log data). HDFS splits large files into blocks distributed and replicated across many nodes, conceptually similar to the sharding and replication ideas from Part 2, but applied to raw files rather than database rows. You don't need deep HDFS expertise for most interviews — just recognize it as the 'big data processing' answer to a problem object storage alone doesn't fully solve (high-throughput parallel processing of huge datasets, not just storage and retrieval).

3.4 Storage Tiers — Matching Cost to Access Frequency

Storage Tiers — Matching Cost to Access Frequency



Figure 22.2 — Data automatically migrates from hot to cold storage as it ages and is accessed less often.

Not all stored data is accessed equally often, and storage providers offer different price/performance tiers to match this reality. Storing rarely-accessed data (like 3-year-old log archives) on the same expensive, low-latency storage as actively-served content (like this week's most-viewed videos) wastes money for no real benefit.

Tier	Characteristics
Hot	Frequently accessed; optimized for low latency; highest cost per GB
Warm	Accessed occasionally; balanced cost and retrieval speed
Cold/Archive	Rarely accessed; cheapest storage cost, but slower (sometimes minutes to hours) retrieval time

Many object storage providers support lifecycle policies — automated rules that move data between tiers based on age or access patterns (e.g., 'move to cold storage after 90 days of no access') without requiring manual intervention, letting cost optimization happen automatically as data naturally ages.

Why this matters: Mentioning storage tiers and lifecycle policies when discussing data storage at scale demonstrates cost-consciousness — a quality interviewers at companies operating at real scale specifically value, since storage costs can become a very real, significant line item as systems grow.

4. Trade-offs Table

Decision	Pros	Cons
Storing files in object storage + reference in DB	Keeps database lean and fast; object storage scales to virtually unlimited size	Requires coordinating two systems (e.g., handling a DB write succeeding but the file upload failing)
Using storage tiers with lifecycle policies	Significant cost savings on infrequently-accessed data	Retrieval from cold storage can be slow — not suitable for data that might suddenly need fast access
Distributed file systems (HDFS) for big-data processing	High-throughput parallel processing across huge datasets	More operational complexity than cloud object storage; less relevant if you don't have heavy batch-processing needs

5. How Real Companies Use This

- Instagram and most photo/video platforms store the actual media files in object storage (often via a CDN in front of it, connecting back to Chapter 16), while their databases store only metadata — captions, likes, timestamps, and a reference URL to the actual file.
- Netflix stores its enormous video catalog in object storage, with content automatically distributed to CDN edge nodes (Open Connect, from Chapter 16) for actual playback delivery.
- Companies with regulatory data-retention requirements (e.g., financial transaction logs that must be kept for 7 years but are almost never accessed) heavily rely on cold/archive storage tiers to keep compliance costs manageable.

6. Common Mistakes & Misconceptions

- Storing large binary files directly in a relational database instead of referencing them from object storage — a classic, costly architectural mistake.

- Forgetting to mention a CDN (Chapter 16) when discussing how object-stored content (like images or videos) actually gets served to end users — object storage and CDN are typically used together, not as alternatives.
- Treating all stored data as equally 'hot' and paying premium storage costs for data that's rarely if ever accessed after initial creation.

7. Interviewer Q&A Bank

Q1. Why wouldn't you store a user's uploaded profile photo directly in your relational database?

A: Storing large binary files directly in a relational database bloats the database, slows down backups and replication, and isn't what the database is optimized for. The standard pattern is to upload the actual photo to object storage (like S3), and store just a reference — a URL or object key — in the database row, keeping the database lean and using object storage for what it's actually built for.

Q2. When would you choose object storage over a traditional database?

A: When the data is a large, unstructured file — images, videos, PDFs, backups — that's retrieved as a whole unit by a known key, rather than queried, filtered, or joined like structured data. In practice, most real systems use both together: the database for structured, queryable metadata, and object storage for the actual large files, rather than choosing one exclusively.

Q3. What are storage tiers, and why would you use them?

A: Storage tiers (hot, warm, cold/archive) offer different price and performance trade-offs based on how frequently data is accessed — hot tiers are fast but expensive, cold tiers are slow to retrieve but very cheap. I'd use lifecycle policies to automatically migrate data to cheaper tiers as it ages and is accessed less often, which can produce significant cost savings for data like old logs or infrequently-viewed archives without manual intervention.

Q4. How does object storage typically work alongside a CDN?

A: Object storage serves as the durable origin store for large files, while a CDN (Chapter 16) caches copies of frequently-accessed files at edge locations close to users, dramatically reducing latency and reducing direct load on the object storage origin. They're complementary, not competing — object storage answers 'where does the file durably live,' and the CDN answers 'how do we serve it to users quickly worldwide.'

Q5. What's the difference between object storage and a distributed file system like HDFS?

A: Object storage (like S3) is primarily designed for durable storage and retrieval of files by key, with a simple API (PUT/GET/DELETE). Distributed file systems like HDFS are designed for high-throughput PARALLEL PROCESSING of huge datasets across many machines, typically for big-data analytics workloads, splitting files into blocks distributed and replicated across nodes. You'd reach for HDFS (or similar) specifically when you need to run large-scale parallel computation over the data, not just store and retrieve it.

8. HLD Mini-Scenario: Storage Design for a Video-Sharing Platform

Prompt: "Design the storage strategy for a video-sharing platform: uploaded videos need to be stored durably, served efficiently to viewers worldwide, and older, rarely-watched videos shouldn't cost as much to store as this week's trending content."

- 1. Store the actual video files in object storage, with the database holding only metadata (title, description, upload date, view count, and a reference to the object storage key) — keeping the database fast and focused on what it's good at.
- 2. Serve videos to viewers through a CDN (Chapter 16) layered in front of object storage, so popular/trending videos are cached at edge nodes close to viewers, while object storage remains the durable origin source.
- 3. Apply a lifecycle policy that migrates videos toward cheaper, colder storage tiers as their view count drops over time, automatically balancing storage cost against the (declining) likelihood that an old video needs fast retrieval.

This integrates four chapters' worth of concepts (database design, object storage, CDN, storage tiers) into one coherent answer — exactly the kind of synthesis that strong HLD answers demonstrate.

9. Questions YOU Should Ask the Interviewer

- What types and sizes of files are we storing, and roughly what total volume should we expect?
- Is there a predictable access pattern over time (e.g., content gets less popular as it ages) that we could exploit for cost savings?
- Are there compliance or data-retention requirements affecting how long data must be kept and how it should be stored?

10. Scenario-Based Practice Bank (New)

Scenario 1: A team stores user-uploaded PDF documents as BLOB columns directly in their PostgreSQL database, and backups have grown unmanageably slow.

How to approach it: This is the classic BLOB-in-database mistake. Migrate the actual PDF files to object storage, keeping only a reference/URL in the database row — this shrinks the database dramatically and restores fast backup/replication, since the database is no longer carrying the large binary payloads.

Scenario 2: A photo-sharing app's database and object-storage upload aren't coordinated, and occasionally a database row references a photo that failed to actually finish uploading.

How to approach it: Flag this as the real coordination cost of the two-system pattern (DB reference + object storage file). Recommend only writing the database reference after confirming the object storage upload succeeded (or using a status field like 'upload_pending' until confirmed), rather than assuming both operations always succeed together.

Scenario 3: A company keeps 7 years of financial transaction logs on the same fast, expensive storage tier as its actively-served application data, and storage costs have become a significant budget line item.

How to approach it: Recommend a lifecycle policy that migrates these rarely-accessed compliance logs to a cold/archive storage tier after some inactivity period, since retrieval speed barely matters for data that's almost never accessed but must be retained — this can produce major cost savings with no real functional downside.

Scenario 4: A video platform's older, rarely-watched videos are stored on the same tier as this week's trending uploads, and storage costs are growing faster than viewership.

How to approach it: Recommend a lifecycle policy tied to view-count/access-recency, automatically migrating videos toward cheaper, colder storage tiers as they age and get watched less, rather than treating all video content as equally 'hot' regardless of actual demand.

Scenario 5: A team is running nightly analytics jobs across petabytes of log data and is debating whether object storage alone is sufficient, or whether they need something else.

How to approach it: Point out that object storage is built for durable storage/retrieval by key, not high-throughput parallel computation across huge datasets. If they need to run large-scale parallel analytics jobs directly against this data, a distributed file system (like HDFS) or a big-data processing framework built to read from object storage is the more appropriate tool for that specific workload.

Scenario 6: An interviewer asks how a video platform's uploaded content actually gets served quickly to viewers around the world, and the candidate's answer only mentions object storage.

How to approach it: Complete the answer by adding a CDN (Chapter 16) in front of the object storage origin — object storage durably holds the master copy, while the CDN caches popular content at edge locations close to viewers, which is the near-universal real-world pairing rather than object storage serving all requests directly from one region.

Chapter 22 — One-Page Revision Sheet

- Object/blob storage = purpose-built for large, unstructured files (images, video, PDFs), accessed by key via simple PUT/GET/DELETE — not for structured queries.
- Never store large binary files directly in a relational database — store the file in object storage, and just a reference/URL in the database row.
- Most real systems use BOTH together: database for structured, queryable metadata; object storage for the actual large files.
- Distributed file systems (HDFS) are for high-throughput PARALLEL PROCESSING of huge datasets (big-data analytics), not just storage/retrieval like object storage.
- Storage tiers (hot/warm/cold) match cost to access frequency — lifecycle policies automate migration to cheaper tiers as data ages and is accessed less.
- Object storage and CDNs work together: object storage is the durable origin; the CDN caches popular content at the edge for fast global delivery.
- Always mention a CDN when discussing how object-stored media actually reaches end users — they're a near-universal pairing in real systems.

Memory hook: A database is a filing cabinet of labeled forms (query anything inside). Object storage is a warehouse of sealed, barcoded boxes (instant retrieval by barcode, but you can't search what's inside without opening one).

CHAPTER 23

HLD Framework & Capacity Estimation Practice

The repeatable method you'll use for every 'design X' question from here on

1. Simple Explanation First

This chapter doesn't introduce new building blocks — it's a practice chapter. You now have every tool needed (Parts 1-4) to solve real 'design X' questions; what's missing is repetitions applying the RADIO framework from Chapter 1 under realistic time pressure. We'll drill capacity estimation specifically, since it's the step candidates most often skip or fumble, and it's also the step that should directly inform every later architecture decision.

Analogy: Think of this chapter like a pilot's pre-flight checklist drill. A pilot doesn't invent a new checklist for every flight — they run the SAME checklist every time, fast and automatically, because repetition has made it second nature. RADIO is your checklist. The case studies in Chapters 24-28 are your flights. This chapter is where you practice running the checklist itself, in isolation, until it's automatic.

2. Why This Topic Exists

Knowing individual concepts (caching, sharding, load balancing) is necessary but not sufficient — interviews test whether you can ASSEMBLE them correctly, under time pressure, for a specific, novel problem. Many technically strong candidates underperform simply because they haven't rehearsed the assembly process itself, freezing on 'where do I even start' rather than the actual technical content. This chapter exists to make the starting move automatic, so all your mental energy in Chapters 24-28 (and in real interviews) can go toward the actual design decisions.

Why this matters: Consider this chapter the bridge between 'I know what a load balancer is' and 'I can confidently design a system that uses one, under a ticking clock, while a stranger watches and asks follow-up questions.' That bridge is built through practice, not additional facts.

3. Core Concepts in Depth

3.1 RADIO, Revisited With Real Depth

Chapter 1 introduced RADIO as a skeleton. Now that you have the full vocabulary from Parts 1-4, here's what each letter should actually contain in a real answer.

Step	What a STRONG Answer Includes Now
Requirements	Functional scope (explicitly include/exclude features) + non-functional targets (scale, latency, consistency needs) — informed by CAP/PACELC (Ch 5) thinking about which parts need strong consistency.
Architecture	Client -> Load Balancer (Ch 13) -> App Servers -> Cache (Ch 11) -> Database (Ch 7), plus Queues (Ch 14) for async work, CDN (Ch 16) for static content, API Gateway (Ch 15) if multiple services.
Data Model	Specific schema/fields, SQL vs NoSQL choice justified by access patterns (Ch 7), sharding key if needed (Ch 10), indexing strategy (Ch 8).

Step	What a STRONG Answer Includes Now
Interface	Concrete API endpoints with request/response shapes, justified API style (REST/gRPC, Ch 4).
Optimize	Named bottlenecks with named fixes: replication (Ch 9) for read scaling, sharding (Ch 10) for write scaling, consistent hashing (Ch 17) for resharding, circuit breakers (Ch 18) for resilience, monitoring (Ch 20) for visibility.

3.2 Capacity Estimation — A Full Worked Drill

Chapter 6 introduced the mechanics of back-of-envelope math. Here we drill it five more times in rapid succession, varying the type of system, so the PATTERN becomes automatic rather than something you have to reconstruct from scratch each time.

Drill 1: Storage for a Photo-Sharing App

- 1. Assume 50 million daily active users, 20% upload one photo per day, average photo size 2MB.
- 2. Daily uploads = $50,000,000 \times 0.20 = 10,000,000$ photos/day.
- 3. Daily storage = $10,000,000 \times 2\text{MB} = 20,000,000 \text{ MB} = 20 \text{ TB/day}$.
- 4. Yearly storage = $20 \text{ TB} \times 365 \sim 7.3 \text{ PB/year}$ — this scale immediately tells us we need object storage (Ch 22), not a database, and that storage tiering (hot/cold) will matter within a year or two as older photos are viewed less.

Drill 2: QPS for a Search Autocomplete Feature

- 1. Assume 10 million daily active users, each performing 5 searches/day, each search triggering ~ 3 autocomplete requests (one per few keystrokes).
- 2. Total autocomplete requests/day = $10,000,000 \times 5 \times 3 = 150,000,000/\text{day}$.
- 3. Average QPS = $150,000,000 / 86,400 \sim 1,735 \text{ QPS}$.
- 4. Peak QPS (2-3x average) $\sim 3,500\text{-}5,200 \text{ QPS}$ — this volume, combined with the need for sub-100ms responses, strongly suggests aggressive caching (Ch 11) of common prefixes rather than hitting a database on every keystroke.

Drill 3: Bandwidth for a Video Streaming Launch

- 1. Assume 1 million concurrent viewers at peak, average stream bitrate 5 Mbps (a common HD streaming rate).
- 2. Total peak bandwidth = $1,000,000 \times 5 \text{ Mbps} = 5,000,000 \text{ Mbps} = 5 \text{ Tbps}$.
- 3. This is an enormous number for any single origin server or even a single data center to serve directly — this single calculation alone justifies a CDN (Ch 16) as non-negotiable, not optional, for this system.

Drill 4: Write Throughput for a Ride-Sharing Location Service

- 1. Assume 5 million active drivers during peak hours, each sending a location update every 4 seconds.
- 2. Updates per second = $5,000,000 / 4 = 1,250,000 \text{ writes/second}$.
- 3. This write volume is far beyond what a single traditional database can handle — it points toward a purpose-built solution: an in-memory store (Ch 11) optimized for fast writes and geospatial queries, likely sharded geographically (Ch 10), rather than a general-purpose relational database.

Drill 5: Connection Count for a Chat Application

- 1. Assume 20 million daily active users, with 10% concurrently online at any peak moment.
- 2. Concurrent WebSocket connections (Ch 3) = $20,000,000 \times 0.10 = 2,000,000$ simultaneous persistent connections.
- 3. If a single server can comfortably hold $\sim 50,000$ concurrent connections, this requires roughly 40 WebSocket gateway servers at minimum — directly informing the horizontal scaling (Ch 6) and load balancing (Ch 13) strategy for the connection layer specifically.

Why this matters: Notice the pattern across all five drills: the math isn't decorative. Each calculation directly produces a specific architectural conclusion (object storage, caching, CDN, in-memory store, server count). This is the habit to build — always close the loop from number to decision.

4. Trade-offs Table

Approach to Estimation	Pros	Cons
Doing rough math out loud, stating assumptions	Shows process and reasoning, which is what's actually graded	Numbers won't be perfectly precise — but precision isn't the point
Skipping estimation, jumping straight to architecture	Saves time upfront	Architecture decisions end up unjustified and may not fit the actual scale
Over-investing time in precise calculations	Numbers are very accurate	Wastes limited interview time that should go toward architecture and trade-off discussion

5. How Real Companies Use This

- Capacity planning teams at large tech companies run exactly this kind of estimation routinely — before launching a new feature or service, to provision the right amount of infrastructure ahead of time rather than reactively scrambling during an outage.
- Major product launches (e.g., a new iPhone announcement driving traffic, a Super Bowl ad) are preceded by capacity estimation exercises specifically to ensure systems are provisioned for the anticipated spike, not just average daily traffic.
- Interview loops at most major tech companies explicitly include capacity estimation as a graded component of the HLD rubric, separate from the architecture diagram itself.

6. Common Mistakes & Misconceptions

- Treating capacity estimation as a one-time, disconnected step at the start of the interview, rather than revisiting it as the design evolves (e.g., when the interviewer changes a constraint mid-interview).
- Getting lost in unnecessary precision (exact byte counts, complex unit conversions) instead of using clean, round numbers that are 'close enough' to inform a decision.

- Forgetting to translate a number into an architectural consequence — calculating QPS but never saying what that QPS number actually implies for the design.

7. Interviewer Q&A Bank

Q1. Why do you do capacity estimation before designing the architecture?

A: Because the right architecture genuinely depends on scale — a system serving 10,000 users and one serving 100 million users can need fundamentally different designs, even with identical functional requirements. Estimating scale upfront lets me justify specific decisions (whether to shard, whether a CDN is necessary, how many servers are needed) with real numbers, rather than guessing or over/under-engineering.

Q2. How precise do these calculations need to be?

A: Not very — the goal is reasoning with sensible assumptions and round numbers to get the right ORDER OF MAGNITUDE, since that's what actually drives architecture decisions. Being off by 20% rarely changes whether you need sharding; being off by 1000x would. I'd rather move quickly with clean approximations than spend interview time on precise arithmetic.

Q3. What's the difference between average and peak load, and why does it matter for capacity planning?

A: Average load is the typical, steady-state traffic level, while peak load accounts for predictable daily/weekly patterns and unpredictable spikes — real traffic isn't evenly distributed across 24 hours. Systems need to be provisioned for peak load, not average, or they'll degrade or fail exactly when traffic (and business impact) is highest, so I always apply a peak multiplier (commonly 2-3x average) to my estimates.

Q4. If the interviewer suddenly says 'now assume 100x the users,' how would you respond?

A: I'd quickly redo the key estimations (storage, QPS, bandwidth) at the new scale, and specifically identify which parts of my original design would break first — usually whichever component was sized closest to its limits — then explain the specific scaling technique (sharding, more aggressive caching, geographic distribution) needed to absorb that growth, rather than redesigning the entire system from scratch.

Q5. What number would you start with when given a vague 'design a popular app' prompt with no scale specified?

A: I'd ask the interviewer for a rough scale target first, since that single number anchors the rest of the design. If they want me to propose one, I'd pick a recognizable reference point (e.g., 'let's assume a scale similar to a successful mid-size app — around 10 million daily active users') and state it explicitly as an assumption, so the interviewer can correct me if they had something different in mind.

8. HLD Mini-Scenario: Rapid-Fire Estimation Under a Changing Constraint

Prompt: "Design a notification service for a food delivery app. Initially assume 5 million daily active users. Halfway through, the interviewer says: 'Actually, assume this needs to support 200 million users globally.'"

- 1. At 5 million users (say, 3 notifications/user/day): ~15 million notifications/day, average QPS ~ 175, comfortably handled by a single queue (Ch 14) and a modest pool of worker servers.
- 2. At 200 million users (40x growth): ~600 million notifications/day, average QPS ~ 6,940, peak QPS potentially 15,000-20,000 — this changes the picture significantly, now requiring a high-

throughput, partitioned message queue (e.g., Kafka, Ch 14), geographically distributed worker pools, and likely regional sharding of the underlying user/device-token database (Ch 10).

- 3. Explicitly name what breaks first at the new scale: a single-region, single-queue setup would become a bottleneck, and third-party push notification provider rate limits (Apple/Google's own APIs) would also need to be accounted for at this volume — a detail invisible at the smaller scale.

This demonstrates the exact skill this chapter is built to drill: redoing estimation quickly, and translating the NEW numbers into specific, named architectural changes, not a vague 'we'd scale it up.'

9. Questions YOU Should Ask the Interviewer

- Is there a target scale in mind, or would you like me to propose reasonable assumptions?
- Should I optimize my estimation for precision, or is an order-of-magnitude approximation sufficient here?
- Are there known traffic patterns (daily peaks, seasonal spikes) I should specifically account for?

10. Scenario-Based Practice Bank (New)

Scenario 1: An interviewer gives you 'Design a photo-sharing app' with zero scale information, and the clock is already running.

***How to approach it:** Don't stall waiting for a number. State a reasonable assumed scale out loud (e.g., 'let's assume 50 million daily active users, similar to a successful mid-size app') and proceed, explicitly flagging it as an assumption the interviewer can correct.*

Scenario 2: Midway through designing a notification service sized for 5 million users, the interviewer says 'now assume 200 million users globally.'

***How to approach it:** Quickly redo the key numbers (total notifications/day, average and peak QPS) at the new scale, and name specifically what breaks first — usually the single-region queue or database — then state the concrete fix (partitioned queue, regional sharding) rather than vaguely saying 'we'd scale it up.'*

Scenario 3: You calculate a video platform needs 5 Tbps of peak bandwidth, but then move on to database schema design without mentioning what that number implies.

***How to approach it:** Close the loop explicitly: a number that big means a CDN isn't optional — it's mandatory, since no single origin server or data center could serve that load directly. Always state the architectural consequence of a calculation, not just the calculation itself.*

Scenario 4: You're asked to estimate storage for a photo app and spend two minutes converting between exact byte units to get a precise figure.

***How to approach it:** This is over-investing in precision that isn't graded. Round aggressively (2MB per photo, 1000 bytes per KB) and move quickly to the order-of-magnitude answer (e.g., '~7 PB/year') — that's sufficient to justify the architecture decision (object storage) and saves time for the parts of the interview that matter more.*

Scenario 5: An interviewer asks you to estimate concurrent WebSocket connections for a chat app with 20 million DAU, 10% concurrently online.

How to approach it: Walk through it cleanly: $20,000,000 \times 0.10 = 2,000,000$ concurrent connections; if one server holds ~50,000 connections, that's roughly 40 WebSocket gateway servers minimum — directly informing the horizontal scaling and load-balancing strategy for that specific stateful layer.

Scenario 6: You estimated average QPS for a system but the interviewer asks why you didn't account for peak traffic.

How to approach it: Acknowledge the gap directly: real traffic isn't evenly distributed across the day, so provisioning only for average load risks failure exactly during the highest-impact moments. Apply a peak multiplier (commonly 2-3x average) and use that number to size the actual infrastructure.

Chapter 23 — One-Page Revision Sheet

- This chapter is a practice chapter, not new content — it drills RADIO (Ch 1) and capacity estimation (Ch 6) until they're automatic.
- Every RADIO step should now be filled with specific, named tools from Parts 1-4 — not generic placeholders.
- Always close the loop: a calculated number (storage, QPS, bandwidth, connections) should directly produce a specific architecture decision, stated explicitly.
- Use round numbers and clean assumptions out loud — precision isn't graded, reasoning process is.
- Always estimate for PEAK load (2-3x average), not just average — systems must survive the busiest moments, not the typical ones.
- When a constraint changes mid-interview (e.g., '10x the scale'), quickly redo the key estimates and name SPECIFICALLY what breaks first and what fixes it.
- If no scale is given, either ask the interviewer or explicitly state a reasonable assumed scale before proceeding.

Memory hook: A pilot's pre-flight checklist isn't reinvented each flight — it's run the same way every time until it's automatic. RADIO is your checklist; the case studies ahead are your flights.

CHAPTER 24

Case Study: Design a URL Shortener (TinyURL)

The classic starter problem — small enough to fully solve, rich enough to teach the full method

1. Simple Explanation First

A URL shortener takes a long URL and produces a short, unique code that redirects back to the original when visited. It's the most commonly used 'first' system design interview question precisely because it's small enough to fully design within 45 minutes, yet touches a surprising number of real concepts: hashing/ID generation, database design, caching, and capacity estimation.

Analogy: Think of a coat check at a theater. You hand over your long, bulky coat (the long URL); the attendant gives you a small numbered ticket (the short code). Later, you hand back the ticket, and the attendant instantly retrieves your exact coat. The ticket itself means nothing on its own — its entire value is as a fast, compact reference to something larger stored elsewhere.

2. Requirements (the 'R' in RADIO)

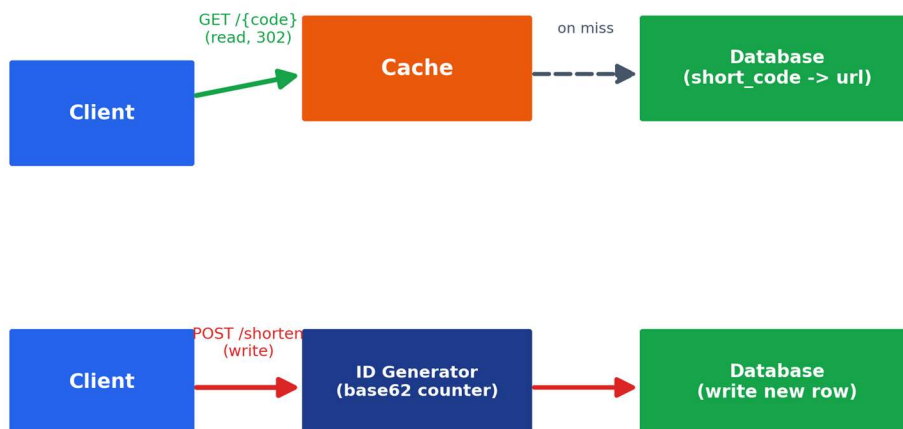
Functional requirements: given a long URL, generate a unique short URL; given a short URL, redirect to the original long URL; optionally, let URLs expire after a set time or custom alias be specified.

Non-functional requirements: extremely high read:write ratio (far more people click links than create them — often estimated at 100:1 or higher); redirects must be very low latency; short codes should be unguessable/unique; the system should be highly available, since broken links are a poor experience (mild AP lean per Chapter 5, since a few seconds of staleness on a redirect is harmless, but downtime is not).

Quick capacity estimation: assume 100 million new URLs created per month, and a 100:1 read:write ratio. Writes/sec $\sim 100,000,000 / (30 \times 86,400) \sim \sim 39$ writes/sec. Reads/sec $\sim 39 \times 100 \sim \sim 3,900$ reads/sec average, with peak likely 2-3x that. These are very manageable numbers for a single well-designed system — this is NOT a system that needs aggressive sharding from day one, which should shape how much complexity we introduce.

3. Architecture (the 'A' in RADIO)

URL Shortener — Read Path vs Write Path



Extreme read:write skew (~100:1) means the cache absorbs nearly all traffic.

Figure 24.1 — Reads hit the cache first; writes go through an ID generator before being stored.

Client requests pass through a load balancer (Chapter 13) to a pool of stateless app servers (Chapter 6). For redirects (the dominant read path), app servers check a cache (Chapter 11) first — given the extreme read:write skew, a cache will absorb the vast majority of traffic, protecting the database almost entirely from direct read load. For shortening requests (writes), the app server calls an ID generator before writing the new mapping to the database.

4. Data Model (the 'D' in RADIO)

A simple table is sufficient at this scale: `short_code` (primary key, indexed — Chapter 8), `long_url`, `created_at`, `expires_at` (nullable), `creator_id` (nullable, if supporting accounts). Given the simple key-based lookup pattern (no complex joins needed), this could be a relational database OR a key-value store (Chapter 7) — either is defensible, with the key-value option leaning slightly more naturally into the access pattern.

4.1 ID Generation — The Most Interesting Design Decision

How do we generate a short, unique code? This is the question interviewers most often dig into.

Approach	How It Works	Trade-off
Hash the long URL (e.g., MD5, take first 7 chars)	Same long URL always produces the same short code	Collisions are possible (two different URLs hashing to the same short prefix) and need detection/handling
Base62 encode an auto-incrementing counter	A counter (e.g., from a single source like a database sequence) is encoded into base62 (a-z, A-Z, 0-9) for compactness	Guarantees uniqueness with no collisions, but a single global counter can become a bottleneck/single point of contention at very high write volume
Pre-generated key pool	A background service pre-generates and stores a pool of unique random codes; app servers simply claim one from the pool when needed	No real-time collision-checking needed at write time, but adds the operational complexity of managing the pool

Given our earlier estimation (~39 writes/sec), a base62-encoded auto-incrementing counter (or several counters partitioned by range, to avoid a single point of contention) is a clean, defensible choice — collision handling for the hash-based approach would be solving a problem we don't actually have at this write volume.

5. Interface (the 'I' in RADIO)

A small, clean REST API (Chapter 4) fits this use case well — there's no complex nested data justifying GraphQL, and this is a public-facing API where REST's simplicity and wide compatibility are valuable.

- POST /api/shorten — body: { long_url, custom_alias (optional), expires_at (optional) } -> returns { short_url }
- GET /{short_code} — returns an HTTP 302 redirect to the original long URL, or 404 if not found/expired

6. Optimize & Scale (the 'O' in RADIO)

- Caching: cache popular short codes (Chapter 11) — given the extreme read skew, even a modestly-sized cache will achieve a very high hit rate, since a small number of links tend to receive a disproportionate share of clicks.
- Database scaling: at the estimated scale, a single primary database with a couple of read replicas (Chapter 9) is more than sufficient — sharding (Chapter 10) would be premature complexity here, though if asked 'what if this were 1000x bigger,' sharding by a hash of short_code with consistent hashing (Chapter 17) would be the next step.
- Analytics (a common follow-up): if click analytics are needed (which links, when, from where), this is a perfect candidate for asynchronous processing (Chapter 14) — log the click event to a queue and process analytics in the background, rather than slowing down the actual redirect.
- Monitoring: track redirect latency (p99, Chapter 20) and cache hit rate specifically, since these two metrics most directly reflect user experience for this system.

7. Common Mistakes & Misconceptions

- Jumping straight to sharding and complex distributed ID generation without first estimating scale — as shown above, this system's actual numbers don't justify that complexity.

- Forgetting to handle short code collisions or exhaustion in the hash-based approach, or forgetting that base62 encoding needs a strategy to avoid a single bottlenecked counter at very high scale.
- Designing the redirect (read) path without caching, missing the single highest-leverage optimization for this specific, extremely read-heavy system.

8. Interviewer Q&A Bank

Q1. How would you generate unique short codes?

A: I'd lean toward base62-encoding an auto-incrementing counter, since it guarantees uniqueness without needing collision detection, and is simple to reason about at our estimated write volume of around 39 writes/second. For much higher write volumes, I'd consider partitioning the counter (e.g., multiple counters with different starting offsets) to avoid a single point of contention, or move to a pre-generated key pool.

Q2. Why would you cache redirects, and what eviction policy would you use?

A: Given the extreme read:write ratio (roughly 100:1), a relatively small cache can absorb the vast majority of redirect traffic, since a disproportionate share of clicks tend to go to a smaller set of popular links. I'd use LRU (Chapter 11) as a sensible default eviction policy, since recently-accessed short codes are a reasonable proxy for likely-to-be-accessed-again codes.

Q3. How would you handle a custom alias request that's already taken?

A: I'd check for existing custom aliases at write time with a uniqueness constraint on the short_code column (or a dedicated lookup before insert), returning a clear error to the client if the requested alias is unavailable, since this is a synchronous, user-facing check that needs to happen before confirming the shorten request succeeded.

Q4. What happens if the cache and database briefly disagree — e.g., a link was deleted but is still cached?

A: Given this system's lean toward availability (Chapter 5), I'd accept brief staleness here as a reasonable trade-off — a deleted link still redirecting for a few more minutes until the cache entry expires (TTL-based invalidation, Chapter 11) is a low-stakes inconsistency, not worth the added complexity of synchronous cache invalidation on every delete.

Q5. How would this design change if we needed real-time click analytics per link?

A: I'd avoid adding analytics processing to the synchronous redirect path, since that would directly slow down the user-facing redirect. Instead, I'd publish a lightweight click event to a message queue (Chapter 14) as a fire-and-forget side effect, and have a separate analytics service consume those events asynchronously, keeping the actual redirect path fast and simple.

9. Questions YOU Should Ask the Interviewer

- Roughly what scale should I design for — millions or billions of URLs created over the system's lifetime?
- Do we need custom aliases, link expiration, or click analytics, or should I focus purely on the core shorten/redirect flow?
- Is there a need for user accounts (so users can manage/view their own created links), or is this fully anonymous?

10. Scenario-Based Practice Bank (New)

Scenario 1: An interviewer asks you to redesign the URL shortener assuming 1000x the estimated write volume (roughly 39,000 writes/sec instead of 39).

***How to approach it:** At this volume, a single auto-incrementing counter becomes a real bottleneck. Move to multiple counters partitioned by range (or a pre-generated key pool), and revisit whether the database now needs sharding by a hash of `short_code` with consistent hashing (Chapter 17), which wasn't justified at the original, much smaller scale.*

Scenario 2: A user reports that a link they just deleted is still redirecting correctly for a few more minutes.

***How to approach it:** This is expected, acceptable behavior given the system's AP lean: the cache entry hasn't expired yet (TTL-based invalidation). Since brief staleness on a deleted link is low-stakes, this doesn't need synchronous cache invalidation — explain this is a deliberate trade-off, not a bug.*

Scenario 3: The product team wants to add real-time analytics showing exactly how many clicks a link got in the last 10 seconds.

***How to approach it:** Keep the analytics pipeline asynchronous (via a queue) rather than synchronous, but note that 'real-time' (a few seconds of lag) is still achievable this way — the key point is never adding this processing directly to the synchronous redirect path, regardless of how fresh the analytics need to be.*

Scenario 4: Two users simultaneously request the same custom alias 'my-brand' at nearly the same instant.

***How to approach it:** Rely on a uniqueness constraint on the `short_code` column at the database level as the actual source of truth, not just an application-level check-then-insert (which has the same race condition risk as Chapter 12's concurrency examples) — the database constraint atomically rejects the second, duplicate insert.*

Scenario 5: An interviewer asks whether this system needs strong consistency anywhere.

***How to approach it:** Point to the one place it genuinely matters: ensuring a given `short_code` maps to exactly one `long_url` with no ambiguity (handled by the uniqueness constraint at write time). Everywhere else — redirect freshness after a delete, analytics — mild eventual consistency is perfectly acceptable and even preferred for availability.*

Scenario 6: A candidate defaults to designing this system with sharding and a distributed ID-generation service from the very first minute.

***How to approach it:** This is the classic 'skip estimation, over-engineer' mistake this chapter warns about. Redirect the conversation back to the actual numbers (~39 writes/sec, ~3,900 reads/sec) and show that a single primary database with read replicas and a cache comfortably handles this — sharding should be reserved for a 'what if this were 1000x bigger' follow-up, not the default starting design.*

Chapter 24 — One-Page Revision Sheet

- Extreme read:write skew (~100:1) is the defining characteristic — caching the redirect path is the single highest-leverage design decision.
- ID generation options: hash-based (needs collision handling), base62 counter (simple, no collisions, watch for counter contention), pre-generated pool (no real-time collision check, more ops complexity).
- At realistic scale (~39 writes/sec, ~3,900 reads/sec), a single primary DB + read replicas + cache is sufficient — sharding would be premature.
- Data model: short_code (PK), long_url, created_at, expires_at — simple enough for SQL or key-value storage.
- API: POST /api/shorten to create, GET /{short_code} returns a 302 redirect.
- Click analytics (if asked) should be asynchronous (queue) — never add it to the synchronous redirect path.
- This system leans AP (Chapter 5): brief staleness on a deleted/updated link is acceptable; downtime on redirects is not.

Memory hook: *A theater coat check: a small ticket (short code) is a fast, compact reference to something larger stored elsewhere (the long URL) — the ticket itself carries no information, just a pointer.*

CHAPTER 25

Case Study: Design a Rate Limiter & a Distributed Cache

Two compact, frequently-asked designs that both hinge on the same core idea: shared state

1. Simple Explanation First

These two case studies are paired because they share a central design challenge: how do multiple independent servers coordinate on shared state (a counter, a cached value) without stepping on each other? A rate limiter needs every server to agree on 'how many requests has this client made recently?' A distributed cache needs every server to agree on 'what's the current cached value for this key?' Solving both teaches the same underlying lesson from two angles.

Analogy: Imagine several ticket booths at one amusement park, all needing to enforce 'no more than 1,000 people in the park at once.' If each booth keeps its own private headcount, the park could easily oversell past 1,000 by the combined error across booths. The booths need a **SHARED**, constantly-updated counter visible to all of them — that shared counter is exactly the role a distributed cache plays for both of these designs.

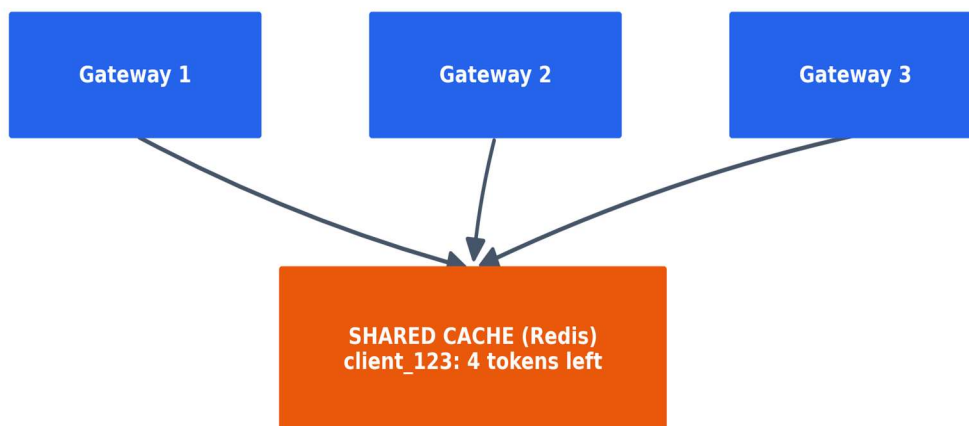
2. Part A: Designing a Rate Limiter

2.1 Requirements

Functional: limit each client (identified by API key, user ID, or IP) to N requests per time window; reject requests over the limit with a clear error. Non-functional: the rate limiter itself must add minimal latency (it sits in the critical path of every request), must work correctly across many gateway/server instances (not just within one), and should fail safe (a rate limiter outage shouldn't take down the whole API, per the trade-offs discussed in Chapter 5).

2.2 Architecture

Rate Limiter — Shared Counter, Not Per-Server



All gateway instances check/decrement the SAME atomic counter — no per-server drift.

Figure 25.1 — Token bucket counters live in a shared cache, not per-server memory, so limits are enforced consistently.

The rate limiter logic typically lives at the API Gateway (Chapter 15), checking each request against the client's current usage before forwarding it onward. The critical design decision: where does the counter live? If each gateway instance kept its own local counter, a client could exceed the intended limit simply by having requests land on different gateway instances (each tracking only a fraction of the client's true total). The fix is a shared, distributed cache (Chapter 11) — typically Redis — holding the authoritative counter state, checked and updated by every gateway instance.

2.3 Algorithm Choice

Chapter 15 covered token bucket, leaky bucket, and sliding window in depth. For this case study, token bucket is the standard choice: each client has a bucket (stored as a key in Redis) holding a token count, decremented on each request and refilled at a steady rate using Redis's atomic increment/decrement operations — critically, these operations need to be atomic (Chapter 12) to avoid a race condition where two simultaneous requests both read '1 token left' and both proceed, oversending past the limit.

Common mistake: Implementing the check-then-decrement logic as two separate operations (read the count, then if it's positive, decrement it) instead of one atomic operation. Under concurrent load, two requests can both pass the 'read' step before either completes the 'decrement' step, both incorrectly succeeding. Redis's atomic operations (or a Lua script for more complex multi-step logic) are the correct way to avoid this exact race condition (echoing Chapter 12's concurrency control concepts).

3. Part B: Designing a Distributed Cache

3.1 Requirements

Functional: store and retrieve key-value pairs quickly; support expiration (TTL); support common operations (get, set, delete, increment). Non-functional: very low latency (sub-millisecond to low-millisecond reads); must scale horizontally as data volume grows beyond one machine's memory capacity; should gracefully handle individual node failures without losing all cached data.

3.2 How Data Is Distributed Across Cache Nodes

This is where Chapter 17's consistent hashing becomes the centerpiece of the design. Rather than every cache node holding every key (wasteful and doesn't scale) or using naive modulo hashing (catastrophic cache invalidation on resizing), keys are distributed across nodes using consistent hashing with virtual nodes, so the cache cluster can grow or shrink with minimal disruption to the existing cached data.

- Each cache node owns a portion of the hash ring (Chapter 17).
- A client (or a routing layer) hashes the requested key to determine which node owns it, then sends the request directly to that node.
- Adding a new cache node only requires redistributing the small slice of keys near its ring position — the rest of the cache remains intact and warm, avoiding a sudden wave of cache misses hitting the database (the exact scenario discussed in Chapter 17's HLD mini-scenario).

3.3 Handling Node Failure

If a cache node fails, the keys it owned become temporarily unavailable from the cache (falling back to the database/cache-aside pattern from Chapter 11, which will then repopulate the cache from a different/recovered node going forward). For higher resilience, cache data can be replicated (Chapter 9's concepts, applied to a cache rather than a database) across more than one node, so a single node failure doesn't cause a full cold-cache scenario for its portion of the keyspace.

4. Trade-offs Table

Decision	Pros	Cons
Centralized counter in Redis (rate limiter)	Accurate, consistent limits across all gateway instances	Adds a network round-trip to every request; Redis becomes a critical dependency
Token bucket via atomic Redis operations	Prevents race conditions, allows controlled bursts	Slightly more complex than a naive counter; needs careful atomic operation usage
Consistent hashing for cache node distribution	Minimal disruption when scaling the cache cluster	More complex than naive hashing; still requires a routing layer aware of the ring

5. How Real Companies Use This

- Stripe, GitHub, and most major API providers implement token-bucket-style rate limiting backed by a shared cache, returning rate-limit-remaining headers so client developers can self-monitor their usage.

- Twitter/X and Facebook operate enormous distributed caching layers (built on consistent hashing principles) to absorb the vast majority of read traffic for their social graphs, since hitting the underlying database for every read at their scale would be infeasible.
- Redis Cluster, a popular real-world implementation, uses a hash-slot mechanism (a variant of the consistent hashing concept) to distribute keys across multiple Redis nodes automatically.

6. Common Mistakes & Misconceptions

- Designing a rate limiter with per-server local counters, which fails to enforce a true global limit once there's more than one server — a very common interview trap.
- Forgetting that the counter check-and-update must be atomic, leaving a race condition that allows limits to be exceeded under concurrent load.
- Using naive modulo hashing to distribute cache keys across nodes, recreating the exact resharding pain Chapter 17 exists to solve.

7. Interviewer Q&A Bank

Q1. Why can't a rate limiter just keep a counter in each server's local memory?

A: Because with multiple gateway/server instances behind a load balancer, a client's requests can land on different instances, each only seeing a fraction of that client's total traffic — a per-server local counter would let the client exceed the intended global limit by spreading requests across instances. The counter needs to live in shared state, like a distributed cache, that every instance reads from and writes to.

Q2. How do you prevent a race condition in the rate limiter's token check?

A: By making the check-and-decrement a single atomic operation rather than two separate steps, since under concurrent requests, two threads could both read 'token available' before either decrements, both incorrectly succeeding past the limit. Redis provides atomic increment/decrement operations, or more complex multi-step logic can be wrapped in a Lua script that Redis executes atomically.

Q3. How would you distribute keys across multiple cache nodes as the cache grows?

A: I'd use consistent hashing with virtual nodes, placing both cache nodes and keys on a hash ring, so each key is owned by its nearest node clockwise. This means adding or removing a cache node only requires redistributing a small fraction of keys, rather than invalidating the entire cache the way naive modulo hashing would when the node count changes.

Q4. What happens to a rate limiter's accuracy if the underlying cache (e.g., Redis) goes down?

A: This is a real trade-off to address explicitly — I'd design the rate limiter to fail open (allow requests through) rather than fail closed (block all requests) during a cache outage, since blocking all traffic due to a rate-limiter dependency failure is usually worse than temporarily allowing unrestricted traffic for a brief outage window. This connects to the broader fault-tolerance principle from Chapter 19 of deciding deliberately how a component should behave during a dependency failure.

Q5. Would you replicate data across distributed cache nodes?

A: For data where a cache miss is cheap (the database can serve it directly), I'd accept that a node failure causes some temporary cold-cache misses without needing replication. For more critical or expensive-to-recompute cached data, I'd replicate it across at least one additional node, accepting the extra memory cost in exchange for avoiding a sudden spike of database load if a node fails.

8. HLD Mini-Scenario: Combining Both Designs for a Public API Platform

Prompt: "A company offers a public API. Design how rate limiting and caching work together to protect backend services while keeping responses fast for well-behaved clients."

- 1. At the API Gateway (Chapter 15), check the client's rate limit using a token bucket counter stored in a shared Redis cluster — distributed across nodes via consistent hashing (Chapter 17) so the rate-limiting layer itself scales horizontally without disruption.
- 2. For requests that pass the rate limit check, serve cacheable responses (e.g., GET requests for relatively static data) from the SAME distributed cache infrastructure, reusing the consistent-hashing-based node distribution already in place for rate limiting.
- 3. If the Redis cluster experiences a partial outage, fail the rate limiter open (allow traffic) but treat cache misses normally (fall back to the database) — explicitly different failure behaviors for two different concerns sharing the same infrastructure.

This shows how the shared infrastructure pattern (a distributed cache) can serve two different purposes (rate limiting state and general caching) in one real system, while still requiring different failure-handling decisions for each.

9. Questions YOU Should Ask the Interviewer

- Should the rate limiter apply per-API-key, per-user, or per-IP — and do different tiers need different limits?
- How should the system behave if the shared cache/counter store becomes temporarily unavailable?
- What's the expected cache hit rate and data size — does this fit comfortably in memory on a few nodes, or do we need to plan for a much larger cluster?

10. Scenario-Based Practice Bank (New)

Scenario 1: A rate limiter deployed across 10 gateway instances is letting some clients make far more requests than their configured limit.

***How to approach it:** This is almost certainly the per-server local counter trap: each instance only sees a fraction of the client's traffic. Move the counter to a shared Redis store checked by every instance, so the limit is enforced globally rather than per-instance.*

Scenario 2: Under a load test with many simultaneous requests from one client, the rate limiter occasionally allows a few more requests through than the configured limit.

***How to approach it:** This points to a non-atomic check-then-decrement race condition. Fix it by using a single atomic Redis operation (or a Lua script for multi-step logic) so the read-and-decrement happens as one indivisible step, closing the window where two concurrent requests both see 'token available.'*

Scenario 3: The Redis cluster backing both the rate limiter and general response cache goes down briefly during a deploy.

***How to approach it:** Apply different failure behavior per concern: let the rate limiter fail open (allow traffic through) since blocking everything is worse than a brief window of unrestricted traffic, while cache misses simply fall back to querying the database directly as they normally would on a miss.*

Scenario 4: A distributed cache cluster needs to grow from 6 to 9 nodes to handle increased load, and the team is worried about a wave of cache misses overwhelming the database during the transition.

***How to approach it:** Confirm the cluster uses consistent hashing with virtual nodes (not naive modulo hashing) — this ensures only the small slice of keys near the new nodes' ring positions gets redistributed, so the vast majority of the cache stays warm and the database isn't hit with a sudden miss storm.*

Scenario 5: A specific cached value that's very expensive to recompute (a complex aggregation query) is lost every time its cache node briefly fails.

***How to approach it:** Recommend replicating this specific expensive-to-recompute data across at least one additional cache node, accepting the extra memory cost — unlike cheap-to-recompute data, a cold-cache miss here causes real, avoidable load on the database.*

Scenario 6: An interviewer asks how rate limiting and caching for a public API platform can share the same underlying Redis infrastructure without interfering with each other.

***How to approach it:** Explain that both use the same consistent-hashing-distributed Redis cluster, but treat their data and failure modes independently — separate key namespaces for rate-limit counters vs cached responses, and distinct failure behavior (fail-open for limits, normal DB fallback for cache misses) even though they share the same physical infrastructure.*

Chapter 25 — One-Page Revision Sheet

- Both designs hinge on the same core challenge: coordinating shared state across multiple independent servers, not keeping local-only state.
- Rate limiter: per-server local counters fail once there's more than one server — use a shared cache (Redis) holding the authoritative counter.
- Token bucket check-and-decrement MUST be atomic (single operation or Lua script) to avoid race conditions that let limits be exceeded.
- Distributed cache: use consistent hashing with virtual nodes (Ch 17) to distribute keys across nodes — avoids naive-hashing's catastrophic resharding pain.
- Decide explicit failure behavior for shared infrastructure: rate limiter should typically fail OPEN (allow traffic) during an outage; cache misses fall back to the database normally.
- Replicate cache data when a miss is expensive to recompute; accept cold-cache misses on node failure when a miss is cheap (DB serves it directly).
- This case study is really Chapter 11 (caching) + Chapter 12 (atomicity) + Chapter 15 (rate limiting) + Chapter 17 (consistent hashing) combined into one coherent system.

Memory hook: Multiple ticket booths at one park, all enforcing 'max 1,000 visitors' — without a SHARED, constantly-updated headcount, each booth's private tally lets the park oversell.

CHAPTER 26

Case Study: Design a Chat / Messaging App (WhatsApp)

Real-time delivery, presence, and message guarantees at massive scale

1. Simple Explanation First

A chat application lets users exchange messages in real time, see when others are online, and trust that sent messages actually arrive. Unlike most of our earlier case studies, this system is fundamentally built around persistent, two-way connections rather than simple request-response — making it the natural home for everything Chapter 3 (WebSockets) and Chapter 14 (Pub-Sub) taught.

Analogy: Think of the difference between mailing a letter and being on a phone call. Most of our earlier case studies (URL shortener, rate limiter) behave like mail — you send a request, you get a response, the connection ends. A chat app behaves like a phone call — both sides stay connected, and either side can speak at any moment without the other having to ask first. Designing for an open phone line requires fundamentally different infrastructure than designing for mail.

2. Requirements (the 'R' in RADIO)

Functional requirements: send/receive one-on-one and group messages; show online/offline presence; show delivery and read receipts; support media (images, videos) attachments; messages should arrive in order within a conversation.

Non-functional requirements: very low latency for message delivery (near-instant feel); high availability (a chat app being down feels especially broken to users); message delivery should be reliable — at-least-once delivery (Chapter 14) is the realistic target, with the client deduplicating using a message ID if needed; ordering within a single conversation matters more than global ordering across all conversations.

Quick capacity estimation: assume 100 million daily active users, each sending 40 messages/day on average. Total messages/day = 4 billion. Average QPS $\sim 4,000,000,000 / 86,400 \sim 46,300$ messages/sec, with peak likely 2-3x that. Concurrent connections (assuming 20% of DAU online at peak) ~ 20 million simultaneous WebSocket connections — this single number is what most shapes the architecture, since it requires many WebSocket gateway servers, each holding a slice of those connections.

3. Architecture (the 'A' in RADIO)

Chat: Cross-Gateway Delivery via Pub-Sub

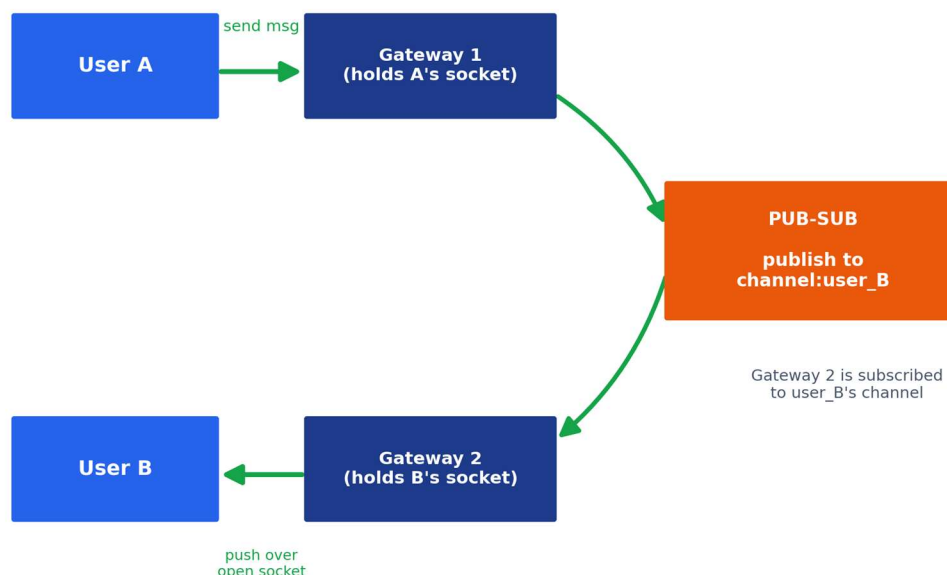


Figure 26.1 — Persistent WebSocket connections let the gateway push messages instantly; pub-sub fans messages out to the right gateway instance.

Users maintain a persistent WebSocket connection (Chapter 3) to a WebSocket Gateway layer — a pool of servers specifically dedicated to holding these long-lived connections, distinct from the stateless request-handling app servers used elsewhere in this book. When User A sends a message, it reaches the gateway instance A is connected to, which forwards it to a Message Service. The Message Service persists the message and needs to deliver it to User B — but B might be connected to a DIFFERENT gateway instance (with 20 million connections spread across many servers, this is the common case, not an edge case).

This is solved with a Pub-Sub layer (Chapter 14): the Message Service publishes the new message to a channel associated with User B; EVERY gateway instance subscribes to channels for the users currently connected to it, so whichever specific instance holds B's connection receives the message and pushes it down B's open WebSocket. A separate Presence Service tracks which users are currently online (and on which gateway instance), used both for showing online status and for the Message Service to know whether to attempt real-time push or queue the message for later delivery.

4. Data Model (the 'D' in RADIO)

Messages are extremely high-volume and naturally partition by conversation, making this a strong fit for sharding (Chapter 10) by conversation_id — the dominant query pattern ('get message history for this conversation') stays within a single shard. A NoSQL wide-column store (Chapter 7) like Cassandra is a common real-world choice here, given the write-heavy, append-mostly access pattern and the natural partition key.

- Messages table: message_id, conversation_id (shard key), sender_id, content, timestamp, status (sent/delivered/read).
- Conversations table: conversation_id, participant_ids, last_message_preview, updated_at — for rendering the conversation list quickly.
- Presence data: user_id -> online status, last_seen, connected gateway instance — this changes extremely frequently and is a natural fit for a fast in-memory store (Chapter 11) rather than a persistent database.

5. Interface (the 'I' in RADIO)

- WebSocket events (not REST, since this is a persistent connection): send_message, message_delivered, message_read, presence_update — these are bidirectional events flowing over the open connection, not discrete HTTP calls.
- REST endpoints for non-real-time needs: GET /conversations (list), GET /conversations/{id}/messages (history, paginated) — these fit the traditional request-response model fine, since loading history isn't inherently a real-time push scenario.

6. Optimize & Scale (the 'O' in RADIO)

- Offline message delivery: if a recipient is offline, the message is persisted and a push notification (via Apple/Google's push services) is sent instead of a WebSocket push; the message delivers via the normal history-fetch flow when the user reconnects.
- Read receipts and delivery status: these are themselves a form of message (a small event flowing the opposite direction), and can reuse the same Pub-Sub delivery mechanism, with eventual consistency (Chapter 5) being perfectly acceptable — a read receipt arriving a second late is harmless.
- Group chats: fan-out to multiple recipients echoes the Pub-Sub fan-out pattern from Chapter 14 directly — one message published once, delivered independently to each group member's gateway connection.
- Monitoring: track WebSocket connection count per gateway instance and message delivery latency (p99, Chapter 20) as the key health signals for this system specifically.

7. Common Mistakes & Misconceptions

- Designing the chat system around traditional REST polling instead of WebSockets — this fundamentally can't deliver the instant, push-based experience users expect from a modern chat app.
- Forgetting the cross-gateway-instance delivery problem — assuming a message can be pushed directly to a recipient without accounting for the fact that they're likely connected to a different server instance than the sender.
- Over-engineering for strict global message ordering across all conversations, when only PER-CONVERSATION ordering actually matters for a correct user experience.

8. Interviewer Q&A Bank

Q1. Why use WebSockets instead of HTTP polling for a chat application?

A: WebSockets provide a persistent, full-duplex connection that lets the server push new messages to a client instantly, without the client needing to repeatedly ask 'anything new?' Polling (even long-polling) adds latency and unnecessary overhead at the scale of millions of concurrent users sending and receiving messages constantly — WebSockets are the natural fit for this genuinely real-time, bidirectional use case.

Q2. How does a message get delivered if the sender and recipient are connected to different gateway servers?

A: I'd use a Pub-Sub layer between the gateway servers — when a message needs to reach a specific user, it's published to a channel associated with that user, and the gateway instance currently holding that user's connection (having subscribed to their channel) receives it and pushes it down their open WebSocket. This decouples 'which gateway instance is the message FROM' from 'which gateway instance does the message need to reach.'

Q3. How would you handle message delivery to a user who's currently offline?

A: I'd persist the message to the message store regardless of the recipient's online status, and additionally trigger a push notification through the relevant platform's push service (APNs for iOS, FCM for Android) to alert them outside the app. When the user later opens the app and reconnects, they'd fetch recent message history through the normal REST history endpoint, which would include the message they missed.

Q4. Why would you shard the message database by conversation_id rather than by user_id or message_id?

A: Because the dominant query pattern is 'fetch message history for this specific conversation,' which stays entirely within a single shard if we partition by conversation_id — sharding by user_id or a random message_id would scatter a single conversation's messages across multiple shards, making the most common read operation expensive and requiring cross-shard queries.

Q5. How would you ensure a recipient doesn't see messages out of order?

A: I'd focus on ordering guarantees within a single conversation, not globally across the whole system, since that's what actually matters for user experience. This can be achieved by including a sequence number or timestamp per conversation and having the client render messages sorted by that value, with the server ensuring messages within one conversation are processed and delivered in the order they were sent — global ordering across unrelated conversations isn't something users would ever notice or care about.

9. HLD Mini-Scenario: Scaling Connection Capacity

Prompt: "Your chat app's WebSocket gateway layer is hitting connection limits as the user base grows past your initial estimate. How do you scale it?"

- 1. Add more WebSocket gateway server instances behind a load balancer (Chapter 13) configured for connection-based distribution, since each new instance adds capacity for more concurrent persistent connections.

- 2. Ensure the Pub-Sub layer connecting gateway instances (Chapter 14) scales accordingly too — more gateway instances means more subscribers and more cross-instance message routing, so the pub-sub broker's own capacity needs to grow in step.
- 3. Monitor per-instance connection count (Chapter 20) as a primary capacity signal, scaling out proactively before any single instance approaches its practical connection ceiling, rather than waiting for it to start rejecting new connections.

This shows that scaling a connection-heavy system is about horizontally scaling the STATEFUL gateway layer specifically (a less common scaling target than the typical stateless app server scaling from Chapter 6), while making sure the coordination layer between instances scales in parallel.

10. Questions YOU Should Ask the Interviewer

- Do we need to support group chats, or is this strictly one-on-one messaging?
- How important are delivery/read receipts — are they core to the experience or a nice-to-have?
- Should message history be retained indefinitely, or is there an expected retention/archival policy?

11. Scenario-Based Practice Bank (New)

Scenario 1: Users occasionally report that a message they sent 'disappeared' and never reached the recipient, though there's no error shown.

How to approach it: Investigate the cross-gateway delivery path first: confirm the Pub-Sub publish-to-recipient's-channel step actually happened and that the recipient's gateway instance was correctly subscribed. This is the most likely failure point in a chat system, more so than the initial send from the sender's side.

Scenario 2: A recipient who was offline for a day reports they never got a message that was sent to them, even after reopening the app.

How to approach it: Check that the message was persisted to durable storage regardless of online status (not just attempted via WebSocket push) and that the client correctly fetches recent history via the REST history endpoint on reconnect — offline delivery must not depend solely on a live push succeeding.

Scenario 3: In a group chat, some members report seeing messages in a different order than others.

How to approach it: Confirm ordering is enforced per-conversation via a sequence number or timestamp assigned server-side, and that clients render strictly by that value rather than by local receipt time — network delays can cause messages to arrive at different clients in different real-time order even when the server processed them correctly.

Scenario 4: The WebSocket gateway layer is approaching its connection ceiling as the user base grows past initial projections.

How to approach it: Scale out the stateful gateway layer horizontally with more instances behind a connection-aware load balancer, and make sure the Pub-Sub layer connecting instances scales in step — more gateways means more cross-instance subscribers and routing load, not just more raw connection capacity.

Scenario 5: An interviewer asks why message history retrieval (GET /conversations/{id}/messages) isn't also built as a WebSocket event like send_message.

How to approach it: Explain that loading history is a traditional request-response operation with no need for a persistent push channel — REST fits it naturally and more simply, while WebSocket events are reserved for genuinely real-time, server-initiated pushes like new incoming messages or presence updates.

Scenario 6: A read receipt updates a second or two after the message was actually read, and a teammate is worried this delay is a bug.

How to approach it: Reassure that this is an acceptable eventual-consistency trade-off (Chapter 5) — read receipts are low-stakes, and a brief delay is invisible to users in practice. This shouldn't be engineered for strict real-time synchronous guarantees at the cost of added complexity.

Chapter 26 — One-Page Revision Sheet

- Chat apps are built around persistent WebSocket connections (Ch 3), not request-response — this is the defining architectural difference from earlier case studies.
- Cross-gateway-instance delivery problem: sender and recipient are often on DIFFERENT gateway servers — solved with a Pub-Sub layer (Ch 14) that fans messages to the right instance.
- Shard message storage by conversation_id — the dominant query pattern ('get this conversation's history') should stay within one shard.
- At-least-once delivery (Ch 14) is the realistic target; client-side deduplication via message ID handles any duplicate delivery.
- Offline users: persist the message + trigger a push notification (APNs/FCM); deliver via normal history fetch on reconnect.
- Ordering matters PER CONVERSATION, not globally — don't over-engineer for system-wide message ordering.
- Scaling target here is different: the STATEFUL WebSocket gateway layer needs horizontal scaling, and the pub-sub coordination layer must scale alongside it.

Memory hook: Most earlier case studies are like mail (request, response, done). A chat app is like an open phone line — both sides stay connected, and either can speak any moment without asking first.

CHAPTER 27

Case Study: Design a Social Media Feed

(Instagram/Twitter)

The hardest part isn't storing posts — it's deciding when to assemble each user's feed

1. Simple Explanation First

A social media feed shows each user a personalized, reverse-chronological (or ranked) stream of posts from accounts they follow. The deceptively hard part isn't storing posts — it's efficiently producing each user's individual feed when millions of users follow overlapping but different sets of accounts, including some accounts (celebrities) with tens of millions of followers.

Analogy: Imagine a newspaper that needs to print a *DIFFERENT* front page for every single subscriber, personalized to exactly which columnists they follow. You could either pre-print every subscriber's personalized paper the moment any columnist publishes a new article (fast to hand out later, but a massive print run every time a popular columnist writes something), or wait until a subscriber actually asks for their paper and assemble it on the spot by pulling the latest articles from each of their followed columnists (no wasted printing, but the subscriber waits while it's assembled). This is exactly the fan-out on write vs fan-out on read decision at the heart of this chapter.

2. Requirements (the 'R' in RADIO)

Functional requirements: users can post content (text/image/video) and follow other users; users see a feed of posts from accounts they follow, typically reverse-chronological or algorithmically ranked; users can like/comment on posts.

Non-functional requirements: feed loading must be fast (users expect near-instant feed refresh); the system must handle extreme follower-count skew gracefully (most users have few followers, a tiny fraction have tens of millions); eventual consistency (Chapter 5) is acceptable — a post appearing in followers' feeds a few seconds late is invisible to users, but the feed being unavailable is not.

Quick capacity estimation: assume 500 million daily active users, posting 0.5 posts/user/day on average = 250 million posts/day. Far more significant: assume each user checks their feed 10 times/day -> 5 billion feed reads/day, average QPS $\sim 5,000,000,000 / 86,400 \sim \sim 58,000$ reads/sec. The read:write ratio here ($\sim 20:1$) is extreme but less skewed than the URL shortener case study — and unlike that case study, EACH read is personalized per user, so a simple shared cache won't directly solve this the same way.

3. Architecture & The Core Design Decision: Fan-Out Strategy

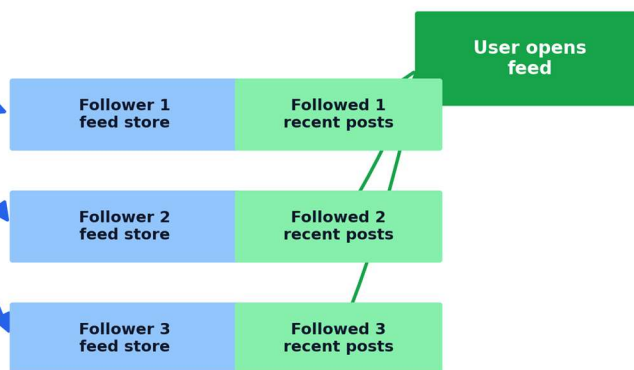
Fan-Out on Write vs Fan-Out on Read

FAN-OUT ON WRITE (push)



fast reads later,
but huge write burst
for celebrities

FAN-OUT ON READ (pull)



no wasted writes,
but assembled live
on every read

Figure 27.1 — Fan-out on write pre-computes every follower's feed; fan-out on read assembles the feed at request time.

Strategy	How It Works	Trade-off
Fan-out on Write (push)	When a user posts, the post is immediately written into the feed of every follower (Chapter 14's fan-out pattern, applied to feed pre-computation)	Very fast reads (feed is pre-assembled) — but a celebrity with 50 million followers triggers 50 million writes per post, which is enormous and slow
Fan-out on Read (pull)	A user's feed is assembled on-demand at read time, by pulling recent posts from everyone they follow and merging/sorting them	No wasted writes for unpopular posts — but assembling a feed live, for every read, is more computational work per request
Hybrid (most real systems)	Fan-out on write for most users (with normal follower counts); fan-out on read specifically for celebrity/high-follower accounts	Gets the speed benefit of pre-computation for the common case, while avoiding the write explosion for the rare, extreme case

Why this matters: This hybrid approach is one of the best examples in this entire book of a senior-level instinct: don't pick ONE strategy for the whole system when the data itself is bimodal (most accounts are small, a few are enormous) — apply different strategies to different segments of the same problem, based on their actual characteristics.

4. Data Model (the 'D' in RADIO)

- Posts table: post_id, author_id, content, media_url (referencing object storage, Chapter 22), created_at — sharded by author_id or post_id.
- Follow graph: follower_id, followee_id — this relationship data, and queries like 'who does user X follow,' fits naturally in a relational model or a graph database (Chapter 7) if relationship-traversal queries (like 'friends of friends' for suggestions) become a major feature.
- Pre-computed feed (for fan-out on write users): a per-user list of post IDs, typically stored in a fast key-value store (Chapter 7/11) like Redis, since it needs to support fast inserts (new post arrives) and fast reads (load my feed) — not complex queries.

5. Interface (the 'I' in RADIO)

- POST /posts — create a new post; triggers the fan-out process asynchronously via a queue (Chapter 14), so the post-creation response returns immediately without waiting for fan-out to complete.
- GET /feed — returns the current user's feed; for fan-out-on-write users, this is a fast read from the pre-computed feed store; for celebrity-following segments, this triggers the fan-out-on-read merge logic.

6. Optimize & Scale (the 'O' in RADIO)

- Async fan-out: the actual fan-out-on-write process (pushing a new post into millions of followers' feeds) should never happen synchronously during the post-creation request — it's queued (Chapter 14) and processed by background workers, so posting feels instant to the author regardless of their follower count.
- Caching: cache fully-assembled feed results, even for fan-out-on-read users, for a short TTL (Chapter 11) — most users don't refresh their feed every second, so a few seconds of caching dramatically reduces redundant computation.
- CDN for media: images and videos attached to posts are served via object storage + CDN (Chapters 16 and 22), never through the application servers directly.
- Ranking (a common follow-up): if the feed isn't strictly reverse-chronological but algorithmically ranked, this typically becomes a separate ranking/scoring service consulted during feed assembly, which is a significant system in its own right (often involving machine learning) — worth acknowledging as a distinct, complex component rather than hand-waving 'and then we rank it.'

7. Common Mistakes & Misconceptions

- Choosing only fan-out on write OR only fan-out on read, without recognizing the bimodal nature of follower counts that makes a hybrid approach clearly superior.
- Performing fan-out synchronously within the post-creation request, which would make posting unacceptably slow for any account with a large following.
- Treating feed assembly as a simple database query, missing that pulling and merging posts from potentially thousands of followed accounts (in the read-fan-out case) is a genuinely nontrivial computational problem worth discussing explicitly.

8. Interviewer Q&A Bank

Q1. What's the difference between fan-out on write and fan-out on read, and when would you use each?

A: Fan-out on write pre-computes and pushes a new post into every follower's feed immediately when it's created, making reads very fast but writes expensive for accounts with huge follower counts. Fan-out on read assembles a user's feed on-demand by pulling from everyone they follow, avoiding wasted writes but making each read more computationally expensive. I'd use fan-out on write for the vast majority of users with normal follower counts, and fan-out on read specifically for celebrity accounts with extremely large followings, combining both in a hybrid approach.

Q2. Why would fan-out on write be a problem for a celebrity account?

A: A celebrity with 50 million followers would trigger 50 million individual write operations every time they post, which is an enormous burst of write load concentrated into a short window — both slow to fully complete and a significant strain on the system. This is exactly the kind of hot-spot scenario discussed in Chapter 10's sharding chapter, and it's why a uniform fan-out-on-write strategy breaks down at this extreme.

Q3. Should fan-out happen synchronously when a user creates a post?

A: No — fan-out should be processed asynchronously via a message queue, so the post-creation request returns success immediately to the author without waiting for potentially millions of downstream feed-insert operations to complete. The author sees their post succeed instantly, while followers' feeds update within a short delay afterward, which is an acceptable eventual-consistency trade-off.

Q4. How would you handle a user who follows both regular accounts and a celebrity account in their feed?

A: Their feed assembly would combine both sources: pulling pre-computed feed entries from the fan-out-on-write path for regular followed accounts, and separately fetching recent posts from any followed celebrity accounts via fan-out on read, then merging and sorting both sets by timestamp (or ranking score) before returning the combined feed.

Q5. How would you scale the system if it needed to support algorithmic (non-chronological) ranking instead of simple reverse-chronological order?

A: I'd introduce a dedicated ranking service that scores candidate posts (pulled via the same fan-out infrastructure) based on relevance signals, consulted during feed assembly rather than baked into the storage layer itself. This is a substantial system in its own right, often involving machine learning models, and I'd treat it as a distinct component with its own scaling and latency considerations rather than assuming it's a simple sort operation.

9. HLD Mini-Scenario: Handling a Sudden Viral Post

Prompt: "A normally-small account's post unexpectedly goes viral, gaining 10 million new followers within an hour. How does your design handle this?"

- 1. Monitor follower count (or post engagement velocity) as a signal (Chapter 20) to dynamically reclassify an account from 'fan-out on write' to 'fan-out on read' once it crosses a threshold — rather than a fixed, permanent classification, this needs to adapt to real-time changes.
- 2. Ensure the queue-based fan-out system (Chapter 14) can handle a sudden burst gracefully — if fan-out for this account was already in progress as a write-fan-out job when it crossed the

threshold, the system needs a way to gracefully cap or redirect the remaining work rather than continuing a now-inappropriate massive write burst.

- 3. Lean on caching (Chapter 11) heavily for this specific viral post's content, since an enormous number of users will be reading it simultaneously regardless of fan-out strategy — this is a classic hot-key scenario benefiting from aggressive, short-TTL caching.

This demonstrates that real systems need DYNAMIC strategies, not just static rules decided once at design time — a strong signal of production-systems thinking.

10. Questions YOU Should Ask the Interviewer

- Should the feed be strictly reverse-chronological, or do we need to support algorithmic ranking?
- What's the expected follower-count distribution — are we designing for a relatively even social graph, or do we need to handle extreme celebrity-scale accounts?
- How fresh does a feed need to be — is a few seconds of delay after a new post acceptable?

11. Scenario-Based Practice Bank (New)

Scenario 1: A celebrity account with 60 million followers posts, and the fan-out-on-write job is still running 20 minutes later, delaying delivery to remaining followers.

***How to approach it:** This is the expected failure mode of applying fan-out on write uniformly. Reclassify this account (based on follower count crossing a threshold) to fan-out on read, so their posts are pulled live by each follower's feed assembly instead of triggering tens of millions of individual writes.*

Scenario 2: A small account unexpectedly goes viral, gaining 10 million followers within an hour, and the write-fan-out job already in progress is now wildly inappropriate for its new scale.

***How to approach it:** This needs dynamic reclassification: monitor follower count/engagement velocity as a live signal, and once the account crosses the fan-out threshold, stop the fan-out-on-write job (or let it complete for followers already queued) and switch newly-joining followers' feed assembly to pull from this account via fan-out on read going forward.*

Scenario 3: A user's feed combines posts from 400 regular follows and 2 celebrity accounts, and the feed loads noticeably slower than users with no celebrity follows.

***How to approach it:** Confirm the design merges both paths correctly: pre-computed feed entries for the 400 regular follows (fast, fan-out-on-write) plus a live pull-and-merge specifically for the 2 celebrity accounts (fan-out-on-read) — the slowdown is expected and can be mitigated with short-TTL caching of the merged result.*

Scenario 4: An interviewer asks what happens to post-creation latency for a celebrity account under a pure fan-out-on-write design.

***How to approach it:** Explain the write-storm problem directly: a celebrity with 50 million followers would need 50 million write operations per post if done synchronously, making posting unacceptably slow — this is exactly why fan-out must be asynchronous (queued) and why celebrity accounts need a different strategy (fan-out on read) entirely.*

Scenario 5: Product wants to switch from reverse-chronological order to an algorithmically ranked feed.

How to approach it: Recommend introducing a dedicated ranking service consulted during feed assembly (scoring candidate posts pulled via the existing fan-out infrastructure), rather than trying to bake ranking logic into the storage layer — this is a substantial, often ML-driven system in its own right, not a simple sort step.

Scenario 6: A team wants one single fan-out strategy for simplicity, even after being shown the celebrity write-storm problem.

How to approach it: Point out this ignores the bimodal nature of the actual data (most accounts have few followers, a small number have tens of millions) — a uniform strategy either wastes enormous write capacity on celebrities (pure write fan-out) or adds unnecessary read-time computation for the vast majority of normal accounts (pure read fan-out). The hybrid approach specifically avoids forcing one-size-fits-all onto genuinely different-shaped data.

Chapter 27 — One-Page Revision Sheet

- The core challenge is WHEN to assemble each user's feed, not how to store posts — fan-out on write (push) vs fan-out on read (pull).
- Fan-out on write: pre-computed, fast reads, but explodes for celebrity accounts with huge follower counts (write storm).
- Fan-out on read: cheap writes, but more computation per read (merging posts from many followed accounts).
- Hybrid is the standard real-world answer: fan-out on write for normal accounts, fan-out on read for celebrity/high-follower accounts.
- Fan-out must be asynchronous (queue, Ch 14) — never block the post-creation response on fanning out to potentially millions of followers.
- Account classification (write-fan-out vs read-fan-out) should be DYNAMIC, adapting as an account's follower count or post velocity changes (e.g., a viral post).
- Ranking (if non-chronological) is a distinct, often ML-driven component consulted during feed assembly — treat it as its own system, not an afterthought sort step.

Memory hook: A newspaper that pre-prints a personalized front page for every subscriber the moment a columnist publishes (fan-out on write) vs assembling each paper only when a subscriber asks for it (fan-out on read) — most subscribers get the fast pre-print, but pre-printing for an author with millions of subscribers would flood the presses.

CHAPTER 28

Case Study: Design a Ride-Sharing App (Uber) & a Video Platform (YouTube/Netflix)

Two large, distinct designs in one chapter — compare the patterns, don't just memorize each separately

1. Simple Explanation First

This final case-study chapter pairs two large, real-world systems that are architecturally very different from each other and from every earlier case study — specifically so you finish Part 5 having seen a genuinely wide range of problem shapes, not just variations on one theme. Ride-sharing is fundamentally a real-time, location-based MATCHING problem. Video platforms are fundamentally a large-file PROCESSING AND DELIVERY problem. Seeing both back-to-back helps cement that system design vocabulary is general-purpose — the right combination of tools changes completely based on what the system actually needs to do.

2. Part A: Designing a Ride-Sharing App (Uber)

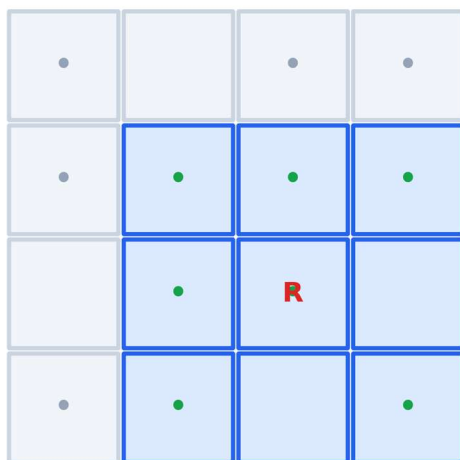
2.1 Requirements

Functional: riders request a ride from their current location; the system matches them with a nearby available driver; both parties can track the ride in real time; the system calculates fare and processes payment. Non-functional: matching must happen within seconds; location updates need to be frequent and low-latency; payment correctness is critical (strong consistency, Chapter 5) while location tracking can tolerate brief staleness (eventual consistency is fine).

Quick capacity estimation: assume 5 million active drivers during peak hours, each sending a location update every 4 seconds (from Chapter 23's Drill 4) — this works out to roughly 1.25 million location writes/second, an enormous, sustained write volume that immediately rules out a traditional relational database for this specific data.

2.2 The Core Design Challenge: Geospatial Matching

Geospatial Indexing — Search Nearby Cells, Not the Whole City



R = rider. Shaded cells = searched (rider's cell + 8 neighbors). Dots = drivers.

Figure 28.1 — Geospatial indexing lets the system query only nearby cells instead of scanning every driver citywide.

Finding 'the nearest available drivers to this rider' is fundamentally a geospatial query, and naively checking the distance from the rider to every single driver in the city would be far too slow at scale. The standard solution is geospatial indexing — dividing the map into a grid of cells (using a technique like geohashing or a quadtree), and indexing which drivers currently occupy each cell. To find nearby drivers, the system queries only the rider's cell and its immediate neighbors, dramatically narrowing the search space.

- Driver location data is held in a fast in-memory store (Chapter 11) like Redis, which has built-in geospatial query support — a strong real-world fit given the extreme write volume and need for fast nearby-search.
- This pattern echoes consistent hashing's core lesson (Chapter 17): instead of a brute-force scan, a smart indexing structure narrows the relevant search space dramatically.

2.3 Architecture & Data Model

- Location Service: ingests extremely high-volume driver location updates, written to the in-memory geospatial store; this is a write-optimized, ephemeral-data component — we don't need to keep a full history of every location ping forever.
- Matching Service: given a ride request, queries the Location Service for nearby available drivers, applies matching logic (distance, driver rating, vehicle type), and offers the ride to a candidate driver.

- Trip & Payment Service: once matched, manages the trip lifecycle and payment — this data needs strong consistency (Chapter 5) and durability, fitting a traditional relational database (Chapter 7), in contrast to the location data's NoSQL/in-memory fit.
- This is a clean example of using DIFFERENT storage technologies for different data within the same system (Chapter 7's core lesson) — location data and payment data have almost opposite consistency and durability needs.

2.4 Optimize & Scale

- Geographic sharding (Chapter 10): partition the Location Service by city or region, since a rider in New York never needs to be matched with a driver in Tokyo — this is a naturally clean sharding key that also keeps related queries within a single shard.
- Real-time updates to riders (tracking their driver's approach) reuse the WebSocket/Pub-Sub pattern from Chapter 26's chat case study directly — the same underlying pattern, applied to a different kind of real-time data.

3. Part B: Designing a Video Platform (YouTube/Netflix)

3.1 Requirements

Functional: creators upload videos; the platform processes them into multiple qualities/formats; viewers can browse, search, and stream videos smoothly. Non-functional: streaming must work well across varying network conditions and devices worldwide; upload-to-availability time should be reasonable but doesn't need to be instant; storage and bandwidth costs at this scale are a first-class design concern, not an afterthought.

Quick capacity estimation: from Chapter 23's Drill 3, 1 million concurrent viewers at 5 Mbps average bitrate requires 5 Tbps of peak bandwidth — a number that, on its own, makes a CDN (Chapter 16) mandatory rather than optional for this system.

3.2 The Core Design Challenge: Upload, Transcode, Distribute

Video Pipeline: Upload -> Async Transcode -> CDN

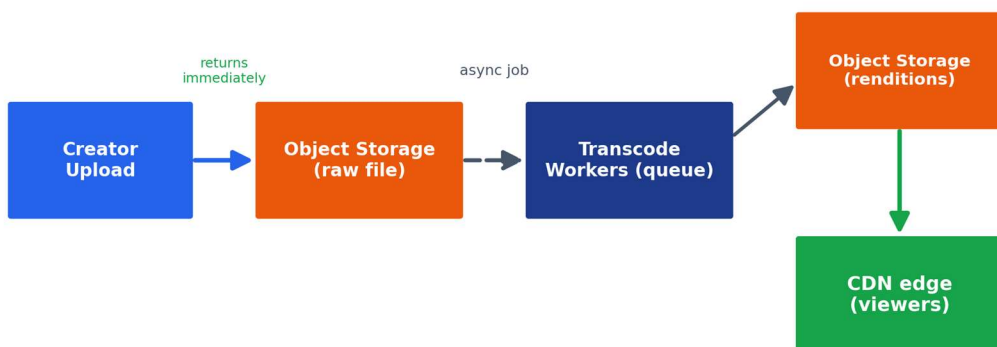


Figure 28.2 — Upload returns immediately; transcoding happens asynchronously before distribution to the CDN.

A raw uploaded video file isn't directly ready for efficient streaming — it needs to be transcoded into multiple resolutions and formats (360p, 720p, 1080p, 4K) so viewers on different devices and network speeds can each get an appropriately-sized stream (a technique called adaptive bitrate streaming). This transcoding work is computationally expensive and takes real time (often minutes), so it must be asynchronous (Chapter 14) — the upload itself returns success quickly, while transcoding happens in the background, with the video becoming available for streaming once processing completes.

- Object storage (Chapter 22) holds both the original uploaded file and every transcoded rendition — this is a textbook large-file storage use case, never appropriate for a traditional database.
- A transcoding service (often a fleet of workers processing jobs from a queue) handles the actual format conversion, which is CPU/GPU-intensive and benefits from horizontal scaling (Chapter 6) of worker instances based on queue depth.
- Once transcoded, renditions are distributed to a CDN (Chapter 16) for actual playback delivery — viewers stream from a nearby edge node, never directly from origin object storage.

3.3 Data Model & Optimize

- Video metadata table: video_id, title, description, uploader_id, status (processing/ready), duration, references to each transcoded rendition's object storage key — a relational or document database (Chapter 7) fits this structured metadata well.
- View counts and engagement data: extremely high-write, simple-increment data, well-suited to an eventually-consistent counter approach (Chapter 5) rather than strongly-consistent exact-real-time counting — a view counter being off by a few for a moment is invisible to users.

- Storage tiering (Chapter 22): older, rarely-watched videos can migrate to cheaper cold storage tiers, while currently-trending content stays in hot storage and aggressively cached at CDN edges — directly reusing the cost-optimization pattern from Chapter 22.

4. Comparing the Two Systems — What This Teaches

Ride-Sharing (Uber)	Video Platform (YouTube/Netflix)
Core challenge: real-time geospatial MATCHING	Core challenge: large-file PROCESSING and global DELIVERY
Extreme write volume of small, ephemeral data (locations)	Moderate write volume of large, durable data (video files)
Strong consistency needed for payments only; location is loosely consistent	Strong consistency needed for metadata; view counts are loosely consistent
Geographic sharding is the natural partition strategy	CDN/edge distribution is the natural scaling strategy

Why this matters: If you take away one lesson from comparing these two systems, let it be this: there is no universal 'correct' system design template. The right architecture is always a direct consequence of what the data actually IS and how it's actually ACCESSED — exactly the diagnostic instinct this entire book has been building toward, one chapter at a time.

5. Interviewer Q&A Bank

Q1. How would you efficiently find nearby drivers for a ride request?

A: I'd use geospatial indexing — dividing the map into a grid of cells (via geohashing or a quadtree) and tracking which drivers are currently in each cell, typically in a fast in-memory store like Redis with built-in geospatial support. To find nearby drivers, the system queries only the rider's cell and its immediate neighbors, rather than calculating distance to every driver in the city, which would be far too slow at scale.

Q2. Why would you use different databases for location data versus payment data in the same ride-sharing system?

A: Location data is high-volume, ephemeral, and tolerant of brief staleness, making a fast in-memory store the right fit. Payment data is lower-volume but needs strong consistency and durability — an incorrect or lost payment record is a serious problem, unlike a slightly-stale driver location. Using a relational database for payments and an in-memory geospatial store for locations matches each data type to its actual requirements, rather than forcing one storage technology to serve both very different needs.

Q3. Why is video transcoding done asynchronously rather than as part of the upload request?

A: Transcoding a video into multiple resolutions is computationally expensive and can take minutes, which would make the upload request unacceptably slow if done synchronously. Instead, the upload itself completes quickly once the raw file is safely stored, and transcoding happens in the background via a queue, with the video becoming available for streaming once processing finishes — keeping the user-facing upload experience fast regardless of backend processing time.

Q4. Why is a CDN considered mandatory rather than optional for a video streaming platform?

A: Because the bandwidth required to serve video at scale — potentially terabits per second for a popular platform — would overwhelm any single origin server or data center if served directly. A CDN distributes that load across many geographically-spread edge nodes, serving the vast majority of streaming traffic without ever reaching the origin, which is the only practical way to deliver video at this scale and with acceptable latency worldwide.

Q5. What do the ride-sharing and video platform case studies have in common, despite being very different systems?

A: Both demonstrate matching the storage and architecture choices to the actual shape and access pattern of each specific type of data within the system, rather than using one uniform approach everywhere — location data and payment data have opposite consistency needs in ride-sharing, just as video files and metadata have very different storage needs in the video platform. The unifying lesson across this entire book is that good system design is diagnostic: understand what the data IS and how it's accessed, then choose tools that fit.

6. HLD Mini-Scenario: A Ride-Sharing Surge Pricing Event

Prompt: "During a major event (a concert ending), ride requests in one small geographic area spike 50x normal volume within minutes. How does your design handle this?"

- 1. Recognize this as a hot-spot scenario (Chapter 10) concentrated in one geographic shard — if location/matching data is sharded by region, this single shard now bears dramatically disproportionate load.
- 2. Apply caching and request queuing (Chapter 14) at the matching layer for this hot region specifically, smoothing the burst rather than letting 50x simultaneous matching requests overwhelm the service synchronously.
- 3. Consider that this is precisely the kind of scenario surge pricing exists to address from a product perspective (reducing demand and incentivizing more drivers into the area) — worth mentioning that the system and the business logic work together to manage extreme localized demand, not just infrastructure scaling alone.

This connects back to Chapter 10's hot-partition discussion, applied to a genuinely real, business-relevant scenario.

7. Questions YOU Should Ask the Interviewer

- For the ride-sharing system: should I focus on the matching algorithm itself, or the broader system architecture?
- For the video platform: should I go deep on the transcoding pipeline, or focus more on the metadata/browsing/search experience?
- Are there specific scale or geographic constraints (single city vs global) that should shape either design?

8. Scenario-Based Practice Bank (New)

Scenario 1: During a large concert's end, ride requests in a single small area spike 50x within minutes, overwhelming the local matching service.

How to approach it: Recognize this as a geographic hot-spot on a region-sharded system. Add caching and request queuing at the matching layer for that specific hot region to smooth the burst, and note that surge pricing is the product-level lever that reduces demand and pulls in more drivers — infrastructure scaling and business logic work together here.

Scenario 2: An interviewer asks why driver location data isn't stored in the same relational database used for trip and payment records.

How to approach it: Explain the mismatch in requirements: location data is extremely high-volume, ephemeral, and tolerant of brief staleness — a fast in-memory geospatial store fits it far better than a relational database, which is instead reserved for trip/payment data that genuinely needs strong consistency and durability.

Scenario 3: A video platform's upload endpoint is taking minutes to respond because transcoding happens synchronously within the request.

How to approach it: Move transcoding to an asynchronous background process via a queue: the upload should return success as soon as the raw file is safely stored in object storage, with transcoding workers processing the format conversion afterward and the video becoming available once that completes.

Scenario 4: A newly-uploaded, suddenly-trending video overwhelms the origin object storage because too many viewers are streaming directly from it.

How to approach it: Confirm the CDN is correctly in front of object storage for this content — viewers should stream from nearby CDN edge nodes, not the origin directly. This is exactly the mandatory-CDN scenario the bandwidth math (potentially terabits per second) already justifies for any popular video platform.

Scenario 5: A view-count feature needs to increment on every single video play across a massively popular platform, and exact real-time accuracy is being requested.

How to approach it: Push back gently: view counts are a natural fit for eventually-consistent counters, since a count being off by a handful for a moment is invisible to users. Enforcing strong, real-time-exact consistency here would add unnecessary complexity and cost for no real user-facing benefit.

Scenario 6: An interviewer asks what these two very different case studies (ride-sharing, video platform) have in common as a closing question.

How to approach it: Name the unifying lesson directly: both systems succeed by matching storage and architecture choices to what each specific piece of data actually IS and how it's accessed — location vs payment data in ride-sharing, video files vs metadata vs view counts in the video platform — rather than applying one uniform technology choice across a system with genuinely different data shapes.

Chapter 28 — One-Page Revision Sheet

- Ride-sharing core challenge: real-time geospatial MATCHING. Geospatial indexing (geohash/quadtree) narrows driver search to nearby cells instead of scanning the whole city.
- Ride-sharing data split: location data (high-volume, ephemeral, eventually consistent, in-memory store) vs payment data (lower-volume, strongly consistent, relational DB).
- Geographic sharding (Ch 10) is the natural partition strategy for ride-sharing — a rider never matches with a driver in a different city.
- Video platform core challenge: large-file PROCESSING and global DELIVERY. Upload returns fast; transcoding is asynchronous (Ch 14); CDN (Ch 16) is mandatory given the bandwidth math.
- Video platform data split: structured metadata (relational/document DB) vs the actual video files (object storage, Ch 22) vs view counts (eventually consistent counters).
- Both systems prove the same lesson: match storage/architecture choices to what each piece of data actually IS and how it's accessed — never one uniform approach for an entire system.
- This closes Part 5: every case study chapter (24-28) has been the same RADIO method, applied to genuinely different problem shapes.

Memory hook: *Uber is a real-time matchmaker reading a constantly-shifting map. YouTube is a factory that processes raw material (video) into finished products (renditions) and ships them worldwide. Different jobs, different tools — same underlying design discipline.*

CHAPTER 29

LLD / OOD Fundamentals

SOLID principles, design patterns, and two complete LLD walkthroughs

1. Simple Explanation First

Low-Level Design (LLD) / Object-Oriented Design (OOD) asks: given one specific component or system, how would you actually design the classes, interfaces, and relationships that implement it? Where HLD draws boxes representing entire services, LLD draws boxes representing individual classes and the methods on them. The interviewer is testing whether your code structure is clean, extensible, and testable — not just whether it works.

Analogy: If HLD is an architect's floor plan showing where the rooms go, LLD is the structural engineer's drawing showing exactly how the load-bearing walls are built, where the pipes run, and how each room's fixtures connect to the building's systems. The floor plan tells you what exists; the structural drawing tells you how it actually works inside.

2. Why This Topic Exists

A system built on poorly-structured classes becomes a nightmare to maintain: changing one thing breaks another, adding a new feature requires touching a dozen existing files, and testing becomes impossible without a real database running. SOLID principles and design patterns are accumulated wisdom about how to avoid these traps — not arbitrary rules, but solutions to specific pain points engineers have encountered repeatedly over decades.

Why this matters: LLD rounds explicitly test whether you can produce maintainable code architecture — a quality impossible to evaluate in a standard coding round. Senior engineers are expected to think about SOLID automatically, not as a checklist to consult. The goal of this chapter is to make these principles feel like second nature.

3. SOLID Principles

SOLID — Five Principles for Clean Classes



Figure 29.1 — SOLID: five principles that keep classes focused, extensible, and easy to test.

3.1 S — Single Responsibility Principle

A class should have only ONE reason to change — meaning it should do ONE job well, not be a grab-bag of unrelated functionality.

Violates SRP	Follows SRP
A UserService that handles authentication, sends emails, AND generates PDF invoices — three unrelated reasons to change	An AuthService, an EmailService, and an InvoiceService — each with one focused responsibility
If the email provider changes, you're forced to touch the same class that handles authentication logic	Each class can change for its own reason without touching the others

3.2 O — Open/Closed Principle

A class should be OPEN for extension (new behavior can be added) but CLOSED for modification (existing, tested code shouldn't need to change when adding new features). The classic implementation: define an abstract interface or base class, and extend it with new concrete classes rather than adding if/else branches to an existing class every time a new variant is needed.

Common mistake: Adding a new 'else if' branch to a PaymentProcessor class every time a new payment method is added (Visa, Mastercard, PayPal, Bitcoin...). This forces modifying a class that's already working and tested. Instead, define a PaymentMethod interface and create a new implementing class per method — the PaymentProcessor never needs to change when a new method is added.

3.3 L — Liskov Substitution Principle

A subclass must be fully substitutable for its parent class without breaking the program. If code works correctly with a base class, it should work identically correctly with any subclass, with no surprising

behavior changes or exceptions. A classic violation: a Square class extending Rectangle, but overriding `setWidth()` and `setHeight()` to keep the sides equal — code that calls `rect.setWidth(5)` and `rect.setHeight(10)` on a Rectangle would get a 10x10 square instead of a 5x10 rectangle, breaking the caller's expectation.

3.4 I — Interface Segregation Principle

No client should be forced to depend on methods it doesn't use. Prefer many small, focused interfaces over one large 'do-everything' interface. If a class only needs to be printable, it should implement a Printable interface — not a giant Document interface that also requires it to implement `save()`, `email()`, `compress()`, and `encrypt()` even though it doesn't care about any of those.

3.5 D — Dependency Inversion Principle

High-level modules should not depend on low-level modules directly — both should depend on abstractions (interfaces). This is what makes code testable: if a UserService has a concrete dependency on MySQLDatabase, you can't test UserService without a real MySQL database running. If UserService depends on a DatabaseInterface abstraction instead, you can inject a MockDatabase in tests, making testing fast, isolated, and reliable.

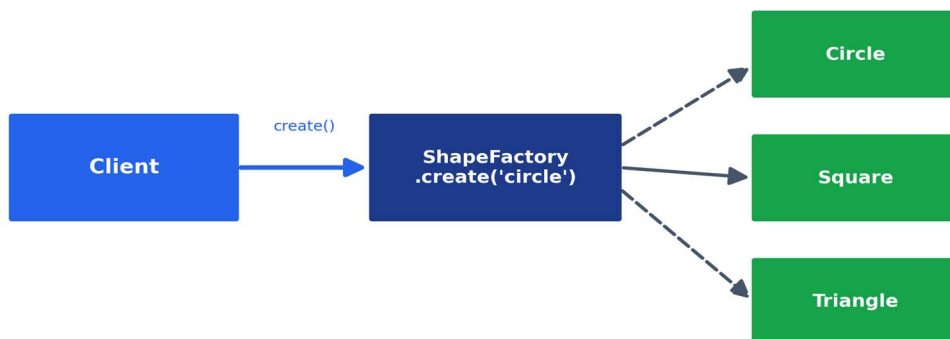
Why this matters: This principle is why dependency injection frameworks (Spring, Guice) exist, and why 'inject your dependencies rather than creating them inside the class' is repeated so often. It's the practical mechanism that makes the entire book's refrain of 'code to interfaces, not implementations' actually work in real codebases.

4. Design Patterns You Must Know

Design patterns are named, reusable solutions to commonly-occurring design problems. You don't need to know all 23 Gang-of-Four patterns — you need to know the handful that come up constantly in both real work and LLD interviews.

4.1 Factory Pattern — Let Me Handle Object Creation

Factory Pattern



The client never calls 'new Circle()' directly — the Factory decides.

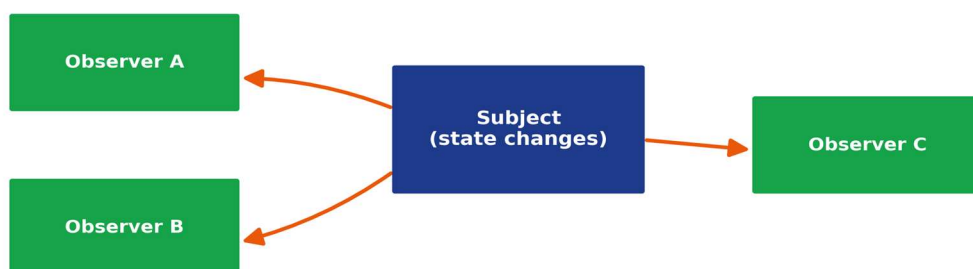
Figure 29.2 — The Factory decides which concrete class to instantiate; the Client never calls new directly.

Problem: a client needs to create objects but shouldn't need to know which specific subclass to instantiate, since that decision depends on configuration, context, or input data. **Solution:** delegate object creation to a Factory class. The client calls `factory.create('circle')` and gets back a `Shape` — it doesn't care that a `Circle` was constructed under the hood.

- Real-world use: a notification system where the factory creates an `EmailNotification`, `SMSNotification`, or `PushNotification` based on user preference settings stored in a database.

4.2 Observer Pattern — Notify Me When Something Changes

Observer Pattern



Subject calls `update()` on every registered Observer — none are hardcoded.

Figure 29.3 — The Subject notifies all registered Observers automatically when its state changes.

Problem: when one object's state changes, zero or more other objects need to be notified automatically — but you don't want to hardcode all those dependencies into the object that changed. **Solution:** the Subject maintains a list of Observers; when its state changes, it calls `update()` on each one. Observers can be added or removed at runtime without touching Subject code.

- Real-world connection: this is the pattern underlying React's `useState` hooks, JavaScript event listeners, and the Pub-Sub architecture from Chapter 14 — three completely different contexts, same fundamental shape.

4.3 Strategy Pattern — Swap Algorithms at Runtime

Problem: you need different variations of an algorithm, and the right one varies by context or configuration. **Solution:** define a Strategy interface, create concrete implementations of each algorithm variant, and inject the right one into the class that uses it. The class itself stays unchanged; you just swap the strategy.

- Real-world use: a sorting system where you can inject `QuickSortStrategy`, `MergeSortStrategy`, or `BucketSortStrategy` depending on dataset size. The `Sorter` class never changes regardless of which algorithm is selected. This directly implements Open/Closed.

4.4 Singleton Pattern — There Can Be Only One

A Singleton ensures exactly ONE instance of a class exists across the entire application, accessible via a static `getInstance()` method with a private constructor. Common use: configuration managers, thread pools, connection pools.

Common mistake: Overusing Singleton as a 'global variable in disguise,' creating hidden dependencies and making classes impossible to test in isolation. Dependency injection (from DIP) is almost always a better solution: pass the shared object explicitly to classes that need it, rather than having them call `ConfigManager.getInstance()` internally — this makes the dependency visible and mockable in tests.

4.5 Decorator Pattern — Add Behavior Without Changing the Class

Problem: you want to add optional behaviors to objects at runtime, without modifying the original class or creating an explosion of subclasses for every possible combination. Solution: wrap the original object in a Decorator class that implements the same interface, adds extra behavior, and delegates the core operation to the wrapped object.

- Real-world use: Java I/O streams (BufferedReader wrapping FileReader), HTTP middleware (auth, logging, rate-limiting decorators wrapping the core request handler), and coffee-shop order systems (MilkDecorator, SugarDecorator wrapping a base Coffee).

5. LLD Walkthrough 1: Parking Lot System

Parking Lot — Class Relationships

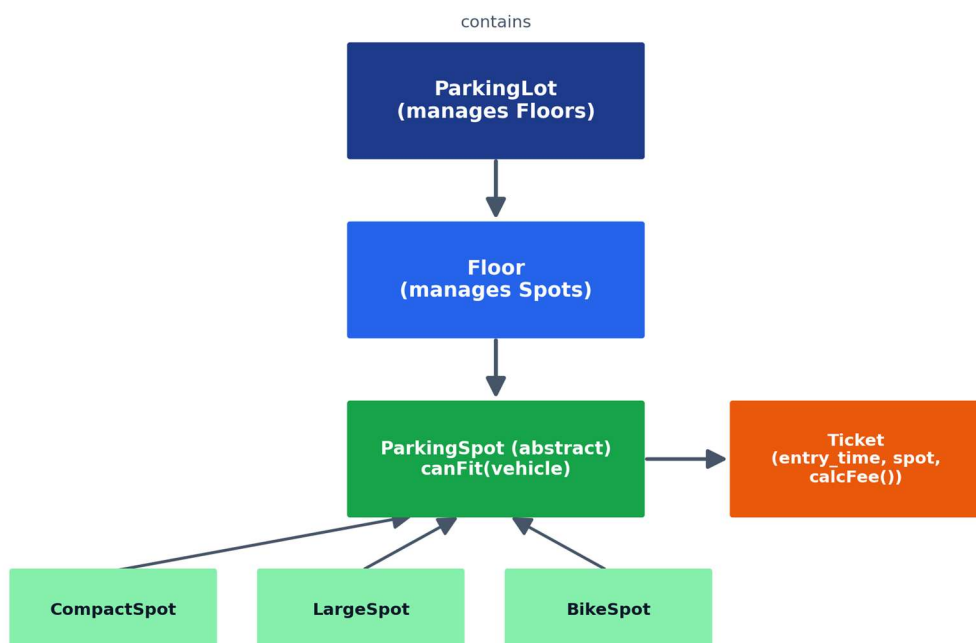


Figure 29.4 — ParkingLot contains Floors, which contain abstract ParkingSpots, which issue Tickets.

5.1 Requirements

A multi-floor parking lot handling different vehicle types (bikes, compact cars, large vehicles). Entry gates issue a ticket when a vehicle parks; exit gates calculate a fee based on duration. The system should know how many spots are available at any time and of what type.

5.2 Key Design Decisions

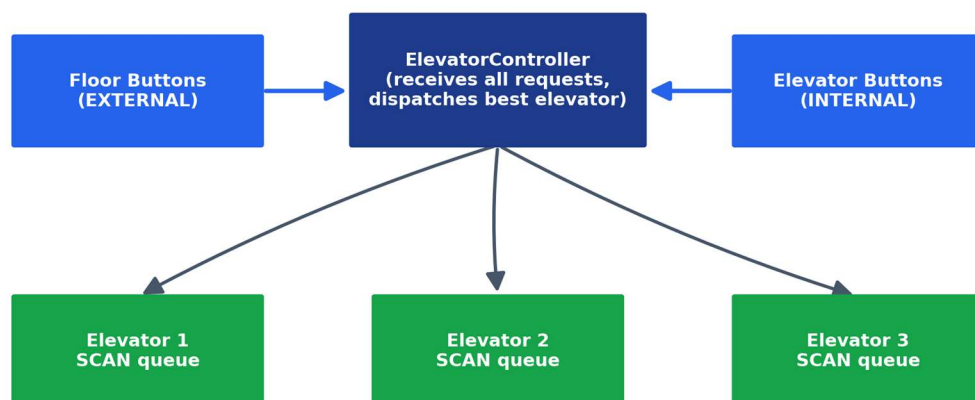
- 1. ParkingSpot is abstract with a canFit(vehicle) method — subclasses (CompactSpot, LargeSpot, BikeSpot) each implement their specific size-matching logic. This follows Open/Closed: adding a HandicappedSpot type means adding a new subclass, not modifying existing ones.
- 2. A Ticket is issued at entry containing the specific ParkingSpot reference and entry time. Fee calculation (calcFee()) lives on the Ticket or a dedicated FeeCalculator — not scattered across multiple classes. This is Single Responsibility.
- 3. ParkingLot delegates spot-availability queries to Floor, which knows its own spots — ParkingLot doesn't need to know the internal layout of each floor. This is proper encapsulation following the Law of Demeter.
- 4. Vehicle type is an enum (BIKE, COMPACT, LARGE) — this is what each spot's canFit() checks, making it easy to add a new vehicle type without changing the ParkingSpot interface itself.

5.3 SOLID Applied

Principle	Where It Appears in This Design
Single Responsibility	ParkingLot manages flow, Floor manages its spots, Ticket calculates fees — no class does more than one thing
Open/Closed	Adding a new spot type = new subclass of ParkingSpot, no existing code touched
Liskov Substitution	Any ParkingSpot subclass works wherever ParkingSpot is expected — canFit() contract is respected
Dependency Inversion	ParkingLot depends on the ParkingSpot interface, not directly on CompactSpot or LargeSpot

6. LLD Walkthrough 2: Elevator System

Elevator System — Controller Dispatches, Each Elevator Schedules



Dispatch algorithm (SCAN, nearest-elevator, etc.) is injected as a Strategy.

Figure 29.5 — ElevatorController dispatches all requests; each Elevator runs its own scheduling queue.

6.1 Requirements

A building with N elevators and M floors. Users press buttons (inside elevators to go to a floor, outside elevators to call one). The system dispatches the best elevator to each request and should minimize average wait time.

6.2 Key Design Decisions

- 1. ElevatorController is the single brain — it receives ALL requests (from floor buttons and inside-elevator buttons) and decides which elevator services each, preventing elevators from independently making conflicting decisions.
- 2. Each Elevator maintains a PriorityQueue of assigned floors and processes them in SCAN/LOOK order: move in one direction servicing all requests along the way, then reverse when no more remain in that direction. This minimizes redundant back-and-forth travel.
- 3. Request has a type (INTERNAL = pressed inside an elevator, EXTERNAL = pressed from a floor button) because dispatch logic differs: internal requests are already assigned to one elevator; external requests need the Controller to pick the best one.
- 4. The dispatch algorithm is injected as a Strategy (section 4.3), so SCANStrategy can be swapped for NearestElevatorStrategy or OptimalStrategy without changing ElevatorController — this directly applies Open/Closed.

6.3 Elevator State Machine

An elevator is a natural state machine — modeling valid states and transitions prevents bugs like doors opening while the elevator is moving.

State	Transitions To (and When)
IDLE	MOVING_UP or MOVING_DOWN — when a new request is dispatched to this elevator
MOVING_UP	STOPPED_AT_FLOOR — when it reaches a requested floor; or MOVING_DOWN when no more requests are above
MOVING_DOWN	STOPPED_AT_FLOOR — when it reaches a requested floor; or MOVING_UP when no more requests are below
STOPPED_AT_FLOOR	DOORS_OPEN — immediately after stopping
DOORS_OPEN	DOORS_CLOSED — after a timeout or explicit close button press
DOORS_CLOSED	MOVING_UP / MOVING_DOWN / IDLE — based on next item in PriorityQueue

7. Common LLD Interview Mistakes

- Designing a god class that does everything (a single ParkingSystem managing floors, spots, vehicles, tickets, fees, and gates) — the most common violation of SRP in LLD rounds.
- Using if/else chains or switch statements where polymorphism belongs — a clear signal the design isn't properly extensible.
- Forgetting to model stateful entities (like an elevator) as state machines — states and valid transitions are often the clearest path to correctness.
- Mixing storage/persistence concerns directly into domain classes — a ParkingLot class shouldn't contain database connection logic (violates SRP and Dependency Inversion equally).

8. Interviewer Q&A Bank

Q1. What is the Single Responsibility Principle and why does it matter?

A: SRP says each class should have only one reason to change — one focused job. It matters because a class doing many things creates tight coupling: changing one behavior risks breaking others, and testing it requires setting up all unrelated dependencies it touches. When each class has one job, changes are localized, tests are focused, and the codebase stays understandable as it grows.

Q2. How does the Open/Closed Principle apply to a payment system?

A: Instead of a PaymentProcessor with if/else branches for each payment method that grows every time a new method is added, I'd define a PaymentMethod interface with a processPayment() method, and create a new implementing class for each method — CreditCardPayment, PayPalPayment, CryptoPayment. The PaymentProcessor depends on the PaymentMethod interface and never needs to change when a new method is added: it's open for extension through new classes, but closed to modification.

Q3. What's the difference between Factory and Strategy patterns?

A: Factory is about CREATION: deciding which concrete class to instantiate, hiding that decision from the caller. Strategy is about BEHAVIOR: swapping different algorithm implementations at runtime. Both

involve an interface and multiple concrete implementations, but Factory is invoked once at object creation time while Strategy can be changed dynamically during the object's lifetime.

Q4. How would you design the scheduling algorithm for the elevator system?

A: I'd implement it as a Strategy interface so different algorithms can be swapped without changing ElevatorController. My recommended default is SCAN: the elevator moves in one direction servicing all requests along the way until there are no more in that direction, then reverses. This minimizes average wait time by preventing constant back-and-forth. LOOK is a similar variant that reverses as soon as there are no more requests in the current direction — slightly more efficient in practice.

Q5. Why is Singleton considered an antipattern in some contexts?

A: Because it creates hidden, global dependencies that make classes hard to test in isolation. A class that internally calls ConfigManager.getInstance() has a hidden dependency on a global object — you can't test it without a fully-initialized singleton. Dependency injection achieves the same 'one shared instance' behavior without the hidden coupling: you pass the shared object explicitly to classes that need it, making the dependency visible and mockable in tests.

9. Questions YOU Should Ask the Interviewer (LLD Round)

- Should I focus on the class structure and relationships, or do you also want me to sketch out method implementations?
- Are there specific design patterns you'd like to see applied, or should I choose based on what fits the problem naturally?
- How important is thread-safety — should I assume concurrent access by multiple users or threads?

10. Scenario-Based Practice Bank (New)

Scenario 1: A candidate's ParkingSystem class handles floor management, spot allocation, ticket issuance, fee calculation, AND payment processing all in one class.

How to approach it: This is a textbook SRP violation — a god class. Split it into focused classes: ParkingLot (overall flow), Floor (its own spots), Ticket (fee calculation), and a separate PaymentProcessor — each with one reason to change.

Scenario 2: Every time a new payment method is added to a checkout system, an engineer has to add another 'else if' branch to the same PaymentProcessor class, and it's grown to 15 branches.

How to approach it: This violates Open/Closed. Introduce a PaymentMethod interface with a processPayment() method, and create one new implementing class per payment type — the PaymentProcessor then depends only on the interface and never needs modification when a new method is added.

Scenario 3: A Square class extends Rectangle and overrides setWidth()/setHeight() to always keep both sides equal, and code that calls rect.setWidth(5); rect.setHeight(10) unexpectedly gets a 10x10 square.

How to approach it: This is a Liskov Substitution violation — Square isn't safely substitutable for Rectangle, since callers reasonably expect setWidth/setHeight to behave independently. Recommend

not modeling Square as a Rectangle subclass at all, or redesigning the abstraction so both share a common Shape interface without assuming interchangeable width/height semantics.

Scenario 4: A class implementing a 'Printable' interface is forced to also implement save(), email(), compress(), and encrypt() methods it doesn't need or use, throwing 'not supported' exceptions for the ones it can't handle.

How to approach it: This is an Interface Segregation violation — the interface is too broad. Split it into smaller, focused interfaces (Printable, Savable, Emailable, etc.) so a class only implements what it actually needs, rather than being forced to stub out irrelevant methods.

Scenario 5: A UserService directly instantiates a MySQLDatabase inside its constructor, and the team can't write a unit test for UserService without spinning up a real database.

How to approach it: This is a Dependency Inversion violation. Have UserService depend on a DatabaseInterface abstraction instead of the concrete MySQLDatabase class, and inject the implementation from outside — in tests, inject a MockDatabase instead, making UserService fast and isolated to test.

Scenario 6: An interviewer asks you to add a new elevator dispatch algorithm (e.g., prioritizing VIP floor requests) without modifying the existing ElevatorController class.

How to approach it: Since the dispatch algorithm is already injected as a Strategy, implement a new VIPPriorityStrategy class implementing the same Strategy interface and inject it in place of SCANStrategy — ElevatorController itself needs zero changes, which is the Open/Closed principle working exactly as designed.

Chapter 29 — One-Page Revision Sheet

- SOLID: Single Responsibility (one class, one job), Open/Closed (extend without modifying), Liskov (subclass replaces parent safely), Interface Segregation (small focused interfaces), Dependency Inversion (depend on abstractions, not concretions).
- Factory: delegate object creation — caller never uses 'new ConcreteClass()' directly.
- Observer: Subject notifies all registered Observers automatically on state change — add/remove observers freely without touching Subject.
- Strategy: inject different algorithm implementations — open/closed applied to behavior variation.
- Singleton: use sparingly — prefer dependency injection; Singleton creates hidden global state that makes testing hard.
- Parking Lot insight: ParkingSpot is abstract with canFit() — each concrete spot type knows its own size constraint. This makes adding new spot types safe and trivial.
- Elevator insight: ElevatorController is the single dispatcher; Elevator runs SCAN/LOOK scheduling via an injected Strategy; model Elevator as a state machine to prevent invalid transitions.

Memory hook: *HLD is the floor plan (what rooms exist). LLD is the structural drawing (how each room is actually built inside). SOLID ensures the internal structure won't collapse when you add another floor.*

CHAPTER 30

The Interview Playbook - Master Cheat Sheet

Everything from all 29 chapters distilled into one actionable reference for interview day

1. What This Chapter Is

This is the final chapter and the payoff chapter. Every concept in this book has been building toward one goal: walking into a system design interview, staying calm, and producing a complete, well-reasoned design under time pressure. This chapter assembles the scaffolding for that performance — not new content to learn, but a structured reference to review the night before and mentally rehearse the morning of.

Analogy: Think of this chapter as a musician's setlist before a gig. Every song has been rehearsed individually across the previous chapters. This is the card that says 'play them in this order, at this tempo, hit these key moments.' The music is in your fingers already — this just ensures you don't freeze on the opening note.

2. Your 45-Minute Interview Flow

The 45-Minute Interview Flow — Five Phases



Run it the same way every time until it's automatic — narrate every phase out loud.

Figure 30.1 - Your five-phase playbook. Run it the same way every time until it is automatic.

The flow is deterministic: always the same five phases, always in this order, always narrated out loud. The interviewer is grading your process as much as your output. A candidate who silently draws a perfect diagram teaches the interviewer nothing; a candidate who narrates every trade-off demonstrates senior-level thinking even if the diagram is imperfect.

Phase 1: Clarify Requirements (0-5 min)

Never start designing before you understand what you're designing. Spend the first five minutes asking focused questions. The answers will shape every subsequent decision.

- Functional scope: which features are in scope and which are explicitly out?
- Scale: daily active users, messages per day, data volume over the system's lifetime.
- Non-functional targets: latency requirements, availability SLA, consistency needs.
- Constraints: geographic distribution, tech stack preferences, regulatory requirements.

Why this matters: Five minutes of clarification prevents thirty minutes of designing the wrong system. Interviewers deliberately leave prompts vague to see if you'll ask — jumping straight to design is itself a red flag.

Phase 2: Capacity Estimation (5-10 min)

Run the numbers. State your assumptions out loud, use round numbers, and critically, close the loop from every number to an architectural consequence. Precision is irrelevant; order-of-magnitude reasoning is everything.

- 1. QPS: $\text{DAU} \times \text{events/user/day} / 86,400 = \text{avg QPS}$. Multiply by 2-3 for peak.
- 2. Storage: $\text{objects/day} \times \text{avg size} \times \text{years} = \text{total}$. Tells you immediately if you need object storage (Ch 22) or a simple database suffices.
- 3. Bandwidth: $\text{concurrent users} \times \text{stream bitrate} = \text{peak bandwidth}$. Tells you immediately if a CDN (Ch 16) is mandatory.
- 4. Connections: $\text{DAU} \times \% \text{ online at peak} = \text{concurrent connections}$. Tells you if WebSockets (Ch 3) and a multi-server gateway (Ch 26) are needed.

Every number should end with a stated consequence: 'At 50,000 writes/sec, a single database can't sustain this — so we need sharding from day one.' That one sentence transforms estimation from box-ticking into real design input.

Phase 3: High-Level Architecture (10-20 min)

Draw the core system diagram. Start with the standard spine and justify every component you add or deliberately omit.

- Standard spine: Client -> Load Balancer (Ch 13) -> App Servers -> Cache (Ch 11) -> Primary Database (Ch 7).
- Add a CDN (Ch 16) if you have static assets or large media serving globally.
- Add an API Gateway (Ch 15) if you have multiple backend services or need auth, rate limiting at the edge.
- Add a Message Queue (Ch 14) for any step that does not need an immediate synchronous response.
- Add Object Storage (Ch 22) for any large binary files — images, video, backups.
- Add WebSocket Gateway (Ch 3/26) if the system has real-time, push-based communication.

Why this matters: You do not add every component to every system. Part of what is being graded is your judgment about what is *NEEDED* versus what is unnecessary complexity. Justify each component in one sentence and explain what you are deliberately leaving out.

Phase 4: Deep Dive and Data Model (20-35 min)

Pick the most interesting or risky component and go deep. This is where you demonstrate real technical depth. Simultaneously, sketch the data model.

- Data model: concrete table names, specific field types, and the access pattern that justifies each index (Ch 8) and SQL vs NoSQL choice (Ch 7).
- API design: specific endpoints with HTTP methods and request/response shape (Ch 4). Concrete: GET /feed?cursor=X, not just 'some feed endpoint.'
- Sharding key (Ch 10): name it explicitly and justify why — this is almost always asked as a follow-up.
- The most interesting component to deep-dive is almost always whichever is hardest: the fan-out engine (Ch 27), geospatial index (Ch 28), ID generator (Ch 24), or WebSocket routing problem (Ch 26).

Phase 5: Optimize and Trade-offs (35-45 min)

Close by proactively addressing scale, reliability, and observability — the three dimensions interviewers always care about.

- Scale: what breaks first at 10x? Name the specific bottleneck and the named fix: 'The database primary becomes the bottleneck — add read replicas (Ch 9) for read-heavy load, shard by X (Ch 10) if write volume grows past Y.'
- Reliability: walk through SPOFs (Ch 19) at every layer. Load balancer, database, cache, queue — name the redundancy strategy for each.
- Observability: answer 'how would you know this system is failing?' before being asked (Ch 20). Name specific metrics for this system — error rate, p99 latency, cache hit ratio — not just 'we would add monitoring.'

3. The Top 20 Mistakes Candidates Make

Requirements Phase

- 1. Starting to design without asking any clarifying questions — designing the wrong system with perfect execution still scores zero.
- 2. Not explicitly defining what is OUT of scope — an open-ended design trying to include everything is unfinishable in 45 minutes.
- 3. Forgetting non-functional requirements — latency target, consistency level, and availability SLA are as important as features.

Estimation Phase

- 4. Skipping estimation entirely and jumping straight to architecture — leaving every design decision unjustified.
- 5. Spending too long on precise arithmetic — rounding 86,400 to 100,000 is fine; spending 5 minutes on exact math wastes time.
- 6. Calculating numbers but never stating consequences — the number only matters if it changes a design decision.
- 7. Estimating only for average load — provision for peak (2-3x average), not typical traffic.

Architecture Phase

- 8. Adding every component (CDN, Kafka, microservices, sharding) regardless of whether the problem needs them — over-engineering is penalised as much as under-engineering.
- 9. Forgetting that the load balancer itself is a potential SPOF — every component needs its own redundancy story.
- 10. Saying 'we'd use a database' without specifying SQL vs NoSQL and why — the choice matters and so does the justification.
- 11. Defaulting to microservices for a system that does not justify the complexity (Ch 18) — a well-structured monolith is the right answer for many prompts.

Data Model and API Phase

- 12. Designing the data model without naming a sharding key — 'we'd shard it' without specifying how is an incomplete answer.
- 13. Designing an API with only happy-path responses — name error cases and appropriate status codes.
- 14. Storing large binary files in a relational database (Ch 22) — a classic, immediately penalised mistake.
- 15. Not mentioning indexes (Ch 8) when describing query patterns — an unindexed query at scale is a full-table scan.

Optimization and Reliability Phase

- 16. Only mentioning redundancy for app servers — forgetting databases, load balancers, queues, and caches also need redundancy.
- 17. Treating monitoring as an afterthought — 'we'd add Datadog' is not an answer; naming specific metrics for THIS system is.
- 18. Never proactively mentioning failure scenarios — waiting to be asked 'what if this fails?' instead of walking through it yourself.
- 19. Claiming exactly-once message delivery without acknowledging its complexity (Ch 14) — at-least-once plus idempotent consumers is the real answer.
- 20. Running out of time before optimization — leaving the most senior-signal part of the interview blank because earlier phases ran long.

4. Master Bank of Questions to Ask the Interviewer

Asking good questions is not just a courtesy — it is a graded skill. These questions signal you understand which dimensions actually drive design decisions.

Scale and Traffic

- What is the target scale — DAU, events per user per day, data volume over the system's lifetime?
- Is the traffic distribution relatively even, or are there known spikes (time of day, seasonal events)?
- Is the read:write ratio heavily skewed in one direction?

Consistency and Correctness

- Does the system require strong consistency, or is eventual consistency acceptable?
- Are there any operations where correctness is non-negotiable even at the cost of availability?
- How should the system behave during a partial outage — fail closed (safe) or fail open (available)?

Geographic and Infrastructure

- Is the user base concentrated in one region, or spread globally?
- Are there compliance or data-residency requirements that restrict where data can be stored?
- Should I assume we are building on a public cloud, or on-premises?

Product and Business Constraints

- What is the actual cost of downtime for this system — minutes, hours, or are there financial penalties?
- Are there existing systems this needs to integrate with?
- Is there a specific part of this system you would like me to prioritise for the deep dive?

5. Compiled One-Page Cheat Sheet — Every Chapter's Key Point

The single most interview-critical insight from each of the 28 building-block chapters. Use this as a last-minute revision pass.

Ch	Topic	The One Thing to Remember
1	RADIO Framework	Requirements -> Architecture -> Data Model -> Interface -> Optimize. Always in this order, always narrated out loud.
2	DNS + Networking	DNS translates names to IPs. TCP is reliable, UDP is fast. HTTP/2 multiplexes many requests over one connection.
3	APIs + Protocols	REST = stateless resources. gRPC = fast binary for internal services. WebSocket = persistent full-duplex for real-time push.
4	API Design	Not every request needs an immediate response. Choose REST vs gRPC based on public-facing vs internal service-to-service.
5	CAP + PACELC	CP = consistent but may be unavailable. AP = available but may be stale. PACELC adds: consistency has a latency cost even without a partition.
6	Scalability	Horizontal scaling = more machines. Make services stateless. Estimate before designing — the numbers drive the decisions.
7	SQL vs NoSQL	SQL: ACID, joins, complex queries. NoSQL: flexible schema, horizontal scale — choose type by access pattern (key-value, document, column, graph).

Ch	Topic	The One Thing to Remember
8	Indexing	An index speeds reads by trading write overhead. Name your indexes explicitly. Composite index column order matters. Never leave a WHERE clause unindexed at scale.
9	Replication	Primary-replica for read scaling. Sync = strong consistency + higher write latency. Async = eventual consistency + lower write latency. Name your lag tolerance.
10	Sharding	Partition data across machines. Choose a shard key whose distribution matches your query pattern. Consistent hashing minimises resharding pain. Beware hot partitions.
11	Caching	Cache-aside is the default. TTL for expiry. LRU eviction. Cache hit rate and p99 latency are the two key metrics. Never cache personal data without a per-user cache key.
12	Transactions	ACID. Two-Phase Commit for distributed atomicity (expensive). Saga pattern (Ch 18) as the practical alternative for microservices. Optimistic vs pessimistic locking.
13	Load Balancing	L4 (IP/port) vs L7 (full HTTP, path/header routing). Health checks are mandatory. LB itself needs redundancy — it is a SPOF if deployed as a single instance.
14	Message Queues	Decouple producer and consumer in time. At-least-once + idempotent consumers. Kafka for streaming/replay. Dead-letter queues for repeatedly failing messages.
15	API Gateway	Single front door: auth, rate limiting, routing, logging. Token bucket rate limiting. Rate limit per-client, not globally. Gateway itself must scale and be redundant.
16	CDN	Geographic caching at the edge. Static content = easy, long TTL. Versioned URLs sidestep invalidation for build assets. Never cache personal data without per-user cache keys.
17	Consistent Hashing	Servers and keys on a ring; each key maps clockwise to the nearest server. Adding a node reshuffles $\sim 1/N$ data only. Virtual nodes fix uneven distribution.
18	Microservices	Monolith first unless justified. Circuit breakers prevent cascading failures. Saga for cross-service transactions. Beware the distributed monolith trap.
19	Availability	99.9% = 8.76 hrs/year downtime. Active-passive vs active-active. RTO = how fast to recover. RPO = how much data loss is acceptable. Check SPOFs at every layer.
20	Observability	Metrics (something is wrong) + Logs (what happened) + Traces (where in services). Track p99 not average. Alert on user-impact signals, not internal details.
21	Security	AuthN = who are you. AuthZ = what can you do. JWT = self-verifying, short-lived + revocable refresh token. Encrypt in transit (TLS) AND at rest. Least privilege.

Ch	Topic	The One Thing to Remember
22	Object Storage	Large files go in object storage (S3-like), never in a relational DB. Store a reference URL in the DB row. Hot/warm/cold tiers cut cost. Always pair with CDN for delivery.
23	Estimation Practice	Close the loop: number -> architectural consequence. Estimate for peak (2-3x average). State your assumptions explicitly before calculating.
24	URL Shortener	100:1 read:write -> cache is the highest-leverage component. Base62 counter for ID generation. AP system: brief staleness on a deleted link is harmless.
25	Rate Limiter + Cache	Per-server local counters fail with multiple servers — shared Redis holds the authoritative counter. Atomic operations prevent race conditions. Consistent hashing distributes cache keys.
26	Chat App	WebSockets for persistent push. Cross-gateway delivery via pub-sub. Shard messages by conversation_id. At-least-once + offline push notifications for disconnected users.
27	Social Feed	Fan-out on write = fast reads but write storm for celebrities. Fan-out on read = cheap writes but slower assembly. Hybrid: write for normal accounts, read for celebrities.
28	Uber + YouTube	Uber: geohash/quadtrees index + in-memory store for locations, relational DB for payments. YouTube: async transcoding via queue, object storage for files, CDN is mandatory.

6. The RADIO Checklist — Interview Day Card

R - Requirements

A - Architecture

D - Data Model

I - Interface

O - Optimize

7. How to Handle Mid-Interview Constraint Changes

Interviewers routinely change the problem mid-interview: 'now assume 100x the scale,' 'what if we need to support 50 countries,' 'what if strong consistency becomes a requirement.' This is deliberate — it probes how you handle shifting requirements under pressure.

- Do not panic and do not discard your whole design. State calmly what changes and what stays the same.
- Re-run the affected estimation section. New number, new architectural consequence — say it out loud.
- Name specifically what breaks first in your current design at the new scale, then name the fix for that specific component only.

- If a consistency requirement changes (eventual -> strong): 'This moves us from AP toward CP, meaning we accept some availability reduction in exchange for consistency guarantees. Concretely, that means synchronous replication (Ch 9) instead of async, which adds write latency.'

8. How to Handle 'I Don't Know' Moments

Every candidate hits a question they don't know the precise answer to. The best candidates don't freeze or guess wildly — they reason transparently.

- 'I am not certain of the exact implementation, but based on first principles I would reason...' — then apply what you do know.
- Name the trade-offs of the options you can think of, even if you cannot recall the standard solution. Trade-off articulation is itself a senior-level signal.
- 'That is a specific detail I would want to verify before a production decision — in an interview context I would lean toward X because...' This is honest and shows good engineering judgment.

9. Final Q&A Bank — Cross-Cutting Questions

Q1. How do you decide whether to use SQL or NoSQL?

A: I start with the access pattern: if the data has complex relationships that benefit from joins and ACID transactions matter, SQL is the default. If the data is document-like with variable structure, or needs to scale horizontally for very high write throughput with simple key-based lookups, I would consider NoSQL. The specific NoSQL type matters too: key-value for fast simple lookups, column-family for time-series or write-heavy data, document for flexible schemas, graph for relationship traversal. I never pick NoSQL as a default just because it 'scales better' — SQL with proper indexes and replication handles most systems at most scales.

Q2. How do you approach designing for reliability from day one?

A: I walk through every architectural layer and ask 'what happens to the whole system if this specific component fails?' For each SPOF I find, I identify the redundancy strategy: multiple app server instances behind a load balancer, database replication with automated failover, redundant load balancers with a floating IP, replicated message queue brokers. I also establish RTO and RPO targets early, because those numbers directly determine how expensive the redundancy strategy needs to be — a system that can tolerate 4 hours of downtime needs a completely different solution than one that must stay up through any single-component failure.

Q3. How would you scale a system experiencing write bottlenecks on its primary database?

A: I would first check whether the bottleneck is genuine write volume or a symptom of missing indexes causing expensive writes. If it is true write volume, the progression is: first, optimize within the single primary (batching, connection pooling, async acknowledgments where consistency allows); then add a cache layer to absorb reads inadvertently hitting the primary; then shard horizontally, choosing a shard key that distributes writes evenly and matches the dominant query pattern. I would use consistent hashing to make future resharding less painful, and I would be explicit about which queries now require cross-shard joins so the trade-off is clear.

Q4. Walk me through how you would explain the CAP theorem to a junior engineer.

A: I would start with a concrete failure scenario rather than the abstract definition: imagine two database nodes that normally sync with each other and the network between them breaks. Now every request must choose: respond using potentially-stale data on the local node (choosing Availability), or refuse to respond until the network recovers and consistency can be confirmed (choosing Consistency). In practice, since network partitions do happen, the real choice is usually between AP (stay up, possibly serve stale data) and CP (stay consistent, possibly refuse requests during a partition). I would then ask which choice their specific feature can tolerate — that makes it immediately practical.

Q5. What is the most important thing a system design answer needs to demonstrate?

A: That you understand WHY each design decision was made, not just what it is. Anyone can memorise 'use a CDN for static assets' — what interviewers are checking is whether you can reason from first principles: 'Our bandwidth math shows 5 Tbps peak, which no origin can serve directly, which is exactly what a CDN is built to absorb.' The insight, the trade-off, and the explicit connection to the problem's constraints are what differentiate a strong candidate from one who is pattern-matching to memorised templates.

10. The Night Before — Your Revision Checklist

- 1. Recite the five RADIO phases and what belongs in each, without looking. If any phase is fuzzy, re-read Phase 1-5 above.
- 2. Mentally walk through the 45-minute flow for one case study you know well — URL shortener or social feed.
- 3. Review the Top 20 Mistakes list. Are there any you have caught yourself making in recent practice?
- 4. Scan the cheat sheet for any row that feels vague. If Chapter N is fuzzy, re-read just that chapter's revision sheet box.
- 5. Practise saying out loud: 'Let me start by clarifying requirements before I begin designing.' Getting that opening sentence automatic removes the biggest source of interview-opening freeze.

11. Scenario-Based Practice Bank (New)

Scenario 1: You're 30 minutes into a 45-minute interview and realize you haven't done capacity estimation yet, having jumped straight into architecture.

How to approach it: Don't abandon the interview or panic — briefly pause and run a fast, rounded estimation now, explicitly connecting at least one number to an architectural decision you already made or need to adjust. A late but present estimation step is far better than skipping it entirely.

Scenario 2: Midway through designing a chat app sized for 20 million users, the interviewer says 'now assume this needs to support 2 billion users across 190 countries.'

How to approach it: Stay calm and re-run the key numbers (concurrent connections, message QPS) at the new scale, then name specifically what breaks first — likely the single-region WebSocket gateway pool and message database — and state the fix (geographic sharding, regional gateway clusters) rather than redesigning from scratch.

Scenario 3: An interviewer asks a very specific question about a technology you've never used (e.g., 'how does DynamoDB's adaptive capacity work internally?').

How to approach it: Don't guess wildly or freeze. Say plainly you haven't worked with that specific mechanism, then reason from first principles about what problem it's likely solving based on similar systems you do know (e.g., hot-partition mitigation), and note you'd verify the exact mechanism before a production decision.

Scenario 4: You've spent 25 minutes on the architecture phase because the interviewer kept asking follow-up questions, and only 20 minutes remain for deep dive and optimization.

How to approach it: Proactively acknowledge the time and re-prioritize out loud: pick the single most important deep-dive topic and move decisively to Phase 5's reliability/observability points even briefly, rather than letting the interview end with zero mention of scaling, SPOFs, or monitoring — that phase carries real signal and shouldn't be silently dropped.

Scenario 5: A candidate describes their design as using 'a database' and 'some caching' without ever naming SQL vs NoSQL or a specific eviction/invalidation strategy.

How to approach it: This is exactly the vagueness the Top 20 Mistakes list warns against. Push for specifics: name the actual database type and justify it by access pattern, and name the cache eviction policy (e.g., LRU) and invalidation approach (TTL, explicit, event-driven) rather than leaving these as unstated placeholders.

Scenario 6: You finish a design with time to spare and the interviewer asks 'what would you add if I said reliability was the most important factor here?'

How to approach it: Walk through SPOFs at every layer systematically (load balancer, app servers, database, cache, queue) and name a redundancy strategy for each, then state concrete RTO/RPO targets this would imply — showing the same layer-by-layer discipline Chapter 19 and this playbook's Phase 5 both emphasize.

Chapter 30 — One-Page Revision Sheet

- Phase 1 (0-5 min): Clarify Requirements. Functional scope, scale numbers, non-functional targets, constraints. ASK before designing.
- Phase 2 (5-10 min): Capacity Estimation. QPS, storage, bandwidth, connections. Every number must produce a stated architectural consequence.
- Phase 3 (10-20 min): High-Level Architecture. Standard spine + justified additions. Narrate every component choice and every deliberate omission.
- Phase 4 (20-35 min): Deep Dive + Data Model. Concrete schema, SQL vs NoSQL justified, sharding key named, specific API endpoints with fields.
- Phase 5 (35-45 min): Optimize + Trade-offs. What breaks first at 10x (name it + fix). SPOFs at every layer (name redundancy strategy). Observability (name specific metrics).
- The 45-minute flow is deterministic — run it the same way every time. The interviewer grades your thinking process, not just the finished diagram.
- Narrate everything out loud. Ask for feedback. Own uncertainty gracefully. Time-box yourself ruthlessly.

Memory hook: A musician's setlist before a gig: every song has been rehearsed. This is the card that says play them in this order, at this tempo, hit these key moments. The music is in your hands already.